

```
=====
FILE: catalogs\__init__.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
=====
FILE: catalogs\db2_sql_types.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.catalogs.db2.db2_sql_types import *
```

```
=====
FILE: catalogs\dclgen_schema.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.catalogs.mapping.dclgen_schema import *
```

```
=====
FILE: catalogs\output_sections.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.catalogs.output.output_sections import *
```

```
=====
FILE: catalogs\sheet_mapping_schema.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.catalogs.mapping.sheet_mapping_schema import *
```

```
=====
FILE: docs\ARCHITECTURE.md
ENCODING: utf-8
=====
```

```
# IDMS DB2 Converter Architecture
```

```
## Purpose
```

This project converts IDMS COBOL source into DB2 embedded SQL COBOL.

The conversion is metadata-driven and conservative.

```
## Inputs
```

- Sheet Mapping Excel or CSV
- One or more DCLGEN files
- Optional Copybook files
- IDMS COBOL source code

```
## Outputs
```

- Converted DB2 COBOL code
- Validation messages
- Metadata overview
- Record summary
- Column summary
- Relationship summary

- Sheet Mapping preview

Core Authority Rules

- Sheet Mapping is the authority for DB2 record/table names.
- Sheet Mapping is the authority for DB2 column names.
- DCLGEN is the authority for COBOL host variable names and PIC clauses.
- Original COBOL is the authority for business flow.
- Final output is resequenced in manual-style COBOL format.

Layer Responsibilities

reference/catalogs

Owns static catalogs:

- Sheet Mapping column names
- Sheet Mapping aliases
- DCLGEN conventions
- SQL type names
- Output section names and generated block markers

No parsing logic.

No runtime conversion logic.

reference/patterns

Owns regex patterns only.

Rules:

- No business rules
- No services
- No parsers
- No generators
- No validators
- No orchestration

reference/rules

Owns business, conversion, validation, layout, and DB2 rules.

Rules:

- No regex
- No parsing logic
- No service logic
- No transformer logic
- No generator logic
- No validator logic

parsers

Own parsing logic only.

Parsers:

- Parse Sheet Mapping
- Parse DCLGEN
- Parse Copybook
- Parse COBOL source operations

Parsers do not convert COBOL and do not generate SQL.

repositories

Own metadata lookup over parsed inputs.

Repositories:

- Provide Sheet Mapping lookup
- Provide DCLGEN lookup
- Provide Copybook lookup

Repositories do not generate SQL and do not transform COBOL.

resolvers

Resolve metadata names and references.

Resolvers:

- Resolve DB2 table names
- Resolve DB2 column names
- Resolve host variables
- Resolve cursor names
- Resolve active record context
- Resolve update SQL plans

Resolvers do not parse files and do not modify COBOL.

transformers

Transform COBOL source.

Transformers:

- Convert IDMS statements into DB2-compatible lines
- Rewrite safe qualified field references
- Apply conservative PIC length fixes

Transformers do not own regex patterns.

generators

Generate DB2 SQL and DB2 COBOL support blocks.

Generators:

- SQL snippets
- SQL error paragraph
- DB2 infrastructure
- Cursor paragraphs

Generators do not hardcode business names.

composers

Compose and normalize generated COBOL.

Composers:

- Manual layout
- Fixed-format output
- Cursor flow
- Cursor ordering
- Final COBOL normalization
- Retrieval program normalization
- Update program normalization
- Restart conversion handling
- SQLCODE wrapper normalization

services

Reusable stateless helpers.

Services:

- Fixed-format line operations
- Name normalization
- Date format normalization
- Program-name synchronization
- Area alignment

validators

Validation only.

Validators:

- Input validation
- Mapping validation

- Production COBOL validation

orchestration

Coordinates the conversion pipeline.

Orchestration:

- Creates repositories
- Creates resolvers
- Creates generators
- Creates transformers
- Runs conversion stages
- Builds final result

Orchestration does not own conversion rules or regex patterns.

ui

Streamlit UI only.

UI:

- Upload files
- Trigger parsing
- Trigger conversion
- Display outputs
- Download generated COBOL text

UI does not implement conversion logic.

Import Rules

Internal modules should use relative imports:

```
```python
from ..domain.models import ConversionInput
from ..reference.rules.db2.cursor_rules import cursor_paragraph_number
from ..reference.patterns.idms.statement_patterns import OBTAIN_CALC_PATTERN
```

```
=====
FILE: docs\DEVELOPER_GUIDE.md
ENCODING: utf-8
=====
```

### # Developer Guide

#### ## Local Setup

From the project root:

```
```powershell
$env:PYTHONPATH = "src"
python -m pip install -e .
python -m pip install pytest streamlit openpyxl
```

```
=====
FILE: patterns\__init__.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
=====
FILE: patterns\cobol_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.patterns.cobol.structure_patterns import *
```

```
=====
FILE: patterns\db2_date_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.db2.date_patterns import *
```

```
=====
FILE: patterns\db2_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.db2.infrastructure_patterns import *
```

```
=====
FILE: patterns\dclgen_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.db2.dclgen_patterns import *
```

```
=====
FILE: patterns\field_reference_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.db2.date_patterns import *
```

```
=====
FILE: patterns\idms_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.idms.statement_patterns import *
from idms_db2_phase2.reference.patterns.idms.declarative_patterns import *
```

```
=====
FILE: patterns\sequence_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.cobol.sequence_patterns import *
```

```
=====
FILE: patterns\sql_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.db2.sql_patterns import *
```

```
=====
FILE: patterns\validation_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.patterns.validation.production_patterns import *
```

```
=====
FILE: pyproject.toml
ENCODING: utf-8
=====
```

```
[project]
name = "idms-db2-code-conversion"
version = "0.1.0"
description = "IDMS COBOL to DB2 embedded SQL COBOL converter"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "openpyxl>=3.1.5",
    "pydantic>=2.0.0",
    "pydantic-settings>=2.0.0",
    "streamlit>=1.30.0",
]

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

```
=====
FILE: README.md
ENCODING: utf-8
=====
```

IDMS DB2 Code Conversion

Clean architecture Python project for converting IDMS COBOL programs into DB2 embedded SQL COBOL.

Inputs

- Sheet Mapping Excel or CSV
- One or more DCLGEN text files
- Optional Copybook text file
- Optional Copybook PDF
- IDMS COBOL source code

Output

- Converted DB2 COBOL code
- Validation messages
- Metadata overview
- Record summary
- Column summary
- Set / relationship summary
- Sheet Mapping preview

Architecture Rules

- `reference/catalogs/` contains static catalog values, column names, aliases, labels, and SQL type lists.
- `reference/patterns/` contains regex patterns only.
- `reference/rules/` contains business, conversion, validation, and layout rules only.
- `parsers/` contains parsing logic only.
- `repositories/` contains parsed metadata lookup logic only.
- `resolvers/` contains table, column, host variable, cursor, and context resolution logic only.
- `analyzers/` contains source analysis logic only.
- `transformers/` contains IDMS-to-DB2 transformation logic only.

- `generators/` contains generated SQL and COBOL block generation only.
- `composers/` contains final assembly and COBOL normalization logic only.
- `services/` contains reusable stateless services only.
- `validators/` contains validation logic only.
- `orchestration/` coordinates the conversion pipeline only.

Run

```
```bash
set PYTHONPATH=src
python -m streamlit run src/idms_db2_phase2/app.py --server.port 8502
```

```
=====
FILE: rules__init__.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
=====
FILE: rules\authority_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.authority.input_authority_rules import *
from idms_db2_phase2.reference.rules.authority.no_hardcode_rules import *
```

```
=====
FILE: rules\business_flow_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.conversion.business_flow_rules import *
```

```
=====
FILE: rules\conversion_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.conversion.idms_to_db2_rules import *
```

```
=====
FILE: rules\cursor_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.db2.cursor_rules import *
```

```
=====
FILE: rules\db2_date_conversion_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.db2.date_conversion_rules import *
```

```
=====
FILE: rules\db2_output_date_conversion_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.db2.output_date_conversion_rules import *
```

```
=====
FILE: rules\db2_validation_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.validation.db2_validation_rules import *
```

```
=====
FILE: rules\field_mapping_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.conversion.field_mapping_rules import *
```

```
=====
FILE: rules\final_cobol_normalization_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.normalization.final_cobol_normalization_rules import *
```

```
=====
FILE: rules\fixed_format_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.cobol.fixed_format_rules import *
```

```
=====
FILE: rules\sql_generation_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.db2.sql_generation_rules import *
```

```
=====
FILE: rules\update_sql_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.reference.rules.db2.update_sql_rules import *
```

```
=====
FILE: rules\validation_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.reference.rules.validation.validation_messages import *
```

```
=====
FILE: run_clean_tests.ps1
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_phase2__init__.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
__all__ = [
 "__version__",
]
```

```
__version__ = "0.1.0"
```

```
=====
FILE: src\idms_db2_phase2\analyzers__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_phase2\analyzers\metadata_summary_builder.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from ..domain.models import SheetMappingRow
from ..services.name_normalizer import NameNormalizer
```

```
class MetadataSummaryBuilder:
```

```
 """
```

```
 Builds metadata summaries from Sheet Mapping rows.
```

```
 Used by orchestration/UI diagnostics to show:
```

- Metadata overview
- Record summary
- Column summary
- Relationship summary
- Sheet Mapping preview

```
 This analyzer does not transform COBOL and does not generate SQL.
```

```
 """
```

```
 def metadata_overview(
 self,
 sheet_mapping_rows: list[SheetMappingRow],
 dclgen_count: int,
 copybook_count: int,
 cobol_text: str,
 target_program_id: str,
) -> dict[str, object]:
 return {
 "sheet_mapping_rows": len(sheet_mapping_rows or []),
 "dclgen_columns": int(dclgen_count or 0),
 "copybook_fields": int(copybook_count or 0),
 "idms_cobol_text_length": len(str(cobol_text or "")),
 "target_program_id": str(target_program_id or ""),
 }
```

```
 def record_summary(
```

```

self,
rows: list[SheetMappingRow],
) -> list[dict[str, object]]:
 grouped: dict[str, list[SheetMappingRow]] = {}

 for row in rows or []:
 record = NameNormalizer.normalize(row.cobol_record_idms)

 if not record:
 continue

 grouped.setdefault(record, []).append(row)

 output: list[dict[str, object]] = []

 for record_name, record_rows in sorted(grouped.items()):
 db2_table = self._first_db2_table(record_rows)
 columns = self._unique(
 [
 NameNormalizer.to_sql(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.to_sql(row.new_db2_field_name)
]
)
 key_columns = self._unique(
 [
 NameNormalizer.to_sql(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.to_sql(row.new_db2_field_name)
 and (
 NameNormalizer.normalize(row.idms_key)
 or NameNormalizer.normalize(row.db2_key)
)
]
)

 output.append(
 {
 "record_name": record_name,
 "db2_table": db2_table,
 "column_count": len(columns),
 "key_columns": key_columns,
 }
)

 return output

def column_summary(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows or []:
 record = NameNormalizer.normalize(row.cobol_record_idms)
 source = NameNormalizer.normalize(row.cobol_zone)
 db2_table = self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_record),
 NameNormalizer.to_sql(row.cross_application_db2_table),
)
 db2_column = self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_field_name),
 NameNormalizer.to_sql(row.cross_application_db2_field_name),
)

 if not any([record, source, db2_table, db2_column]):
 continue

 output.append(
 {
 "record_name": record,
 "source_field": source,
 "db2_table": db2_table,
 "db2_column": db2_column,
 "idms_key": NameNormalizer.normalize(row.idms_key),
 "db2_key": NameNormalizer.normalize(row.db2_key),
 }
)

```

```

 "relation": NameNormalizer.normalize(row.relation),
 }
)

return output

def relationship_summary(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows or []:
 relation = NameNormalizer.normalize(row.relation)

 if not relation:
 continue

 output.append(
 {
 "record_name": NameNormalizer.normalize(row.cobol_record_idms),
 "relation": relation,
 "db2_table": self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_record),
 NameNormalizer.to_sql(row.cross_application_db2_table),
),
 "db2_column": self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_field_name),
 NameNormalizer.to_sql(row.cross_application_db2_field_name),
),
 "cross_application_db2_table": NameNormalizer.to_sql(
 row.cross_application_db2_table
),
 "cross_application_db2_field_name": NameNormalizer.to_sql(
 row.cross_application_db2_field_name
),
 }
)

 return output

def sheet_mapping_preview(
 self,
 rows: list[SheetMappingRow],
 limit: int = 20,
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in list(rows or [])[:limit]:
 output.append(
 {
 "Cobol Record IDMS": row.cobol_record_idms,
 "Cobol Zone": row.cobol_zone,
 "IDMS Key": row.idms_key,
 "IDMS PIC Clause": row.idms_pic_clause,
 "Length of Field Bytes": row.length_of_field_bytes,
 "Field end position": row.field_end_position,
 "DB2 Key": row.db2_key,
 "New DB2 Record": row.new_db2_record,
 "New DB2 Field name": row.new_db2_field_name,
 "New DB2 Data Type": row.new_db2_data_type,
 "Hopex Expression Type": row.hopex_expression_type_remark,
 "Remarks": row.remarks,
 "Relation": row.relation,
 "Reference Field Name": row.reference_field_name_copybook,
 "Reference Field PIC Clause": row.reference_field_pic_clause,
 "Cross Application DB2 Table": row.cross_application_db2_table,
 "Cross Application DB2 Field Name": (
 row.cross_application_db2_field_name
),
 "Cross Appln DB2 Data Type": (
 row.cross_application_db2_data_type
),
 "Basetype": row.basetype,
 }
)
)

```

```

 return output

 def _first_db2_table(
 self,
 rows: list[SheetMappingRow],
) -> str:
 for row in rows:
 table = self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_record),
 NameNormalizer.to_sql(row.cross_application_db2_table),
)

 if table:
 return table

 return ""

 def _first_non_empty(
 self,
 *values: str,
) -> str:
 for value in values:
 text = str(value or "").strip()

 if text:
 return text

 return ""

 def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```
=====
FILE: src\idms_db2_phase2\app.py
```

```
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from pathlib import Path
import sys
```

```
import streamlit as st
```

```
CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[1]
PROJECT_ROOT = CURRENT_FILE.parents[2]
```

```
for path in [PROJECT_ROOT, SRC_DIR]:
 if str(path) not in sys.path:
 sys.path.insert(0, str(path))
```

```
from idms_db2_phase2.config.application_settings import APP_DESCRIPTION, APP_NAME
from idms_db2_phase2.ui.main_page import render_main_page
```

```
def main() -> None:
 st.set_page_config(
 page_title=APP_NAME,
 layout="wide",
)
 st.title(APP_NAME)
 st.caption(APP_DESCRIPTION)

 render_main_page()
```

```
if __name__ == "__main__":
 main()
```

```
=====
FILE: src\idms_db2_phase2\composers__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\composers\cobol_formatter.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
class CobolFormatter:
```

```
 """
```

```
 Preserve-first COBOL formatter.
```

```
 This formatter intentionally does not re-indent or restructure
 the full COBOL program.
```

```
 It only:
```

- Normalizes Windows/Unix line endings.
- Removes trailing spaces.
- Limits excessive blank lines.
- Ensures one final newline.

```
 """
```

```
 def format(
```

```
 self,
```

```
 text: str,
```

```
) -> str:
```

```
 if not text:
```

```
 return ""
```

```
 normalized = self._normalize_line_endings(text)
```

```
 normalized = self._rstrip_lines(normalized)
```

```
 normalized = self._normalize_excess_blank_lines(normalized)
```

```
 return normalized.rstrip() + "\n"
```

```
 def _normalize_line_endings(
```

```
 self,
```

```
 text: str,
```

```
) -> str:
```

```
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")
```

```
 def _rstrip_lines(
```

```
 self,
```

```
 text: str,
```

```
) -> str:
```

```
 return "\n".join(line.rstrip() for line in str(text or "").splitlines())
```

```
 def _normalize_excess_blank_lines(
```

```
 self,
```

```
 text: str,
```

```
) -> str:
```

```
 output = str(text or "")
```

```

while "\n\n\n" in output:
 output = output.replace("\n\n\n", "\n\n")

return output

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from dataclasses import dataclass

from ..reference.patterns.cobol.cursor_flow_patterns import (
 CONTINUE_PATTERN,
 CURSOR_PARAGRAPH_HEADER_PATTERN,
 PERFORM_BUSINESS_PARAGRAPH_PATTERN,
 PERFORM_CURSOR_PATTERN,
 UNTIL_SQLCODE_100_PATTERN,
 WHEN_ZERO_PATTERN,
)
from ..reference.patterns.cobol.sequence_patterns import strip_sequence_numbers

```

```

@dataclass(slots=True)
class CursorFlowPlan:
 cursor_name: str
 open_paragraph: str
 fetch_paragraph: str
 close_paragraph: str
 eoc_condition: str
 business_paragraph: str
 open_index: int
 fetch_index: int
 business_index: int
 until_index: int

```

```

class CursorFlowComposer:
 """
 Normalizes generated cursor execution flow.

 Converts mechanical flow:

```

```

 PERFORM 710-OPEN-CURSOR.
 PERFORM 720-FETCH-CURSOR.
 PERFORM BUSINESS-PARA
 UNTIL SQLCODE = 100.

```

into:

```

 PERFORM 710-OPEN-CURSOR.
 PERFORM 720-FETCH-CURSOR UNTIL WS-CURSOR-EOC-YES.
 PERFORM 730-CLOSE-CURSOR.

```

It also updates generated FETCH paragraphs so WHEN ZERO performs the business paragraph instead of only CONTINUE.

This composer does not hardcode program names, record names, table names, cursor names, or business paragraph names.

```

"""

```

```

LOOKAHEAD_LIMIT = 120

```

```

def __init__(
 self,
) -> None:
 self.messages: list[str] = []

```

```

def compose(
 self,
 cobol_text: str,
 operations: list[object] | None = None,

```

```

) -> str:
 self.messages = []
 text = str(cobol_text or "")

 if not text:
 return ""

 lines = text.splitlines()
 plans = self._cursor_flow_plans(lines)

 if not plans:
 self.messages.append("Cursor flow: no cursor flow normalization required.")
 return text.rstrip() + "\n"

 lines = self._rewrite_call_sites(lines, plans)
 lines = self._rewrite_fetch_paragraphs(lines, plans)

 self.messages.append(
 f"Cursor flow: normalized {len(plans)} cursor flow block(s)."
)
 return "\n".join(lines).rstrip() + "\n"

def _cursor_flow_plans(
 self,
 lines: list[str],
) -> list[CursorFlowPlan]:
 plans: list[CursorFlowPlan] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])
 open_match = PERFORM_CURSOR_PATTERN.match(logical)

 if not open_match:
 index += 1
 continue

 if open_match.group("operation").upper() != "OPEN":
 index += 1
 continue

 cursor_name = open_match.group("cursor").upper()
 open_number = int(open_match.group("number"))
 fetch_number = open_number + 10
 close_number = open_number + 20

 fetch_index = self._find_fetch_after_open(
 lines=lines,
 start_index=index + 1,
 cursor_name=cursor_name,
 fetch_number=fetch_number,
)

 if fetch_index < 0:
 index += 1
 continue

 business_info = self._find_business_loop_after_fetch(
 lines=lines,
 start_index=fetch_index + 1,
)

 if business_info is None:
 index += 1
 continue

 business_index, until_index, business_paragraph = business_info

 plans.append(
 CursorFlowPlan(
 cursor_name=cursor_name,
 open_paragraph=f"{open_number:03d}-OPEN-{cursor_name}",
 fetch_paragraph=f"{fetch_number:03d}-FETCH-{cursor_name}",
 close_paragraph=f"{close_number:03d}-CLOSE-{cursor_name}",
 eoc_condition=f"WS-{cursor_name}-EOC-YES",
 business_paragraph=business_paragraph,
)
)

```

```

 open_index=index,
 fetch_index=fetch_index,
 business_index=business_index,
 until_index=until_index,
)
)

index = until_index + 1

return plans

def _find_fetch_after_open(
 self,
 lines: list[str],
 start_index: int,
 cursor_name: str,
 fetch_number: int,
) -> int:
 end_index = min(
 len(lines),
 start_index + self.LOOKAHEAD_LIMIT,
)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])
 match = PERFORM_CURSOR_PATTERN.match(logical)

 if not match:
 continue

 if match.group("operation").upper() != "FETCH":
 continue

 if match.group("cursor").upper() != cursor_name:
 continue

 if int(match.group("number")) != fetch_number:
 continue

 return index

 return -1

def _find_business_loop_after_fetch(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[int, int, str] | None:
 end_index = min(
 len(lines),
 start_index + self.LOOKAHEAD_LIMIT,
)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])
 business_match = PERFORM_BUSINESS_PARAGRAPH_PATTERN.match(logical)

 if not business_match:
 continue

 if index + 1 >= len(lines):
 continue

 until_logical = self._logical(lines[index + 1])

 if not UNTIL_SQLCODE_100_PATTERN.match(until_logical):
 continue

 paragraph = business_match.group("paragraph").upper()
 return index, index + 1, paragraph

 return None

def _rewrite_call_sites(
 self,
 lines: list[str],

```

```

 plans: list[CursorFlowPlan],
) -> list[str]:
 remove_indexes: set[int] = set()
 replacements: dict[int, str] = {}

 for plan in plans:
 replacements[plan.fetch_index] = (
 f"PERFORM {plan.fetch_paragraph} "
 f"UNTIL {plan.eoc_condition}."
)
 remove_indexes.add(plan.business_index)
 remove_indexes.add(plan.until_index)

 close_line = f"PERFORM {plan.close_paragraph}."
 replacements[plan.until_index] = close_line
 remove_indexes.discard(plan.until_index)

 output: list[str] = []

 for index, line in enumerate(lines):
 if index in replacements:
 output.append(self._with_reference_indent(line, replacements[index]))
 continue

 if index in remove_indexes:
 continue

 output.append(line)

 return output

 def _rewrite_fetch_paragraphs(
 self,
 lines: list[str],
 plans: list[CursorFlowPlan],
) -> list[str]:
 output = list(lines)

 for plan in plans:
 paragraph_index = self._find_paragraph_header(
 lines=output,
 paragraph_name=plan.fetch_paragraph,
)

 if paragraph_index < 0:
 continue

 output = self._replace_when_zero_continue(
 lines=output,
 paragraph_index=paragraph_index,
 business_paragraph=plan.business_paragraph,
)

 return output

 def _find_paragraph_header(
 self,
 lines: list[str],
 paragraph_name: str,
) -> int:
 target = paragraph_name.upper().rstrip(".") + "."

 for index, line in enumerate(lines):
 logical = self._logical(line).upper()

 if logical == target:
 return index

 return -1

 def _replace_when_zero_continue(
 self,
 lines: list[str],
 paragraph_index: int,
 business_paragraph: str,
) -> list[str]:

```

```

output = list(lines)
end_index = min(len(output), paragraph_index + self.LOOKAHEAD_LIMIT)

for index in range(paragraph_index, end_index):
 logical = self._logical(output[index])

 if CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical) and index != paragraph_index:
 return output

 if not WHEN_ZERO_PATTERN.match(logical):
 continue

 continue_index = index + 1

 if continue_index >= len(output):
 return output

 continue_logical = self._logical(output[continue_index])

 if not CONTINUE_PATTERN.match(continue_logical):
 return output

 output[continue_index] = self._with_reference_indent(
 output[continue_index],
 f"PERFORM {business_paragraph}.",
)
 return output

return output

def _with_reference_indent(
 self,
 reference_line: str,
 replacement_body: str,
) -> str:
 indent = self._leading_spaces(reference_line)
 return f"{indent}{replacement_body}"

def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

def _leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_order_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.cobol.sequence_patterns import strip_sequence_numbers

```

```

class CursorOrderComposer:
 """
 Normalizes ORDER BY clauses in generated cursor declarations.

```

Generic rule:

- A cursor declaration without a WHERE clause is treated as parent/root.
- Parent/root cursor ORDER BY is removed.
- A cursor declaration with a WHERE clause is treated as child/dependent and keeps ORDER BY.

This composer does not hardcode cursor names, table names, record names, or business fields.

"""

```
def __init__(
 self,
) -> None:
 self.messages: list[str] = []

def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 lines = source.splitlines()
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self._logical(lines[index])

 if not self._is_exec_sql_start(logical):
 output.append(lines[index])
 index += 1
 continue

 block, next_index = self._collect_exec_sql_block(lines, index)

 if not self._is_cursor_declare_block(block):
 output.extend(block)
 index = next_index
 continue

 updated_block = self._normalize_cursor_order_block(block)

 if updated_block != block:
 changed = True

 output.extend(updated_block)
 index = next_index

 if changed:
 self.messages.append("Cursor order: removed parent cursor ORDER BY clause.")

 return "\n".join(output).rstrip() + "\n"

def _collect_exec_sql_block(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 output: list[str] = []
 index = start_index

 while index < len(lines):
 output.append(lines[index])
 logical = self._logical(lines[index])
 index += 1

 if self._is_exec_sql_end(logical):
 break

 return output, index
```

```

def _normalize_cursor_order_block(
 self,
 block: list[str],
) -> list[str]:
 if not self._has_order_by(block):
 return block

 if self._has_where_before_order_by(block):
 return block

 output: list[str] = []
 skipping_order_by = False

 for line in block:
 logical = self._logical(line).upper()

 if self._is_order_by_start(logical):
 skipping_order_by = True
 continue

 if skipping_order_by and self._is_order_by_end(logical):
 skipping_order_by = False
 output.append(line)
 continue

 if skipping_order_by:
 continue

 output.append(line)

 return output

def _is_cursor_declare_block(
 self,
 block: list[str],
) -> bool:
 text = "\n".join(self._logical(line).upper() for line in block)
 return "DECLARE " in text and "CURSOR " in text

def _has_order_by(
 self,
 block: list[str],
) -> bool:
 return any(
 self._is_order_by_start(self._logical(line).upper())
 for line in block
)

def _has_where_before_order_by(
 self,
 block: list[str],
) -> bool:
 found_where = False

 for line in block:
 logical = self._logical(line).upper()

 if logical == "WHERE" or logical.startswith("WHERE "):
 found_where = True

 if self._is_order_by_start(logical):
 return found_where

 return False

def _is_order_by_start(
 self,
 logical: str,
) -> bool:
 if logical == "ORDER BY":
 return True

 if logical.startswith("ORDER BY "):
 return True

 return False

```

```

def _is_order_by_end(
 self,
 logical: str,
) -> bool:
 if logical == "FOR READ ONLY":
 return True

 if logical.startswith("FOR READ ONLY"):
 return True

 if logical == "END-EXEC":
 return True

 if logical == "END-EXEC.":
 return True

 return False

def _is_exec_sql_start(
 self,
 logical: str,
) -> bool:
 return logical.upper().startswith("EXEC SQL")

def _is_exec_sql_end(
 self,
 logical: str,
) -> bool:
 upper = logical.upper()
 return upper == "END-EXEC" or upper == "END-EXEC."

def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

```

```

=====
FILE: src\idms_db2_phase2\composers\db2_date_comparison_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.db2.date_patterns import (
 DATE_WORKING_STORAGE_MARKER_PATTERN,
 DB2_DATE_COMPARISON_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 WORKING_STORAGE_SECTION_PATTERN,
)
from ..reference.rules.db2.date_conversion_rules import (
 DB2_DATE_BASE_WORKING_STORAGE_LINES,
 DB2_DATE_COMPARISON_WS_MARKER,
 DB2_DATE_CONVERSION_LINE_TEMPLATES,
 DB2_DATE_HELPER_FIELD_TEMPLATE,
 DB2_DATE_HIGH_NUMERIC_LITERAL,
 DB2_DATE_HIGH_VALUE_LITERAL,
 DB2_DATE_IF_REPLACEMENT_TEMPLATE,
 DB2_DATE_LOW_VALUE_LITERAL,
)
from ..services.name_normalizer import NameNormalizer

class Db2DateComparisonComposer:
 """
 Realigns DB2 date host fields before comparing them with numeric
 COBOL date fields such as PARMDATE.

 No program name, table name, DCLGEN group name, or business field name
 is hardcoded.
 """

 def __init__(
 self,

```

```

) -> None:
 self.messages: list[str] = []

def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 lines = source.splitlines()
 output: list[str] = []
 helpers: list[str] = []

 for line in lines:
 match = DB2_DATE_COMPARISON_PATTERN.match(line)

 if not match:
 output.append(line)
 continue

 indent = match.group("indent")
 field_name = match.group("field").upper()
 group_name = match.group("group").upper()
 condition = match.group("condition")
 helper = self._helper_name(field_name)

 helpers.append(helper)

 output.extend(
 self._conversion_lines(
 indent=indent,
 field_name=field_name,
 group_name=group_name,
 helper=helper,
)
)
 output.append(
 DB2_DATE_IF_REPLACEMENT_TEMPLATE.format(
 indent=indent,
 helper=helper,
 condition=condition,
)
)

 updated = "\n".join(output).rstrip() + "\n"

 if helpers:
 updated = self._ensure_working_storage(
 text=updated,
 helpers=helpers,
)
 self.messages.append(
 f"DB2 date comparison: generated {len(set(helpers))} helper field(s)."
)

 return updated

def _conversion_lines(
 self,
 indent: str,
 field_name: str,
 group_name: str,
 helper: str,
) -> list[str]:
 return [
 template.format(
 indent=indent,
 field_name=field_name,
 group_name=group_name,
 helper=helper,
 low_value=DB2_DATE_LOW_VALUE_LITERAL,
 high_value=DB2_DATE_HIGH_VALUE_LITERAL,

```

```

 high_numeric=DB2_DATE_HIGH_NUMERIC_LITERAL,
)
 for template in DB2_DATE_CONVERSION_LINE_TEMPLATES
]

def _ensure_working_storage(
 self,
 text: str,
 helpers: list[str],
) -> str:
 if DATE_WORKING_STORAGE_MARKER_PATTERN.search(text):
 return text

 helper_lines = [
 DB2_DATE_HELPER_FIELD_TEMPLATE.format(helper=helper)
 for helper in self._unique(helpers)
]

 block = "\n".join(
 [
 *DB2_DATE_BASE_WORKING_STORAGE_LINES,
 *helper_lines,
]
)

 lines = text.splitlines()
 insert_index = self._working_storage_insert_index(lines)

 if insert_index < 0:
 return f"{block}\n{text.rstrip()}\n"

 output = lines[:insert_index]
 output.append(block)
 output.extend(lines[insert_index:])

 return "\n".join(output).rstrip() + "\n"

def _working_storage_insert_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if WORKING_STORAGE_SECTION_PATTERN.match(line.strip()):
 return index + 1

 for index, line in enumerate(lines):
 if PROCEDURE_DIVISION_PATTERN.match(line.strip()):
 return index

 return -1

def _helper_name(
 self,
 field_name: str,
) -> str:
 return f"HELP-{NameNormalizer.to_cobol(field_name)}"

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(value)

```

return output

```
=====
FILE: src\idms_db2_phase2\composers\final_cobol_normalization_composer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass

from ..reference.patterns.normalization.final_cobol_patterns import (
 DB2_COLON_COMMENT_BODY_PATTERN,
 DB2_SPACE_COMMENT_BODY_PATTERN,
 STRING_END_PATTERN,
 STRING_INTRO_PATTERN,
 STRING_START_PATTERN,
)
from ..reference.rules.normalization.final_cobol_normalization_rules import (
 DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
)
from ..services.cobol_area_alignment_service import CobolAreaAlignmentService
from ..services.cursor_select_minimizer_service import CursorSelectMinimizerService
from ..services.db2_date_format_service import Db2DateFormatService
from ..services.final_sequence_resequencer_service import (
 FinalSequenceResequencerService,
)
from ..services.fixed_format_line_service import FixedFormatLineService
from ..services.program_name_sync_service import ProgramNameSyncService
from ..services.update_procedure_normalization_service import (
 UpdateProcedureNormalizationService,
)
```

```
@dataclass(frozen=True)
class FinalCobolNormalizationConfig:
 """
 Configuration for final COBOL normalization.
 """

 db2_date_external_format: str = DEFAULT_DB2_DATE_EXTERNAL_FORMAT
 require_order_by_columns_in_select: bool = ORDER_BY_COLUMNS_IN_SELECT_DEFAULT
 resequence_final_output: bool = True
```

```
class FinalCobolNormalizationComposer:
 """
 Applies final generic COBOL normalization.

 Responsibilities:
 - Program-name synchronization.
 - DB2 date format correction.
 - Cursor SELECT/FETCH minimization.
 - Update procedure normalization.
 - Procedure Division Area B alignment.
 - Generated comment normalization.
 - STRING block manual-style formatting.
 - Final manual-style resequencing.
 """

 def __init__(
 self,
 config: FinalCobolNormalizationConfig | None = None,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.config = config or FinalCobolNormalizationConfig()
 self.fixed_format = fixed_format or FixedFormatLineService()

 self.program_name_sync = ProgramNameSyncService()
 self.date_format_service = Db2DateFormatService(
 date_external_format=self.config.db2_date_external_format,
)
 self.cursor_select_minimizer = CursorSelectMinimizerService(
 fixed_format=self.fixed_format,
```

```

 require_order_by_columns_in_select=(
 self.config.require_order_by_columns_in_select
),
)
 self.update_procedure_normalization = UpdateProcedureNormalizationService(
 fixed_format=self.fixed_format,
)
 self.area_alignment = CobolAreaAlignmentService(
 fixed_format=self.fixed_format,
)
 self.sequence_resequencer = FinalSequenceResequencerService(
 fixed_format=self.fixed_format,
)
 self.messages: list[str] = []

def compose(
 self,
 text: str,
) -> str:
 output = str(text or "")

 if not output:
 return ""

 self.messages = []

 output = self.program_name_sync.sync(output)
 output = self.date_format_service.apply(output)
 output = self.cursor_select_minimizer.minimize(output)
 output = self.update_procedure_normalization.apply(output)
 output = self.area_alignment.align(output)
 output = self._format_generated_comments(output)
 output = self._format_string_blocks(output)

 if self.config.resequence_final_output:
 output = self.sequence_resequencer.resequence(output)

 return output.rstrip() + "\n"

def _format_generated_comments(
 self,
 text: str,
) -> str:
 lines = str(text or "").splitlines()
 output: list[str] = []

 for line in lines:
 if not self.fixed_format.is_fixed_line(line):
 output.append(line)
 continue

 left, indicator, body, right = self.fixed_format.split(line)

 if indicator != "":
 output.append(line)
 continue

 clean_body = body.strip()

 if DB2_COLON_COMMENT_BODY_PATTERN.match(clean_body):
 normalized = DB2_COLON_COMMENT_BODY_PATTERN.sub(
 "DB2:",
 clean_body,
 count=1,
).strip()
 output.append(
 self.fixed_format.build(
 body=normalized,
 left_sequence=left,
 right_sequence=right,
 indicator=indicator,
)
)
 continue

 if DB2_SPACE_COMMENT_BODY_PATTERN.match(clean_body):

```

```

 normalized = DB2_SPACE_COMMENT_BODY_PATTERN.sub(
 "DB2 ",
 clean_body,
 count=1,
).strip()
 output.append(
 self.fixed_format.build(
 body=normalized,
 left_sequence=left,
 right_sequence=right,
 indicator=indicator,
)
)
 continue

 output.append(line)

 return "\n".join(output).rstrip() + "\n"

def _format_string_blocks(
 self,
 text: str,
) -> str:
 lines = str(text or "").splitlines()
 output: list[str] = []
 inside_string = False

 for line in lines:
 logical = self._logical(line)

 if STRING_START_PATTERN.match(logical):
 inside_string = True
 output.append(line)
 continue

 if inside_string and STRING_INTRO_PATTERN.match(logical):
 output.append(self._indent_line(line, " "))
 continue

 if inside_string and STRING_END_PATTERN.match(logical):
 inside_string = False
 output.append(line)
 continue

 if inside_string:
 output.append(self._indent_line(line, " "))
 continue

 output.append(line)

 return "\n".join(output).rstrip() + "\n"

def _indent_line(
 self,
 line: str,
 indent: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 body = self.fixed_format.body(line).strip()
 return self.fixed_format.replace_body(line, f"{indent}{body}")

 return f"{indent}{str(line or '').strip()}"

def _logical(
 self,
 line: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 return self.fixed_format.body(line).strip()

 return str(line or "").strip()

```

```
=====
FILE: src\idms_db2_phase2\composers\fixed_format_body_parser.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from ..reference.rules.cobol.fixed_format_comment_rules import (
 COMMENT_BODY_MARKERS,
 DEFAULT_SOURCE_INDICATOR,
 DEBUG_BODY_MARKERS,
 NORMALIZED_DEBUG_INDICATOR,
 PAGE_BODY_MARKERS,
 SUPPORTED_SOURCE_INDICATORS,
)
from ..reference.rules.cobol.fixed_format_rules import (
 BODY_START_INDEX,
 INDICATOR_INDEX,
 LEFT_SEQUENCE_END_INDEX,
 LEFT_SEQUENCE_START_INDEX,
 RIGHT_SEQUENCE_END_INDEX,
 RIGHT_SEQUENCE_START_INDEX,
)
from ..services.fixed_format_line_service import FixedFormatLineService

class FixedFormatBodyParser:
 """
 Parses a source line into fixed-format indicator and body.

 Responsibilities:
 - Preserve existing column 7 indicator.
 - Restore comment/page/debug indicators shifted into body.
 - Remove old right sequence numbers from partially fixed lines.
 - Return only indicator and COBOL body.

 This class does not wrap lines and does not generate sequence numbers.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 def parse(
 self,
 line: str,
) -> tuple[str, str]:
 text = str(line or "").rstrip("\n")

 if self.fixed_format.is_fixed_line(text):
 return self._from_existing_fixed_line(text)

 if self._has_left_sequence(text):
 return self._from_partially_fixed_line(text)

 return self._from_unsequenced_line(text)

 def _from_existing_fixed_line(
 self,
 line: str,
) -> tuple[str, str]:
 _left, indicator, body, _right = self.fixed_format.split(line)

 safe_indicator = self._safe_indicator(indicator)
 clean_body = str(body or "").rstrip()

 if safe_indicator == DEFAULT_SOURCE_INDICATOR:
 restored_indicator, restored_body = self._restore_indicator_from_body(
 clean_body
)

 if restored_indicator != DEFAULT_SOURCE_INDICATOR:
 return restored_indicator, restored_body
```

```

 return safe_indicator, clean_body

def _from_partially_fixed_line(
 self,
 line: str,
) -> tuple[str, str]:
 text = str(line or "").rstrip()

 indicator = text[INDICATOR_INDEX : INDICATOR_INDEX + 1]
 indicator = self._safe_indicator(indicator)

 body = text[BODY_START_INDEX:]
 body = self._remove_trailing_right_sequence(body)

 if indicator == DEFAULT_SOURCE_INDICATOR:
 restored_indicator, restored_body = self._restore_indicator_from_body(
 body
)

 if restored_indicator != DEFAULT_SOURCE_INDICATOR:
 return restored_indicator, restored_body

 return indicator, body.rstrip()

def _from_unsequenced_line(
 self,
 line: str,
) -> tuple[str, str]:
 text = str(line or "").rstrip()
 stripped = text.lstrip()

 if not stripped:
 return DEFAULT_SOURCE_INDICATOR, ""

 restored_indicator, restored_body = self._restore_indicator_from_body(
 stripped
)

 if restored_indicator != DEFAULT_SOURCE_INDICATOR:
 return restored_indicator, restored_body

 return DEFAULT_SOURCE_INDICATOR, text.rstrip()

def _restore_indicator_from_body(
 self,
 body: str,
) -> tuple[str, str]:
 text = str(body or "").rstrip()
 stripped = text.lstrip()

 if not stripped:
 return DEFAULT_SOURCE_INDICATOR, ""

 first_character = stripped[:1]

 if first_character in COMMENT_BODY_MARKERS:
 return first_character, stripped[1:].lstrip()

 if first_character in PAGE_BODY_MARKERS:
 return first_character, stripped[1:].lstrip()

 if self._is_debug_body(stripped):
 return NORMALIZED_DEBUG_INDICATOR, stripped[1:].lstrip()

 return DEFAULT_SOURCE_INDICATOR, text.rstrip()

def _has_left_sequence(
 self,
 line: str,
) -> bool:
 text = str(line or "")

 if len(text) < LEFT_SEQUENCE_END_INDEX:
 return False

 left = text[LEFT_SEQUENCE_START_INDEX:LEFT_SEQUENCE_END_INDEX]

```

```

 return left.isdigit()

def _remove_trailing_right_sequence(
 self,
 body: str,
) -> str:
 text = str(body or "").rstrip()
 right_width = RIGHT_SEQUENCE_END_INDEX - RIGHT_SEQUENCE_START_INDEX

 if len(text) < right_width:
 return text

 candidate = text[-right_width:]

 if candidate.isdigit():
 return text[:-right_width].rstrip()

 return text

def _is_debug_body(
 self,
 text: str,
) -> bool:
 stripped = str(text or "").lstrip()

 if not stripped:
 return False

 first_character = stripped[:1]

 if first_character not in DEBUG_BODY_MARKERS:
 return False

 if len(stripped) == 1:
 return True

 return stripped[1:2].isspace()

def _safe_indicator(
 self,
 indicator: str,
) -> str:
 value = str(indicator or DEFAULT_SOURCE_INDICATOR)[:1]

 if value == "d":
 return NORMALIZED_DEBUG_INDICATOR

 if value not in SUPPORTED_SOURCE_INDICATORS:
 return DEFAULT_SOURCE_INDICATOR

 return value

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_body_wrapper.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.cobol.fixed_format_comment_rules import (
 COMMENT_BODY_MARKERS,
 PAGE_BODY_MARKERS,
)
from ..reference.rules.cobol.fixed_format_rules import BODY_WIDTH
from ..reference.rules.cobol.fixed_format_wrapping_rules import (
 COMMENT_CONTINUATION_INDENT,
 DEFAULT_CONTINUATION_INDENT,
 MOVE_CONTINUATION_INDENT,
 MOVE_KEYWORD,
 MOVE_TARGET_SEPARATOR,
 READABLE_BODY_WIDTH,
 SQL_ASSIGNMENT_SEPARATOR,
 SQL_CONTINUATION_INDENT,
 SQL_SPLIT_PREFIXES,
)

```

```

)
from .fixed_format_word_wrapper import FixedFormatWordWrapper

class FixedFormatBodyWrapper:
 """
 Wraps fixed-format COBOL body text.

 Responsibilities:
 - Keep body content within columns 8 through 72.
 - Wrap long comments safely.
 - Wrap long MOVE statements safely.
 - Wrap long SQL assignment and predicate lines safely.
 - Preserve existing indentation wherever possible.

 This class does not parse indicators and does not generate sequence numbers.
 """

 def __init__(
 self,
 word_wrapper: FixedFormatWordWrapper | None = None,
) -> None:
 self.word_wrapper = word_wrapper or FixedFormatWordWrapper()

 def wrap(
 self,
 indicator: str,
 body: str,
) -> list[str]:
 clean_body = str(body or "").rstrip()

 if len(clean_body) <= READABLE_BODY_WIDTH:
 return [clean_body]

 if indicator in COMMENT_BODY_MARKERS or indicator in PAGE_BODY_MARKERS:
 return self._wrap_comment_body(clean_body)

 move_lines = self._wrap_move_body(clean_body)

 if move_lines:
 return move_lines

 sql_lines = self._wrap_sql_body(clean_body)

 if sql_lines:
 return sql_lines

 return self.word_wrapper.wrap_by_words(
 text=clean_body,
 continuation_indent=self._continuation_indent_for_body(
 body=clean_body,
 default=DEFAULT_CONTINUATION_INDENT,
),
)

 def _wrap_comment_body(
 self,
 body: str,
) -> list[str]:
 clean_body = str(body or "").strip()

 if not clean_body:
 return [""]

 if self._is_repeated_character_line(clean_body):
 return self.word_wrapper.hard_wrap(
 text=clean_body,
 continuation_indent=COMMENT_CONTINUATION_INDENT,
)

 return self.word_wrapper.wrap_by_words(
 text=clean_body,
 continuation_indent=COMMENT_CONTINUATION_INDENT,
)

 def _wrap_move_body(

```

```

 self,
 body: str,
) -> list[str]:
 original = str(body or "").rstrip()
 logical = original.strip()
 upper = logical.upper()

 if not upper.startswith(MOVE_KEYWORD):
 return []

 separator_index = upper.find(MOVE_TARGET_SEPARATOR)

 if separator_index <= 0:
 return []

 indent = self.word_wrapper.leading_spaces(
 original,
 default=DEFAULT_CONTINUATION_INDENT,
)

 source = logical[len(MOVE_KEYWORD) : separator_index].strip()
 target = logical[separator_index + len(MOVE_TARGET_SEPARATOR) :].strip()

 if not source or not target:
 return []

 one_line = f"{indent}MOVE {source} TO {target}"

 if len(one_line) <= READABLE_BODY_WIDTH:
 return [one_line]

 output: list[str] = []

 output.extend(
 self._safe_body_lines(
 text=f"{indent}MOVE {source}",
 continuation_indent=indent + MOVE_CONTINUATION_INDENT,
)
)

 output.extend(
 self._safe_body_lines(
 text=f"{indent}TO {target}",
 continuation_indent=indent + MOVE_CONTINUATION_INDENT,
)
)

 return output

def _wrap_sql_body(
 self,
 body: str,
) -> list[str]:
 original = str(body or "").rstrip()
 logical = original.strip()
 upper = logical.upper()

 if not self._starts_with_any(
 value=upper,
 prefixes=SQL_SPLIT_PREFIXES,
):
 return []

 indent = self.word_wrapper.leading_spaces(
 original,
 default=DEFAULT_CONTINUATION_INDENT,
)

 separator_index = logical.find(SQL_ASSIGNMENT_SEPARATOR)

 if separator_index < 0:
 return self.word_wrapper.wrap_by_words(
 text=original,
 continuation_indent=indent + SQL_CONTINUATION_INDENT,
)

```

```

left_side = logical[: separator_index + len(SQL_ASSIGNMENT_SEPARATOR)].rstrip()
right_side = logical[separator_index + len(SQL_ASSIGNMENT_SEPARATOR) :].strip()

if not left_side or not right_side:
 return []

output: list[str] = []

output.extend(
 self._safe_body_lines(
 text=f"{indent}{left_side}",
 continuation_indent=indent + SQL_CONTINUATION_INDENT,
)
)

output.extend(
 self._safe_body_lines(
 text=f"{indent}{SQL_CONTINUATION_INDENT}{right_side}",
 continuation_indent=indent + SQL_CONTINUATION_INDENT,
)
)

return output

def _safe_body_lines(
 self,
 text: str,
 continuation_indent: str,
) -> list[str]:
 clean_text = str(text or "").rstrip()

 if len(clean_text) <= READABLE_BODY_WIDTH:
 return [clean_text]

 if len(clean_text) <= BODY_WIDTH and self._should_keep_one_line(clean_text):
 return [clean_text]

 return self.word_wrapper.wrap_by_words(
 text=clean_text,
 continuation_indent=continuation_indent,
)

def _should_keep_one_line(
 self,
 text: str,
) -> bool:
 upper = str(text or "").strip().upper()

 if upper.startswith("MOVE "):
 return False

 if self._starts_with_any(value=upper, prefixes=SQL_SPLIT_PREFIXES):
 return False

 return False

def _continuation_indent_for_body(
 self,
 body: str,
 default: str,
) -> str:
 leading = self.word_wrapper.leading_spaces(body, default=default)

 if leading:
 return leading + DEFAULT_CONTINUATION_INDENT

 return default

def _starts_with_any(
 self,
 value: str,
 prefixes: tuple[str, ...],
) -> bool:
 text = str(value or "")

 for prefix in prefixes:

```

```

 if text.startswith(prefix):
 return True

 return False

def _is_repeated_character_line(
 self,
 text: str,
) -> bool:
 clean = str(text or "").strip()

 if not clean:
 return False

 return all(character == clean[:1] for character in clean)

=====
FILE: src\idms_db2_phase2\composers\fixed_format_composer.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..reference.rules.cobol.fixed_format_blank_line_rules import (
 BLANK_LINE_BODY,
 BLANK_LINE_INDICATOR,
 PRESERVE_BLANK_LINES_AS_SEQUENCED_LINES,
)
from ..reference.rules.cobol.fixed_format_rules import (
 DEFAULT_LEFT_START,
 DEFAULT_LEFT_STEP,
 DEFAULT_RIGHT_START,
 DEFAULT_RIGHT_STEP,
)
from ..services.fixed_format_line_service import FixedFormatLineService
from .fixed_format_body_parser import FixedFormatBodyParser
from .fixed_format_body_wrapper import FixedFormatBodyWrapper

class FixedFormatComposer:
 """
 Final fixed-format COBOL composer.

 This class coordinates final physical COBOL formatting only.

 Responsibilities:
 - Normalize line endings.
 - Preserve blank lines as sequenced fixed-format lines when configured.
 - Parse indicator/body from each input line.
 - Wrap long bodies through FixedFormatBodyWrapper.
 - Generate left and right sequence numbers.
 - Build final 80-column manual-style COBOL lines.

 Detailed formatting rules are kept outside this coordinator.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 body_parser: FixedFormatBodyParser | None = None,
 body_wrapper: FixedFormatBodyWrapper | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.body_parser = body_parser or FixedFormatBodyParser(
 fixed_format=self.fixed_format,
)
 self.body_wrapper = body_wrapper or FixedFormatBodyWrapper()
 self.messages: list[str] = []

 def format(
 self,
 text: str,
 left_start: int = DEFAULT_LEFT_START,
 left_step: int = DEFAULT_LEFT_STEP,
 right_start: int = DEFAULT_RIGHT_START,

```

```

 right_step: int = DEFAULT_RIGHT_STEP,
 preserve_blank_lines: bool = True,
) -> str:
 self.messages = []

 source = str(text or "")

 if not source:
 return ""

 output: list[str] = []
 left_value = int(left_start)
 right_value = int(right_start)

 for raw_line in self._normalized_lines(source):
 if not raw_line.strip():
 blank_result = self._format_blank_line(
 preserve_blank_lines=preserve_blank_lines,
 left_value=left_value,
 right_value=right_value,
)

 if blank_result:
 output.append(blank_result)
 left_value += int(left_step)
 right_value += int(right_step)

 continue

 indicator, body = self.body_parser.parse(raw_line)
 wrapped_bodies = self.body_wrapper.wrap(
 indicator=indicator,
 body=body,
)

 for wrapped_body in wrapped_bodies:
 output.append(
 self.fixed_format.build(
 body=wrapped_body,
 left_sequence=left_value,
 right_sequence=right_value,
 indicator=indicator,
)
)

 left_value += int(left_step)
 right_value += int(right_step)

 self.messages.append(
 "Fixed-format composer: generated manual-style COBOL lines."
)

 return "\n".join(output).rstrip() + "\n"

def _format_blank_line(
 self,
 preserve_blank_lines: bool,
 left_value: int,
 right_value: int,
) -> str:
 if not preserve_blank_lines:
 return ""

 if not PRESERVE_BLANK_LINES_AS_SEQUENCED_LINES:
 return ""

 return self.fixed_format.build(
 body=BLANK_LINE_BODY,
 left_sequence=left_value,
 right_sequence=right_value,
 indicator=BLANK_LINE_INDICATOR,
)

def _normalized_lines(
 self,
 text: str,

```

```

) -> list[str]:
 return (
 str(text or "")
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .splitlines()
)

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_word_wrapper.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.cobol.fixed_format_rules import BODY_WIDTH
from ..reference.rules.cobol.fixed_format_wrapping_rules import (
 READABLE_BODY_WIDTH,
)

```

```

class FixedFormatWordWrapper:
 """
 Generic word wrapping helper for fixed-format COBOL bodies.

 This helper does not know COBOL business rules.
 It only keeps body text readable and inside the fixed body area.
 """

 def wrap_by_words(
 self,
 text: str,
 continuation_indent: str,
 preferred_width: int = READABLE_BODY_WIDTH,
) -> list[str]:
 raw = str(text or "").rstrip()

 if len(raw) <= preferred_width:
 return [raw]

 leading_indent = self.leading_spaces(raw, default="")
 continuation = str(continuation_indent or "")
 words = raw.strip().split()

 if not words:
 return [""]

 lines: list[str] = []
 current = leading_indent

 for word in words:
 candidate = self._candidate_with_word(
 current=current,
 word=word,
 leading_indent=leading_indent,
)

 if len(candidate) <= preferred_width:
 current = candidate
 continue

 if current.strip():
 lines.append(current.rstrip())

 current = self._next_current_for_word(
 word=word,
 continuation=continuation,
 lines=lines,
 preferred_width=preferred_width,
)

 if current.strip():
 lines.append(current.rstrip())

 return self.force_lines_to_body_width(

```

```

 lines=lines,
 continuation_indent=continuation,
)

def hard_wrap(
 self,
 text: str,
 continuation_indent: str,
 preferred_width: int = READABLE_BODY_WIDTH,
) -> list[str]:
 raw = str(text or "").rstrip()

 if len(raw) <= preferred_width:
 return [raw]

 continuation = str(continuation_indent or "")
 continuation_width = preferred_width - len(continuation)

 if continuation_width <= 0:
 continuation_width = preferred_width

 output: list[str] = []
 remaining = raw

 output.append(remaining[:preferred_width].rstrip())
 remaining = remaining[preferred_width:]

 while remaining:
 part = remaining[:continuation_width]
 output.append((continuation + part).rstrip())
 remaining = remaining[continuation_width:]

 return self.force_lines_to_body_width(
 lines=output,
 continuation_indent=continuation,
)

def force_lines_to_body_width(
 self,
 lines: list[str],
 continuation_indent: str,
) -> list[str]:
 output: list[str] = []

 for line in lines:
 text = str(line or "").rstrip()

 if len(text) <= BODY_WIDTH:
 output.append(text)
 continue

 output.extend(
 self.hard_wrap(
 text=text,
 continuation_indent=continuation_indent,
 preferred_width=BODY_WIDTH,
)
)

 return output or [""]

def leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

```

```

 return value[:count]

 def _candidate_with_word(
 self,
 current: str,
 word: str,
 leading_indent: str,
) -> str:
 if str(current or "").strip():
 return str(current).rstrip() + " " + str(word or "")

 return str(leading_indent or "") + str(word or "")

 def _next_current_for_word(
 self,
 word: str,
 continuation: str,
 lines: list[str],
 preferred_width: int,
) -> str:
 if len(continuation + word) <= preferred_width:
 return continuation + word

 hard_parts = self.hard_wrap(
 text=word,
 continuation_indent=continuation,
 preferred_width=preferred_width,
)

 if hard_parts:
 lines.extend(hard_parts[:-1])
 return hard_parts[-1]

 return word[:preferred_width]

```

```

=====
FILE: src\idms_db2_phase2\composers\manual_layout_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.catalogs.output.output_sections import (
 DB2_CURSOR_PARAGRAPH_MARKER,
 DB2_INFRASTRUCTURE_MARKER,
)
from ..reference.patterns.cobol.structure_patterns import END_PROGRAM_PATTERN

```

```

class ManualLayoutComposer:
 """
 Keeps generated DB2 blocks in a manual-style production layout.

 Required output order:
 1. Original COBOL header starts first.
 2. Original DATA and PROCEDURE structure remains.
 3. Generated DB2 infrastructure remains one complete block.
 4. Generated cursor OPEN/FETCH/CLOSE paragraphs are near the end.
 5. SQLERROR paragraph is near the end.
 6. END PROGRAM remains last.

 This composer is conservative and does not rewrite business logic.
 """

 def __init__(
 self,
) -> None:
 self.messages: list[str] = []

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []

```

```

 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()
 lines = self._rstrip_blank_lines(lines)

 if not lines:
 return ""

 end_program_line = ""
 end_program_index = self._find_end_program_index(lines)

 if end_program_index >= 0:
 end_program_line = lines[end_program_index]
 lines = lines[:end_program_index] + lines[end_program_index + 1 :]

 lines = self._normalize_generated_block_spacing(lines)

 if end_program_line:
 lines = self._append_end_program(lines, end_program_line)

 result = "\n".join(lines)
 result = self._normalize_blank_lines(result)
 return result.strip() + "\n"

def _find_end_program_index(
 self,
 lines: list[str],
) -> int:
 for index in range(len(lines) - 1, -1, -1):
 if END_PROGRAM_PATTERN.match(lines[index].strip()):
 return index

 return -1

def _append_end_program(
 self,
 lines: list[str],
 end_program_line: str,
) -> list[str]:
 output = self._rstrip_blank_lines(lines)

 if output:
 output.append("")

 output.append(end_program_line)
 return output

def _normalize_generated_block_spacing(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []

 for line in lines:
 stripped = str(line or "").strip()

 if stripped in {
 f"* {DB2_INFRASTRUCTURE_MARKER}",
 f"* {DB2_CURSOR_PARAGRAPH_MARKER}",
 }:
 if output and output[-1].strip():
 output.append("")

 output.append(line.rstrip())

 return output

def _rstrip_blank_lines(
 self,
 lines: list[str],
) -> list[str]:
 output = list(lines)

 while output and not output[-1].strip():
 output.pop()

```

```

 return output

def _normalize_blank_lines(
 self,
 text: str,
) -> str:
 output = str(text or "")

 while "\n\n\n\n" in output:
 output = output.replace("\n\n\n\n", "\n\n\n")

 return output

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

=====
FILE: src\idms_db2_phase2\composers\manual_style_preserver.py
ENCODING: utf-8
=====

from __future__ import annotations

class ManualStylePreserver:
 """
 Preserves manual COBOL style after conversion.

 Requirement:
 - Output should remain close to original IDMS COBOL style.
 - Only DB2 conversion changes should be carried out.
 - Existing comments should remain where possible.
 """

 def preserve(
 self,
 original_text: str,
 converted_text: str,
) -> str:
 if not converted_text:
 return ""

 text = self._preserve_comment_density(converted_text)
 text = self._normalize_excess_blank_lines(text)
 return text.rstrip() + "\n"

 def _preserve_comment_density(
 self,
 text: str,
) -> str:
 output_lines: list[str] = []

 for line in str(text or "").splitlines():
 output_lines.append(line.rstrip())

 return "\n".join(output_lines)

 def _normalize_excess_blank_lines(
 self,
 text: str,
) -> str:
 output = str(text or "")

 while "\n\n\n\n" in output:
 output = output.replace("\n\n\n\n", "\n\n\n")

 return output

```

```
=====
FILE: src\idms_db2_phase2\composers\retrieval_program_normalization_composer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from ..repositories.dclgen_repository import DclgenRepository
from ..services.procedure_indentation_service import ProcedureIndentationService
from ..services.retrieval_statement_normalization_service import (
 RetrievalStatementNormalizationService,
)

class RetrievalProgramNormalizationComposer:
 """
 Coordinates retrieval/output-program normalization.

 Responsibilities:
 - Ensure DCLGEN INCLUDE exists when a DCLGEN group is referenced.
 - Replace residual IDMS ERROR-STATUS control when a DB2 flag exists.
 - Add INITIALIZE before output-record WRITE when safe.
 - Normalize Procedure Division indentation.

 This composer does not own regex patterns, rules, or business-specific
 mapping logic.
 """

 def __init__(
 self,
 dclgen_repository: DclgenRepository,
 statement_normalization: RetrievalStatementNormalizationService | None = None,
 procedure_indentation: ProcedureIndentationService | None = None,
) -> None:
 self.dclgen_repository = dclgen_repository
 self.statement_normalization = (
 statement_normalization
 or RetrievalStatementNormalizationService(dclgen_repository)
)
 self.procedure_indentation = (
 procedure_indentation or ProcedureIndentationService()
)
 self.messages: list[str] = []

 def compose(
 self,
 original_text: str = "",
 converted_text: str = "",
) -> str:
 self.messages = []
 output = str(converted_text or original_text or "")

 if not output.strip():
 return output

 output = self.statement_normalization.normalize(output)
 self.messages.extend(self.statement_normalization.messages)

 output = self.procedure_indentation.normalize(output)
 self.messages.extend(self.procedure_indentation.messages)

 return output.rstrip() + "\n"
```

```
=====
FILE: src\idms_db2_phase2\composers\sqlcode_wrapper_normalization_composer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from ..reference.patterns.normalization.sqlcode_wrapper_patterns import (
 END_IF_PATTERN,
 IF_NOT_SQLCODE_100_PATTERN,
 IF_SQLCODE_100_PATTERN,
 MOVE_DCLGEN_TO_TARGET_PATTERN,
```

```

 REMOVED_REDUNDANT_OBTAIN_NEXT_COMMENT_PATTERN,
)
from ..services.sqlcode_guard_normalization_service import (
 SqlcodeGuardNormalizationService,
)

class SqlcodeWrapperNormalizationComposer:
 """
 Normalizes generated SQLCODE wrappers.

 This composer keeps the existing cursor-loop wrapper behavior and also
 delegates orphaned SQLCODE guard cleanup to the shared SQLCODE guard
 normalization service.

 This composer does not own regex patterns and does not hardcode
 program names, table names, cursor names, records, columns, or DCLGEN
 groups.
 """

 def __init__(
 self,
 sqlcode_guard: SqlcodeGuardNormalizationService | None = None,
) -> None:
 self.sqlcode_guard = sqlcode_guard or SqlcodeGuardNormalizationService()
 self.messages: list[str] = []

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 output = self._remove_redundant_cursor_wrappers(source)
 output = self.sqlcode_guard.normalize(output)
 self.messages.extend(self.sqlcode_guard.messages)

 return output.rstrip() + "\n"

 def _remove_redundant_cursor_wrappers(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self._logical(lines[index])

 if not IF_NOT_SQLCODE_100_PATTERN.match(logical):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if index + 1 >= len(lines):
 output.append(lines[index].rstrip())
 index += 1
 continue

 next_logical = self._logical(lines[index + 1])

 if not REMOVED_REDUNDANT_OBTAIN_NEXT_COMMENT_PATTERN.match(next_logical):
 output.append(lines[index].rstrip())
 index += 1
 continue

 output.append(
 self._with_body(
 lines[index],
 "*DB2: Removed redundant SQLCODE wrapper after cursor EOC loop.",
)
)

```

```

)
)
 recovered_lines, next_index = self._recover_non_sqlcode_body(lines, index + 1)
 output.extend(recovered_lines)
 index = next_index
 changed = True

if changed:
 self.messages.append(
 "SQLCODE wrapper normalization: removed redundant generated wrapper."
)

return "\n".join(output).rstrip() + "\n"

def _recover_non_sqlcode_body(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 output: list[str] = []
 index = start_index
 nested_if_count = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 if index == start_index:
 index += 1
 continue

 if IF_SQLCODE_100_PATTERN.match(logical):
 nested_if_count += 1
 index += 1
 continue

 if END_IF_PATTERN.match(logical):
 if nested_if_count > 0:
 nested_if_count -= 1
 index += 1
 continue

 index += 1
 break

 if MOVE_DCLGEN_TO_TARGET_PATTERN.match(logical):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if "SQLCODE" not in logical.upper():
 output.append(lines[index].rstrip())

 index += 1

 return output, index

def _logical(
 self,
 line: str,
) -> str:
 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

def _with_body(
 self,
 original_line: str,
 body: str,
) -> str:
 line = str(original_line or "")
 normalized_body = str(body or "").rstrip()

 if len(line) >= 7 and line[:6].isdigit():

```

```

 prefix = line[:7]
 suffix = line[72:] if len(line) > 72 else ""
 return f"{prefix}{normalized_body[:65].ljust(65)}{suffix}"

```

```

 return normalized_body

```

```

=====
FILE: src\idms_db2_phase2\composers\update_procedure_structure_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from ..reference.patterns.normalization.update_program_patterns import (
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 INCLUDE_PATTERN,
 STOP_RUN_PATTERN,
 TIMESTAMP_PARAGRAPH_PATTERN,
)

```

```

from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver

```

```

class UpdateProcedureStructureComposer:

```

```

 """

```

```

 Normalizes update-program procedure structure.

```

```

 Current responsibilities:

```

- Keep EXEC SQL INCLUDE blocks out of unsafe procedural locations.
- Prevent timestamp paragraph fallthrough by moving STOP RUN before it when both are present.

```

 This composer does not hardcode business names.

```

```

 """

```

```

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

```

```

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self._prevent_timestamp_fallthrough(output)

 return output.rstrip() + "\n"

```

```

 def _prevent_timestamp_fallthrough(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 timestamp_index = -1
 stop_index = -1

 for index, line in enumerate(lines):
 logical = line.strip()

```

```

 if timestamp_index < 0 and TIMESTAMP_PARAGRAPH_PATTERN.match(logical):
 timestamp_index = index

 if stop_index < 0 and STOP_RUN_PATTERN.match(logical):
 stop_index = index

 if timestamp_index < 0 or stop_index < 0:
 return text

 if stop_index < timestamp_index:
 return text

 stop_line = lines.pop(stop_index)

 if stop_index < timestamp_index:
 timestamp_index -= 1

 lines.insert(timestamp_index, stop_line)

 self.messages.append(
 "Update procedure structure: moved STOP RUN before timestamp paragraph."
)
 return "\n".join(lines).rstrip() + "\n"

def _collect_exec_sql_block(
 self,
 lines: list[str],
 index: int,
) -> tuple[list[str], int]:
 if index >= len(lines):
 return [], index

 if not EXEC_SQL_START_PATTERN.match(lines[index].strip()):
 return [], index

 output: list[str] = []

 while index < len(lines):
 output.append(lines[index])
 logical = lines[index].strip()
 index += 1

 if EXEC_SQL_END_PATTERN.match(logical):
 break

 return output, index

def _include_name_from_block(
 self,
 block: list[str],
) -> str:
 for line in block:
 match = INCLUDE_PATTERN.match(line.strip())

 if match:
 return str(match.group("include") or "").strip().upper()

 return ""

```

```

=====
FILE: src\idms_db2_phase2\composers\update_program_normalization_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from typing import Any

```

```

from .update_procedure_structure_composer import UpdateProcedureStructureComposer
from ..services.fixed_format_line_service import FixedFormatLineService
from ..services.update_procedure_normalization_service import (
 UpdateProcedureNormalizationService,
)

```

```

class UpdateProgramNormalizationComposer:
 """
 Coordinates update-program normalization.

 This composer receives metadata dependencies from ComponentFactory and
 passes them to update normalization services.

 This composer does not define regex patterns and does not hardcode
 program names, table names, records, columns, DCLGEN groups, or host
 variables.
 """

 def __init__(
 self,
 mapping_repository: Any | None = None,
 dclgen_repository: Any | None = None,
 table_name_resolver: Any | None = None,
 host_variable_resolver: Any | None = None,
 fixed_format: FixedFormatLineService | None = None,
 procedure_normalization: UpdateProcedureNormalizationService | None = None,
 structure_normalization: UpdateProcedureStructureComposer | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.fixed_format = fixed_format or FixedFormatLineService()

 self.procedure_normalization = (
 procedure_normalization
 or UpdateProcedureNormalizationService(
 fixed_format=self.fixed_format,
 mapping_repository=self.mapping_repository,
 dclgen_repository=self.dclgen_repository,
 table_name_resolver=self.table_name_resolver,
 host_variable_resolver=self.host_variable_resolver,
)
)

 self.structure_normalization = (
 structure_normalization
 or self._create_structure_normalization()
)

 self.messages: list[str] = []

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self.procedure_normalization.normalize(output)
 self.messages.extend(self.procedure_normalization.messages)

 if self.structure_normalization is not None:
 output = self.structure_normalization.compose(output)
 self.messages.extend(
 getattr(
 self.structure_normalization,
 "messages",
 [],
)
)

 output = self.procedure_normalization.normalize(output)
 self.messages.extend(self.procedure_normalization.messages)

 return output.rstrip() + "\n"

 def normalize(

```

```

 self,
 text: str,
) -> str:
 return self.compose(text)

 def apply(
 self,
 text: str,
) -> str:
 return self.compose(text)

 def _create_structure_normalization(
 self,
) -> UpdateProcedureStructureComposer | None:
 if self.mapping_repository is None:
 return None

 if self.dclgen_repository is None:
 return None

 if self.table_name_resolver is None:
 return None

 if self.host_variable_resolver is None:
 return None

 return UpdateProcedureStructureComposer(
 mapping_repository=self.mapping_repository,
 dclgen_repository=self.dclgen_repository,
 table_name_resolver=self.table_name_resolver,
 host_variable_resolver=self.host_variable_resolver,
)

```

```

=====
FILE: src\idms_db2_phase2\composers\update_restart_conversion_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.normalization.update_restart_patterns import (
 CONVERTED_FOR_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 REMOVED_OBTAIN_CALC_PATTERN,
 SKIPPED_FOR_PATTERN,
 SQLCODE_IF_PATTERN,
)
from ..services.name_normalizer import NameNormalizer

class UpdateRestartConversionComposer:
 """
 Converts unsafe missing-mapping restart/control generated blocks into
 manual redesign comments.

 Generic rule:
 - Do not invent DB2 restart SQL.
 - If metadata is missing, preserve valid COBOL flow using CONTINUE.
 - Remove immediate SQL blocks and SQLCODE checks after skipped conversion
 because no SQL was safely executed.
 """

 def __init__(
 self,
) -> None:
 self.messages: list[str] = []

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

```

```

if not source:
 return ""

lines = source.splitlines()
output: list[str] = []
index = 0
changed = False

while index < len(lines):
 logical = lines[index].strip()
 match = SKIPPED_FOR_PATTERN.match(logical)

 if not match:
 match = REMOVED_OBTAIN_CALC_PATTERN.match(logical)

 if not match:
 output.append(lines[index])
 index += 1
 continue

 record = NameNormalizer.to_cobol(match.group("record"))
 indent = self._leading_spaces(lines[index])

 output.extend(
 [
 f"{indent}*DB2: Manual redesign required for {record}.",
 f"{indent}*DB2: Missing restart/control mapping metadata.",
 f"{indent}CONTINUE.",
]
)

 index = self._skip_generated_sqlcode_area(lines, index + 1)
 changed = True

if changed:
 self.messages.append(
 "Update restart conversion: replaced unsafe generated block with manual redesign comment."
)

return "\n".join(output).rstrip() + "\n"

def _skip_generated_sqlcode_area(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index
 inside_exec_sql = False

 while index < len(lines):
 logical = lines[index].strip()

 if EXEC_SQL_START_PATTERN.match(logical):
 inside_exec_sql = True
 index += 1
 continue

 if inside_exec_sql:
 index += 1

 if EXEC_SQL_END_PATTERN.match(logical):
 inside_exec_sql = False

 continue

 if SQLCODE_IF_PATTERN.match(logical):
 index += 1
 continue

 break

 return index

def _leading_spaces(
 self,
 text: str,

```

```

) -> str:
 value = str(text or "")
 count = len(value) - len(value.lstrip(" "))
 return value[:count]

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_normalization_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from ..reference.patterns.normalization.update_program_patterns import (
 CONVERTED_OBTAIN_CALC_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
)
from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

```

```

class UpdateSqlNormalizationComposer:
 """
 Normalizes generated SQL blocks for update programs.

 Current responsibility:
 - Replace generated OBTAIN CALC SELECT blocks with a conservative
 direct-update comment when table and key metadata exists.
 - If metadata is missing, emit a skipped-conversion block.

 This composer does not hardcode business names, tables, columns,
 DCLGEN groups, or host variables.
 """

```

```

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

```

```

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self._rewrite_obtain_calc_selects(output)

 return output.rstrip() + "\n"

```

```

 def _rewrite_obtain_calc_selects(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = lines[index].strip()

```

```

match = CONVERTED_OBTAIN_CALC_PATTERN.match(logical)

if not match:
 output.append(lines[index])
 index += 1
 continue

record = NameNormalizer.normalize(match.group("record"))
cobol_record = NameNormalizer.to_cobol(record)
indent = self._leading_spaces(lines[index])
table = self.table_name_resolver.table_for_record(record)
key_columns = self.mapping_repository.key_columns_for_record(record)

index = self._skip_generated_sql_and_sqlcode(lines, index + 1)

if not table:
 output.extend(
 [
 f"{indent}*DB2: OBTAIN CALC conversion skipped for {cobol_record}.",
 f"{indent}*DB2: Missing Sheet Mapping or DCLGEN table metadata.",
 f"{indent}CONTINUE.",
]
)
 changed = True
 continue

if not key_columns:
 output.extend(
 [
 f"{indent}*DB2: OBTAIN CALC conversion skipped for {cobol_record}.",
 f"{indent}*DB2: Missing key column metadata.",
 f"{indent}CONTINUE.",
]
)
 changed = True
 continue

output.extend(
 [
 f"{indent}*DB2: Removed OBTAIN CALC SELECT for {cobol_record}.",
 f"{indent}*DB2: Direct UPDATE will use mapped composite key WHERE clause.",
 f"{indent}CONTINUE.",
]
)
changed = True

if changed:
 self.messages.append(
 "Update SQL normalization: removed redundant OBTAIN CALC SELECT block."
)

return "\n".join(output).rstrip() + "\n"

def _skip_generated_sql_and_sqlcode(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index
 inside_exec_sql = False

 while index < len(lines):
 logical = lines[index].strip()

 if EXEC_SQL_START_PATTERN.match(logical):
 inside_exec_sql = True
 index += 1
 continue

 if inside_exec_sql:
 index += 1

 if EXEC_SQL_END_PATTERN.match(logical):
 inside_exec_sql = False

 continue

```

```

 if logical.startswith("IF SQLCODE") or logical.startswith("END-IF"):
 index += 1
 continue

 break

 return index

def _leading_spaces(
 self,
 text: str,
) -> str:
 value = str(text or "")
 count = len(value) - len(value.lstrip(" "))
 return value[:count]

```

```

=====
FILE: src\idms_db2_phase2\config__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\config\application_settings.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

APP_NAME = "IDMS DB2 Code Conversion"
APP_DESCRIPTION = (
 "Python automation for converting IDMS COBOL programs "
 "into DB2 embedded SQL COBOL."
)

```

```

DEFAULT_STREAMLIT_PORT = 8502
DEFAULT_AUTO_FIX_PIC_LENGTH_MISMATCHES = False
DEFAULT_DB2_DATE_EXTERNAL_FORMAT = "DD.MM.YYYY"
DEFAULT_ORDER_BY_COLUMNS_IN_SELECT = False

```

```

=====
FILE: src\idms_db2_phase2\config\logging_settings.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

LOG_FILE_PREFIX = "idms_db2_conversion"
LOG_DATE_TIME_FORMAT = "%d-%m-%Y_%H%M%S"
LOG_MESSAGE_FORMAT = "%(asctime)s | %(levelname)s | %(name)s | %(message)s"
LOG_DATE_FORMAT = "%d-%m-%Y %H:%M:%S"
DEFAULT_LOG_LEVEL = "INFO"

```

```

=====
FILE: src\idms_db2_phase2\config\path_settings.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from pathlib import Path
TEMP_PROJECT_ROOT = Path(r"C:\S\S-Input")

PROJECT_ROOT = Path(__file__).resolve().parents[3]
SRC_DIR = PROJECT_ROOT / "src"
LOGS_DIR = PROJECT_ROOT / "logs"

DEFAULT_OUTPUT_DIR = TEMP_PROJECT_ROOT / "output"

```

```

DEFAULT_SHEET_MAPPING_PATH = TEMP_PROJECT_ROOT / "Excel_Sheet_mapping.csv"
DEFAULT_RETRIEVAL_SOURCE_PATH = TEMP_PROJECT_ROOT / "Retrieval.txt"
DEFAULT_UPDATE_SOURCE_PATH = TEMP_PROJECT_ROOT / "Update.txt"

```

```

DEFAULT_DCLGEN_CANDIDATE_PATHS = [
 TEMP_PROJECT_ROOT / "DCLGENS_BEFF.txt",
 TEMP_PROJECT_ROOT / "DCLGENS_BFAR.txt",
 TEMP_PROJECT_ROOT / "DCLGENS_EVEF.txt",
]

```

```

DEFAULT_COPYBOOK_CANDIDATE_PATHS = [
 TEMP_PROJECT_ROOT / "Copybook.txt",
]

```

```

def existing_files(
 paths: list[Path],
) -> list[Path]:
 return [
 path
 for path in paths
 if path.exists() and path.is_file()
]

```

```

def ensure_runtime_directories() -> None:
 LOGS_DIR.mkdir(parents=True, exist_ok=True)
 DEFAULT_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

```

```

=====
FILE: src\idms_db2_phase2\domain__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\domain\models.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field
from typing import Any

```

```

@dataclass(slots=True)
class SheetMappingRow:
 idms_to_db2_mapping: str = ""
 cobol_record_idms: str = ""
 cobol_zone: str = ""
 idms_key: str = ""
 idms_pic_clause: str = ""
 length_of_field_bytes: str = ""
 field_end_position: str = ""
 db2_key: str = ""
 new_db2_record: str = ""
 new_db2_field_name: str = ""
 new_db2_data_type: str = ""
 hopex_expression_type_remark: str = ""
 remarks: str = ""
 relation: str = ""
 reference_field_name_copybook: str = ""
 reference_field_pic_clause: str = ""
 cross_application_db2_table: str = ""
 cross_application_db2_field_name: str = ""
 cross_application_db2_data_type: str = ""
 basetype: str = ""

```

```

@dataclass(slots=True)
class DclgenColumn:

```

```

table_name: str = ""
group_name: str = ""
column_name: str = ""
host_variable: str = ""
level: str = ""
pic_clause: str = ""
usage: str = ""
raw_line: str = ""

```

```
@dataclass(slots=True)
```

```

class CopybookField:
 field_name: str = ""
 level: str = ""
 pic_clause: str = ""
 usage: str = ""
 parent_name: str = ""
 raw_line: str = ""

```

```
@dataclass(slots=True)
```

```

class IdmsOperation:
 operation: str
 record_name: str = ""
 set_name: str = ""
 source_line: str = ""
 line_number: int = 0

```

```
@dataclass(slots=True)
```

```

class ConversionInput:
 sheet_mapping_rows: list[SheetMappingRow] = field(default_factory=list)
 dclgen_columns: list[DclgenColumn] = field(default_factory=list)
 copybook_fields: list[CopybookField] = field(default_factory=list)
 idms_cobol_text: str = ""
 target_program_id: str = ""
 auto_fix_pic_length_mismatches: bool = False

```

```
@dataclass(slots=True)
```

```

class ConversionResult:
 converted_cobol: str = ""
 validation_messages: list[str] = field(default_factory=list)
 metadata_overview: dict[str, Any] = field(default_factory=dict)
 record_summary: list[dict[str, Any]] = field(default_factory=list)
 column_summary: list[dict[str, Any]] = field(default_factory=list)
 relationship_summary: list[dict[str, Any]] = field(default_factory=list)
 sheet_mapping_preview: list[dict[str, Any]] = field(default_factory=list)
 operations: list[IdmsOperation] = field(default_factory=list)

```

```
@dataclass(slots=True)
```

```

class FieldUsage:
 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 moved_fields: set[str] = field(default_factory=set)
 key_fields: set[str] = field(default_factory=set)
 update_fields: set[str] = field(default_factory=set)
 ordering_fields: set[str] = field(default_factory=set)

```

```
@dataclass(slots=True)
```

```

class ProgramFlowAnalysis:
 diagnostics: list[str] = field(default_factory=list)
 field_usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)

```

```

=====
FILE: src\idms_db2_phase2\generators__init__.py
ENCODING: utf-8
=====

```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_phase2\generators\cursor_paragraph_generator.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from ..domain.models import IdmsOperation
from ..generators.db2_infrastructure_generator import Db2InfrastructureGenerator
from ..generators.sql_error_generator import SqlErrorGenerator
from ..reference.catalogs.output.output_sections import DB2_CURSOR_PARAGRAPH_MARKER
from ..reference.patterns.cobol.structure_patterns import END_PROGRAM_PATTERN
from ..resolvers.host_variable_resolver import HostVariableResolver

class CursorParagraphGenerator:
 """
 Generates DB2 cursor OPEN / FETCH / CLOSE paragraphs.

 Cursor declarations are generated by Db2InfrastructureGenerator.
 This generator reuses Db2InfrastructureGenerator.last_cursor_specs
 whenever available so FETCH INTO host variables match SELECT columns.

 This generator does not hardcode program names, records, DB2 tables,
 DB2 columns, DCLGEN groups, or host variables.
 """

 SQLERROR_PARAGRAPH_NAME = "SQLERROR"

 def __init__(
 self,
 db2_infrastructure_generator: Db2InfrastructureGenerator,
 host_variable_resolver: HostVariableResolver,
 sql_error_generator: SqlErrorGenerator,
) -> None:
 self.db2_infrastructure_generator = db2_infrastructure_generator
 self.host_variable_resolver = host_variable_resolver
 self.sql_error_generator = sql_error_generator
 self.messages: list[str] = []

 def apply(
 self,
 cobol_text: str,
 operations: list[IdmsOperation],
) -> tuple[str, list[str]]:
 self.messages = []
 text = str(cobol_text or "")

 if not text:
 return text, self.messages

 cursor_specs = self._cursor_specs(operations)

 if not cursor_specs:
 self.messages.append("DB2 cursor paragraphs: no cursor operations found.")
 return text, self.messages

 if DB2_CURSOR_PARAGRAPH_MARKER in text:
 self.messages.append(
 "DB2 cursor paragraphs: generated cursor paragraph block already exists."
)
 return text.rstrip() + "\n", self.messages

 block = self.paragraph_block(
 cursor_specs=cursor_specs,
 sql_error_paragraph=self.SQLERROR_PARAGRAPH_NAME,
)
 updated_text = self._insert_paragraph_block(
 text=text,
 block=block,
)

 self.messages.append(
 f"DB2 cursor paragraphs: generated {len(cursor_specs)} cursor paragraph set(s)."
```

```

for spec in cursor_specs:
 if not list(spec.get("host_variables", [])):
 self.messages.append(
 "DB2 cursor paragraphs: no FETCH host variables resolved for "
 f"cursor {spec.get('cursor_name', '')} "
 f"record {spec.get('record_name', '')} "
 f"table {spec.get('table_name', '')}."
)

return updated_text.rstrip() + "\n", self.messages

def paragraph_block(
 self,
 cursor_specs: list[dict[str, object]],
 sql_error_paragraph: str,
) -> str:
 lines: list[str] = []

 lines.extend(self._comment_block(DB2_CURSOR_PARAGRAPH_MARKER))
 lines.append("")

 for spec in cursor_specs:
 cursor_name = str(spec.get("cursor_name") or "")
 open_paragraph = str(spec.get("open_paragraph") or "")
 fetch_paragraph = str(spec.get("fetch_paragraph") or "")
 close_paragraph = str(spec.get("close_paragraph") or "")
 eoc_flag = str(spec.get("eoc_flag") or "")
 host_variables = list(spec.get("host_variables") or [])

 if not cursor_name:
 continue

 if open_paragraph:
 lines.extend(
 self.open_paragraph(
 cursor_name=cursor_name,
 paragraph_name=open_paragraph,
 eoc_flag=eoc_flag,
 sql_error_paragraph=sql_error_paragraph,
)
)
 lines.append("")

 if fetch_paragraph:
 lines.extend(
 self.fetch_paragraph(
 cursor_name=cursor_name,
 paragraph_name=fetch_paragraph,
 eoc_flag=eoc_flag,
 host_variables=host_variables,
 sql_error_paragraph=sql_error_paragraph,
)
)
 lines.append("")

 if close_paragraph:
 lines.extend(
 self.close_paragraph(
 cursor_name=cursor_name,
 paragraph_name=close_paragraph,
 sql_error_paragraph=sql_error_paragraph,
)
)
 lines.append("")

 return "\n".join(lines).rstrip() + "\n"

def open_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 eoc_flag: str,
 sql_error_paragraph: str,
) -> list[str]:
 eoc_no_condition = f"{eoc_flag}-NO" if eoc_flag else ""

```

```

lines = [
 f"{paragraph_name}.",
 f" MOVE '{paragraph_name}' TO SQL-LOCATION.",
]

if eoc_no_condition:
 lines.append(f" SET {eoc_no_condition} TO TRUE.")

lines.extend(
 [
 " EXEC SQL",
 f" OPEN {cursor_name}",
 " END-EXEC.",
 " EVALUATE SQLCODE",
 " WHEN ZERO",
 " CONTINUE",
 " WHEN OTHER",
 f" DISPLAY 'ERROR WHILE OPENING CURSOR {cursor_name}'.",
 f" PERFORM {sql_error_paragraph}.",
 " END-EVALUATE.",
]
)

return lines

def fetch_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 eoc_flag: str,
 host_variables: list[str],
 sql_error_paragraph: str,
) -> list[str]:
 eoc_yes_condition = f"{eoc_flag}-YES" if eoc_flag else ""
 clean_hosts = self._clean_host_variables(host_variables)

 lines: list[str] = [
 f"{paragraph_name}.",
 f" MOVE '{paragraph_name}' TO SQL-LOCATION.",
 " EXEC SQL",
 f" FETCH {cursor_name}",
]

 if clean_hosts:
 lines.append(" INTO")
 lines.extend(self._fetch_host_lines(clean_hosts))

 lines.extend(
 [
 " END-EXEC.",
 " EVALUATE SQLCODE",
 " WHEN ZERO",
 " CONTINUE",
 " WHEN 100",
]
)

 if eoc_yes_condition:
 lines.append(f" SET {eoc_yes_condition} TO TRUE.")
 else:
 lines.append(" CONTINUE")

 lines.extend(
 [
 " WHEN OTHER",
 f" DISPLAY 'ERROR WHILE FETCHING CURSOR {cursor_name}'.",
 f" PERFORM {sql_error_paragraph}.",
 " END-EVALUATE.",
]
)

 return lines

def close_paragraph(
 self,
 cursor_name: str,

```

```

 paragraph_name: str,
 sql_error_paragraph: str,
) -> list[str]:
 return [
 f"{{paragraph_name}}.",
 f" MOVE '{{paragraph_name}}' TO SQL-LOCATION.",
 " EXEC SQL",
 f" CLOSE {{cursor_name}}",
 " END-EXEC.",
 " EVALUATE SQLCODE",
 " WHEN ZERO",
 " CONTINUE",
 " WHEN OTHER",
 f" DISPLAY 'ERROR WHILE CLOSING CURSOR {{cursor_name}}'.",
 f" PERFORM {{sql_error_paragraph}}.",
 " END-EVALUATE.",
]

 def _cursor_specs(
 self,
 operations: list[IdmsOperation],
) -> list[dict[str, object]]:
 cached_specs = getattr(
 self.db2_infrastructure_generator,
 "last_cursor_specs",
 [],
)

 if cached_specs:
 return cached_specs

 return self.db2_infrastructure_generator.cursor_specs(operations)

 def _insert_paragraph_block(
 self,
 text: str,
 block: str,
) -> str:
 lines = str(text or "").splitlines()
 insert_index = self._find_end_program_index(lines)

 if insert_index < 0:
 insert_index = len(lines)

 output = lines[:insert_index]

 if output and output[-1].strip():
 output.append("")

 output.extend(block.rstrip().splitlines())

 if insert_index < len(lines):
 output.append("")
 output.extend(lines[insert_index:])

 return "\n".join(output).rstrip() + "\n"

 def _find_end_program_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if END_PROGRAM_PATTERN.match(line.strip()):
 return index

 return -1

 def _fetch_host_lines(
 self,
 host_variables: list[str],
) -> list[str]:
 output: list[str] = []

 for index, host_variable in enumerate(host_variables):
 comma = "," if index < len(host_variables) - 1 else ""
 output.append(f" {host_variable}{comma}")

```

```

 return output

 def _clean_host_variables(
 self,
 host_variables: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for host_variable in host_variables:
 text = str(host_variable or "").strip()

 if not text:
 continue

 if not text.startswith(":"):
 text = f":{text}"

 normalized = text.upper()

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(text)

 return output

 def _comment_block(
 self,
 title: str,
) -> list[str]:
 return [
 f"* {title}",
]

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure_generator.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import IdmsOperation
from ..reference.catalogs.output.output_sections import (
 DB2_CURSOR_DECLARATIONS_MARKER,
 DB2_CURSOR_FLAGS_MARKER,
 DB2_INFRASTRUCTURE_MARKER,
 DB2_SQL_ERROR_LOCATION_MARKER,
 DEFAULT_CURSOR_EOC_VALUE,
 DEFAULT_CURSOR_NOT_EOC_VALUE,
 SQL_LOCATION_FIELD_NAME,
 SQL_LOCATION_PICTURE,
 SQLCA_INCLUDE_NAME,
 SQLERRWS_INCLUDE_NAME,
)
from ..reference.patterns.cobol.structure_patterns import (
 DATA_DIVISION_PATTERN,
 END_PROGRAM_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 WORKING_STORAGE_PATTERN,
)
from ..reference.rules.db2.cursor_rules import cursor_paragraph_number
from ..resolvers.column_name_resolver import ColumnNameResolver
from ..resolvers.cursor_name_resolver import CursorNameResolver
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

```

```

class Db2InfrastructureGenerator:
 """
 Generates DB2 infrastructure in DATA DIVISION.

```

Generic behavior:

- Include SQLCA.
- Include SQLERRWS.
- Include DCLGEN views/tables used by cursor specs or DB2 operations.
- Generate SQL-LOCATION.
- Generate cursor EOC flags.
- Generate cursor declarations using WITH HOLD.
- Add FOR READ ONLY for cursor declarations.

This generator does not hardcode business records, DB2 tables, DB2 columns, DCLGEN groups, or host variables.

"""

```
CURSOR_OPERATIONS = {
 "OBTAIN_FIRST",
 "OBTAIN_NEXT",
 "FIND_FIRST",
}

def __init__(
 self,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 host_variable_resolver: HostVariableResolver,
 cursor_name_resolver: CursorNameResolver,
) -> None:
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.cursor_name_resolver = cursor_name_resolver
 self.messages: list[str] = []
 self.last_cursor_specs: list[dict[str, object]] = []

def apply(
 self,
 cobol_text: str,
 operations: list[IdmsOperation],
) -> tuple[str, list[str]]:
 self.messages = []
 text = str(cobol_text or "")

 if not text:
 return "", self.messages

 cursor_specs = self.cursor_specs(operations)
 self.last_cursor_specs = cursor_specs

 used_tables = self._used_tables(
 operations=operations,
 cursor_specs=cursor_specs,
)

 if not used_tables and not self._contains_exec_sql(text):
 self.messages.append("DB2 infrastructure: no DB2 usage detected.")
 return text.rstrip() + "\n", self.messages

 if DB2_INFRASTRUCTURE_MARKER in text:
 self.messages.append("DB2 infrastructure: block already exists.")
 return text.rstrip() + "\n", self.messages

 infrastructure_block = self.infrastructure_block(
 table_names=used_tables,
 cursor_specs=cursor_specs,
)
 updated = self._insert_before_procedure_or_end(
 text=text,
 block=infrastructure_block,
)

 self.messages.append("DB2 infrastructure: generated DB2 support block.")
 return updated.rstrip() + "\n", self.messages

def cursor_specs(
 self,
 operations: list[IdmsOperation],
) -> list[dict[str, object]]:
```

```

output: list[dict[str, object]] = []
seen_tables: set[str] = set()

for operation in operations:
 if operation.operation not in self.CURSOR_OPERATIONS:
 continue

 record = NameNormalizer.normalize(operation.record_name)

 if not record:
 continue

 table = self.table_name_resolver.table_for_record(record)

 if not table:
 self.messages.append(
 f"DB2 infrastructure: cursor skipped for {record}; "
 "table metadata not resolved."
)
 continue

 if table in seen_tables:
 continue

 seen_tables.add(table)
 order = len(output) + 1
 cursor_name = self.cursor_name_resolver.cursor_name_from_table(
 table_name=table,
 cursor_order=order,
)
 select_columns = self.column_name_resolver.columns_for_record(record)
 host_variables = self.host_variable_resolver.host_references_for_columns(
 table_name=table,
 columns=select_columns,
)

 output.append(
 {
 "record_name": record,
 "table_name": table,
 "cursor_name": cursor_name,
 "select_columns": select_columns,
 "host_variables": host_variables,
 "open_paragraph": f"{cursor_paragraph_number(order, 'open')}-OPEN-{{cursor_name}}",
 "fetch_paragraph": f"{cursor_paragraph_number(order, 'fetch')}-FETCH-{{cursor_name}}",
 "close_paragraph": f"{cursor_paragraph_number(order, 'close')}-CLOSE-{{cursor_name}}",
 "eoc_flag": f"WS-{{cursor_name}}-EOC",
 }
)

return output

def infrastructure_block(
 self,
 table_names: list[str],
 cursor_specs: list[dict[str, object]],
) -> str:
 lines: list[str] = []

 lines.extend(self._comment_block(DB2_INFRASTRUCTURE_MARKER))
 lines.extend(
 [
 "EXEC SQL",
 f" INCLUDE {SQLCA_INCLUDE_NAME}",
 "END-EXEC.",
 "EXEC SQL",
 f" INCLUDE {SQLERRWS_INCLUDE_NAME}",
 "END-EXEC.",
]
)

 for table_name in table_names:
 include_name = NameNormalizer.to_sql(table_name)

 if not include_name:
 continue

```

```

 lines.extend(
 [
 "EXEC SQL",
 f" INCLUDE {include_name}",
 "END-EXEC.",
]
)

 lines.append("")
 lines.extend(self._comment_block(DB2_SQL_ERROR_LOCATION_MARKER))
 lines.append(f"01 {SQL_LOCATION_FIELD_NAME} {SQL_LOCATION_PICTURE}")

 if cursor_specs:
 lines.append("")
 lines.extend(self._cursor_flag_block(cursor_specs))
 lines.append("")
 lines.extend(self._cursor_declaration_block(cursor_specs))

 return "\n".join(lines).rstrip() + "\n"

def _cursor_flag_block(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 lines: list[str] = []
 lines.extend(self._comment_block(DB2_CURSOR_FLAGS_MARKER))

 for spec in cursor_specs:
 flag_name = str(spec.get("eoc_flag") or "")

 if not flag_name:
 continue

 lines.extend(
 [
 f"01 {flag_name} PIC X VALUE '{DEFAULT_CURSOR_NOT_EOC_VALUE}'.",
 f" 88 {flag_name}-YES VALUE '{DEFAULT_CURSOR_EOC_VALUE}'.",
 f" 88 {flag_name}-NO VALUE '{DEFAULT_CURSOR_NOT_EOC_VALUE}'.",
]
)

 return lines

def _cursor_declaration_block(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 lines: list[str] = []
 lines.extend(self._comment_block(DB2_CURSOR_DECLARATIONS_MARKER))

 for spec in cursor_specs:
 cursor_name = str(spec.get("cursor_name") or "")
 table_name = str(spec.get("table_name") or "")
 select_columns = list(spec.get("select_columns") or [])

 if not cursor_name or not table_name or not select_columns:
 continue

 lines.extend(
 [
 "EXEC SQL",
 f" DECLARE {cursor_name} CURSOR WITH HOLD FOR",
 " SELECT",
]
)
 lines.extend(self._comma_lines(select_columns, indent=" "))
 lines.extend(
 [
 f" FROM {table_name}",
 " FOR READ ONLY",
 "END-EXEC.",
]
)

 return lines

```

```

def _used_tables(
 self,
 operations: list[IdmsOperation],
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 values: list[str] = []

 for operation in operations:
 record = NameNormalizer.normalize(operation.record_name)

 if not record:
 continue

 table = self.table_name_resolver.table_for_record(record)

 if table:
 values.append(table)

 for spec in cursor_specs:
 table = str(spec.get("table_name") or "")

 if table:
 values.append(table)

 return self._unique(values)

def _insert_before_procedure_or_end(
 self,
 text: str,
 block: str,
) -> str:
 lines = str(text or "").splitlines()
 insert_index = self._find_working_storage_insert_index(lines)

 if insert_index < 0:
 insert_index = self._find_procedure_index(lines)

 if insert_index < 0:
 insert_index = self._find_end_program_index(lines)

 if insert_index < 0:
 insert_index = len(lines)

 output = lines[:insert_index]
 output.append("")
 output.extend(block.rstrip().splitlines())
 output.append("")
 output.extend(lines[insert_index:])

 return "\n".join(output).rstrip() + "\n"

def _find_working_storage_insert_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if WORKING_STORAGE_PATTERN.match(line.strip()):
 return index + 1

 for index, line in enumerate(lines):
 if DATA_DIVISION_PATTERN.match(line.strip()):
 return index + 1

 return -1

def _find_procedure_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if PROCEDURE_DIVISION_PATTERN.match(line.strip()):
 return index

 return -1

```

```

def _find_end_program_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if END_PROGRAM_PATTERN.match(line.strip()):
 return index

 return -1

def _contains_exec_sql(
 self,
 text: str,
) -> bool:
 return "EXEC SQL" in str(text or "").upper()

def _comma_lines(
 self,
 values: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []

 for index, value in enumerate(values):
 comma = "," if index < len(values) - 1 else ""
 output.append(f"{indent}{value}{comma}")

 return output

def _comment_block(
 self,
 title: str,
) -> list[str]:
 return [
 f"* {title}",
]

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.to_sql(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\generators\sql_error_generator.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.db2.sql_error_patterns import (
 END_PROGRAM_PATTERN,
 SQLERROR_INCLUDE_PATTERN,
 SQLERROR_PARAGRAPH_PATTERN,
 SQLERROR_PERFORM_PATTERN,
)

```

```

class SqlErrorGenerator:
 """

```

Generates and ensures generic SQLERROR support.

Rules:

- Use SQLERROR, not SQL-ERROR.
- Generated DB2 cursor paragraphs and SQL blocks perform SQLERROR.
- SQLERROR paragraph wraps EXEC SQL INCLUDE SQLERROR END-EXEC.
- Do not hardcode program names, DB2 tables, columns, DCLGEN groups, or host variables.

```
DEFAULT_PARAGRAPH_NAME = "SQLERROR"
```

```
def sqlcode_check(
 self,
 sql_error_paragraph: str = DEFAULT_PARAGRAPH_NAME,
) -> list[str]:
 paragraph = self._normalize_paragraph_name(sql_error_paragraph)

 return [
 "IF SQLCODE NOT = 0",
 " AND SQLCODE NOT = 100",
 f" PERFORM {paragraph}",
 "END-IF.",
]

def not_found_check(
 self,
 statement: str = "CONTINUE",
) -> list[str]:
 action = str(statement or "CONTINUE").strip()

 return [
 "IF SQLCODE = 100",
 f" {action}",
 "END-IF.",
]

def paragraph(
 self,
 paragraph_name: str = DEFAULT_PARAGRAPH_NAME,
) -> list[str]:
 name = self._normalize_paragraph_name(paragraph_name)

 return [
 f"{name}.",
 " EXEC SQL",
 " INCLUDE SQLERROR",
 " END-EXEC.",
]

def ensure_sql_error_paragraph(
 self,
 cobol_text: str,
 paragraph_name: str = DEFAULT_PARAGRAPH_NAME,
) -> str:
 """
 Ensure a standard SQLERROR paragraph exists.

 If SQLERROR already exists, normalize SQL-ERROR references to SQLERROR
 and return the text.

 If missing, insert SQLERROR before END PROGRAM when possible.
 """

 source = str(cobol_text or "")

 if not source:
 return ""

 normalized = self.normalize_sqlerror_references(source)
 paragraph = self._normalize_paragraph_name(paragraph_name)

 if SQLERROR_PARAGRAPH_PATTERN.search(normalized):
 return normalized.rstrip() + "\n"

 block = "\n".join(self.paragraph(paragraph))
```

```

lines = normalized.splitlines()
insert_index = self._find_end_program_index(lines)

if insert_index < 0:
 if lines and lines[-1].strip():
 lines.append("")
 lines.extend(block.splitlines())
 return "\n".join(lines).rstrip() + "\n"

output = lines[:insert_index]

if output and output[-1].strip():
 output.append("")

output.extend(block.splitlines())
output.append("")
output.extend(lines[insert_index:])

return "\n".join(output).rstrip() + "\n"

def normalize_sqlerror_references(
 self,
 cobol_text: str,
) -> str:
 """
 Normalize generated SQL-ERROR references to SQLERROR.
 """

 source = str(cobol_text or "")

 if not source:
 return ""

 updated = source.replace("SQL-ERROR", "SQLERROR")
 updated = SQLERROR_PERFORM_PATTERN.sub("PERFORM SQLERROR", updated)

 return updated

def has_sqlerror_include(
 self,
 cobol_text: str,
) -> bool:
 return bool(SQLERROR_INCLUDE_PATTERN.search(str(cobol_text or "")))

def _find_end_program_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 if END_PROGRAM_PATTERN.match(line.strip()):
 return index

 return -1

def _normalize_paragraph_name(
 self,
 paragraph_name: str,
) -> str:
 name = str(paragraph_name or self.DEFAULT_PARAGRAPH_NAME).strip()
 name = name.rstrip(".").upper()
 name = name.replace("SQL-ERROR", "SQLERROR")

 if not name:
 return self.DEFAULT_PARAGRAPH_NAME

 return name

```

```

=====
FILE: src\idms_db2_phase2\generators\sql_generator.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.column_name_resolver import ColumnNameResolver
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..resolvers.update_sql_plan_resolver import UpdateSqlPlanResolver
from ..services.name_normalizer import NameNormalizer

class SqlGenerator:
 """
 Generates DB2 embedded SQL snippets.

 Authority:
 - Sheet Mapping determines intended DB2 table and column names.
 - DCLGEN determines final table availability, host variable spelling,
 and group names.
 - TableNameResolver resolves TB/TV mismatches using DCLGEN.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 host_variable_resolver: HostVariableResolver,
 update_sql_plan_resolver: UpdateSqlPlanResolver | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.update_sql_plan_resolver = update_sql_plan_resolver

 def select_by_key(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 table = self.table_name_resolver.table_for_record(record)

 if not table:
 return self._skipped_block(
 operation="SELECT",
 record_name=record,
 reason="Missing Sheet Mapping or DCLGEN table metadata.",
)

 key_columns = self.column_name_resolver.key_columns_for_record(record)

 if not key_columns:
 return self._skipped_block(
 operation="SELECT",
 record_name=record,
 reason="Missing key column metadata.",
)

 select_columns = self.column_name_resolver.columns_for_record(record)

 if not select_columns:
 select_columns = key_columns

 host_references = self.host_variable_resolver.host_references_for_columns(
 table_name=table,
 columns=select_columns,
)

 if not host_references:
 return self._skipped_block(
 operation="SELECT",
 record_name=record,
 reason="Missing DCLGEN host variables.",
)

```

```

where_lines = self._where_lines(
 table_name=table,
 key_columns=key_columns,
)

if not where_lines:
 return self._skipped_block(
 operation="SELECT",
 record_name=record,
 reason="Missing DCLGEN host variables for key columns.",
)

return [
 "EXEC SQL",
 " SELECT",
 *self._comma_lines(select_columns, indent=" "),
 " INTO",
 *self._comma_lines(host_references, indent=" "),
 f" FROM {table}",
 *where_lines,
 "END-EXEC.",
]

def insert(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 table = self.table_name_resolver.table_for_record(record)

 if not table:
 return self._skipped_block(
 operation="INSERT",
 record_name=record,
 reason="Missing Sheet Mapping or DCLGEN table metadata.",
)

 columns = self.column_name_resolver.columns_for_record(record)

 if not columns:
 return self._skipped_block(
 operation="INSERT",
 record_name=record,
 reason="Missing DB2 column metadata.",
)

 host_references = self.host_variable_resolver.host_references_for_columns(
 table_name=table,
 columns=columns,
)

 if not host_references:
 return self._skipped_block(
 operation="INSERT",
 record_name=record,
 reason="Missing DCLGEN host variables.",
)

 return [
 "EXEC SQL",
 f" INSERT INTO {table}",
 " (",
 *self._comma_lines(columns, indent=" "),
 ")",
 " VALUES",
 " (",
 *self._comma_lines(host_references, indent=" "),
 ")",
 "END-EXEC.",
]

def update(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:

```

```

record = NameNormalizer.normalize(record_name)

if self.update_sql_plan_resolver:
 plan = self.update_sql_plan_resolver.resolve(
 record_name=record,
 changed_source_fields=changed_source_fields,
)
 table = plan.table_name
 key_columns = plan.key_columns
 update_columns = plan.update_columns
else:
 table = self.table_name_resolver.table_for_record(record)
 key_columns = self.column_name_resolver.key_columns_for_record(record)
 update_columns = self.column_name_resolver.update_columns_for_record(
 record_name=record,
 changed_source_fields=changed_source_fields,
)

if not table:
 return self._skipped_block(
 operation="UPDATE",
 record_name=record,
 reason="Missing Sheet Mapping or DCLGEN table metadata.",
)

if not key_columns:
 return self._skipped_block(
 operation="UPDATE",
 record_name=record,
 reason="Missing key column metadata.",
)

if not update_columns:
 return self._skipped_block(
 operation="UPDATE",
 record_name=record,
 reason="Missing changed-field metadata.",
)

set_lines = self._set_lines(
 table_name=table,
 update_columns=update_columns,
)
where_lines = self._where_lines(
 table_name=table,
 key_columns=key_columns,
)

if not set_lines or not where_lines:
 return self._skipped_block(
 operation="UPDATE",
 record_name=record,
 reason="Missing DCLGEN host variables.",
)

return [
 "EXEC SQL",
 f" UPDATE {table}",
 " SET",
 *set_lines,
 *where_lines,
 "END-EXEC.",
]

def delete(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 table = self.table_name_resolver.table_for_record(record)

 if not table:
 return self._skipped_block(
 operation="DELETE",
 record_name=record,
 reason="Missing Sheet Mapping or DCLGEN table metadata.",

```

```

)

 key_columns = self.column_name_resolver.key_columns_for_record(record)
 where_lines = self._where_lines(
 table_name=table,
 key_columns=key_columns,
)

 if not where_lines:
 return self._skipped_block(
 operation="DELETE",
 record_name=record,
 reason="Missing key column metadata or DCLGEN host variables.",
)

 return [
 "EXEC SQL",
 f" DELETE FROM {table}",
 *where_lines,
 "END-EXEC.",
]

def commit(
 self,
) -> list[str]:
 return [
 "EXEC SQL",
 " COMMIT",
 "END-EXEC.",
]

def rollback(
 self,
) -> list[str]:
 return [
 "EXEC SQL",
 " ROLLBACK",
 "END-EXEC.",
]

def changed_field_move(
 self,
 record_name: str,
 source_value: str,
 target_source_field: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 table = self.table_name_resolver.table_for_record(record)
 column = self.column_name_resolver.column_for_field(
 record_name=record,
 source_field_name=target_source_field,
)

 if not table or not column:
 return [
 f"*DB2: MOVE conversion skipped for {NameNormalizer.to_cobol(record)}.",
 "*DB2: Missing Sheet Mapping or DCLGEN metadata.",
 "CONTINUE.",
]

 host_reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table,
 column_name=column,
)

 if not host_reference:
 return [
 f"*DB2: MOVE conversion skipped for {NameNormalizer.to_cobol(record)}.",
 "*DB2: Missing DCLGEN host variable metadata.",
 "CONTINUE.",
]

 host_without_colon = host_reference[1:] if host_reference.startswith(":") else host_reference

 return [
 f"MOVE {source_value} TO {host_without_colon}.",
]

```

```

]

def _set_lines(
 self,
 table_name: str,
 update_columns: list[str],
) -> list[str]:
 output: list[str] = []

 for index, column in enumerate(update_columns):
 host_reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=column,
)

 if not host_reference:
 continue

 comma = "," if index < len(update_columns) - 1 else ""
 output.append(f" {column} = {host_reference}{comma}")

 return output

def _where_lines(
 self,
 table_name: str,
 key_columns: list[str],
) -> list[str]:
 output: list[str] = []

 for index, column in enumerate(key_columns):
 host_reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=column,
)

 if not host_reference:
 continue

 keyword = " WHERE" if index == 0 else " AND"
 output.append(f"{keyword} {column} = {host_reference}")

 return output

def _comma_lines(
 self,
 values: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []

 for index, value in enumerate(values):
 comma = "," if index < len(values) - 1 else ""
 output.append(f"{indent}{value}{comma}")

 return output

def _skipped_block(
 self,
 operation: str,
 record_name: str,
 reason: str,
) -> list[str]:
 record = NameNormalizer.to_cobol(record_name)

 return [
 f"*DB2: {operation} conversion skipped for {record}.",
 f"*DB2: {reason}",
 "CONTINUE.",
]

```

```
=====
FILE: src\idms_db2_phase2\infrastructure__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\infrastructure\file_loader.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from io import BytesIO

class FileLoader:
 """
 Loads readable text from uploaded-file-like objects.

 Expected uploaded-file-like object:
 - .name
 - .getvalue()

 Supported:
 - Text-like files through UTF-8 decode with errors ignored.
 - PDF files when pdfplumber is installed.

 This infrastructure class does not parse business metadata.
 """

 def read_uploaded_text(
 self,
 uploaded_file,
) -> str:
 if uploaded_file is None:
 return ""

 raw_bytes = uploaded_file.getvalue()
 file_name = str(getattr(uploaded_file, "name", "") or "").lower()

 if file_name.endswith(".pdf"):
 return self.read_pdf_bytes(raw_bytes)

 return raw_bytes.decode(
 "utf-8",
 errors="ignore",
)

 def read_pdf_bytes(
 self,
 raw_bytes: bytes,
) -> str:
 try:
 import pdfplumber
 except ImportError as exc:
 raise RuntimeError(
 "PDF input requires pdfplumber. Install it with: pip install pdfplumber"
) from exc

 lines: list[str] = []

 with pdfplumber.open(BytesIO(raw_bytes)) as pdf:
 for page in pdf.pages:
 page_text = page.extract_text() or ""

 for raw_line in page_text.splitlines():
 cleaned_line = self._clean_line(raw_line)

 if cleaned_line:
 lines.append(cleaned_line)

 return "\n".join(lines).strip()
```

```

def _clean_line(
 self,
 line: str | None,
) -> str:
 if line is None:
 return ""

 cleaned = str(line).rstrip()
 cleaned = cleaned.replace("\t", " ")
 cleaned = cleaned.replace("\u00a0", " ")

 while " " in cleaned:
 cleaned = cleaned.replace(" ", " ")

 return cleaned.strip()

```

```

=====
FILE: src\idms_db2_phase2\infrastructure\local_uploaded_file.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

class LocalUploadedFile:
 """
 Local file wrapper.

 Parser flow expects uploaded-file-like objects with:
 - .name
 - .getvalue()

 This wrapper allows local runner scripts to reuse the same parser logic
 used by UI upload flows.
 """

 def __init__(
 self,
 file_path: Path,
) -> None:
 self.file_path = Path(file_path)
 self.name = self.file_path.name

 def getvalue(
 self,
) -> bytes:
 return self.file_path.read_bytes()

```

```

=====
FILE: src\idms_db2_phase2\infrastructure\logger_factory.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import logging
from datetime import datetime
from pathlib import Path

from ..config.logging_settings import (
 DEFAULT_LOG_LEVEL,
 LOG_DATE_FORMAT,
 LOG_DATE_TIME_FORMAT,
 LOG_FILE_PREFIX,
 LOG_MESSAGE_FORMAT,
)
from ..config.path_settings import LOGS_DIR

class LoggerFactory:
 """
 Creates production-style loggers that write to the logs folder.

```

One log file is created per run:  
 logs/idms\_db2\_conversion\_DD-MM-YYYY\_HHMMSS.log  
 ""

```
@staticmethod
def create_logger(
 name: str,
 logs_dir: Path | None = None,
 level: str = DEFAULT_LOG_LEVEL,
) -> logging.Logger:
 target_logs_dir = Path(logs_dir or LOGS_DIR)
 target_logs_dir.mkdir(
 parents=True,
 exist_ok=True,
)

 timestamp = datetime.now().strftime(LOG_DATE_TIME_FORMAT)
 log_file = target_logs_dir / f"{LOG_FILE_PREFIX}_{timestamp}.log"

 logger = logging.getLogger(name)
 logger.setLevel(LoggerFactory._level_value(level))
 logger.handlers.clear()
 logger.propagate = False

 formatter = logging.Formatter(
 fmt=LOG_MESSAGE_FORMAT,
 datefmt=LOG_DATE_FORMAT,
)

 file_handler = logging.FileHandler(
 log_file,
 encoding="utf-8",
)
 file_handler.setLevel(LoggerFactory._level_value(level))
 file_handler.setFormatter(formatter)

 console_handler = logging.StreamHandler()
 console_handler.setLevel(LoggerFactory._level_value(level))
 console_handler.setFormatter(formatter)

 logger.addHandler(file_handler)
 logger.addHandler(console_handler)

 return logger

@staticmethod
def _level_value(
 level: str,
) -> int:
 text = str(level or DEFAULT_LOG_LEVEL).upper()

 return {
 "DEBUG": logging.DEBUG,
 "INFO": logging.INFO,
 "WARNING": logging.WARNING,
 "ERROR": logging.ERROR,
 "CRITICAL": logging.CRITICAL,
 }.get(text, logging.INFO)
```

```
=====
FILE: src\idms_db2_phase2\infrastructure\runtime_context.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path

@dataclass(slots=True)
class RuntimeContext:
 """
 Runtime context for local executions.
```

Keep runtime paths outside parser and service logic.  
Parser and service classes should receive data, not know where files came from.  
"""

```
project_root: Path
input_dir: Path
output_dir: Path
sheet_mapping_path: Path | None = None
dclgen_paths: list[Path] = field(default_factory=list)
copybook_paths: list[Path] = field(default_factory=list)
idms_cobol_source_path: Path | None = None
target_program_id: str = ""
auto_fix_pic_length_mismatches: bool = False

def ensure_output_dir(
 self,
) -> None:
 self.output_dir.mkdir(
 parents=True,
 exist_ok=True,
)

def validate_required_files(
 self,
) -> None:
 if self.sheet_mapping_path is None:
 raise ValueError("Sheet Mapping path is required.")

 if self.idms_cobol_source_path is None:
 raise ValueError("IDMS COBOL source path is required.")

 self._validate_file_exists(
 self.sheet_mapping_path,
 "Sheet Mapping",
)
 self._validate_file_exists(
 self.idms_cobol_source_path,
 "IDMS COBOL Source",
)

 if not self.dclgen_paths:
 raise ValueError("At least one DCLGEN file path is required.")

 for index, dclgen_path in enumerate(self.dclgen_paths, start=1):
 self._validate_file_exists(
 dclgen_path,
 f"DCLGEN {index}",
)

 for index, copybook_path in enumerate(self.copybook_paths, start=1):
 self._validate_file_exists(
 copybook_path,
 f"Copybook {index}",
)

 self.ensure_output_dir()

def _validate_file_exists(
 self,
 file_path: Path,
 label: str,
) -> None:
 if not file_path.exists():
 raise FileNotFoundError(f"{label} file not found: {file_path}")

 if not file_path.is_file():
 raise FileNotFoundError(f"{label} path is not a file: {file_path}")
```

```
=====
FILE: src\idms_db2_phase2\orchestration__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\orchestration\component_factory.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from typing import Any

from ..analyzers.metadata_summary_builder import MetadataSummaryBuilder
from ..composers.cobol_formatter import CobolFormatter
from ..composers.cursor_flow_composer import CursorFlowComposer
from ..composers.cursor_order_composer import CursorOrderComposer
from ..composers.db2_date_comparison_composer import Db2DateComparisonComposer
from ..composers.final_cobol_normalization_composer import (
 FinalCobolNormalizationComposer,
 FinalCobolNormalizationConfig,
)
from ..composers.fixed_format_composer import FixedFormatComposer
from ..composers.manual_layout_composer import ManualLayoutComposer
from ..composers.manual_style_preserver import ManualStylePreserver
from ..composers.retrieval_program_normalization_composer import (
 RetrievalProgramNormalizationComposer,
)
from ..composers.sqlcode_wrapper_normalization_composer import (
 SqlcodeWrapperNormalizationComposer,
)
from ..composers.update_program_normalization_composer import (
 UpdateProgramNormalizationComposer,
)
from ..composers.update_restart_conversion_composer import (
 UpdateRestartConversionComposer,
)
from ..domain.models import ConversionInput
from ..generators.cursor_paragraph_generator import CursorParagraphGenerator
from ..generators.db2_infrastructure_generator import Db2InfrastructureGenerator
from ..generators.sql_error_generator import SqlErrorGenerator
from ..generators.sql_generator import SqlGenerator
from ..repositories.copybook_repository import CopybookRepository
from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.column_name_resolver import ColumnNameResolver
from ..resolvers.cursor_name_resolver import CursorNameResolver
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.record_context_resolver import RecordContextResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..resolvers.update_sql_plan_resolver import UpdateSqlPlanResolver
from ..transformers.cobol_transformer import CobolTransformer
from ..transformers.field_reference_rewriter import FieldReferenceRewriter
from ..transformers.idms_statement_transformer import IdmsStatementTransformer
from ..transformers.pic_length_auto_fixer import PicLengthAutoFixer
from ..validators.input_validator import InputValidator
from ..validators.mapping_validator import MappingValidator
from ..validators.production_validator import ProductionValidator
```

```
class ComponentFactory:
 """
 Creates conversion components for one conversion run.

 This factory wires dependencies only.
 It must not contain conversion rules, regex patterns,
 DB2 names, COBOL field names, or DCLGEN host variables.
 """

 def create_repositories(
 self,
 conversion_input: ConversionInput,
) -> dict[str, Any]:
 return {
 "mapping": MappingRepository(conversion_input.sheet_mapping_rows),
 "dclgen": DclgenRepository(conversion_input.dclgen_columns),
 "copybook": CopybookRepository(conversion_input.copybook_fields),
 }
```

```

def create_resolvers(
 self,
 repositories: dict[str, Any],
) -> dict[str, Any]:
 table_name_resolver = TableNameResolver(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
)
 column_name_resolver = ColumnNameResolver(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=table_name_resolver,
)
 host_variable_resolver = HostVariableResolver(
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=table_name_resolver,
)
 cursor_name_resolver = CursorNameResolver()
 record_context_resolver = RecordContextResolver(
 mapping_repository=repositories["mapping"],
 table_name_resolver=table_name_resolver,
)
 update_sql_plan_resolver = UpdateSqlPlanResolver(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=table_name_resolver,
 column_name_resolver=column_name_resolver,
 host_variable_resolver=host_variable_resolver,
)

 return {
 "table_name": table_name_resolver,
 "column_name": column_name_resolver,
 "host_variable": host_variable_resolver,
 "cursor_name": cursor_name_resolver,
 "record_context": record_context_resolver,
 "update_sql_plan": update_sql_plan_resolver,
 }

def create_generators(
 self,
 repositories: dict[str, Any],
 resolvers: dict[str, Any],
) -> dict[str, Any]:
 sql_error_generator = SqlErrorGenerator()

 sql_generator = SqlGenerator(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
 column_name_resolver=resolvers["column_name"],
 host_variable_resolver=resolvers["host_variable"],
 update_sql_plan_resolver=resolvers["update_sql_plan"],
)

 db2_infrastructure_generator = Db2InfrastructureGenerator(
 table_name_resolver=resolvers["table_name"],
 column_name_resolver=resolvers["column_name"],
 host_variable_resolver=resolvers["host_variable"],
 cursor_name_resolver=resolvers["cursor_name"],
)

 cursor_paragraph_generator = CursorParagraphGenerator(
 db2_infrastructure_generator=db2_infrastructure_generator,
 host_variable_resolver=resolvers["host_variable"],
 sql_error_generator=sql_error_generator,
)

 return {
 "sql": sql_generator,
 "sql_error": sql_error_generator,
 "db2_infrastructure": db2_infrastructure_generator,
 "cursor_paragraph": cursor_paragraph_generator,
 }

```

```

def create_transformers(
 self,
 repositories: dict[str, Any],
 resolvers: dict[str, Any],
 generators: dict[str, Any],
) -> dict[str, Any]:
 idms_statement_transformer = IdmsStatementTransformer(
 sql_generator=generators["sql"],
 sql_error_generator=generators["sql_error"],
 table_name_resolver=resolvers["table_name"],
 cursor_name_resolver=resolvers["cursor_name"],
)

 return {
 "idms_statement": idms_statement_transformer,
 "cobol": CobolTransformer(
 idms_statement_transformer=idms_statement_transformer,
),
 "field_reference": FieldReferenceRewriter(
 mapping_repository=repositories["mapping"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
),
 "pic_length": PicLengthAutoFixer(),
 }

def create_composers(
 self,
 repositories: dict[str, Any],
 resolvers: dict[str, Any],
) -> dict[str, Any]:
 return {
 "formatter": CobolFormatter(),
 "cursor_flow": CursorFlowComposer(),
 "cursor_order": CursorOrderComposer(),
 "date_compare": Db2DateComparisonComposer(),
 "fixed_format": FixedFormatComposer(),
 "manual_layout": ManualLayoutComposer(),
 "style_preserver": ManualStylePreserver(),
 "retrieval_program": RetrievalProgramNormalizationComposer(
 dclgen_repository=repositories["dclgen"],
),
 "final_cobol": FinalCobolNormalizationComposer(
 config=FinalCobolNormalizationConfig(
 db2_date_external_format="DD.MM.YYYY",
 require_order_by_columns_in_select=False,
 resequence_final_output=True,
),
),
 "sqlcode_wrapper": SqlcodeWrapperNormalizationComposer(),
 "update_program": UpdateProgramNormalizationComposer(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
),
 "update_restart": UpdateRestartConversionComposer(),
 }

def create_validators(
 self,
 repositories: dict[str, Any],
 resolvers: dict[str, Any],
) -> dict[str, Any]:
 return {
 "input": InputValidator(),
 "mapping": MappingValidator(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
),
 "production": ProductionValidator(
 dclgen_repository=repositories["dclgen"],
),
 }

```

```

def create_summary_builder(
 self,
) -> MetadataSummaryBuilder:
 return MetadataSummaryBuilder()

```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion_context.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from ..domain.models import ConversionInput

```

```

@dataclass(slots=True)
class ConversionContext:
 """
 Runtime component context for one stateless conversion run.
 """

 conversion_input: ConversionInput
 repositories: dict[str, Any]
 resolvers: dict[str, Any]
 generators: dict[str, Any]
 transformers: dict[str, Any]
 composers: dict[str, Any]
 validators: dict[str, Any]
 summary_builder: Any
 validation_messages: list[str]

```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion_pipeline.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Any

from ..domain.models import ConversionInput, ConversionResult, IdmsOperation
from .component_factory import ComponentFactory
from .conversion_context import ConversionContext

```

```

class ConversionPipeline:
 """
 Executes one stateless IDMS-to-DB2 conversion run.

 Pipeline stages:
 1. Input validation
 2. Repository creation
 3. Resolver creation
 4. Mapping validation
 5. COBOL transformation
 6. Field-reference rewrite
 7. DB2 infrastructure generation
 8. Cursor paragraph generation
 9. Cursor flow normalization
 10. Cursor ORDER normalization
 11. Date comparison normalization
 12. SQLERROR paragraph enforcement
 13. Retrieval program normalization
 14. Update program normalization only for update-style programs
 15. Restart conversion normalization
 16. SQLCODE wrapper normalization
 17. Manual layout preservation
 18. Fixed-format composition
 19. Final COBOL normalization
 20. Production validation
 21. Result packaging
 """

```

"""

```
UPDATE_STYLE_OPERATIONS = {
 "MODIFY",
 "UPDATE",
 "ERASE",
 "DELETE",
}

def __init__(
 self,
 component_factory: ComponentFactory | None = None,
) -> None:
 self.component_factory = component_factory or ComponentFactory()

def run(
 self,
 conversion_input: ConversionInput,
) -> ConversionResult:
 context = self._create_context(conversion_input)

 self._validate_input(context)

 if context.validation_messages:
 return self._build_result(
 context=context,
 converted_cobol="",
 operations=[],
)

 self._validate_mapping(context)

 converted_cobol, transform_messages, operations = context.transformers[
 "cobol"
].transform(
 cobol_text=conversion_input.idms_cobol_text,
 target_program_id=conversion_input.target_program_id,
)
 context.validation_messages.extend(transform_messages)

 converted_cobol = context.transformers["field_reference"].rewrite(
 converted_cobol,
)
 self._collect_component_messages(
 context,
 context.transformers["field_reference"],
)

 converted_cobol, infrastructure_messages = context.generators[
 "db2_infrastructure"
].apply(
 cobol_text=converted_cobol,
 operations=operations,
)
 context.validation_messages.extend(infrastructure_messages)

 converted_cobol, cursor_messages = context.generators[
 "cursor_paragraph"
].apply(
 cobol_text=converted_cobol,
 operations=operations,
)
 context.validation_messages.extend(cursor_messages)

 converted_cobol = context.composers["cursor_flow"].compose(
 cobol_text=converted_cobol,
 operations=operations,
)
 self._collect_component_messages(
 context,
 context.composers["cursor_flow"],
)

 converted_cobol = context.composers["cursor_order"].compose(
 converted_cobol,
)
```

```

self._collect_component_messages(
 context,
 context.composers["cursor_order"],
)

converted_cobol = context.composers["date_compare"].compose(
 converted_cobol,
)
self._collect_component_messages(
 context,
 context.composers["date_compare"],
)

converted_cobol = context.generators["sql_error"].ensure_sql_error_paragraph(
 converted_cobol,
)

converted_cobol = context.composers["retrieval_program"].compose(
 original_text=conversion_input.idms_cobol_text,
 converted_text=converted_cobol,
)
self._collect_component_messages(
 context,
 context.composers["retrieval_program"],
)

if self._has_update_style_operations(operations):
 converted_cobol = context.composers["update_program"].compose(
 converted_cobol,
)
 self._collect_component_messages(
 context,
 context.composers["update_program"],
)

converted_cobol = context.composers["update_restart"].compose(
 converted_cobol,
)
self._collect_component_messages(
 context,
 context.composers["update_restart"],
)

converted_cobol = context.composers["sqlcode_wrapper"].compose(
 converted_cobol,
)
self._collect_component_messages(
 context,
 context.composers["sqlcode_wrapper"],
)

converted_cobol = context.composers["manual_layout"].compose(
 converted_cobol,
)
self._collect_component_messages(
 context,
 context.composers["manual_layout"],
)

converted_cobol = context.composers["style_preserver"].preserve(
 original_text=conversion_input.idms_cobol_text,
 converted_text=converted_cobol,
)

if conversion_input.auto_fix_pic_length_mismatches:
 converted_cobol = context.transformers["pic_length"].fix(
 source_cobol_text=conversion_input.idms_cobol_text,
 converted_cobol_text=converted_cobol,
)
 self._collect_component_messages(
 context,
 context.transformers["pic_length"],
)

converted_cobol = context.composers["fixed_format"].format(
 converted_cobol,

```

```

)
 self._collect_component_messages(
 context,
 context.composers["fixed_format"],
)

 converted_cobol = context.composers["final_cobol"].compose(
 converted_cobol,
)
 self._collect_component_messages(
 context,
 context.composers["final_cobol"],
)

 converted_cobol = context.composers["formatter"].format(
 converted_cobol,
)

 context.validation_messages.extend(
 context.validators["production"].validate(converted_cobol),
)
 context.validation_messages = self._unique_messages(
 context.validation_messages,
)

 return self._build_result(
 context=context,
 converted_cobol=converted_cobol,
 operations=operations,
)

def _create_context(
 self,
 conversion_input: ConversionInput,
) -> ConversionContext:
 repositories = self.component_factory.create_repositories(
 conversion_input,
)
 resolvers = self.component_factory.create_resolvers(repositories)
 generators = self.component_factory.create_generators(
 repositories=repositories,
 resolvers=resolvers,
)
 transformers = self.component_factory.create_transformers(
 repositories=repositories,
 resolvers=resolvers,
 generators=generators,
)
 composers = self.component_factory.create_composers(
 repositories=repositories,
 resolvers=resolvers,
)
 validators = self.component_factory.create_validators(
 repositories=repositories,
 resolvers=resolvers,
)
 summary_builder = self.component_factory.create_summary_builder()

 return ConversionContext(
 conversion_input=conversion_input,
 repositories=repositories,
 resolvers=resolvers,
 generators=generators,
 transformers=transformers,
 composers=composers,
 validators=validators,
 summary_builder=summary_builder,
 validation_messages=[],
)

def _validate_input(
 self,
 context: ConversionContext,
) -> None:
 context.validation_messages.extend(
 context.validators["input"].validate(context.conversion_input),
)

```

```

)

def _validate_mapping(
 self,
 context: ConversionContext,
) -> None:
 context.validation_messages.extend(
 context.validators["mapping"].validate(),
)

def _has_update_style_operations(
 self,
 operations: list[IdmsOperation],
) -> bool:
 for operation in operations:
 operation_name = str(operation.operation or "").strip().upper()

 if operation_name in self.UPDATE_STYLE_OPERATIONS:
 return True

 return False

def _collect_component_messages(
 self,
 context: ConversionContext,
 component: Any,
) -> None:
 messages = getattr(component, "messages", [])

 if callable(messages):
 messages = messages()

 if not messages:
 return

 context.validation_messages.extend(messages)

def _build_result(
 self,
 context: ConversionContext,
 converted_cobol: str,
 operations: list[Any],
) -> ConversionResult:
 conversion_input = context.conversion_input
 rows = conversion_input.sheet_mapping_rows
 summary_builder = context.summary_builder

 return ConversionResult(
 converted_cobol=converted_cobol,
 validation_messages=self._unique_messages(context.validation_messages),
 metadata_overview=summary_builder.metadata_overview(
 sheet_mapping_rows=rows,
 dclgen_count=len(conversion_input.dclgen_columns),
 copybook_count=len(conversion_input.copybook_fields),
 cobol_text=conversion_input.idms_cobol_text,
 target_program_id=conversion_input.target_program_id,
),
 record_summary=summary_builder.record_summary(rows),
 column_summary=summary_builder.column_summary(rows),
 relationship_summary=summary_builder.relationship_summary(rows),
 sheet_mapping_preview=summary_builder.sheet_mapping_preview(rows),
 operations=operations,
)

def _unique_messages(
 self,
 messages: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for message in messages:
 clean_message = str(message or "").strip()

 if not clean_message:
 continue

```

```

 if clean_message in seen:
 continue

 seen.add(clean_message)
 output.append(clean_message)

 return output

```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import ConversionInput, ConversionResult
from .conversion_pipeline import ConversionPipeline

class ConversionService:
 """
 Public conversion service.

 Existing UI, runners, and API callers should use this class.
 Actual orchestration is delegated to ConversionPipeline.
 """

 def __init__(
 self,
 pipeline: ConversionPipeline | None = None,
) -> None:
 self.pipeline = pipeline or ConversionPipeline()

 def convert(
 self,
 conversion_input: ConversionInput,
) -> ConversionResult:
 return self.pipeline.run(conversion_input)

```

```

=====
FILE: src\idms_db2_phase2\parsers__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\parsers\base_text_parser.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

class BaseTextParser:
 """
 Shared parser helper methods.

 This base class intentionally avoids business rules, schema names,
 and regex definitions. It contains only generic text cleanup utilities.
 """

 def normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def normalize_spaces(
 self,
 text: str,
) -> str:

```

```

output = str(text or "").replace("\t", " ")
output = output.replace("\u00a0", " ")

while " " in output:
 output = output.replace(" ", " ")

return output

def clean_text(
 self,
 text: str,
) -> str:
 output = self.normalize_line_endings(text)
 output = self.normalize_spaces(output)
 output = output.replace("'", "'").replace("'''", '"""')
 output = output.replace('"', '"').replace("'''", '"""')
 return output

def clean_cell(
 self,
 value: object,
) -> str:
 if value is None:
 return ""

 text = str(value)
 text = text.replace("\uffff", "")
 text = text.replace("\xa0", " ")
 text = self.normalize_line_endings(text)
 text = self.normalize_spaces(text)
 return text.strip()

def strip_sequence_area(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if len(text) > 6:
 indicator = text[6:7]

 if indicator in ("*", "/"):
 return indicator + text[7:]

 if text[:6].strip().isdigit():
 return text[6:]

 return text

def logical_line(
 self,
 line: str,
) -> str:
 return self.strip_sequence_area(line).strip()

```

```

=====
FILE: src\idms_db2_phase2\parsers\cobol_parser.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import IdmsOperation
from ..reference.patterns.cobol.structure_patterns import PROGRAM_ID_PATTERN
from ..reference.patterns.idms.statement_patterns import (
 ERASE_PATTERN,
 FIND_CURRENT_PATTERN,
 FIND_FIRST_PATTERN,
 MODIFY_PATTERN,
 OBTAIN_CALC_PATTERN,
 OBTAIN_CALC_REVERSED_PATTERN,
 OBTAIN_FIRST_NEXT_PATTERN,
 OBTAIN_OWNER_PATTERN,
 READY_UPDATE_PATTERN,
 STORE_PATTERN,
)

```

```

)
from .base_text_parser import BaseTextParser

class CobolParser(BaseTextParser):
 """
 Parses IDMS COBOL source and identifies IDMS database operations.

 This parser does not convert COBOL. It only identifies operations for
 later transformation, generation, diagnostics, and validation.
 """

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def program_id(
 self,
 cobol_text: str,
) -> str:
 match = PROGRAM_ID_PATTERN.search(cobol_text or "")

 if not match:
 return ""

 return match.group(1).upper()

 def analyze(
 self,
 cobol_text: str,
) -> list[IdmsOperation]:
 self.diagnostics = []
 clean_text = self.normalize_line_endings(cobol_text)
 operations: list[IdmsOperation] = []

 for line_number, raw_line in enumerate(clean_text.splitlines(), start=1):
 logical = self.logical_line(raw_line)

 if not logical:
 continue

 if logical.startswith("*") or logical.startswith("/"):
 continue

 operation = self._operation_from_line(
 logical=logical,
 source_line=raw_line,
 line_number=line_number,
)

 if operation:
 operations.append(operation)

 self.diagnostics.append(f"COBOL IDMS operation count: {len(operations)}")
 return operations

 def _operation_from_line(
 self,
 logical: str,
 source_line: str,
 line_number: int,
) -> IdmsOperation | None:
 match = OBTAIN_CALC_PATTERN.search(logical)

 if match:
 return IdmsOperation(
 operation="OBTAIN_CALC",
 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

 match = OBTAIN_CALC_REVERSED_PATTERN.search(logical)

 if match:
 return IdmsOperation(
 operation="OBTAIN_CALC",

```

```

 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = OBTAIN_FIRST_NEXT_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation=f"OBTAIN_{match.group('mode').upper()}",
 record_name=match.group("record").upper(),
 set_name=match.group("set").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = OBTAIN_OWNER_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="OBTAIN_OWNER",
 set_name=match.group("set").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = FIND_FIRST_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="FIND_FIRST",
 record_name=str(match.group("record") or "").upper(),
 set_name=match.group("set").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = FIND_CURRENT_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="FIND_CURRENT",
 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = STORE_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="STORE",
 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = MODIFY_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="MODIFY",
 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

match = ERASE_PATTERN.search(logical)

if match:
 return IdmsOperation(
 operation="ERASE",
 record_name=match.group("record").upper(),
 source_line=source_line,
 line_number=line_number,
)

```

```

 if READY_UPDATE_PATTERN.search(logical):
 return IdmsOperation(
 operation="READY_UPDATE",
 source_line=source_line,
 line_number=line_number,
)

 return None

```

```

=====
FILE: src\idms_db2_phase2\parsers\copybook_parser.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import CopybookField
from ..reference.patterns.cobol.copybook_patterns import (
 COPYBOOK_COMMENT_OR_SKIP_PATTERN,
 COPYBOOK_FIELD_PATTERN,
 COPYBOOK_PIC_PATTERN,
 COPYBOOK_USAGE_PATTERN,
)
from ..services.name_normalizer import NameNormalizer
from .base_text_parser import BaseTextParser

class CopybookParser(BaseTextParser):
 """
 Parses COBOL copybook field declarations.

 Copybook is optional. This parser does not perform conversion.
 """

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def parse(
 self,
 text: str,
) -> list[CopybookField]:
 self.diagnostics = []
 clean_text = self.clean_text(text)

 if not clean_text.strip():
 self.diagnostics.append("Copybook text is empty.")
 return []

 statements = self._logical_statements(clean_text)
 output: list[CopybookField] = []
 parent_stack: list[CopybookField] = []

 for statement in statements:
 match = COPYBOOK_FIELD_PATTERN.match(statement)

 if not match:
 continue

 level = match.group("level")
 field_name = NameNormalizer.to_cobol(match.group("name"))
 body = str(match.group("body") or "")

 if not field_name:
 continue

 pic_clause = self._extract_pic_clause(body)
 usage = self._extract_usage(body)
 parent_name = self._current_parent_name(parent_stack, level)

 field = CopybookField(
 field_name=field_name,
 level=level,
 pic_clause=pic_clause,
 usage=usage,

```

```

 parent_name=parent_name,
 raw_line=statement,
)

 if pic_clause:
 output.append(field)

 self._update_parent_stack(parent_stack, field)

self.diagnostics.append(f"Copybook parsed field count: {len(output)}")
return output

def _logical_statements(
 self,
 text: str,
) -> list[str]:
 output: list[str] = []
 buffer = ""

 for raw_line in text.splitlines():
 line = self.logical_line(raw_line)

 if not line.strip():
 continue

 if COPYBOOK_COMMENT_OR_SKIP_PATTERN.search(line):
 continue

 if buffer:
 buffer = f"{buffer} {line.strip()}"
 else:
 buffer = line.strip()

 if "." in line:
 parts = buffer.split(".")

 for part in parts[:-1]:
 clean = part.strip()

 if clean:
 output.append(clean + ".")

 buffer = parts[-1].strip()

 if buffer.strip():
 output.append(buffer.strip())

 return output

def _extract_pic_clause(
 self,
 text: str,
) -> str:
 match = COPYBOOK_PIC_PATTERN.search(text)

 if not match:
 return ""

 return f"PIC {match.group('pic').upper()}"

def _extract_usage(
 self,
 text: str,
) -> str:
 match = COPYBOOK_USAGE_PATTERN.search(text)

 if not match:
 return ""

 return match.group("usage").upper()

def _current_parent_name(
 self,
 parent_stack: list[CopybookField],
 level: str,
) -> str:

```

```

 current_level = int(level)

 for candidate in reversed(parent_stack):
 if int(candidate.level) < current_level:
 return candidate.field_name

 return ""

 def _update_parent_stack(
 self,
 parent_stack: list[CopybookField],
 field: CopybookField,
) -> None:
 current_level = int(field.level)

 while parent_stack and int(parent_stack[-1].level) >= current_level:
 parent_stack.pop()

 if not field.pic_clause:
 parent_stack.append(field)

=====
FILE: src\idms_db2_phase2\parsers\dclgen_parser.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..domain.models import DclgenColumn
from ..reference.catalogs.db2.db2_sql_types import (
 DB2_SQL_SKIP_WORDS,
 DB2_SQL_TYPE_STARTERS,
)
from ..reference.catalogs.mapping.dclgen_schema import (
 DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE,
)
from ..reference.patterns.db2.dclgen_patterns import (
 COBOL_FIELD_PATTERN,
 COBOL_GROUP_PATTERN,
 DECLARE_TABLE_ALT_PATTERN,
 DECLARE_TABLE_PATTERN,
 DCLGEN_PIC_PATTERN,
 DCLGEN_TABLE_COMMENT_PATTERN,
 DCLGEN_USAGE_PATTERN,
 MULTIPLE_UNDERSCORE_PATTERN,
 SQL_NAME_CLEANUP_PATTERN,
 VALID_SQL_COLUMN_NAME_PATTERN,
)
from ..services.name_normalizer import NameNormalizer
from .base_text_parser import BaseTextParser

class DclgenParser(BaseTextParser):
 """
 Parses DB2 DCLGEN text files.

 Static content is externalized:
 - SQL type names: reference/catalogs/db2/db2_sql_types.py
 - DCLGEN conventions: reference/catalogs/mapping/dclgen_schema.py
 - Regex patterns: reference/patterns/db2/dclgen_patterns.py
 """

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def parse_many_texts(
 self,
 texts: list[str],
) -> list[DclgenColumn]:
 self.diagnostics = []
 output: list[DclgenColumn] = []

 self.diagnostics.append(f"DCLGEN input file count: {len(texts)}")

 for index, text in enumerate(texts, start=1):

```

```

 parsed = self.parse(
 text=text,
 source_label=f"DCLGEN-{index}",
)
 output.extend(parsed)

 self.diagnostics.append(f"DCLGEN parsed column count: {len(output)}")
 return output

def parse(
 self,
 text: str,
 source_label: str = "",
) -> list[DclgenColumn]:
 clean_text = self.clean_text(text)

 if not clean_text.strip():
 self.diagnostics.append(f"{source_label}: DCLGEN text is empty.")
 return []

 table_name = self._extract_table_name(clean_text)
 group_name = self._extract_group_name(clean_text)

 if not table_name:
 self.diagnostics.append(f"{source_label}: DCLGEN table not detected.")

 if not group_name:
 self.diagnostics.append(f"{source_label}: DCLGEN group not detected.")

 columns = self._parse_sql_columns(
 text=clean_text,
 table_name=table_name,
 group_name=group_name,
)

 cobol_fields = self._parse_cobol_fields(clean_text)

 if cobol_fields:
 columns = self._merge_host_metadata(
 columns=columns,
 cobol_fields=cobol_fields,
 table_name=table_name,
 group_name=group_name,
)

 return columns

def _extract_table_name(
 self,
 text: str,
) -> str:
 for pattern in (
 DECLARE_TABLE_PATTERN,
 DECLARE_TABLE_ALT_PATTERN,
 DCLGEN_TABLE_COMMENT_PATTERN,
):
 match = pattern.search(text)

 if match:
 return NameNormalizer.to_sql(match.group(1))

 return ""

def _extract_group_name(
 self,
 text: str,
) -> str:
 for line in text.splitlines():
 logical = self.logical_line(line)
 match = COBOL_GROUP_PATTERN.match(logical)

 if match:
 return NameNormalizer.to_cobol(match.group(1))

 return ""

```

```

def _parse_sql_columns(
 self,
 text: str,
 table_name: str,
 group_name: str,
) -> list[DclgenColumn]:
 output: list[DclgenColumn] = []
 in_declare_table = False

 for line in text.splitlines():
 logical = self.logical_line(line)
 upper = logical.upper()

 if DECLARE_TABLE_PATTERN.search(logical) or DECLARE_TABLE_ALT_PATTERN.search(
 logical
):
 in_declare_table = True
 continue

 if in_declare_table and logical.startswith("("):
 in_declare_table = False
 continue

 if not in_declare_table:
 continue

 column = self._parse_sql_column_line(
 line=logical,
 table_name=table_name,
 group_name=group_name,
)

 if column:
 output.append(column)

 return output

def _parse_sql_column_line(
 self,
 line: str,
 table_name: str,
 group_name: str,
) -> DclgenColumn | None:
 text = str(line or "").strip().rstrip(",")

 if not text:
 return None

 parts = text.split()

 if len(parts) < 2:
 return None

 column_name = self._clean_sql_name(parts[0])
 type_name = parts[1].upper().split("(")[0]

 if type_name not in DB2_SQL_TYPE_STARTERS:
 return None

 if column_name in DB2_SQL_SKIP_WORDS:
 return None

 if not VALID_SQL_COLUMN_NAME_PATTERN.match(column_name):
 return None

 return DclgenColumn(
 table_name=table_name,
 group_name=group_name,
 column_name=column_name,
 host_variable="",
 level="",
 pic_clause="",
 usage="",
 raw_line=line,
)

```

```

def _parse_cobol_fields(
 self,
 text: str,
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for line in text.splitlines():
 logical = self.logical_line(line)
 match = COBOL_FIELD_PATTERN.match(logical)

 if not match:
 continue

 host_variable = NameNormalizer.to_cobol(match.group("host"))

 if self._should_ignore_host(host_variable):
 continue

 body = str(match.group("body") or "")
 pic_clause = self._extract_pic_clause(body)
 usage = self._extract_usage(body)

 output.append(
 {
 "level": match.group("level"),
 "host_variable": host_variable,
 "pic_clause": pic_clause,
 "usage": usage,
 "raw_line": logical,
 }
)

 return output

def _merge_host_metadata(
 self,
 columns: list[DclgenColumn],
 cobol_fields: list[dict[str, str]],
 table_name: str,
 group_name: str,
) -> list[DclgenColumn]:
 output: list[DclgenColumn] = []

 for index, field in enumerate(cobol_fields):
 column_name = ""

 if index < len(columns):
 column_name = columns[index].column_name

 output.append(
 DclgenColumn(
 table_name=table_name,
 group_name=group_name,
 column_name=column_name,
 host_variable=field["host_variable"],
 level=field["level"],
 pic_clause=field["pic_clause"],
 usage=field["usage"],
 raw_line=field["raw_line"],
)
)

 return output

def _extract_pic_clause(
 self,
 text: str,
) -> str:
 match = DCLGEN_PIC_PATTERN.search(text)

 if not match:
 return ""

 return f"PIC {match.group('pic').upper()}"

def _extract_usage(

```

```

 self,
 text: str,
) -> str:
 match = DCLGEN_USAGE_PATTERN.search(text)

 if not match:
 return ""

 return match.group("usage").upper()

 def _clean_sql_name(
 self,
 value: str,
) -> str:
 text = NameNormalizer.normalize(value)
 text = SQL_NAME_CLEANUP_PATTERN.sub("_", text)
 text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)
 return text.strip("_")

 def _should_ignore_host(
 self,
 host_variable: str,
) -> bool:
 normalized = NameNormalizer.normalize(host_variable)

 for suffix in DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE:
 if normalized.endswith(f"-{suffix}") or normalized.endswith(f"_{suffix}"):
 return True

 return False

```

```

=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping_parser.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import csv
from io import BytesIO, StringIO
from typing import Any

from openpyxl import load_workbook

from ..domain.models import SheetMappingRow
from ..reference.catalogs.mapping.sheet_mapping_schema import (
 SHEET_MAPPING_FIELD_ALIASES,
 SHEET_MAPPING_HEADER_DETECTION_GROUPS,
 SHEET_MAPPING_MODEL_FIELD_MAP,
)
from ..reference.patterns.mapping.sheet_mapping_patterns import (
 HEADER_NON_ALPHANUMERIC_PATTERN,
 HEADER_WHITESPACE_PATTERN,
)
from .base_text_parser import BaseTextParser

```

```

class SheetMappingParser(BaseTextParser):
 """
 Parses Sheet Mapping files from CSV text or XLSX bytes.

 This parser does not own Sheet Mapping column names, aliases,
 header detection rules, or regex patterns.
 """

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def parse_csv_text(
 self,
 text: str,
) -> list[SheetMappingRow]:
 self.diagnostics = []

 clean_text = self.clean_text(text)

```

```

if not clean_text.strip():
 self.diagnostics.append("Sheet Mapping CSV text is empty.")
 return []

reader = csv.reader(StringIO(clean_text))
rows = list(reader)

if not rows:
 self.diagnostics.append("Sheet Mapping CSV has no rows.")
 return []

header_index = self._find_header_index(rows)

if header_index < 0:
 self.diagnostics.append("Sheet Mapping header row was not detected.")
 return []

headers = rows[header_index]
column_indexes = self._resolve_column_indexes(headers)
output: list[SheetMappingRow] = []

for raw_row in rows[header_index + 1 :]:
 row = self._row_from_values(raw_row, column_indexes)

 if self._is_empty_row(row):
 continue

 output.append(row)

self._append_diagnostics(output)
return output

def parse_xlsx_bytes(
 self,
 content: bytes,
) -> list[SheetMappingRow]:
 self.diagnostics = []

 workbook = load_workbook(
 filename=BytesIO(content),
 read_only=True,
 data_only=True,
)
 worksheet = workbook.active

 rows = [
 [self.clean_cell(cell) for cell in row]
 for row in worksheet.iter_rows(values_only=True)
]

 if not rows:
 self.diagnostics.append("Sheet Mapping workbook has no rows.")
 return []

 header_index = self._find_header_index(rows)

 if header_index < 0:
 self.diagnostics.append("Sheet Mapping header row was not detected.")
 return []

 headers = rows[header_index]
 column_indexes = self._resolve_column_indexes(headers)
 output: list[SheetMappingRow] = []

 for raw_row in rows[header_index + 1 :]:
 row = self._row_from_values(raw_row, column_indexes)

 if self._is_empty_row(row):
 continue

 output.append(row)

 self._append_diagnostics(output)
 return output

```

```

def parse_uploaded_file(
 self,
 uploaded_file: Any,
) -> list[SheetMappingRow]:
 self.diagnostics = []

 if uploaded_file is None:
 self.diagnostics.append("No Sheet Mapping file supplied.")
 return []

 file_name = str(getattr(uploaded_file, "name", "") or "").lower()
 raw_bytes = uploaded_file.getvalue()

 self.diagnostics.append(f"Sheet Mapping file name: {file_name}")
 self.diagnostics.append(f"Sheet Mapping file size bytes: {len(raw_bytes)}")

 if file_name.endswith(".xlsx"):
 return self.parse_xlsx_bytes(raw_bytes)

 if file_name.endswith(".xls"):
 self.diagnostics.append(
 "Unsupported .xls file detected. Save the file as .xlsx or .csv."
)
 return []

 text = raw_bytes.decode("utf-8-sig", errors="replace")
 return self.parse_csv_text(text)

def _find_header_index(
 self,
 rows: list[list[object]],
) -> int:
 for index, row in enumerate(rows):
 normalized_values = {
 self._normalize_header(value)
 for value in row
 if self.clean_cell(value)
 }

 if self._matches_header_groups(normalized_values):
 return index

 return -1

def _matches_header_groups(
 self,
 normalized_values: set[str],
) -> bool:
 matched_groups = 0

 for group in SHEET_MAPPING_HEADER_DETECTION_GROUPS:
 normalized_group = {
 self._normalize_header(value)
 for value in group
 }

 if normalized_values.intersection(normalized_group):
 matched_groups += 1

 return matched_groups >= 2

def _resolve_column_indexes(
 self,
 headers: list[object],
) -> dict[str, int]:
 normalized_headers = {
 self._normalize_header(header): index
 for index, header in enumerate(headers)
 if self.clean_cell(header)
 }

 column_indexes: dict[str, int] = {}

 for model_field, canonical_column in SHEET_MAPPING_MODEL_FIELD_MAP.items():
 candidate_names = [canonical_column]
 candidate_names.extend(

```

```

 SHEET_MAPPING_FIELD_ALIASES.get(canonical_column, [])
)

 for candidate in candidate_names:
 normalized_candidate = self._normalize_header(candidate)

 if normalized_candidate in normalized_headers:
 column_indexes[model_field] = normalized_headers[
 normalized_candidate
]
 break

 return column_indexes

def _row_from_values(
 self,
 values: list[object],
 column_indexes: dict[str, int],
) -> SheetMappingRow:
 data: dict[str, str] = {}

 for model_field in SHEET_MAPPING_MODEL_FIELD_MAP:
 column_index = column_indexes.get(model_field)

 if column_index is None:
 data[model_field] = ""
 continue

 if column_index >= len(values):
 data[model_field] = ""
 continue

 data[model_field] = self.clean_cell(values[column_index])

 return SheetMappingRow(**data)

def _normalize_header(
 self,
 value: object,
) -> str:
 text = self.clean_cell(value).upper()
 text = HEADER_WHITESPACE_PATTERN.sub(" ", text)
 text = HEADER_NON_ALPHANUMERIC_PATTERN.sub("", text)
 return text

def _is_empty_row(
 self,
 row: SheetMappingRow,
) -> bool:
 return not any(
 self.clean_cell(getattr(row, field_name))
 for field_name in SHEET_MAPPING_MODEL_FIELD_MAP
)

def _append_diagnostics(
 self,
 rows: list[SheetMappingRow],
) -> None:
 record_count = sum(
 1
 for row in rows
 if str(row.cobol_record_idms or "").strip()
)
 source_field_count = sum(
 1
 for row in rows
 if str(row.cobol_zone or "").strip()
 or str(row.reference_field_name_copybook or "").strip()
)
 db2_table_count = sum(
 1
 for row in rows
 if str(row.new_db2_record or "").strip()
 or str(row.cross_application_db2_table or "").strip()
)
 db2_column_count = sum(

```

```

1
for row in rows
if str(row.new_db2_field_name or "").strip()
or str(row.cross_application_db2_field_name or "").strip()
)
useful_context_count = sum(
1
for row in rows
if (
 str(row.cobol_record_idms or "").strip()
 and (
 str(row.cobol_zone or "").strip()
 or str(row.reference_field_name_copybook or "").strip()
)
 and (
 str(row.new_db2_record or "").strip()
 or str(row.cross_application_db2_table or "").strip()
)
 and (
 str(row.new_db2_field_name or "").strip()
 or str(row.cross_application_db2_field_name or "").strip()
)
)
)

self.diagnostics.append(f"Sheet Mapping parsed rows: {len(rows)}")
self.diagnostics.append(
 f"Sheet Mapping populated Cobol Record IDMS rows: {record_count}"
)
self.diagnostics.append(
 f"Sheet Mapping populated source field rows: {source_field_count}"
)
self.diagnostics.append(
 f"Sheet Mapping populated DB2 table rows: {db2_table_count}"
)
self.diagnostics.append(
 f"Sheet Mapping populated DB2 column rows: {db2_column_count}"
)
self.diagnostics.append(
 f"Sheet Mapping useful source-to-target rows: {useful_context_count}"
)

```

```

=====
FILE: src\idms_db2_phase2\parsers\text_loader.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..infrastructure.file_loader import FileLoader

class TextLoader(FileLoader):
 """
 Backward-compatible parser-side alias for FileLoader.

 Clean architecture keeps actual file loading in infrastructure.
 """

```

```

=====
FILE: src\idms_db2_phase2\reference__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\reference\catalogs__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\catalogs\db2__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\catalogs\db2\db2_sql_types.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
DB2_SQL_TYPE_STARTERS = {
 "CHAR",
 "CHARACTER",
 "VARCHAR",
 "LONG",
 "GRAPHIC",
 "VARGRAPHIC",
 "SMALLINT",
 "INTEGER",
 "INT",
 "BIGINT",
 "DECIMAL",
 "DEC",
 "NUMERIC",
 "NUM",
 "FLOAT",
 "REAL",
 "DOUBLE",
 "DATE",
 "TIME",
 "TIMESTAMP",
 "BLOB",
 "CLOB",
 "DBCLOB",
 "XML",
}
```

```
DB2_SQL_SKIP_WORDS = {
 "EXEC",
 "SQL",
 "DECLARE",
 "TABLE",
 "END-EXEC",
 "END",
 "NOT",
 "NULL",
 "WITH",
 "DEFAULT",
 "PRIMARY",
 "FOREIGN",
 "KEY",
 "CONSTRAINT",
 "UNIQUE",
 "CHECK",
 "REFERENCES",
 "CREATE",
 "IN",
 "IS",
 "THE",
 "DCLGEN",
 "COMMAND",
 "THAT",
 "MADE",
 "FOLLOWING",
 "STATEMENTS",
}
```

```
=====
FILE: src\idms_db2_phase2\reference\catalogs\mapping__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\catalogs\mapping\dclgen_schema.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
DCLGEN_GROUP_PREFIX = "DCL"
```

```
DCLGEN_COLUMN_FIELDS = [
 "table_name",
 "group_name",
 "column_name",
 "host_variable",
 "level",
 "pic_clause",
 "usage",
 "raw_line",
]
```

```
DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE = {
 "IND",
 "NULL",
 "NULLIND",
 "INDICATOR",
}
```

```
COBOL_LEVEL_NUMBERS = {
 "01",
 "02",
 "03",
 "04",
 "05",
 "06",
 "07",
 "08",
 "09",
 "10",
 "11",
 "12",
 "13",
 "14",
 "15",
 "16",
 "17",
 "18",
 "19",
 "20",
 "21",
 "22",
 "23",
 "24",
 "25",
 "26",
 "27",
 "28",
 "29",
 "30",
 "31",
 "32",
 "33",
 "34",
 "35",
 "36",
 "37",
 "38",
 "39",
}
```

```

"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"77",
"88",
}

```

```

=====
FILE: src\idms_db2_phase2\reference\catalogs\mapping\sheet_mapping_schema.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING = "IDMS to DB2 Mapping"
SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS = "Cobol Record IDMS"
SHEET_MAPPING_COLUMN_COBOL_ZONE = "Cobol Zone"
SHEET_MAPPING_COLUMN_IDMS_KEY = "IDMS Key"
SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE = "IDMS PIC Clause"
SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES = "Length of Field Bytes"
SHEET_MAPPING_COLUMN_FIELD_END_POSITION = "Field end position"
SHEET_MAPPING_COLUMN_DB2_KEY = "DB2 Key"
SHEET_MAPPING_COLUMN_NEW_DB2_RECORD = "New DB2 Record"
SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME = "New DB2 Field name"
SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE = "New DB2 Data Type"
SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK = "Hopex Expression TypeRemark"
SHEET_MAPPING_COLUMN_REMARKS = "Remarks"
SHEET_MAPPING_COLUMN_RELATION = "Relation"
SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK = "Reference Field Name (Copybook)"
SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE = "Reference Field PIC Clause"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE = "Cross Application DB2 table"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME = (
 "Cross Application DB2 Field Name"
)
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE = (
 "Cross Appln DB2 Data Type"
)
SHEET_MAPPING_COLUMN_BASETYPE = "Basetype"

```

```

SHEET_MAPPING_CANONICAL_COLUMNS = [
 SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING,
 SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
 SHEET_MAPPING_COLUMN_COBOL_ZONE,
 SHEET_MAPPING_COLUMN_IDMS_KEY,
 SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE,
 SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
 SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
 SHEET_MAPPING_COLUMN_DB2_KEY,
 SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
 SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
 SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
 SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK,
 SHEET_MAPPING_COLUMN_REMARKS,
 SHEET_MAPPING_COLUMN_RELATION,
 SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK,
 SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE,
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME,
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE,
 SHEET_MAPPING_COLUMN_BASETYPE,
]

```

```

SHEET_MAPPING_MODEL_FIELD_MAP = {
 "idms_to_db2_mapping": SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING,
 "cobol_record_idms": SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
 "cobol_zone": SHEET_MAPPING_COLUMN_COBOL_ZONE,
 "idms_key": SHEET_MAPPING_COLUMN_IDMS_KEY,
}

```

```

"idsm_pic_clause": SHEET_MAPPING_COLUMN_IDS_PIC_CLAUSE,
"length_of_field_bytes": SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
"field_end_position": SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
"db2_key": SHEET_MAPPING_COLUMN_DB2_KEY,
"new_db2_record": SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
"new_db2_field_name": SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
"new_db2_data_type": SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
"hopex_expression_type_remark": (
 SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK
),
"remarks": SHEET_MAPPING_COLUMN_REMARKS,
"relation": SHEET_MAPPING_COLUMN_RELATION,
"reference_field_name_copybook": (
 SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK
),
"reference_field_pic_clause": SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
"cross_application_db2_table": (
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE
),
"cross_application_db2_field_name": (
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME
),
"cross_application_db2_data_type": (
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE
),
"basetype": SHEET_MAPPING_COLUMN_BASETYPE,
}

```

```

SHEET_MAPPING_FIELD_ALIASES = {
 SHEET_MAPPING_COLUMN_IDS_TO_DB2_MAPPING: [
 "IDMS to DB2 Mapping",
 "IDMS DB2 Mapping",
 "IDMS_DB2_MAPPING",
],
 SHEET_MAPPING_COLUMN_COBOL_RECORD_IDS: [
 "Cobol Record IDMS",
 "COBOL RECORD IDMS",
 "Cobol Recrd IDMS",
 "COBOL RECRD IDMS",
 "Cobol Rec IDMS",
 "COBOL REC IDMS",
 "Cobol Rord IDMS",
 "COBOL RCRD IDMS",
 "Cobol Recrd",
 "COBOL RECRD",
 "IDMS Record",
 "IDMS_RECORD",
],
 SHEET_MAPPING_COLUMN_COBOL_ZONE: [
 "Cobol Zone",
 "COBOL Zone",
 "COBOL ZONE",
 "Cobol Field",
 "COBOL Field",
 "IDMS Field",
 "IDMS COBOL Zone",
 "IDMS COBOL Field",
 "Zone",
],
 SHEET_MAPPING_COLUMN_IDS_KEY: [
 "IDMS Key",
 "IDMS KEY",
 "Key IDMS",
 "IDMS_Key",
],
 SHEET_MAPPING_COLUMN_IDS_PIC_CLAUSE: [
 "IDMS PIC Clause",
 "IDMS PIC",
 "PIC Clause",
 "Picture",
 "PIC",
],
 SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES: [
 "Length of Field Bytes",
 "Length",
 "Field Length",
],
}

```

```

 "Length Bytes",
 "Length of Field",
],
 SHEET_MAPPING_COLUMN_FIELD_END_POSITION: [
 "Field end position",
 "Field End Position",
 "End Position",
 "Field End",
],
 SHEET_MAPPING_COLUMN_DB2_KEY: [
 "DB2 Key",
 "DB2 key",
 "DB2 KEY",
 "Key DB2",
 "DB2_key",
 "DB2_Key",
 "DB2KEY",
],
 SHEET_MAPPING_COLUMN_NEW_DB2_RECORD: [
 "New DB2 Record",
 "New DB2 Record ",
 "DB2 Record",
 "DB2 Table",
 "New DB2 Table",
 "Table",
 "DB2_Table",
 "New DB2_Record",
],
 SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME: [
 "New DB2 Field name",
 "New DB2 Field Name",
 "New DB2_Field name",
 "New DB2_Field Name",
 "New DB2 Field",
 "New DB2_Field",
 "DB2 Field",
 "DB2 Column",
 "New DB2 Column",
],
 SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE: [
 "New DB2 Data Type",
 "New DB2 DataType",
 "DB2 Data Type",
 "DB2 Datatype",
 "DB2 Type",
],
 SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK: [
 "Hopex Expression TypeRemark",
 "Hopex Expression Type Remark",
 "Expression Type Remark",
],
 SHEET_MAPPING_COLUMN_REMARKS: [
 "Remarks",
 "Remark",
 "Comments",
],
 SHEET_MAPPING_COLUMN_RELATION: [
 "Relation",
 "Relationship",
 "Set Relation",
],
 SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK: [
 "Reference Field Name (Copybook)",
 "Reference Field Name Copybook",
 "Reference Field Name",
 "Copybook Field Name",
],
 SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE: [
 "Reference Field PIC Clause",
 "Reference PIC Clause",
 "Copybook PIC Clause",
],
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE: [
 "Cross Application DB2 table",
 "Cross Application DB2 Table",
 "Cross App DB2 Table",
]

```

```

],
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME: [
 "Cross Application DB2 Field Name",
 "Cross App DB2 Field Name",
 "Cross Application DB2 Column",
],
 SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE: [
 "Cross Appln DB2 Data Type",
 "Cross Application DB2 Data Type",
],
 SHEET_MAPPING_COLUMN_BASETYPE: [
 "Basetype",
 "Base Type",
 "BASE_TYPE",
],
}

```

```

SHEET_MAPPING_HEADER_DETECTION_GROUPS = [
 [
 SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
 "Cobol Recrd IDMS",
 "IDMS Record",
],
 [
 SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
 "DB2 Table",
 "DB2 Record",
],
 [
 SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
 "New DB2_Field name",
 "DB2 Column",
],
]

```

```

=====
FILE: src\idms_db2_phase2\reference\catalogs\output__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\reference\catalogs\output\output_sections.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

DB2_INFRASTRUCTURE_MARKER = (
 "DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS"
)

```

```

DB2_SQL_ERROR_LOCATION_MARKER = "DB2 SQL ERROR LOCATION"
DB2_CURSOR_FLAGS_MARKER = "DB2 CURSOR END-OF-CURSOR FLAGS"
DB2_CURSOR_DECLARATIONS_MARKER = "DB2 CURSOR DECLARATIONS"
DB2_CURSOR_PARAGRAPH_MARKER = "DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS"

```

```

SQLCA_INCLUDE_NAME = "SQLCA"
SQLERRWS_INCLUDE_NAME = "SQLERRWS"

```

```

SQL_LOCATION_FIELD_NAME = "SQL-LOCATION"
SQL_LOCATION_PICTURE = "PIC X(40) VALUE SPACES."

```

```

DEFAULT_CURSOR_EOC_VALUE = "Y"
DEFAULT_CURSOR_NOT_EOC_VALUE = "N"

```

```
=====
FILE: src\idms_db2_phase2\reference\patterns__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\patterns\cobol__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\patterns\cobol\copybook_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import re
```

```
COPYBOOK_FIELD_PATTERN = re.compile(
 r"^\s*(?P<level>0[1-9]|[1-4][0-9]|77|88)\s+"
 r"(?P<name>[A-Z][A-Z0-9-]*)"
 r"(?:\s+(?P<body>.*))?\.\?\s*$",
 flags=re.IGNORECASE,
)
```

```
COPYBOOK_PIC_PATTERN = re.compile(
 r"\bPIC\s+(?P<pic>[A-Z0-9S().+\\-V]+)",
 flags=re.IGNORECASE,
)
```

```
COPYBOOK_USAGE_PATTERN = re.compile(
 r"\b(?:P<usage>COMP-3|COMP-5|COMP|BINARY|DISPLAY)\b",
 flags=re.IGNORECASE,
)
```

```
COPYBOOK_COMMENT_OR_SKIP_PATTERN = re.compile(
 r"^\s*(?:\s*/|EJECT|SKIP|TITLE)\b",
 flags=re.IGNORECASE,
)
```

```
=====
FILE: src\idms_db2_phase2\reference\patterns\cobol\cursor_flow_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import re
```

```
PERFORM_CURSOR_PATTERN = re.compile(
 r"^\s*PERFORM\s+"
 r"(?P<number>\d{3})-"
 r"(?P<operation>OPEN|FETCH|CLOSE)-"
 r"(?P<cursor>[A-Z0-9-]+)"
 r"\.\?\s*$",
 flags=re.IGNORECASE,
)
```

```
PERFORM_BUSINESS_PARAGRAPH_PATTERN = re.compile(
 r"^\s*PERFORM\s+"
 r"(?P<paragraph>[A-Z0-9][A-Z0-9-]*)"
 r"\.\?\s*$",
 flags=re.IGNORECASE,
)
```

```
UNTIL_SQLCODE_100_PATTERN = re.compile(
```

```

 r"^\\s*UNTIL\\s+SQLCODE\\s*=\\s*100\\.?\\s*$",
 flags=re.IGNORECASE,
)

CURSOR_PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^\\s*"
 r"(?P<number>\\d{3})-"
 r"(?P<operation>OPEN|FETCH|CLOSE)-"
 r"(?P<cursor>[A-Z0-9-]+)"
 r"\\.\\s*$",
 flags=re.IGNORECASE,
)

ANY_PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^\\s*[A-Z0-9][A-Z0-9-]*\\.\\s*$",
 flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
 r"^\\s*WHEN\\s+ZERO\\s*$",
 flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
 r"^\\s*CONTINUE\\.?\\s*$",
 flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN = re.compile(
 r"^\\s**DB2:\\s*Converted\\s+OBTAIN\\s+NEXT\\b",
 flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_COMMENT_PATTERN = re.compile(
 r"^\\s**DB2:\\s*Converted\\s+OBTAIN\\b",
 flags=re.IGNORECASE,
)

=====
FILE: src\\idms_db2_phase2\\reference\\patterns\\cobol\\field_reference_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

QUALIFIED_REFERENCE_PATTERN = re.compile(
 r"\\b(?:P<field>[A-Z][A-Z0-9-]*)\\s+(?:OF|IN)\\s+"
 r"(?P<record>[A-Z][A-Z0-9-]*)\\b",
 flags=re.IGNORECASE,
)

DCLGEN_OF_REFERENCE_PATTERN = re.compile(
 r":?\\s*(?P<field>[A-Z][A-Z0-9-]*)\\s+OF\\s+"
 r"(?P<group>DCL[A-Z0-9-]+)",
 flags=re.IGNORECASE,
)

DCLGEN_DOT_REFERENCE_PATTERN = re.compile(
 r":?\\s*(?P<group>DCL[A-Z0-9-]+)\\.?(?P<field>[A-Z][A-Z0-9-]*)",
 flags=re.IGNORECASE,
)

MOVE_STATEMENT_PATTERN = re.compile(
 r"^\\s*MOVE\\s+(?P<source>.+?)\\s+TO\\s+(?P<target>.+?)(?:\\.|\\s*)$",
 flags=re.IGNORECASE,
)

CONDITION_STATEMENT_PATTERN = re.compile(
 r"^\\s*(IF|WHEN|UNTIL|EVALUATE)\\b",
 flags=re.IGNORECASE,
)

```

```

OUTPUT_STATEMENT_PATTERN = re.compile(
 r"^\s*(DISPLAY|WRITE|REWRITE|RETURN|GOBACK|EXIT|CALL)\b",
 flags=re.IGNORECASE,
)

COMMENT_OR_BLANK_PATTERN = re.compile(
 r"^\s*(?:$|\s*)",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\cobol\pic_clause_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

WORKING_STORAGE_PIC_PATTERN = re.compile(
 r"^(?P<prefix>\s*(?:0[1-9]|[1-4][0-9]|77)\s+"
 r"(?P<name>[A-Z][A-Z0-9-]*)\s+PIC\s+"
 r"(?P<pic>[A-Z0-9S().+\-V]+)"
 r"(?P<suffix>.*$)",
 flags=re.IGNORECASE,
)

MOVE_STATEMENT_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<source>[A-Z][A-Z0-9-]*)\s+TO\s+"
 r"(?P<target>[A-Z][A-Z0-9-]*)\.\s*$",
 flags=re.IGNORECASE,
)

PIC_LENGTH_PATTERN = re.compile(
 r"(?P<type>[X9])(?:$(?P<length>\d+)$(?P<single>\d*))",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\cobol\sequence_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

LEFT_SEQUENCE_PATTERN = re.compile(
 r"^\d{6}",
)

RIGHT_SEQUENCE_PATTERN = re.compile(
 r"\d{8}$",
)

FIXED_FORMAT_LINE_PATTERN = re.compile(
 r"^(?P<left>\d{6})(?P<indicator>.)?(?P<body>.{0,65})(?P<right>\d{8})?$",
)

SEQUENCE_ONLY_PATTERN = re.compile(
 r"^\d{6}\s*\d{8}?$",
)

```

```

def strip_sequence_numbers(
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if len(text) >= 7 and text[:6].isdigit():
 text = text[6:]

 if len(text) >= 8 and text[-8:].isdigit():
 text = text[:-8]

 return text.rstrip()

=====
FILE: src\idms_db2_phase2\reference\patterns\cobol\structure_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

DIVISION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?"
 r"(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)"
 r"s+DIVISION\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)

IDENTIFICATION_DIVISION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?IDENTIFICATION\s+DIVISION\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

ENVIRONMENT_DIVISION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?ENVIRONMENT\s+DIVISION\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

DATA_DIVISION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?DATA\s+DIVISION\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

WORKING_STORAGE_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

LINKAGE_SECTION_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

PROGRAM_ID_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?PROGRAM-ID\.\s*([A-Z0-9-]+\.)\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

END_PROGRAM_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?END\s+PROGRAM\b.*\.?s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

STOP_RUN_PATTERN = re.compile(
 r"^s*(?:\d{6}\s+)?STOP\s+RUN\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)

```

)

```
=====
FILE: src\idms_db2_phase2\reference\patterns\db2__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\patterns\db2\date_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
DB2_DATE_COMPARISON_PATTERN = re.compile(
 r"^(?P<indent>\s*)IF\s+"
 r"(?P<field>(?:DA|DT)-[A-Z0-9-]+)"
 r"\s+OF\s+"
 r"(?P<group>DCL[A-Z0-9-]+)"
 r"\s+"
 r"(?P<condition>.\bPARMDATE\b.*)$",
 flags=re.IGNORECASE,
)
```

```
DATE_WORKING_STORAGE_MARKER_PATTERN = re.compile(
 r"DB2 DATE(?:COMPARISON|CONVERSION) WORKING STORAGE",
 flags=re.IGNORECASE,
)
```

```
DB2_SHARED_DATE_HELPER_USAGE_PATTERN = re.compile(
 r"\b(?:DA-CCYYMMDD-R|DA-CCYYMMDD|DA-DD-MM-CCYY)\b",
 flags=re.IGNORECASE,
)
```

```
DB2_DATE_WORKING_STORAGE_BASE_PATTERN = re.compile(
 r"\b(?:01\s+WS-DATUMVELDEN|DA-CCYYMMDD|DA-DD-MM-CCYY)\b",
 flags=re.IGNORECASE,
)
```

```
DATE_HELPER_FIELD_PATTERN = re.compile(
 r"\bHELP-(?:DA|DT)-[A-Z0-9-]+\b",
 flags=re.IGNORECASE,
)
```

```
WORKING_STORAGE_SECTION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)
```

```
=====
FILE: src\idms_db2_phase2\reference\patterns\db2\dclgen_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```

DECLARE_TABLE_PATTERN = re.compile(
 r"EXEC\s+SQL\s+DECLARE\s+([A-Z0-9_.$@]+)\s+TABLE",
 flags=re.IGNORECASE,
)

DECLARE_TABLE_ALT_PATTERN = re.compile(
 r"\bDECLARE\s+([A-Z0-9_.$@]+)\s+TABLE\b",
 flags=re.IGNORECASE,
)

DCLGEN_TABLE_COMMENT_PATTERN = re.compile(
 r"\bDCLGEN\s+TABLE\s*[:=]? \s*([A-Z0-9_.$@]+)\s*$",
 flags=re.IGNORECASE,
)

COBOL_GROUP_PATTERN = re.compile(
 r"^\s*(?P<level>0[2-9]|[1-4][0-9]|77)\s+"
 r"(?P<host>[A-Z][A-Z0-9-]*)\s+"
 r"(?P<body>.*)$",
 flags=re.IGNORECASE,
)

COBOL_FIELD_PATTERN = re.compile(
 r"^\s*(?P<level>0[2-9]|[1-4][0-9]|77)\s+"
 r"(?P<host>[A-Z][A-Z0-9-]*)\s+"
 r"(?P<body>.*)$",
 flags=re.IGNORECASE,
)

DCLGEN_PIC_PATTERN = re.compile(
 r"\bPIC\s+(?P<pic>[A-Z0-9S().+ \-V]+)",
 flags=re.IGNORECASE,
)

DCLGEN_USAGE_PATTERN = re.compile(
 r"\b(?P<usage>COMP-3|COMP-5|COMP|BINARY|DISPLAY)\b",
 flags=re.IGNORECASE,
)

VALID_SQL_COLUMN_NAME_PATTERN = re.compile(
 r"^[A-Z][A-Z0-9_]*$",
 flags=re.IGNORECASE,
)

SQL_NAME_CLEANUP_PATTERN = re.compile(
 r"^[^A-Z0-9_]+",
 flags=re.IGNORECASE,
)

MULTIPLE_UNDERSCORE_PATTERN = re.compile(
 r"_+",
)

```

```

=====
FILE: src\idms_db2_phase2\reference\patterns\db2\infrastructure_patterns.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

import re

```

```

SQLCA_TOKEN_PATTERN = re.compile(
 r"\bSQLCA\b",
 flags=re.IGNORECASE,
)

```

```

SQLERRWS_TOKEN_PATTERN = re.compile(
 r"\bSQLERRWS\b",
 flags=re.IGNORECASE,
)

```

```

SQL_LOCATION_PATTERN = re.compile(
 r"\bSQL-LOCATION\b",
 flags=re.IGNORECASE,
)

```

```

)

SQL_ERROR_PARAGRAPH_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?(SQL-ERROR|SQLERROR)\s*\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

SQL_ERROR_PERFORM_PATTERN = re.compile(
 r"\bPERFORM\s+(SQL-ERROR|SQLERROR)\s*\.\s*",
 flags=re.IGNORECASE,
)

DB2_CURSOR_BLOCK_MARKER_PATTERN = re.compile(
 r"DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS",
 flags=re.IGNORECASE,
)

DB2_INFRASTRUCTURE_MARKER_PATTERN = re.compile(
 r"DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",
 flags=re.IGNORECASE,
)

DB2_CURSOR_DECLARATIONS_MARKER_PATTERN = re.compile(
 r"DB2 GENERATED CURSOR DECLARATIONS",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\db2\sql_error_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

SQLERROR_PARAGRAPH_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?(?:SQLERROR|SQL-ERROR)\s*\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

SQLERROR_PERFORM_PATTERN = re.compile(
 r"\bPERFORM\s+(?:SQLERROR|SQL-ERROR)\s*\.\s*",
 flags=re.IGNORECASE,
)

SQLERROR_INCLUDE_PATTERN = re.compile(
 r"\bINCLUDE\s+SQLERROR\b",
 flags=re.IGNORECASE,
)

CUSTOM_SQLERROR_DISPLAY_PATTERN = re.compile(
 r"DISPLAY\s+['\"]DB2\s+SQL\s+ERROR",
 flags=re.IGNORECASE,
)

END_PROGRAM_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?END\s+PROGRAM\b.*\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\db2\sql_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

EXEC_SQL_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",

```

```

 flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
 r"^\s*END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

UPDATE_SQL_PATTERN = re.compile(
 r"\bUPDATE\s+(?P<table>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

INSERT_SQL_PATTERN = re.compile(
 r"\bINSERT\s+INTO\s+(?P<table>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

DELETE_SQL_PATTERN = re.compile(
 r"\bDELETE\s+FROM\s+(?P<table>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

SELECT_SQL_PATTERN = re.compile(
 r"\bSELECT\b",
 flags=re.IGNORECASE,
)

HOST_REFERENCE_PATTERN = re.compile(
 r":(?P<host>[A-Z][A-Z0-9-]*(?:\s+OF\s+DCL[A-Z0-9-]+)?)",
 flags=re.IGNORECASE,
)

OPEN_CURSOR_PATTERN = re.compile(
 r"\bOPEN\s+(?P<cursor_name>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

FETCH_CURSOR_PATTERN = re.compile(
 r"\bFETCH\s+(?P<cursor_name>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

CLOSE_CURSOR_PATTERN = re.compile(
 r"\bCLOSE\s+(?P<cursor_name>[A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

SQLCODE_PATTERN = re.compile(
 r"\bSQLCODE\b",
 flags=re.IGNORECASE,
)

SQLCODE_SUCCESS_PATTERN = re.compile(
 r"\bSQLCODE\s*=\s*0\b",
 flags=re.IGNORECASE,
)

SQLCODE_NOT_FOUND_PATTERN = re.compile(
 r"\bSQLCODE\s*=\s*100\b",
 flags=re.IGNORECASE,
)

SQLCODE_ERROR_PATTERN = re.compile(
 r"\bSQLCODE\s*(?:NOT\s*=\s*0|<\s*0|>\s*0)\b",
 flags=re.IGNORECASE,
)

COMMIT_PATTERN = re.compile(
 r"\bCOMMIT\b",
 flags=re.IGNORECASE,
)

ROLLBACK_PATTERN = re.compile(
 r"\bROLLBACK\b",
 flags=re.IGNORECASE,
)

```

)

```
=====
FILE: src\idms_db2_phase2\reference\patterns\idms__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\patterns\idms\declarative_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
IDMS_DECLARATIVE_PATTERNS = [
 re.compile(
 r"^IDMS-CONTROL\S*\s+SECTION\?.?$",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^PROTOCOL\b.*$",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^IDMS-RECORDS\S+WITHIN\S+WORKING-STORAGE\S+SECTION\?.?$",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^SCHEMA\S*\s+SECTION\?.?$",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^DB\S*[A-Z0-9-]+\s+WITHIN\S+[A-Z0-9-]+\?.?$",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^DB[A-Z0-9-]+\s+WITHIN\S+[A-Z0-9-]+\b",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^COPY\S+IDMS\b",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^COPY\S+IDMS\S+IDMS-STATUS\b",
 flags=re.IGNORECASE,
),
 re.compile(
 r"^COPY\S*IDMS\S+SUBSCHEMA-BINDS\b",
 flags=re.IGNORECASE,
),
]
```

```
=====
FILE: src\idms_db2_phase2\reference\patterns\idms\statement_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
OBTAIN_CALC_PATTERN = re.compile(
 r"\bOBTAIN\S+(?:KEEP\S+)?(?:P<record>[A-Z0-9-]+\s+CALC\b",
 flags=re.IGNORECASE,
)
```

```

OBTAIN_CALC_REVERSED_PATTERN = re.compile(
 r"\bOBTAIN\s+(?:KEEP\s+)?CALC\s+(?P<record>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

OBTAIN_FIRST_NEXT_PATTERN = re.compile(
 r"\bOBTAIN\s+(?:KEEP\s+)?(?P<mode>FIRST|NEXT)\s+"
 r"(?P<record>[A-Z0-9-]+\s+)\s+WITHIN\s+(?P<set>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

OBTAIN_OWNER_PATTERN = re.compile(
 r"\bOBTAIN\s+OWNER\s+WITHIN\s+(?P<set>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

FIND_FIRST_PATTERN = re.compile(
 r"\bFIND\s+FIRST\s+(?P<record>[A-Z0-9-]+\s+)\s*\s*"
 r"WITHIN\s+(?P<set>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

FIND_CURRENT_PATTERN = re.compile(
 r"\bFIND\s+CURRENT\s+(?P<record>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

STORE_PATTERN = re.compile(
 r"\bSTORE\s+(?P<record>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

MODIFY_PATTERN = re.compile(
 r"\bMODIFY\s+(?P<record>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

ERASE_PATTERN = re.compile(
 r"\bERASE\s+(?P<record>[A-Z0-9-]+\s+)\b",
 flags=re.IGNORECASE,
)

READY_PATTERN = re.compile(
 r"^\s*READY\b",
 flags=re.IGNORECASE,
)

READY_UPDATE_PATTERN = re.compile(
 r"^\s*READY\b.*\bUPDATE\b",
 flags=re.IGNORECASE,
)

BIND_STATEMENT_PATTERN = re.compile(
 r"^\s*BIND\b",
 flags=re.IGNORECASE,
)

USAGE_MODE_PATTERN = re.compile(
 r"^\s*USAGE\s+MODE\b",
 flags=re.IGNORECASE,
)

CONNECT_STATEMENT_PATTERN = re.compile(
 r"^\s*CONNECT\b",
 flags=re.IGNORECASE,
)

DISCONNECT_STATEMENT_PATTERN = re.compile(
 r"^\s*DISCONNECT\b",
 flags=re.IGNORECASE,
)

COMMIT_PATTERN = re.compile(
 r"^\s*COMMIT\b",
 flags=re.IGNORECASE,
)

```

```

FINISH_PATTERN = re.compile(
 r"^\s*FINISH\b",
 flags=re.IGNORECASE,
)

ON_DB_REC_NOT_FOUND_PATTERN = re.compile(
 r"\bON\s+DB-REC-NOT-FOUND\b",
 flags=re.IGNORECASE,
)

DB_REC_NOT_FOUND_TOKEN_PATTERN = re.compile(
 r"\bDB-REC-NOT-FOUND\b",
 flags=re.IGNORECASE,
)

DB_END_OF_SET_TOKEN_PATTERN = re.compile(
 r"\bDB-END-OF-SET\b",
 flags=re.IGNORECASE,
)

IDMS_ABORT_PERFORM_PATTERN = re.compile(
 r"\bPERFORM\s+IDMS-ABORT\b",
 flags=re.IGNORECASE,
)

IDMS_STATUS_PERFORM_PATTERN = re.compile(
 r"\bPERFORM\s+IDMS-STATUS\b",
 flags=re.IGNORECASE,
)

IDMS_DECLARATIVE_OR_CONTROL_PATTERNS = [
 READY_PATTERN,
 BIND_STATEMENT_PATTERN,
 USAGE_MODE_PATTERN,
]

=====
FILE: src\idms_db2_phase2\reference\patterns\mapping\sheet_mapping_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

CELL_WHITESPACE_PATTERN = re.compile(
 r"\s+",
)

HEADER_WHITESPACE_PATTERN = re.compile(
 r"\s+",
)

HEADER_NON_ALPHANUMERIC_PATTERN = re.compile(
 r"^[A-Z0-9]+",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\final_cobol_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

```

```

import re

STRING_START_PATTERN = re.compile(
 r"^STRING\b",
 flags=re.IGNORECASE,
)

STRING_END_PATTERN = re.compile(
 r"^END-STRING\.$",
 flags=re.IGNORECASE,
)

STRING_INTRO_PATTERN = re.compile(
 r"^INTO\b",
 flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_TOKEN_PATTERN = re.compile(
 r"^PROCEDURE\s+DIVISION\b",
 flags=re.IGNORECASE,
)

DIVISION_SECTION_HEADER_PATTERN = re.compile(
 r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b|"
 r"^[A-Z0-9-]+\s+SECTION\.$",
 flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^[A-Z0-9][A-Z0-9-]*\.\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
 r"^END-EXEC\.$",
 flags=re.IGNORECASE,
)

MOVE_TO_PROGRAM_NAME_PATTERN = re.compile(
 r"(?P<prefix>\bMOVE\s+['\"])[A-Z0-9-]+(?P<suffix>['\"])\s+TO\s+PROGRAM-NAME\.$",
 flags=re.IGNORECASE,
)

PROGRAM_ID_PATTERN = re.compile(
 r"PROGRAM-ID\.\s*(?P<program>[A-Z0-9-]+\.\.$",
 flags=re.IGNORECASE,
)

DECLARE_CURSOR_PATTERN = re.compile(
 r"\bDECLARE\s+(?P<cursor>[A-Z0-9-]+\s+Cursors\b",
 flags=re.IGNORECASE,
)

FETCH_CURSOR_PATTERN = re.compile(
 r"\bFETCH\s+(?P<cursor>[A-Z0-9-]+\b",
 flags=re.IGNORECASE,
)

SELECT_KEYWORD_PATTERN = re.compile(
 r"^SELECT$",
 flags=re.IGNORECASE,
)

FROM_PATTERN = re.compile(
 r"^FROM\b",
 flags=re.IGNORECASE,
)

ORDER_BY_PATTERN = re.compile(

```

```

 r"^ORDER\s+BY\b",
 flags=re.IGNORECASE,
)

FOR_READ_ONLY_PATTERN = re.compile(
 r"^FOR\s+READ\s+ONLY\b",
 flags=re.IGNORECASE,
)

HOST_REFERENCE_PATTERN = re.compile(
 r":\s*(?P<host>[A-Z0-9-]+(?:\s+OF\s+DCL[A-Z0-9-]+|(?:\s*\.\s*[A-Z0-9-]+)?))",
 flags=re.IGNORECASE,
)

SELECT_ITEM_PATTERN = re.compile(
 r"^\s*,?\s*(?P<column>[A-Z][A-Z0-9_]+)(?:\s*,)?\s*$",
 flags=re.IGNORECASE,
)

SQL_NAME_TOKEN_PATTERN = re.compile(
 r"\b[A-Z][A-Z0-9_]+\b",
 flags=re.IGNORECASE,
)

ASC_DESC_PATTERN = re.compile(
 r"\b(?:ASC|DESC)\b",
 flags=re.IGNORECASE,
)

DB2_COLON_COMMENT_BODY_PATTERN = re.compile(
 r"^\s*DB2\s*:\s*",
 flags=re.IGNORECASE,
)

DB2_SPACE_COMMENT_BODY_PATTERN = re.compile(
 r"^\s*DB2\s+",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\include_placement_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

INCLUDE_INLINE_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\s+INCLUDE\s+(?P<include>[A-Z0-9_]+)\s+END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

INCLUDE_BODY_PATTERN = re.compile(
 r"^\s*INCLUDE\s+(?P<include>[A-Z0-9_]+)\b",
 flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\s*PROCEDURE\s+DIVISION\b",
 flags=re.IGNORECASE,
)

WORKING_STORAGE_PATTERN = re.compile(
 r"^\s*WORKING-STORAGE\s+SECTION\.\s*$",

```

```

 flags=re.IGNORECASE,
)

LINKAGE_SECTION_PATTERN = re.compile(
 r"^\\s*LINKAGE\\s+SECTION\\.?\\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\\idms_db2_phase2\\reference\\patterns\\normalization\\procedure_indentation_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\\s*PROCEDURE\\s+DIVISION\\b",
 flags=re.IGNORECASE,
)

DIVISION_HEADER_PATTERN = re.compile(
 r"^\\s*[A-Z][A-Z0-9-]*\\s+DIVISION\\.?\\s*$",
 flags=re.IGNORECASE,
)

SECTION_HEADER_PATTERN = re.compile(
 r"^\\s*[A-Z0-9][A-Z0-9-]*\\s+SECTION\\.?\\s*$",
 flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^\\s*[A-Z0-9][A-Z0-9-]*\\.?\\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^\\s*EXEC\\s+SQL\\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\\s*END-EXEC\\.?\\s*$",
 flags=re.IGNORECASE,
)

IF_PATTERN = re.compile(
 r"^\\s*IF\\b",
 flags=re.IGNORECASE,
)

ELSE_PATTERN = re.compile(
 r"^\\s*ELSE\\b",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^\\s*END-IF\\.?\\s*$",
 flags=re.IGNORECASE,
)

EVALUATE_PATTERN = re.compile(
 r"^\\s*EVALUATE\\b",
 flags=re.IGNORECASE,
)

WHEN_PATTERN = re.compile(
 r"^\\s*WHEN\\b",
 flags=re.IGNORECASE,
)

END_EVALUATE_PATTERN = re.compile(
 r"^\\s*END-EVALUATE\\.?\\s*$",
 flags=re.IGNORECASE,
)

```

```

)

COMMENT_PATTERN = re.compile(
 r"^\s**",
 flags=re.IGNORECASE,
)

BLANK_PATTERN = re.compile(
 r"^\s*$",
 flags=re.IGNORECASE,
)

END_PROGRAM_PATTERN = re.compile(
 r"^\s*END\s+PROGRAM\b",
 flags=re.IGNORECASE,
)

SINGLE_WORD_EXECUTABLE_PATTERN = re.compile(
 r"^\s*(CONTINUE|COMMIT|ROLLBACK|STOP\s+RUN)\.?\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\retrieval_program_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

EXEC_SQL_INCLUDE_PATTERN = re.compile(
 r"\bINCLUDE\s+(?P<include>[A-Z0-9]+\)\b",
 flags=re.IGNORECASE,
)

DCL_GROUP_DOT_REFERENCE_PATTERN = re.compile(
 r"\b(?P<group>DCL[A-Z0-9]+\)\. (?P<host>[A-Z][A-Z0-9-]*)\b",
 flags=re.IGNORECASE,
)

DCL_GROUP_OF_REFERENCE_PATTERN = re.compile(
 r"\b(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+(?P<group>DCL[A-Z0-9]+\)\b",
 flags=re.IGNORECASE,
)

WORKING_STORAGE_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s*)?WORKING-STORAGE\s+SECTION\.",
 flags=re.IGNORECASE,
)

LINKAGE_SECTION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s*)?LINKAGE\s+SECTION\.",
 flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s*)?PROCEDURE\s+DIVISION\b",
 flags=re.IGNORECASE,
)

ERROR_STATUS_MOVE_PATTERN = re.compile(
 r"^\s*MOVE\s+['\"]?[A-Z0-9]+['\"]?\s+TO\s+ERROR-STATUS\.?\s*$",
 flags=re.IGNORECASE,
)

SW_STATUS_D_PIC_PATTERN = re.compile(
 r"\bSW-STATUS-D\b.+ \bPIC\b",
 flags=re.IGNORECASE,
)

WRITE_PATTERN = re.compile(
 r"^\s*WRITE\s+(?P<record>[A-Z0-9-]+\)\.?\s*$",
 flags=re.IGNORECASE,
)

```

```

)

INITIALIZE_PATTERN = re.compile(
 r"^s*INITIALIZE\s+(?P<record>[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\retrieval_statement_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

WORKING_STORAGE_PATTERN = re.compile(
 r"^s*WORKING-STORAGE\s+SECTION\.?\s*$",
 flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^s*PROCEDURE\s+DIVISION\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_INCLUDE_PATTERN = re.compile(
 r"^s*EXEC\s+SQL\s+INCLUDE\s+(?P<table>[A-Z0-9_]+)\s+END-EXEC\.?\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_PATTERN = re.compile(
 r"^s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

DCL_GROUP_OF_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<field>[A-Z0-9-]+)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

DCL_GROUP_DOT_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<group>DCL[A-Z0-9-]+)\.(?P<field>[A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

ERROR_STATUS_MOVE_PATTERN = re.compile(
 r"^s*MOVE\s+ERROR-STATUS\s+TO\s+(?P<target>[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

SW_STATUS_D_PIC_PATTERN = re.compile(
 r"^s*(?P<level>01|05|77)\s+(?P<name>SW-[A-Z0-9-]*STATUS[A-Z0-9-]*)\s+PIC\s+X",
 flags=re.IGNORECASE,
)

INITIALIZE_PATTERN = re.compile(
 r"^s*INITIALIZE\s+(?P<target>[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

WRITE_PATTERN = re.compile(
 r"^s*WRITE\s+(?P<target>[A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

MOVE_TO_OUTPUT_PATTERN = re.compile(
 r"^s*MOVE\s+.\s+\s+TO\s+(?P<target>[A-Z0-9-]+)(?:\s+OF\s+[A-Z0-9-]+)?\s*\s*\s*",
 flags=re.IGNORECASE,
)

COMMENT_PATTERN = re.compile(
 r"^s*\s*",
 flags=re.IGNORECASE,
)

```

```

BLANK_PATTERN = re.compile(
 r"^\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\sqlcode_guard_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\?\s*$",
 flags=re.IGNORECASE,
)

IF_SQLCODE_NOT_ZERO_PATTERN = re.compile(
 r"^\s*IF\s+SQLCODE\s+NOT\s*=\s*0\b",
 flags=re.IGNORECASE,
)

IF_SQLCODE_EQUALS_NOT_FOUND_PATTERN = re.compile(
 r"^\s*IF\s+SQLCODE\s*=\s*100\b",
 flags=re.IGNORECASE,
)

IF_NOT_SQLCODE_NOT_FOUND_PATTERN = re.compile(
 r"^\s*IF\s+NOT\s+SQLCODE\s*=\s*100\b",
 flags=re.IGNORECASE,
)

AND_SQLCODE_NOT_FOUND_PATTERN = re.compile(
 r"^\s*AND\s+SQLCODE\s+NOT\s*=\s*100\b",
 flags=re.IGNORECASE,
)

PERFORM_SQLERROR_PATTERN = re.compile(
 r"^\s*PERFORM\s+(?:SQLERROR|SQLERREND-IF)\.\?\s*$",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^\s*END-IF\.\?\s*$",
 flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
 r"^\s*CONTINUE\.\?\s*$",
 flags=re.IGNORECASE,
)

DB2_COMMENT_PATTERN = re.compile(
 r"^\s**DB2:",
 flags=re.IGNORECASE,
)

DB2_MANUAL_REDESIGN_COMMENT_PATTERN = re.compile(
 r"^\s**DB2:\s*Manual redesign required\b",
 flags=re.IGNORECASE,
)

DB2_MISSING_MAPPING_COMMENT_PATTERN = re.compile(
 r"^\s**DB2:\s*Missing\b",
 flags=re.IGNORECASE,
)

```

```

DB2_REMOVED_OR_SKIPPED_COMMENT_PATTERN = re.compile(
 r"^.*\s*(Removed|Skipped|Missing|Manual redesign required)\b",
 flags=re.IGNORECASE,
)

SQL_MUTATION_STATEMENT_PATTERN = re.compile(
 r"^.*(INSERT|UPDATE|DELETE|SELECT|FETCH|OPEN|CLOSE|COMMIT|ROLLBACK)\b",
 flags=re.IGNORECASE,
)

SQLCODE_REFERENCE_PATTERN = re.compile(
 r"\bSQLCODE\b",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\sqlcode_wrapper_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

IF_NOT_SQLCODE_100_PATTERN = re.compile(
 r"^.*IF\s+NOT\s+SQLCODE\s*=\s*100\.\.?\s*$",
 flags=re.IGNORECASE,
)

IF_SQLCODE_100_PATTERN = re.compile(
 r"^.*IF\s+SQLCODE\s*=\s*100\.\.?\s*$",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^.*END-IF\.\.?\s*$",
 flags=re.IGNORECASE,
)

REMOVED_REDUNDANT_OBTAIN_NEXT_COMMENT_PATTERN = re.compile(
 r"^.*\s*\s*(Removed\s+redundant\s+OBTAIN\s+NEXT)\b",
 flags=re.IGNORECASE,
)

MOVE_DCLGEN_TO_TARGET_PATTERN = re.compile(
 r"^(?P<indent>\s*)MOVE\s+(?P<source>[A-Z0-9-]+\s+OF\s+DCL[A-Z0-9-]+)"
 r"\s+TO\s+(?P<target>[A-Z0-9-]+\s+)\.\.\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\update_direct_sql_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

CONVERTED_OBTAIN_CALC_COMMENT_PATTERN = re.compile(
 r"^.*\s*\s*(?:Converted|Removed)\s+OBTAIN\s+CALC"
 r"(?:\s+SELECT)?\s+for\s+(?P<record>[A-Z0-9-]+\s+)\.\.\s*$",
 flags=re.IGNORECASE,
)

CONVERTED_MODIFY_COMMENT_PATTERN = re.compile(
 r"^.*\s*\s*(Converted\s+MODIFY\s+for\s+"
 r"(?P<record>[A-Z0-9-]+\s+)\.\.\s*$",
 flags=re.IGNORECASE,
)

DIRECT_UPDATE_COMMENT_PATTERN = re.compile(
 r"^.*\s*\s*(Direct\s+UPDATE\s+will\s+use\s+mapped\s+"

```

```

 r"composite\s+key\s+WHERE\s+clause\.\?\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\?\s*$",
 flags=re.IGNORECASE,
)

SELECT_STATEMENT_PATTERN = re.compile(
 r"^\s*SELECT\b",
 flags=re.IGNORECASE,
)

UPDATE_STATEMENT_PATTERN = re.compile(
 r"^\s*UPDATE\s+(?P<table>[A-Z0-9_]+\)\b",
 flags=re.IGNORECASE,
)

IF_SQLCODE_NOT_ZERO_PATTERN = re.compile(
 r"^\s*IF\s+SQLCODE\s+NOT\s*=\s*0\b",
 flags=re.IGNORECASE,
)

AND_SQLCODE_NOT_FOUND_PATTERN = re.compile(
 r"^\s*AND\s+SQLCODE\s+NOT\s*=\s*100\b",
 flags=re.IGNORECASE,
)

PERFORM_SQLERROR_PATTERN = re.compile(
 r"^\s*PERFORM\s+SQLERROR(?:END)?-?IF\.\?\s*$",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^\s*END-IF\.\?\s*$",
 flags=re.IGNORECASE,
)

NON_SQL_HOST_TARGET_PATTERN = re.compile(
 r"^(?P<prefix>\s*TO\s+):(?P<host>[A-Z0-9-]+)"
 r"(?P<suffix>\s+OF\s+[A-Z0-9-]+\.\?\s*)$",
 flags=re.IGNORECASE,
)

MOVE_TO_SOURCE_FIELD_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<source>[A-Z0-9-]+\)\s+TO\s+"
 r"(?P<target>[A-Z0-9-]+\)\.\?\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_DCL_FIELD_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<source>.+?)\s+TO\s+"
 r"(?P<host>[A-Z0-9-]+\)\s+OF\s+(?P<group>DCL[A-Z0-9-]+\)\.\?\s*$",
 flags=re.IGNORECASE,
)

UPDATE_AUDIT_ASSIGNMENT_PATTERN = re.compile(
 r"^(?P<indent>\s*)"
 r"(?P<column_prefix>TS_UPDATE|ID_USERID|NR_USERID|ID_USER|NR_USER)"
 r"_(?P<suffix>[A-Z0-9_]+)"
 r"\s*=\s*:"
 r"(?P<host>[A-Z0-9-]+\)"
 r"(?P<trailing>.*)$",
 flags=re.IGNORECASE,
)

SQL_SET_KEYWORD_PATTERN = re.compile(
 r"^\s*SET\b",
 flags=re.IGNORECASE,
)

```

```

SQL_WHERE_KEYWORD_PATTERN = re.compile(
 r"^\s*WHERE\b",
 flags=re.IGNORECASE,
)

SQL_AND_KEYWORD_PATTERN = re.compile(
 r"^\s*AND\b",
 flags=re.IGNORECASE,
)

SQL_QUERYNO_PATTERN = re.compile(
 r"^\s*QUERYNO\b",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\update_procedure_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

EXEC_SQL_START_PATTERN = re.compile(
 r"^\EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\END-EXEC\.\?$",
 flags=re.IGNORECASE,
)

UPDATE_STATEMENT_PATTERN = re.compile(
 r"^\UPDATE\s+(?P<table>[A-Z0-9_+])\b",
 flags=re.IGNORECASE,
)

FROM_STATEMENT_PATTERN = re.compile(
 r"^\FROM\s+(?P<table>[A-Z0-9_+])\b",
 flags=re.IGNORECASE,
)

SET_KEYWORD_PATTERN = re.compile(
 r"^\SET$",
 flags=re.IGNORECASE,
)

WHERE_KEYWORD_PATTERN = re.compile(
 r"^\WHERE$",
 flags=re.IGNORECASE,
)

SQL_SET_ASSIGNMENT_PATTERN = re.compile(
 r"^(?P<column>[A-Z][A-Z0-9_+])\s*=\s*:"
 r"(?P<group>DCL[A-Z0-9_+])\."
 r"(?P<host>[A-Z][A-Z0-9_+]*)"
 r"(?P<comma>,)\?$",
 flags=re.IGNORECASE,
)

NON_SQL_DCL_DOT_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<group>DCL[A-Z0-9_+])\."
 r"(?P<host>[A-Z][A-Z0-9_+]*)\b",
 flags=re.IGNORECASE,
)

DCLGEN_OF_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<host>[A-Z][A-Z0-9_+]*)\s+OF\s+"
 r"(?P<group>DCL[A-Z0-9_+])\b",
 flags=re.IGNORECASE,
)

```

```

MOVE_TO_DCL_OF_REFERENCE_PATTERN = re.compile(
 r"^(?P<source>.+?)\s+TO\s+"
 r"(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
 r"(?P<group>DCL[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_BARE_RECORD_PATTERN = re.compile(
 r"^(?P<source>SPACES?|SPACE)\s+TO\s+"
 r"(?P<target>[A-Z][A-Z0-9-]*)\.\.?s*$",
 flags=re.IGNORECASE,
)

INITIALIZE_DCL_PATTERN = re.compile(
 r"^(?P<group>DCL[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

SQL_LOCATION_DB_OPERATION_PATTERN = re.compile(
 r"^(?P<operation>SELECT|UPDATE|INSERT|DELETE)-"
 r"(?P<label>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

CONVERTED_DB2_OPERATION_COMMENT_PATTERN = re.compile(
 r"^(?P<prefix>*\s*DB2:\s+Converted\s+"
 r"(?P<operation>OBTAIN\s+CALC|MODIFY|STORE|ERASE|FIND|OBTAIN)"
 r"\s+for\s+"
 r"(?P<label>[A-Z0-9-]+\.)"
 r"(?P<suffix>\.\.?s*)$",
 flags=re.IGNORECASE,
)

=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\update_program_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

CONVERTED_OBTAIN_CALC_PATTERN = re.compile(
 r"^(?P<record>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

CONVERTED_MODIFY_PATTERN = re.compile(
 r"^(?P<record>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^(?P<record>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^(?P<record>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

INCLUDE_PATTERN = re.compile(
 r"^(?P<include>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

SQLCODE_IF_PATTERN = re.compile(
 r"^(?P<record>[A-Z0-9-]+\.\.?s*$",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(

```

```

 r"^\\s*END-IF\\.?\\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_BARE_FIELD_PATTERN = re.compile(
 r"^\\s*MOVE\\s+(?P<source>\\.+?)\\s+TO\\s+(?P<target>[A-Z][A-Z0-9-]+)\\.?\\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_DCL_HOST_PATTERN = re.compile(
 r"^\\s*MOVE\\s+(?P<source>\\.+?)\\s+TO\\s+"
 r"(?P<host>[A-Z][A-Z0-9-]*)\\s+OF\\s+(?P<group>DCL[A-Z0-9-]+)\\.?\\s*$",
 flags=re.IGNORECASE,
)

TIMESTAMP_PARAGRAPH_PATTERN = re.compile(
 r"^\\s*600-GET-TIMESTAMP\\.?\\s*$",
 flags=re.IGNORECASE,
)

STOP_RUN_PATTERN = re.compile(
 r"^\\s*STOP\\s+RUN\\.?\\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\\idms_db2_phase2\\reference\\patterns\\normalization\\update_restart_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

CONVERTED_FOR_PATTERN = re.compile(
 r"^\\s**?DB2:\\s*Converted\\s+"
 r"(?P<operation>OBTAIN\\s+CALC|STORE|MODIFY|ERASE|DELETE|UPDATE|INSERT)"
 r"\\s+(?:for\\s+)?(?P<record>[A-Z0-9-]+)\\.?\\s*$",
 flags=re.IGNORECASE,
)

SKIPPED_FOR_PATTERN = re.compile(
 r"^\\s**?DB2:\\s*"
 r"(?P<operation>SELECT|INSERT|UPDATE|DELETE|STORE|MODIFY|OBTAIN\\s+CALC)"
 r"\\s+conversion\\s+skipped\\s+for\\s+(?P<record>[A-Z0-9-]+)\\.?\\s*$",
 flags=re.IGNORECASE,
)

REMOVED_OBTAIN_CALC_PATTERN = re.compile(
 r"^\\s**?DB2:\\s*Removed\\s+OBTAIN\\s+CALC\\s+SELECT\\s+for\\s+"
 r"(?P<record>[A-Z0-9-]+)\\.?\\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^\\s*EXEC\\s+SQL\\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\\s*END-EXEC\\.?\\s*$",
 flags=re.IGNORECASE,
)

SQLCODE_IF_PATTERN = re.compile(
 r"^\\s*IF\\s+SQLCODE\\b",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^\\s*END-IF\\.?\\s*$",
 flags=re.IGNORECASE,
)

```

```
=====
FILE: src\idms_db2_phase2\reference\patterns\normalization\update_statement_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import re

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

UPDATE_STATEMENT_PATTERN = re.compile(
 r"^\s*UPDATE\s+(?P<table>[A-Z0-9_]+\)\b",
 flags=re.IGNORECASE,
)

SET_KEYWORD_PATTERN = re.compile(
 r"^\s*SET\b",
 flags=re.IGNORECASE,
)

WHERE_KEYWORD_PATTERN = re.compile(
 r"^\s*WHERE\b",
 flags=re.IGNORECASE,
)

SQL_ASSIGNMENT_PATTERN = re.compile(
 r"^\s*(?P<column>[A-Z0-9_]+\s*=\s*(?P<host>[A-Z0-9-]+)(?:\s+OF\s+(?P<group>[A-Z0-9-]+))?,?\s*$",
 flags=re.IGNORECASE,
)

INSERT_ONLY_AUDIT_HOST_PATTERN = re.compile(
 r":(?P<host>TS-CREATE-[A-Z0-9-]+)(?:\s+OF\s+(?P<group>[A-Z0-9-]+))?",
 flags=re.IGNORECASE,
)

UPDATE_AUDIT_COLUMN_PATTERN = re.compile(
 r"^\s*TS_UPDATE_[A-Z0-9_]+\s*=",
 flags=re.IGNORECASE,
)

USER_AUDIT_COLUMN_PATTERN = re.compile(
 r"^\s*(ID_USERID|NR_USERID|ID_USER|NR_USER)_[A-Z0-9_]+\s*=",
 flags=re.IGNORECASE,
)

DCLGEN_DOT_REFERENCE_PATTERN = re.compile(
 r"\b(?P<group>DCL[A-Z0-9-]+)\. (?P<field>[A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

MOVE_TO_BARE_RECORD_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<value>SPACES|ZEROS|ZEROS|0)\s+TO\s+(?P<target>[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_DCLGEN_REFERENCE_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<source>.+?)\s+TO\s+(?P<field>[A-Z0-9-]+)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

CONVERTED_MODIFY_COMMENT_PATTERN = re.compile(
 r"^\s**DB2:\s*Converted\s+MODIFY\s+for\s+(?P<record>[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

MANUAL_REDESIGN_COMMENT_PATTERN = re.compile(
```

```

 r"^\s**DB2:\s*Manual redesign required\b",
 flags=re.IGNORECASE,
)

```

```

=====
FILE: src\idms_db2_phase2\reference\patterns\validation__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\reference\patterns\validation\production_patterns.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re

TODO_HOST_VARIABLE_PATTERN = re.compile(
 r"TODO\s+HOST",
 flags=re.IGNORECASE,
)

TODO_DB2_PATTERN = re.compile(
 r"TODO\s+DB2",
 flags=re.IGNORECASE,
)

ERROR_DB2_PATTERN = re.compile(
 r"ERROR\s+DB2",
 flags=re.IGNORECASE,
)

UNABLE_TO_DECLARE_CURSOR_PATTERN = re.compile(
 r"UNABLE\s+TO\s+DECLARE\s+Cursors",
 flags=re.IGNORECASE,
)

NO_FETCH_HOST_VARIABLES_PATTERN = re.compile(
 r"FETCH\s+[A-Z0-9_-]+\s+INTO\s*(?:END-EXEC|\.)",
 flags=re.IGNORECASE,
)

FORBIDDEN_EXECUTABLE_IDMS_PATTERNS = [
 re.compile(r"\bOBTAIN\b", flags=re.IGNORECASE),
 re.compile(r"\bSTORE\b", flags=re.IGNORECASE),
 re.compile(r"\bMODIFY\b", flags=re.IGNORECASE),
 re.compile(r"\bERASE\b", flags=re.IGNORECASE),
 re.compile(r"\bFIND\b", flags=re.IGNORECASE),
 re.compile(r"\bCONNECT\b", flags=re.IGNORECASE),
 re.compile(r"\bDISCONNECT\b", flags=re.IGNORECASE),
]

FORBIDDEN_IDMS_DECLARATIVE_PATTERNS = [
 re.compile(r"\bIDMS-CONTROL\b", flags=re.IGNORECASE),
 re.compile(r"\bSCHEMA\b", flags=re.IGNORECASE),
 re.compile(r"\bPROTOCOL\b", flags=re.IGNORECASE),
 re.compile(r"\bCOPY\s+IDMS\b", flags=re.IGNORECASE),
 re.compile(r"\bBIND\b", flags=re.IGNORECASE),
 re.compile(r"\bREADY\b", flags=re.IGNORECASE),
]

```

```

=====
FILE: src\idms_db2_phase2\reference\rules__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\rules\authority__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\reference\rules\authority\input_authority_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
INPUT_AUTHORITY_RULES = [
 "Sheet Mapping is the authority for DB2 record/table names.",
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable names and PIC clauses.",
 "Original COBOL is the authority for business flow.",
 "Final output is resequenced in manual-style COBOL format.",
]
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\authority\no_hardcode_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
NO_HARDCODE_RULES = [
 "Do not hardcode business records in services.",
 "Do not hardcode DB2 tables in services.",
 "Do not hardcode DB2 columns in services.",
 "Do not hardcode DCLGEN names in services.",
 "Do not hardcode host variables in services.",
 "Do not hardcode parser column aliases inside parser classes.",
 "Do not hardcode regex patterns inside parser or service classes.",
]
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\cobol__init__.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\cobol\fixed_format_blank_line_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
Fixed-format COBOL blank-line rules.
```

```
This file contains constants only.
No regex.
No parser logic.
No composer logic.
No service logic.
No business hardcoding.
```

```
Purpose:
```

- Prevent raw empty physical lines in final generated COBOL.
- Represent preserved blank lines as valid 80-column fixed-format lines.
- Keep final output consistently sequenced from columns 1-6 and 73-80.

```
"""
```

```
PRESERVE_BLANK_LINES_AS_SEQUENCED_LINES = True
```

```
BLANK_LINE_BODY = ""
```

```
BLANK_LINE_INDICATOR = " "
```

```
FIXED_FORMAT_BLANK_LINE_RULES = [
 "Do not emit raw empty physical lines in final generated COBOL.",
 "If blank lines are preserved, emit them as sequenced fixed-format blank body lines.",
 "Preserved blank lines must still have left and right sequence numbers.",
 "Blank-line handling must not change COBOL business logic.",
]
```

```
=====
```

```
FILE: src\idms_db2_phase2\reference\rules\cobol\fixed_format_comment_rules.py
```

```
ENCODING: utf-8
```

```
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Fixed-format COBOL comment and indicator preservation rules.
```

```
This file contains rule constants only.
```

```
No regex.
```

```
No parser logic.
```

```
No composer logic.
```

```
No service logic.
```

```
No business hardcoding.
```

```
Purpose:
```

- Preserve original COBOL comment/page/debug indicators.
- Prevent original comments from becoming executable COBOL statements.
- Support final manual-style fixed-format output.

```
"""
```

```
COMMENT_BODY_MARKERS = {
 "/*",
}
```

```
PAGE_BODY_MARKERS = {
 "/",
}
```

```
DEBUG_BODY_MARKERS = {
 "D",
 "d",
}
```

```
COMMENT_INDICATORS = {
 "/*",
 "/",
}
```

```
SUPPORTED_SOURCE_INDICATORS = {
 " ",
 "/*",
 "/",
 "D",
 "d",
 "-",
}
```

```
NORMALIZED_DEBUG_INDICATOR = "D"
```

```
DEFAULT_SOURCE_INDICATOR = " "
```

```
PRESERVE_FIXED_FORMAT_COMMENT_RULES = [
 "Preserve existing fixed-format column 7 comment indicator.",
 "Preserve existing fixed-format column 7 page indicator.",
 "Preserve existing fixed-format column 7 debug indicator.",
 "Do not convert original COBOL comment lines into executable COBOL.",
 "When an unsequenced line starts with a comment marker, place the marker in column 7.",
 "When a sequenced line has a comment marker in the body because column 7 was lost, restore it to column 7.",
]
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\cobol\fixed_format_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
TOTAL_WIDTH = 80
```

```
LEFT_SEQUENCE_WIDTH = 6
```

```
INDICATOR_WIDTH = 1
```

```
BODY_WIDTH = 65
```

```
RIGHT_SEQUENCE_WIDTH = 8
```

```
LEFT_SEQUENCE_START_INDEX = 0
```

```
LEFT_SEQUENCE_END_INDEX = 6
```

```
INDICATOR_INDEX = 6
```

```
BODY_START_INDEX = 7
```

```
BODY_END_INDEX = 72
```

```
RIGHT_SEQUENCE_START_INDEX = 72
```

```
RIGHT_SEQUENCE_END_INDEX = 80
```

```
DEFAULT_LEFT_START = 10
```

```
DEFAULT_LEFT_STEP = 10
```

```
DEFAULT_RIGHT_START = 1000
```

```
DEFAULT_RIGHT_STEP = 1000
```

```
SPACE_INDICATOR = " "
```

```
COMMENT_INDICATOR = "/*"
```

```
DEBUG_INDICATOR = "D"
```

```
PAGE_INDICATOR = "/"
```

```
VALID_INDICATORS = {
```

```
 SPACE_INDICATOR,
```

```
 COMMENT_INDICATOR,
```

```
 DEBUG_INDICATOR,
```

```
 PAGE_INDICATOR,
```

```
}
```

```
AREA_A_BODY_INDENT = ""
```

```
AREA_B_BODY_INDENT = " "
```

```
SQL_BODY_INDENT = " "
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\cobol\fixed_format_wrapping_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Fixed-format COBOL wrapping rules.
```

```
This file contains constants only.
```

```
No regex.
```

```
No parser logic.
```

```
No composer logic.
```

```
No service logic.
```

```
No business hardcoding.
```

```
Purpose:
```

```
- Keep final COBOL within the 80-column physical layout.
```

```
- Avoid visually touching the right sequence number.
```

```
- Split long MOVE, SQL predicate, SQL assignment, and comment lines safely.
```

```
"""
```

```
READABLE_BODY_WIDTH = 60
```

```
COMMENT_CONTINUATION_INDENT = ""
```

```
DEFAULT_CONTINUATION_INDENT = " "
```

```
SQL_CONTINUATION_INDENT = " "
```

```
MOVE_CONTINUATION_INDENT = " "
```

```
MOVE_KEYWORD = "MOVE "
```

```

MOVE_TARGET_SEPARATOR = " TO "

SQL_ASSIGNMENT_SEPARATOR = " = "

SQL_SPLIT_PREFIXES = (
 "WHERE ",
 "AND ",
 "OR ",
 "SET ",
)

FIXED_FORMAT_WRAPPING_RULES = [
 "Never truncate long generated COBOL body text when it can be wrapped.",
 "Wrap long generated comment lines before they visually touch the right sequence.",
 "Wrap long MOVE statements by placing the target on a following body line.",
 "Wrap long SQL predicates after the comparison operator.",
 "Wrap long SQL assignments after the assignment operator.",
 "Preserve original body indentation wherever possible.",
 "Keep every generated physical line inside columns 8 through 72.",
 "Prefer readable body width before the hard fixed-format body limit.",
]

=====
FILE: src\idms_db2_phase2\reference\rules\cobol>manual_layout_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

MANUAL_LAYOUT_RULES = [
 "Final COBOL should be manual-style sequenced.",
 "Columns 1-6 contain the left sequence number.",
 "Columns 8-72 contain the COBOL body.",
 "Columns 73-80 contain the right sequence number.",
 "Generated output must preserve COBOL headers and division structure.",
]

=====
FILE: src\idms_db2_phase2\reference\rules\cobol\statement_classification_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

NON_PARAGRAPH_SINGLE_WORDS = {
 "ACCEPT",
 "ADD",
 "ALTER",
 "CALL",
 "CANCEL",
 "CLOSE",
 "COMMIT",
 "COMPUTE",
 "CONTINUE",
 "DELETE",
 "DISPLAY",
 "DIVIDE",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",
 "END-EVALUATE",
 "END-EXEC",
 "END-IF",
 "END-MULTIPLY",
 "END-PERFORM",
 "END-READ",
 "END-RETURN",
 "EVALUATE",
 "EXEC",
 "EXIT",

```

```

 "FETCH",
 "GOBACK",
 "IF",
 "INITIALIZE",
 "INSERT",
 "MOVE",
 "OPEN",
 "PERFORM",
 "READ",
 "RETURN",
 "ROLLBACK",
 "SET",
 "STOP",
 "STRING",
 "SUBTRACT",
 "UPDATE",
 "WHEN",
 "WRITE",
}

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\conversion__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\reference\rules\conversion\business_flow_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

BUSINESS_FLOW_RULES = [
 "Preserve original COBOL business logic unless replacing IDMS database statements.",
 "Do not rewrite unrelated COBOL statements.",
 "Do not corrupt COBOL headers such as DATE-WRITTEN, AUTHOR, or PROGRAM-ID.",
 "Replace only IDMS database access logic with DB2 embedded SQL logic.",
 "Retain paragraph structure unless a database replacement requires a local block.",
]

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\conversion\field_mapping_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

FIELD_MAPPING_RULES = [
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable spelling.",
 "Do not invent DB2 field names.",
 "Do not use IDMS record names in final DB2 procedure logic when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite only safe qualified references unless strong context exists.",
]

```

```

QUALIFIED_REFERENCE_REWRITE_RULES = [
 "Rewrite FIELD OF IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite FIELD IN IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Do not rewrite bare words globally.",
 "When mapping metadata is missing, leave a clear conversion-skipped comment.",
]

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\conversion\idms_to_db2_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

CONVERSION_RULES = [
 "Preserve original COBOL business flow.",
 "Replace IDMS database operations with DB2-compatible SQL logic.",
 "Do not fabricate DB2 SQL when Sheet Mapping metadata is missing.",
 "When mapping metadata is missing, generate a clear DB2 conversion-skipped comment.",
 "When removing executable PROCEDURE DIVISION IDMS code, add CONTINUE.",
]

MISSING_MAPPING_RULES = [
 "If Sheet Mapping entry is missing, conversion must be skipped with a clear comment.",
 "Missing Sheet Mapping metadata is an input-data blocker, not a Python logic failure.",
]

=====
FILE: src\idms_db2_phase2\reference\rules\conversion\name_resolution_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

MIN_SOURCE_FIELD_SIMILARITY = 0.86

TABLE_SUFFIX_ALTERNATES = [
 ("TB", "TV"),
 ("TV", "TB"),
]

DCLGEN_GROUP_PREFIX = "DCL"

HOST_REFERENCE_PREFIX = ":"
HOST_REFERENCE_QUALIFIER = " OF "

DEFAULT_CURSOR_ORDER = 1
CURSOR_NAME_SUFFIX = "C"

=====
FILE: src\idms_db2_phase2\reference\rules\db2__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\reference\rules\db2\cursor_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

CURSOR_AUTHORITY_RULES = [
 "Cursor names are derived from Sheet Mapping DB2 record/table names.",
 "Cursor names are not derived from IDMS set names.",
 "Generated cursor OPEN, FETCH, and CLOSE paragraphs are placed near the end.",
 "SQL-ERROR paragraph is required when cursor code performs SQL error handling.",
]

CURSOR_PARAGRAPH_NUMBERING = {
 1: {
 "open": 710,
 "fetch": 720,
 "close": 730,
 },
 2: {
 "open": 810,
 "fetch": 820,
 "close": 830,
 },
 3: {
 "open": 910,

```

```

 "fetch": 920,
 "close": 930,
 },
}

def cursor_paragraph_number(
 cursor_order: int,
 operation: str,
) -> int:
 operation_key = str(operation or "").strip().lower()

 if cursor_order not in CURSOR_PARAGRAPH_NUMBERING:
 raise ValueError(f"Unsupported cursor order: {cursor_order}")

 if operation_key not in CURSOR_PARAGRAPH_NUMBERING[cursor_order]:
 raise ValueError(f"Unsupported cursor operation: {operation}")

 return CURSOR_PARAGRAPH_NUMBERING[cursor_order][operation_key]

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\db2\date_conversion_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

DB2_DATE_COMPARISON_RULES = [
 "DB2 DATE values must be realigned before numeric comparison.",
 "Generate date conversion only when a DA-/DT- DCLGEN host field is compared with PARMDATE.",
 "Generated helper field names are derived from the detected DB2 host field name.",
 "Date conversion logic must not hardcode program names, table names, record names, or business fields.",
 "Initialize generated date helper fields before moving converted date values into them.",
]

```

```

DB2_DATE_COMPARISON_WS_MARKER = "DB2 DATE CONVERSION WORKING STORAGE"

```

```

DB2_DATE_BASE_WORKING_STORAGE_LINES = [
 f"* {DB2_DATE_COMPARISON_WS_MARKER}",
 "01 WS-DATUMVELDEN.",
 " 03 DA-CCYYMMDD.",
 " 05 CC PIC 99 VALUE ZERO.",
 " 05 YY PIC 99 VALUE ZERO.",
 " 05 MM PIC 99 VALUE ZERO.",
 " 05 DD PIC 99 VALUE ZERO.",
 " 03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.",
 " 05 CCYY PIC 9999.",
 " 05 MM PIC 99.",
 " 05 DD PIC 99.",
 " 03 DA-DD-MM-CCYY.",
 " 05 DD PIC 9(02).",
 " 05 TE-MARKER5 PIC X VALUE '.'. ",
 " 05 MM PIC 9(02).",
 " 05 TE-MARKER6 PIC X VALUE '.'. ",
 " 05 CCYY PIC 9(04).",
]

```

```

DB2_DATE_HELPER_FIELD_TEMPLATE = (
 "02 {helper:<30} PIC 9(8) VALUE ZEROES."
)

```

```

DB2_DATE_LOW_VALUE_LITERAL = "01.01.0001"
DB2_DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DB2_DATE_HIGH_NUMERIC_LITERAL = "99999999"

```

```

DB2_DATE_CONVERSION_LINE_TEMPLATES = [
 "{indent}MOVE ZEROES TO DA-CCYYMMDD",
 "{indent}MOVE ZEROES TO {helper}",
 "{indent}MOVE {field_name} OF {group_name} TO DA-DD-MM-CCYY",
 "{indent}EVALUATE TRUE",
 "{indent} WHEN DA-DD-MM-CCYY = '{low_value}'",
 "{indent} MOVE ZEROES TO DA-CCYYMMDD-R",
 "{indent} WHEN DA-DD-MM-CCYY = '{high_value}'",
 "{indent} MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
]

```

```

 "{indent} WHEN OTHER",
 "{indent} MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 "{indent} MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 "{indent} MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "{indent}END-EVALUATE",
 "{indent}MOVE DA-CCYYMMDD-R TO {helper}",
]

```

```
DB2_DATE_IF_REPLACEMENT_TEMPLATE = "{indent}IF {helper} {condition}"
```

```

=====
FILE: src\idms_db2_phase2\reference\rules\db2\output_date_conversion_rules.py
ENCODING: utf-8
=====

```

```
from __future__ import annotations
```

```

DB2_OUTPUT_DATE_CONVERSION_RULES = [
 "DB2 date values must be realigned before moving to output date fields.",
 "Generate output date conversion only when a DA-/DT- DCLGEN host field is moved to an output date field.",
 "Initialize shared DB2 date work fields before date realignment.",
 "Clear the output date target before moving the converted date value.",
 "Do not hardcode program names, table names, DCLGEN group names, or business field names.",
 "Use shared date work fields when already generated in WORKING-STORAGE.",
]

```

```

DB2_OUTPUT_DATE_LOW_VALUE_LITERAL = "01.01.0001"
DB2_OUTPUT_DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DB2_OUTPUT_DATE_HIGH_NUMERIC_LITERAL = "99999999"

```

```

DB2_OUTPUT_DATE_CONVERSION_LINE_TEMPLATES = [
 "{indent}MOVE ZEROES TO DA-CCYYMMDD",
 "{indent}MOVE SPACES TO {target_name}",
 "{indent}MOVE {field_name} OF {group_name} TO DA-DD-MM-CCYY",
 "{indent}EVALUATE TRUE",
 "{indent} WHEN DA-DD-MM-CCYY = '{low_value}'",
 "{indent} MOVE ZEROES TO DA-CCYYMMDD-R",
 "{indent} WHEN DA-DD-MM-CCYY = '{high_value}'",
 "{indent} MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
 "{indent} WHEN OTHER",
 "{indent} MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 "{indent} MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 "{indent} MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "{indent}END-EVALUATE",
 "{indent}MOVE DA-CCYYMMDD-R TO {target_name}",
]

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\db2\sql_generation_rules.py
ENCODING: utf-8
=====

```

```
from __future__ import annotations
```

```

SQL_GENERATION_RULES = [
 "Sheet Mapping is the authority for DB2 record/table names.",
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable spelling and group names.",
 "If Sheet Mapping uses TB but uploaded DCLGEN uses TV, generated SQL uses TV.",
 "UPDATE generation is conservative/manual-style, not broad all-column update.",
]

```

```

TABLE_RESOLUTION_RULES = [
 "Generated SELECT FROM table name must match resolved DCLGEN table name.",
 "Generated UPDATE table name must match resolved DCLGEN table name.",
]

```

```
=====
FILE: src\idms_db2_phase2\reference\rules\db2\update_sql_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
CONSERVATIVE_UPDATE_RULES = [
 "UPDATE generation is conservative/manual-style, not broad all-column update.",
 "Only columns changed by original COBOL business logic should be updated.",
 "UPDATE audit columns may be added only when present in Sheet Mapping and DCLGEN.",
 "For UPDATE flows, generate TS_UPDATE and USER-ID moves only.",
 "TS_CREATE is insert-only and must not be generated for update-only flows.",
 "For IDMS CALC records, WHERE clause must use Sheet Mapping key metadata only.",
 "Do not add relationship foreign keys to CALC-record WHERE clauses unless marked as key metadata.",
]
```

```
UPDATE_AUDIT_COLUMN_PREFIXES = [
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
]
```

```
INSERT_ONLY_AUDIT_COLUMN_PREFIXES = [
 "TS_CREATE",
]
```

```
KEY_TEXT_MARKERS = [
 "PRIMARY",
 "PRIMARY KEY",
 "KEY",
 "CALC",
]
```

```
DATE_COLUMN_PREFIXES = [
 "DA_",
 "DT_",
]
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\normalization__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\normalization\final_cobol_normalization_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
FINAL_COBOL_NORMALIZATION_RULES = [
 "Fix Procedure Division executable statements to begin in Area B.",
 "Synchronize PROGRAM-NAME from final PROGRAM-ID only in safe MOVE statements.",
 "Use configurable DB2 date external format for generated date conversion.",
 "Remove ORDER-BY-only columns from cursor SELECT and FETCH when safe.",
 "Do not hardcode program names, DB2 tables, columns, DCLGEN groups, or host variables.",
 "Do not remove existing business logic or already fixed functionality.",
]
```

```
DEFAULT_DB2_DATE_EXTERNAL_FORMAT = "DD.MM.YYYY"
ORDER_BY_COLUMNS_IN_SELECT_DEFAULT = False
```

```
SUPPORTED_DB2_DATE_FORMATS = {
 "YYYY-MM-DD",
 "DD.MM.YYYY",
 "YYYYMMDD",
}
```

```

DATE_FORMAT_ALIASES = {
 "ISO": "YYYY-MM-DD",
 "YYYY-MM-DD": "YYYY-MM-DD",
 "EUR": "DD.MM.YYYY",
 "EUROPEAN": "DD.MM.YYYY",
 "DD.MM.YYYY": "DD.MM.YYYY",
 "DD.MM.CCYY": "DD.MM.YYYY",
 "NUMERIC": "YYYYMMDD",
}

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\normalization\include_placement_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

INCLUDE_PLACEMENT_RULES = [
 "DCLGEN INCLUDE blocks must not be emitted after LINKAGE SECTION.",
 "DCLGEN INCLUDE blocks must not be emitted inside PROCEDURE DIVISION.",
 "SQLCA, SQLERRWS, and DCLGEN includes belong in the DB2 infrastructure area.",
 "Duplicate includes must be removed.",
 "Do not remove SQLERROR include paragraph.",
 "Do not hardcode program names, records, DB2 columns, or host variables.",
]

SYSTEM_INCLUDE_NAMES = (
 "SQLCA",
 "SQLERRWS",
 "SQLERROR",
)

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\normalization\procedure_indentation_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

PROCEDURE_INDENTATION_RULES = [
 "Procedure Division executable statements must begin in Area B.",
 "Paragraph headers remain in Area A.",
 "Section headers remain in Area A.",
 "SQL lines inside EXEC SQL blocks use SQL indentation.",
 "IF, ELSE, END-IF, EVALUATE, WHEN, and END-EVALUATE remain readable.",
 "Do not change business logic while normalizing indentation.",
 "Do not truncate fixed-format COBOL body content.",
 "Do not hardcode program names, DB2 tables, columns, DCLGEN groups, or host variables.",
]

AREA_A_INDENT = ""

AREA_B_INDENT = " "

NESTED_BODY_INDENT = " "

SQL_BODY_INDENT = " "

MAX_COBOL_BODY_WIDTH = 65

SEQUENCE_PREFIX_WIDTH = 6

INDICATOR_COLUMN_INDEX = 6

RIGHT_SEQUENCE_START_INDEX = 72

```

```

=====
FILE: src\idms_db2_phase2\reference\rules\normalization\retrieval_statement_normalization_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

RETRIEVAL_STATEMENT_NORMALIZATION_RULES = [
 "Ensure DCLGEN INCLUDE exists when a DCLGEN group is referenced.",
 "Replace residual IDMS ERROR-STATUS usage only when a DB2 status flag exists.",
 "Add INITIALIZE before output WRITE when safe.",
 "Do not duplicate INITIALIZE statements.",
 "Do not hardcode program names, table names, record names, DCLGEN groups, or business fields.",
 "Preserve original retrieval business logic unless a generated DB2 normalization is required.",
]

DCLGEN_GROUP_PREFIX = "DCL"

DEFAULT_RETRIEVAL_STATUS_VALUES = (
 "N",
 "Y",
 "C",
)

WRITE_LOOKBACK_LIMIT = 12

INCLUDE_INSERT_MARKER = "WORKING-STORAGE SECTION."

RETRIEVAL_INITIALIZE_MESSAGE = (
 "Retrieval normalization: added INITIALIZE before WRITE."
)

RETRIEVAL_INCLUDE_MESSAGE = (
 "Retrieval normalization: added missing DCLGEN INCLUDE."
)

RETRIEVAL_STATUS_MESSAGE = (
 "Retrieval normalization: replaced residual ERROR-STATUS usage."
)

=====
FILE: src\idms_db2_phase2\reference\rules\normalization\sqlcode_guard_normalization_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

SQLCODE_GUARD_NORMALIZATION_RULES = [
 "Preserve SQLCODE checks that immediately follow a real EXEC SQL block.",
 "Remove orphaned AND SQLCODE lines when no owning IF SQLCODE line exists.",
 "Remove generated SQLCODE guards after manual-redesign comments when no SQL was emitted.",
 "Do not invent SQL for restart or update logic when mapping metadata is missing.",
 "Normalize generated SQLCODE checks into a complete readable IF block.",
 "Do not hardcode program names, table names, column names, DCLGEN groups, or host variables.",
]

SQLCODE_SUCCESS_VALUE = "0"
SQLCODE_NOT_FOUND_VALUE = "100"

SQLCODE_ERROR_PARAGRAPH_NAMES = (
 "SQLERROR",
 "SQLERREOREND-IF",
)

SQLCODE_RECENT_SQL_LOOKBACK_LIMIT = 12

SQLCODE_MANUAL_CONTEXT_LOOKBACK_LIMIT = 8

NORMALIZED_SQLCODE_GUARD_LINES = (
 "IF SQLCODE NOT = 0",
 "AND SQLCODE NOT = 100",
 "PERFORM SQLERREOREND-IF",
 "END-IF.",
)

```

```
=====
FILE: src\idms_db2_phase2\reference\rules\normalization\update_direct_sql_normalization_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

UPDATE_DIRECT_SQL_NORMALIZATION_RULES = [
 "Remove generated OBTAIN CALC SELECT blocks before direct UPDATE when a MODIFY update follows.",
 "Do not generate broad all-column SELECT statements for update-only flows.",
 "Direct UPDATE SQL must remain conservative and use mapped key metadata.",
 "Changed source fields must resolve through Sheet Mapping to DB2 columns.",
 "DCLGEN must decide final host variable spelling and group names.",
 "Date values moved to DB2 DA or DT columns must be converted to DD.MM.CCYY layout.",
 "Do not use TS_CREATE host variables in update-only flows.",
 "For update-only flows, use TS_UPDATE and user-id audit host variables.",
 "Do not invent restart-table SQL when restart/control metadata is missing.",
 "Do not hardcode program names, DB2 tables, DB2 columns, DCLGEN groups, or host variables.",
]

AUDIT_COLUMN_PREFIXES = (
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
)

AUDIT_COLUMN_TO_HOST_PREFIX = {
 "TS_UPDATE": "TS-UPDATE",
 "ID_USERID": "ID-USERID",
 "NR_USERID": "NR-USERID",
 "ID_USER": "ID-USER",
 "NR_USER": "NR-USER",
}

INSERT_ONLY_AUDIT_HOST_PREFIXES = (
 "TS-CREATE",
)

DATE_COLUMN_PREFIXES = (
 "DA_",
 "DT_",
)

OBTAIN_SELECT_SCAN_LIMIT = 180

SQLCODE_GUARD_SCAN_LIMIT = 10

MODIFY_LOOKBACK_LIMIT = 80
```

```
=====
FILE: src\idms_db2_phase2\reference\rules\normalization\update_statement_normalization_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

UPDATE_STATEMENT_NORMALIZATION_RULES = [
 "Normalize update-program COBOL without hardcoding business names.",
 "Do not generate broad all-column UPDATE statements.",
 "Only preserve SQL that was generated from available Sheet Mapping and DCLGEN metadata.",
 "Do not leave orphaned SQLCODE checks after manual-redesign comments.",
 "Do not use TS_CREATE host variables in update-only flows.",
 "Use DCLGEN host-variable spelling where available.",
 "Convert non-SQL DCLGEN dot references to COBOL OF qualification.",
 "Keep SQL blocks syntactically readable and compiler-oriented.",
]

UPDATE_AUDIT_COLUMN_PREFIXES = (
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
)
```

```

)

INSERT_ONLY_AUDIT_HOST_PREFIXES = (
 "TS-CREATE",
)

UPDATE_TIMESTAMP_HOST_PREFIXES = (
 "TS-UPDATE",
)

USER_AUDIT_HOST_PREFIXES = (
 "ID-USERID",
 "NR-USERID",
 "ID-USER",
 "NR-USER",
)

UPDATE_SQL_KEYWORDS = (
 "UPDATE",
 "SET",
 "WHERE",
 "AND",
)

DEFAULT_UPDATE_USER_HOST_CANDIDATES = (
 "ID-USERID",
 "NR-USERID",
 "ID-USER",
 "NR-USER",
)

=====
FILE: src\idms_db2_phase2\reference\rules\validation__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\reference\rules\validation\db2_validation_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

REQUIRED_DB2_TOKENS = [
 "EXEC SQL",
 "SQLCA",
 "END-EXEC",
]

=====
FILE: src\idms_db2_phase2\reference\rules\validation\validation_messages.py
ENCODING: utf-8
=====

from __future__ import annotations

INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": (
 "IDMS COBOL source text file is required and must contain readable COBOL code."
),
}

MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",

```

```

 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}

```

```

PRODUCTION_VALIDATION_MESSAGES = {
 "empty_converted_cobol": "Production validation: converted COBOL text is empty.",
 "todo_host_variable": (
 "Production validation: TODO host variable remains in generated COBOL."
),
 "todo_db2": (
 "Production validation: TODO DB2 conversion marker remains in generated COBOL."
),
 "error_db2": (
 "Production validation: generated DB2 error marker remains in generated COBOL."
),
 "unable_to_declare_cursor": (
 "Production validation: unable-to-declare-cursor marker remains in generated COBOL."
),
 "no_fetch_host_variables": (
 "Production validation: cursor FETCH has no host variables."
),
 "no_dclgen_hosts": (
 "Production validation: no valid DCLGEN host variables are available for validation."
),
 "residual_executable_idms": (
 "Production validation: residual executable IDMS statement remains"
),
 "residual_idms_declarative": (
 "Production validation: residual IDMS declarative/control statement remains"
),
 "unknown_dclgen_host": (
 "Production validation: generated DCLGEN host variable was not found in uploaded DCLGEN columns"
),
}

```

```

=====
FILE: src\idms_db2_phase2\repositories__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\repositories\copybook_repository.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import CopybookField
from ..services.name_normalizer import NameNormalizer

class CopybookRepository:
 """
 Repository wrapper for optional Copybook fields.

 Copybook metadata is optional and must not override Sheet Mapping
 or DCLGEN authority.
 """

 def __init__(
 self,
 fields: list[CopybookField] | None = None,
) -> None:
 self.fields = fields or []

 def all(
 self,
) -> list[CopybookField]:
 return list(self.fields)

 def count(
 self,

```

```

) -> int:
 return len(self.fields)

def find(
 self,
 field_name: str,
) -> CopybookField | None:
 target = NameNormalizer.to_cobol(field_name)

 if not target:
 return None

 for field in self.fields:
 if NameNormalizer.to_cobol(field.field_name) == target:
 return field

 return None

def has_field(
 self,
 field_name: str,
) -> bool:
 return self.find(field_name) is not None

def picture_for(
 self,
 field_name: str,
) -> str:
 field = self.find(field_name)

 if not field:
 return ""

 return str(field.pic_clause or "").strip()

def usage_for(
 self,
 field_name: str,
) -> str:
 field = self.find(field_name)

 if not field:
 return ""

 return str(field.usage or "").strip()

def fields_under_parent(
 self,
 parent_name: str,
) -> list[CopybookField]:
 target = NameNormalizer.to_cobol(parent_name)

 if not target:
 return []

 return [
 field
 for field in self.fields
 if NameNormalizer.to_cobol(field.parent_name) == target
]

```

```

=====
FILE: src\idms_db2_phase2\repositories\dclgen_repository.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import DclgenColumn
from ..reference.rules.conversion.name_resolution_rules import (
 HOST_REFERENCE_PREFIX,
 HOST_REFERENCE_QUALIFIER,
)
from ..services.name_normalizer import NameNormalizer

```

```

class DclgenRepository:
 """
 Repository wrapper for DCLGEN columns.

 Authority:
 - DCLGEN provides COBOL host variable spelling.
 - DCLGEN provides group names.
 - DCLGEN provides PIC clauses and usage.
 """

 def __init__(
 self,
 columns: list[DclgenColumn] | None = None,
) -> None:
 self.columns = columns or []

 def all(
 self,
) -> list[DclgenColumn]:
 return list(self.columns)

 def count(
 self,
) -> int:
 return len(self.columns)

 def tables(
 self,
) -> list[str]:
 values = [
 NameNormalizer.to_sql(column.table_name)
 for column in self.columns
 if NameNormalizer.to_sql(column.table_name)
]
 return self._unique(values)

 def groups(
 self,
) -> list[str]:
 values = [
 NameNormalizer.to_cobol(column.group_name)
 for column in self.columns
 if NameNormalizer.to_cobol(column.group_name)
]
 return self._unique(values)

 def columns_for_table(
 self,
 table_name: str,
) -> list[DclgenColumn]:
 target = NameNormalizer.to_sql(table_name)

 if not target:
 return []

 return [
 column
 for column in self.columns
 if NameNormalizer.to_sql(column.table_name) == target
]

 def column_names_for_table(
 self,
 table_name: str,
) -> list[str]:
 values = [
 NameNormalizer.to_sql(column.column_name)
 for column in self.columns_for_table(table_name)
 if NameNormalizer.to_sql(column.column_name)
]
 return self._unique(values)

 def group_for_table(
 self,
 table_name: str,

```

```

) -> str:
 for column in self.columns_for_table(table_name):
 group_name = NameNormalizer.to_cobol(column.group_name)

 if group_name:
 return group_name

 return ""

def find_column(
 self,
 table_name: str,
 column_name: str,
) -> DclgenColumn | None:
 target_column = NameNormalizer.to_sql(column_name)

 if not target_column:
 return None

 for column in self.columns_for_table(table_name):
 current_column = NameNormalizer.to_sql(column.column_name)

 if current_column == target_column:
 return column

 return None

def has_table(
 self,
 table_name: str,
) -> bool:
 target = NameNormalizer.to_sql(table_name)

 if not target:
 return False

 return target in self.tables()

def has_column(
 self,
 table_name: str,
 column_name: str,
) -> bool:
 return self.find_column(table_name, column_name) is not None

def host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 column = self.find_column(
 table_name=table_name,
 column_name=column_name,
)

 if not column:
 return ""

 return NameNormalizer.to_cobol(column.host_variable)

def host_reference_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 group_name = self.group_for_table(table_name)
 host_variable = self.host_for_column(
 table_name=table_name,
 column_name=column_name,
)

 if not group_name or not host_variable:
 return ""

 return (
 f"{HOST_REFERENCE_PREFIX}"

```

```

 f"{host_variable}"
 f"{HOST_REFERENCE_QUALIFIER}"
 f"{group_name}"
)

def pic_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 column = self.find_column(
 table_name=table_name,
 column_name=column_name,
)

 if not column:
 return ""

 return str(column.pic_clause or "").strip()

def usage_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 column = self.find_column(
 table_name=table_name,
 column_name=column_name,
)

 if not column:
 return ""

 return str(column.usage or "").strip()

def valid_host_references(
 self,
) -> set[str]:
 output: set[str] = set()

 for column in self.columns:
 table = NameNormalizer.to_sql(column.table_name)
 group = self.group_for_table(table)
 host = NameNormalizer.to_cobol(column.host_variable)

 if group and host:
 output.add(f"{group}.{host}")
 output.add(f"{group} {host}")
 output.add(f"{host} OF {group}")
 output.add(host)

 return output

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```
=====
FILE: src\idms_db2_phase2\repositories\mapping_repository.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from ..domain.models import SheetMappingRow
from ..reference.rules.conversion.name_resolution_rules import (
 MIN_SOURCE_FIELD_SIMILARITY,
)
from ..reference.rules.db2.update_sql_rules import (
 INSERT_ONLY_AUDIT_COLUMN_PREFIXES,
 KEY_TEXT_MARKERS,
 UPDATE_AUDIT_COLUMN_PREFIXES,
)
from ..services.name_normalizer import NameNormalizer

class MappingRepository:
 """
 Repository wrapper for Sheet Mapping rows.

 Authority:
 - Sheet Mapping provides DB2 table names.
 - Sheet Mapping provides DB2 column names.
 - Sheet Mapping source-field metadata resolves IDMS/copybook fields.

 This repository does not generate SQL and does not transform COBOL.
 """

 def __init__(
 self,
 rows: list[SheetMappingRow] | None = None,
) -> None:
 self.rows = rows or []

 def all(
 self,
) -> list[SheetMappingRow]:
 return list(self.rows)

 def count(
 self,
) -> int:
 return len(self.rows)

 def records(
 self,
) -> list[str]:
 values = [
 NameNormalizer.normalize(row.cobol_record_idms)
 for row in self.rows
 if NameNormalizer.normalize(row.cobol_record_idms)
]
 return self._unique(values)

 def tables(
 self,
) -> list[str]:
 values: list[str] = []

 for row in self.rows:
 table = self._row_db2_table(row)

 if table:
 values.append(table)

 return self._unique(values)

 def rows_for_record(
 self,
 record_name: str,
) -> list[SheetMappingRow]:
 target = NameNormalizer.normalize(record_name)
```

```

 if not target:
 return []

 return [
 row
 for row in self.rows
 if NameNormalizer.normalize(row.cobol_record_idms) == target
]

def rows_for_table(
 self,
 table_name: str,
) -> list[SheetMappingRow]:
 target = NameNormalizer.normalize(table_name)

 if not target:
 return []

 return [
 row
 for row in self.rows
 if NameNormalizer.normalize(self._row_db2_table(row)) == target
]

def db2_table_for_record(
 self,
 record_name: str,
) -> str:
 for row in self.rows_for_record(record_name):
 table_name = self._row_db2_table(row)

 if table_name:
 return table_name

 return ""

def db2_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 values = [
 self._row_db2_column(row)
 for row in self.rows_for_record(record_name)
 if self._row_db2_column(row)
]
 return self._unique(values)

def db2_columns_for_table(
 self,
 table_name: str,
) -> list[str]:
 values = [
 self._row_db2_column(row)
 for row in self.rows_for_table(table_name)
 if self._row_db2_column(row)
]
 return self._unique(values)

def db2_column_for_source_field(
 self,
 record_name: str,
 source_field_name: str,
) -> str:
 record_rows = self.rows_for_record(record_name)
 source = NameNormalizer.normalize(source_field_name)

 if not source:
 return ""

 exact_match = self._exact_source_field_match(
 rows=record_rows,
 source_field_name=source,
)

 if exact_match:
 return exact_match

```

```

 similar_match = self._similar_source_field_match(
 rows=record_rows,
 source_field_name=source,
)

 return similar_match

def key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 values: list[str] = []

 for row in self.rows_for_record(record_name):
 column = self._row_db2_column(row)

 if not column:
 continue

 if self._is_key_row(row) and not self._is_relationship_row(row):
 values.append(column)

 return self._unique(values)

def update_columns_for_record(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 changed_fields = [
 NameNormalizer.normalize(value)
 for value in changed_source_fields or []
 if NameNormalizer.normalize(value)
]

 values: list[str] = []

 for row in self.rows_for_record(record_name):
 column = self._row_db2_column(row)

 if not column:
 continue

 if self._is_key_row(row):
 continue

 if self._is_insert_only_audit_column(column):
 continue

 if self._is_update_audit_column(column):
 values.append(column)
 continue

 if not changed_fields:
 continue

 source_names = self._row_source_names(row)

 if any(source_name in changed_fields for source_name in source_names):
 values.append(column)

 return self._unique(values)

def relation_rows_for_record(
 self,
 record_name: str,
) -> list[SheetMappingRow]:
 return [
 row
 for row in self.rows_for_record(record_name)
 if self._is_relationship_row(row)
]

def table_for_column(
 self,

```

```

 column_name: str,
) -> str:
 target = NameNormalizer.to_sql(column_name)

 if not target:
 return ""

 for row in self.rows:
 column = self._row_db2_column(row)

 if NameNormalizer.to_sql(column) == target:
 return self._row_db2_table(row)

 return ""

def has_record(
 self,
 record_name: str,
) -> bool:
 return bool(self.rows_for_record(record_name))

def has_table(
 self,
 table_name: str,
) -> bool:
 return bool(self.rows_for_table(table_name))

def has_column_for_table(
 self,
 table_name: str,
 column_name: str,
) -> bool:
 target = NameNormalizer.to_sql(column_name)

 if not target:
 return False

 return target in {
 NameNormalizer.to_sql(column)
 for column in self.db2_columns_for_table(table_name)
 }

def record_summary(
 self,
) -> list[dict[str, object]]:
 output: list[dict[str, object]] = []

 for record in self.records():
 output.append(
 {
 "record_name": record,
 "db2_table": self.db2_table_for_record(record),
 "column_count": len(self.db2_columns_for_record(record)),
 "key_columns": self.key_columns_for_record(record),
 }
)

 return output

def column_summary(
 self,
) -> list[dict[str, object]]:
 output: list[dict[str, object]] = []

 for row in self.rows:
 record = NameNormalizer.normalize(row.cobol_record_ids)
 source = NameNormalizer.normalize(row.cobol_zone)
 table = self._row_db2_table(row)
 column = self._row_db2_column(row)

 if not any([record, source, table, column]):
 continue

 output.append(
 {
 "record_name": record,

```

```

 "source_field": source,
 "db2_table": table,
 "db2_column": column,
 "idms_key": NameNormalizer.normalize(row.idms_key),
 "db2_key": NameNormalizer.normalize(row.db2_key),
 }
)

return output

def relationship_summary(
 self,
) -> list[dict[str, object]]:
 output: list[dict[str, object]] = []

 for row in self.rows:
 relation = NameNormalizer.normalize(row.relation)

 if not relation:
 continue

 output.append(
 {
 "record_name": NameNormalizer.normalize(row.cobol_record_idms),
 "relation": relation,
 "db2_table": self._row_db2_table(row),
 "db2_column": self._row_db2_column(row),
 "cross_application_db2_table": NameNormalizer.to_sql(
 row.cross_application_db2_table
),
 "cross_application_db2_field_name": NameNormalizer.to_sql(
 row.cross_application_db2_field_name
),
 }
)

 return output

def _exact_source_field_match(
 self,
 rows: list[SheetMappingRow],
 source_field_name: str,
) -> str:
 for row in rows:
 source_names = self._row_source_names(row)

 if source_field_name in source_names:
 return self._row_db2_column(row)

 return ""

def _similar_source_field_match(
 self,
 rows: list[SheetMappingRow],
 source_field_name: str,
) -> str:
 best_column = ""
 best_score = 0.0

 for row in rows:
 column = self._row_db2_column(row)

 if not column:
 continue

 for source_name in self._row_source_names(row):
 score = NameNormalizer.similarity(source_field_name, source_name)

 if score > best_score:
 best_score = score
 best_column = column

 if best_score >= MIN_SOURCE_FIELD_SIMILARITY:
 return best_column

 return ""

```

```

def _row_db2_table(
 self,
 row: SheetMappingRow,
) -> str:
 return self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_record),
 NameNormalizer.to_sql(row.cross_application_db2_table),
)

def _row_db2_column(
 self,
 row: SheetMappingRow,
) -> str:
 return self._first_non_empty(
 NameNormalizer.to_sql(row.new_db2_field_name),
 NameNormalizer.to_sql(row.cross_application_db2_field_name),
)

def _row_source_names(
 self,
 row: SheetMappingRow,
) -> list[str]:
 values = [
 NameNormalizer.normalize(row.cobol_zone),
 NameNormalizer.normalize(row.reference_field_name_copybook),
]
 return self._unique([value for value in values if value])

def _is_key_row(
 self,
 row: SheetMappingRow,
) -> bool:
 text = " ".join(
 [
 NameNormalizer.normalize(row.idms_key),
 NameNormalizer.normalize(row.db2_key),
]
)

 return any(marker in text for marker in KEY_TEXT_MARKERS)

def _is_relationship_row(
 self,
 row: SheetMappingRow,
) -> bool:
 text = " ".join(
 [
 NameNormalizer.normalize(row.relation),
 NameNormalizer.normalize(row.db2_key),
]
)

 return "FOREIGN" in text or "RELATION" in text or "PARENT" in text

def _is_update_audit_column(
 self,
 column_name: str,
) -> bool:
 normalized = NameNormalizer.to_sql(column_name)
 return any(
 normalized.startswith(prefix)
 for prefix in UPDATE_AUDIT_COLUMN_PREFIXES
)

def _is_insert_only_audit_column(
 self,
 column_name: str,
) -> bool:
 normalized = NameNormalizer.to_sql(column_name)
 return any(
 normalized.startswith(prefix)
 for prefix in INSERT_ONLY_AUDIT_COLUMN_PREFIXES
)

def _first_non_empty(

```

```

 self,
 *values: str,
) -> str:
 for value in values:
 text = str(value or "").strip()

 if text:
 return text

 return ""

 def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\resolvers\column_name_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

class ColumnNameResolver:
 """
 Resolves DB2 column names.

 Authority:
 - Sheet Mapping provides DB2 column names.
 - DCLGEN validates whether a column exists for the resolved table.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

 def columns_for_record(
 self,
 record_name: str,
) -> list[str]:

```

```

) -> list[str]:
 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return []

 mapped_table = self.mapping_repository.db2_table_for_record(record_name)
 mapped_columns = self.mapping_repository.db2_columns_for_table(mapped_table)

 if not mapped_columns:
 mapped_columns = self.mapping_repository.db2_columns_for_record(record_name)

 return self._filter_to_dclgen_columns(
 table_name=table_name,
 columns=mapped_columns,
)

def column_for_field(
 self,
 record_name: str,
 source_field_name: str,
) -> str:
 mapped_column = self.mapping_repository.db2_column_for_source_field(
 record_name=record_name,
 source_field_name=source_field_name,
)

 if not mapped_column:
 return ""

 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return mapped_column

 if self.dclgen_repository.has_column(table_name, mapped_column):
 return NameNormalizer.to_sql(mapped_column)

 return ""

def key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return []

 key_columns = self.mapping_repository.key_columns_for_record(record_name)

 return self._filter_to_dclgen_columns(
 table_name=table_name,
 columns=key_columns,
)

def update_columns_for_record(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return []

 update_columns = self.mapping_repository.update_columns_for_record(
 record_name=record_name,
 changed_source_fields=changed_source_fields,
)

 return self._filter_to_dclgen_columns(
 table_name=table_name,
 columns=update_columns,
)

```

```

def _filter_to_dclgen_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in columns:
 normalized = NameNormalizer.to_sql(column)

 if not normalized:
 continue

 if not self.dclgen_repository.has_column(table_name, normalized):
 continue

 output.append(normalized)

 return self._unique(output)

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.to_sql(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_name_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.conversion.name_resolution_rules import (
 CURSOR_NAME_SUFFIX,
 DEFAULT_CURSOR_ORDER,
 TABLE_SUFFIX_ALTERNATES,
)
from ..services.name_normalizer import NameNormalizer

class CursorNameResolver:
 """
 Resolves generated DB2 cursor names from resolved DB2 table/view names.

 Cursor names are derived from DB2 table/view names, not IDMS set names.
 """

 def cursor_name_from_table(
 self,
 table_name: str,
 cursor_order: int = DEFAULT_CURSOR_ORDER,
) -> str:
 normalized_table = NameNormalizer.to_sql(table_name)

 if not normalized_table:
 return ""

 base_name = self._cursor_base_name(normalized_table)

```

```

 return f"{base_name}{CURSOR_NAME_SUFFIX}{cursor_order}"

def _cursor_base_name(
 self,
 table_name: str,
) -> str:
 for left_suffix, right_suffix in TABLE_SUFFIX_ALTERNATES:
 if table_name.endswith(left_suffix):
 return table_name[: -len(left_suffix)]

 if table_name.endswith(right_suffix):
 return table_name[: -len(right_suffix)]

 return table_name

=====
FILE: src\idms_db2_phase2\resolvers\host_variable_resolver.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..repositories.dclgen_repository import DclgenRepository
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

class HostVariableResolver:
 """
 Resolves COBOL host variables from DCLGEN metadata.

 Authority:
 - DCLGEN provides host variable spelling.
 - DCLGEN provides group names.
 """

 def __init__(
 self,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

 def host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return ""

 return self.dclgen_repository.host_for_column(
 table_name=resolved_table,
 column_name=column_name,
)

 def host_reference_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return ""

 return self.dclgen_repository.host_reference_for_column(
 table_name=resolved_table,
 column_name=column_name,
)

 def group_for_table(

```

```

 self,
 table_name: str,
) -> str:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return ""

 return self.dclgen_repository.group_for_table(resolved_table)

 def has_host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> bool:
 return bool(
 self.host_for_column(
 table_name=table_name,
 column_name=column_name,
)
)

 def host_references_for_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in columns:
 host_reference = self.host_reference_for_column(
 table_name=table_name,
 column_name=column,
)

 if host_reference:
 output.append(host_reference)

 return self._unique(output)

 def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(value)

 return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers\record_context_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..domain.models import IdmsOperation
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

```

```

class RecordContextResolver:

```

```

"""
Resolves active record context from source fields and IDMS operations.

This resolver does not rewrite COBOL and does not generate SQL.
"""

def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver

def record_for_field(
 self,
 field_name: str,
) -> str:
 source = NameNormalizer.normalize(field_name)

 if not source:
 return ""

 for record in self.mapping_repository.records():
 column = self.mapping_repository.db2_column_for_source_field(
 record_name=record,
 source_field_name=source,
)

 if column:
 return record

 return ""

def table_for_field(
 self,
 field_name: str,
) -> str:
 record = self.record_for_field(field_name)

 if not record:
 return ""

 return self.table_name_resolver.table_for_record(record)

def last_record_from_operations(
 self,
 operations: list[IdmsOperation],
) -> str:
 for operation in reversed(operations):
 record = NameNormalizer.normalize(operation.record_name)

 if record:
 return record

 return ""

def active_record(
 self,
 explicit_record: str = "",
 active_operations: list[IdmsOperation] | None = None,
 source_field_name: str = "",
) -> str:
 explicit = NameNormalizer.normalize(explicit_record)

 if explicit:
 return explicit

 if source_field_name:
 field_record = self.record_for_field(source_field_name)

 if field_record:
 return field_record

 if active_operations:
 return self.last_record_from_operations(active_operations)

```

```
return ""
```

```
=====
FILE: src\idms_db2_phase2\resolvers\tbl_name_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from ..reference.rules.conversion.name_resolution_rules import (
 TABLE_SUFFIX_ALTERNATES,
)
from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..services.name_normalizer import NameNormalizer
```

```
class TableNameResolver:
```

```
 """
```

```
 Resolves DB2 table/view names.
```

```
 Authority:
```

```
 - Sheet Mapping gives the intended DB2 table name.
```

```
 - DCLGEN decides which table/view name is available to generated SQL.
```

```
 """
```

```
 def __init__(
```

```
 self,
```

```
 mapping_repository: MappingRepository,
```

```
 dclgen_repository: DclgenRepository,
```

```
) -> None:
```

```
 self.mapping_repository = mapping_repository
```

```
 self.dclgen_repository = dclgen_repository
```

```
 def table_for_record(
```

```
 self,
```

```
 record_name: str,
```

```
) -> str:
```

```
 mapped_table = self.mapping_repository.db2_table_for_record(record_name)
```

```
 return self.resolve_table(mapped_table)
```

```
 def resolve_table(
```

```
 self,
```

```
 table_name: str,
```

```
) -> str:
```

```
 requested = NameNormalizer.to_sql(table_name)
```

```
 if not requested:
```

```
 return ""
```

```
 if self.dclgen_repository.has_table(requested):
```

```
 return requested
```

```
 compact_requested = NameNormalizer.compact(requested)
```

```
 for available_table in self.dclgen_repository.tables():
```

```
 if NameNormalizer.compact(available_table) == compact_requested:
```

```
 return available_table
```

```
 alternate = self._alternate_suffix_table(requested)
```

```
 if alternate and self.dclgen_repository.has_table(alternate):
```

```
 return alternate
```

```
 compact_alternate = NameNormalizer.compact(alternate)
```

```
 if compact_alternate:
```

```
 for available_table in self.dclgen_repository.tables():
```

```
 if NameNormalizer.compact(available_table) == compact_alternate:
```

```
 return available_table
```

```
 return ""
```

```
 def has_resolved_table(
```

```

 self,
 table_name: str,
) -> bool:
 return bool(self.resolve_table(table_name))

 def _alternate_suffix_table(
 self,
 table_name: str,
) -> str:
 requested = NameNormalizer.to_sql(table_name)

 if not requested:
 return ""

 for source_suffix, target_suffix in TABLE_SUFFIX_ALTERNATES:
 if requested.endswith(source_suffix):
 return requested[: -len(source_suffix)] + target_suffix

 return ""

=====
FILE: src\idms_db2_phase2\resolvers\update_sql_plan_resolver.py
ENCODING: utf-8
=====

from __future__ import annotations

from dataclasses import dataclass, field

from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.column_name_resolver import ColumnNameResolver
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

@dataclass(slots=True)
class UpdateSqlPlan:
 """
 Conservative UPDATE SQL metadata plan.

 This object carries resolved table, key columns, update columns,
 and diagnostics. It does not generate SQL text.
 """

 record_name: str = ""
 table_name: str = ""
 key_columns: list[str] = field(default_factory=list)
 update_columns: list[str] = field(default_factory=list)
 diagnostics: list[str] = field(default_factory=list)

 @property
 def is_complete(
 self,
) -> bool:
 return bool(
 self.record_name
 and self.table_name
 and self.key_columns
 and self.update_columns
)

class UpdateSqlPlanResolver:
 """
 Resolves conservative UPDATE SQL plans.

 Rules:
 - Sheet Mapping decides DB2 table and column names.
 - DCLGEN decides whether host variables exist.
 - DB2 primary-key metadata decides WHERE columns.
 - Composite primary keys are supported.
 - Relationship columns are not included in WHERE unless marked as key.
 - UPDATE columns come from changed source fields and update audit fields.

```

"""

```
def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver

def resolve(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> UpdateSqlPlan:
 record = NameNormalizer.normalize(record_name)
 plan = UpdateSqlPlan(record_name=record)

 if not record:
 plan.diagnostics.append("Update SQL plan: record name is empty.")
 return plan

 resolved_table = self.table_name_resolver.table_for_record(record)

 if not resolved_table:
 plan.diagnostics.append(
 f"Update SQL plan: no resolved DB2 table for record {record}."
)
 return plan

 plan.table_name = resolved_table

 key_columns = self.column_name_resolver.key_columns_for_record(record)

 if not key_columns:
 plan.diagnostics.append(
 f"Update SQL plan: no key columns resolved for record {record}."
)
 return plan

 plan.key_columns = self._columns_with_hosts(
 table_name=resolved_table,
 columns=key_columns,
)

 if not plan.key_columns:
 plan.diagnostics.append(
 f"Update SQL plan: no DCLGEN host variables for key columns "
 f"on table {resolved_table}."
)
 return plan

 update_columns = self.column_name_resolver.update_columns_for_record(
 record_name=record,
 changed_source_fields=changed_source_fields,
)

 if not update_columns:
 plan.diagnostics.append(
 f"Update SQL plan: no update columns resolved for record {record}."
)
 return plan

 plan.update_columns = self._columns_with_hosts(
 table_name=resolved_table,
 columns=update_columns,
)

 if not plan.update_columns:
 plan.diagnostics.append(
```

```

 f"Update SQL plan: no DCLGEN host variables for update columns "
 f"on table {resolved_table}."
)

 return plan

def _columns_with_hosts(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in columns:
 host_reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=column,
)

 if not host_reference:
 continue

 output.append(NameNormalizer.to_sql(column))

 return self._unique(output)

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.to_sql(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\services__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\services\cobol_area_alignment_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.normalization.final_cobol_patterns import (
 DIVISION_SECTION_HEADER_PATTERN,
 END_EXEC_PATTERN,
 EXEC_SQL_START_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_TOKEN_PATTERN,
)
from ..reference.rules.cobol.fixed_format_rules import (
 AREA_B_BODY_INDENT,
 SQL_BODY_INDENT,
)
from ..reference.rules.cobol.statement_classification_rules import (
 NON_PARAGRAPH_SINGLE_WORDS,

```

```

)
from .fixed_format_line_service import FixedFormatLineService

class CobolAreaAlignmentService:
 """
 Aligns Procedure Division executable statements to Area B when safe.

 Safety:
 - Does not change 80-column sequencing.
 - Does not change business logic.
 - Does not blindly truncate fixed-format body content.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 def align(
 self,
 text: str,
) -> str:
 source = str(text or "")

 if not source:
 return ""

 output: list[str] = []
 in_procedure = False
 in_exec_sql = False

 for line in source.splitlines():
 logical = self._logical(line)

 if PROCEDURE_DIVISION_TOKEN_PATTERN.match(logical):
 in_procedure = True
 output.append(line)
 continue

 if DIVISION_SECTION_HEADER_PATTERN.match(logical) and not PROCEDURE_DIVISION_TOKEN_PATTERN.match(logical):
 if logical.upper().endswith("DIVISION."):
 in_procedure = False

 if not in_procedure:
 output.append(line)
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 output.append(self._replace_body_indent(line, SQL_BODY_INDENT))
 continue

 if END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 output.append(self._replace_body_indent(line, SQL_BODY_INDENT))
 continue

 if in_exec_sql:
 output.append(self._replace_body_indent(line, SQL_BODY_INDENT))
 continue

 if self._is_area_a(logical):
 output.append(line)
 continue

 if self.fixed_format.is_comment_line(line):
 output.append(line)
 continue

 output.append(self._replace_body_indent(line, AREA_B_BODY_INDENT))

 return "\n".join(output).rstrip() + "\n"

 def _replace_body_indent(

```

```

 self,
 line: str,
 indent: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 body = self.fixed_format.body(line).strip()

 if not body:
 return line

 return self.fixed_format.replace_body(line, f"{indent}{body}")

 stripped = str(line or "").strip()

 if not stripped:
 return line

 return f"{indent}{stripped}"

def _is_area_a(
 self,
 logical: str,
) -> bool:
 text = str(logical or "").strip()
 upper = text.upper()

 if not text:
 return False

 if DIVISION_SECTION_HEADER_PATTERN.match(text):
 return True

 if not PARAGRAPH_HEADER_PATTERN.match(text):
 return False

 paragraph_name = upper.rstrip(".").strip()

 if paragraph_name in NON_PARAGRAPH_SINGLE_WORDS:
 return False

 return True

def _logical(
 self,
 line: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 return self.fixed_format.body(line).strip()

 return str(line or "").strip()

```

```

=====
FILE: src\idms_db2_phase2\services\cursor_select_minimizer_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.cobol.sequence_patterns import strip_sequence_numbers
from ..reference.patterns.normalization.final_cobol_patterns import (
 ASC_DESC_PATTERN,
 DECLARE_CURSOR_PATTERN,
 FETCH_CURSOR_PATTERN,
 FOR_READ_ONLY_PATTERN,
 FROM_PATTERN,
 HOST_REFERENCE_PATTERN,
 ORDER_BY_PATTERN,
 SELECT_ITEM_PATTERN,
 SELECT_KEYWORD_PATTERN,
)
from ..reference.rules.normalization.final_cobol_normalization_rules import (
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
)
from .fixed_format_line_service import FixedFormatLineService

```

```

class CursorSelectMinimizerService:
 """
 Removes order-by-only columns from cursor SELECT/FETCH when safe.

 Rules from Phase 2:
 - Remove ORDER-BY-only columns from cursor SELECT and FETCH when safe.
 - Keep the ORDER BY clause itself.
 - Keep the ORDER BY column line itself.
 - SELECT comma normalization is scoped only to SELECT item lines between
 SELECT and FROM.
 - FETCH comma normalization is scoped only to host variables between
 INTO and END-EXEC.
 - SQL keywords such as WHERE, FROM, ORDER BY, and FOR READ ONLY are never
 treated as selected columns.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 require_order_by_columns_in_select: bool = ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.require_order_by_columns_in_select = require_order_by_columns_in_select

 def minimize(
 self,
 text: str,
) -> str:
 source = str(text or "")

 if not source:
 return ""

 if self.require_order_by_columns_in_select:
 return source

 lines = source.splitlines()
 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 if not self._is_exec_sql_start(logical):
 output.append(lines[index])
 index += 1
 continue

 block, next_index = self._collect_exec_sql_block(lines, index)

 if self._is_cursor_declare_block(block):
 output.extend(self._minimize_declare_block(block))
 index = next_index
 continue

 if self._is_fetch_block(block):
 output.extend(self._minimize_fetch_block(block))
 index = next_index
 continue

 output.extend(block)
 index = next_index

 return "\n".join(output).rstrip() + "\n"

 def _collect_exec_sql_block(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 output: list[str] = []
 index = start_index

 while index < len(lines):
 output.append(lines[index])

```

```

 logical = self._logical(lines[index])
 index += 1

 if self._is_exec_sql_end(logical):
 break

 return output, index

def _is_cursor_declare_block(
 self,
 block: list[str],
) -> bool:
 return any(
 DECLARE_CURSOR_PATTERN.search(self._logical(line))
 for line in block
)

def _is_fetch_block(
 self,
 block: list[str],
) -> bool:
 return any(
 FETCH_CURSOR_PATTERN.search(self._logical(line))
 for line in block
)

def _minimize_declare_block(
 self,
 block: list[str],
) -> list[str]:
 order_columns = self._order_by_columns(block)

 if not order_columns:
 return block

 select_columns = self._select_columns(block)

 if not select_columns:
 return block

 removable_columns = [
 column
 for column in order_columns
 if column in select_columns and len(select_columns) > 1
]

 if not removable_columns:
 return block

 output: list[str] = []
 inside_select = False

 for line in block:
 logical = self._logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 inside_select = True
 output.append(line)
 continue

 if inside_select and FROM_PATTERN.match(logical):
 inside_select = False
 output.append(line)
 continue

 if inside_select:
 column = self._line_column(line)

 if column in removable_columns:
 continue

 output.append(line)
 continue

 output.append(line)

```

```

 return self._normalize_select_commas(output)

def _minimize_fetch_block(
 self,
 block: list[str],
) -> list[str]:
 """
 Placeholder-safe FETCH minimization.

 Batch 8 only needs SELECT minimization. Full FETCH position-sync
 minimization will be handled when cursor SELECT/FETCH specs are carried
 through the full pipeline.
 """
 return block

def _normalize_select_commas(
 self,
 block: list[str],
) -> list[str]:
 output: list[str] = []
 select_item_indexes: list[int] = []
 inside_select = False

 for index, line in enumerate(block):
 logical = self._logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 inside_select = True
 output.append(line)
 continue

 if inside_select and FROM_PATTERN.match(logical):
 inside_select = False
 output.append(line)
 continue

 if inside_select:
 column = self._line_column(line)

 if column:
 select_item_indexes.append(len(output))

 output.append(line)
 continue

 output.append(line)

 for position, output_index in enumerate(select_item_indexes):
 line = output[output_index]
 column = self._line_column(line)

 if not column:
 continue

 indent = self._leading_spaces(self._body(line))
 comma = "," if position < len(select_item_indexes) - 1 else ""
 rebuilt_body = f"{indent}{column}{comma}"
 output[output_index] = self._replace_body(line, rebuilt_body)

 return output

def _select_columns(
 self,
 block: list[str],
) -> list[str]:
 output: list[str] = []
 inside_select = False

 for line in block:
 logical = self._logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 inside_select = True
 continue

 if inside_select and FROM_PATTERN.match(logical):

```

```

 break

 if inside_select:
 column = self._line_column(line)

 if column:
 output.append(column)

 return output

def _order_by_columns(
 self,
 block: list[str],
) -> list[str]:
 output: list[str] = []
 inside_order = False

 for line in block:
 logical = self._logical(line)

 if ORDER_BY_PATTERN.match(logical):
 inside_order = True
 continue

 if inside_order and FOR_READ_ONLY_PATTERN.match(logical):
 break

 if inside_order:
 column = self._line_column(line)

 if column:
 output.append(column)

 return output

def _line_column(
 self,
 line: str,
) -> str:
 logical = self._logical(line)
 logical = ASC_DESC_PATTERN.sub("", logical).strip()
 match = SELECT_ITEM_PATTERN.match(logical)

 if not match:
 return ""

 column = match.group("column").upper()

 if column in {
 "SELECT",
 "FROM",
 "WHERE",
 "ORDER",
 "GROUP",
 "HAVING",
 "FETCH",
 "INTO",
 "FOR",
 "READ",
 "ONLY",
 "END-EXEC",
 }:
 return ""

 return column

def _is_exec_sql_start(
 self,
 logical: str,
) -> bool:
 return logical.upper().startswith("EXEC SQL")

def _is_exec_sql_end(
 self,
 logical: str,
) -> bool:

```

```

 upper = logical.upper()
 return upper == "END-EXEC" or upper == "END-EXEC."

def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

def _body(
 self,
 line: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 return self.fixed_format.body(line)

 return str(line or "")

def _replace_body(
 self,
 line: str,
 body: str,
) -> str:
 if self.fixed_format.is_fixed_line(line):
 return self.fixed_format.replace_body(line, body)

 return body

def _leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

```

```

=====
FILE: src\idms_db2_phase2\services\db2_date_format_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.normalization.final_cobol_normalization_rules import (
 DATE_FORMAT_ALIASES,
 DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
 SUPPORTED_DB2_DATE_FORMATS,
)

class Db2DateFormatService:
 """
 Applies configured generated DB2 date external format.

 This service only changes generated shared date literals and helper names.
 It does not inspect business records or table names.
 """

 def __init__(
 self,
 date_external_format: str = DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
) -> None:
 self.date_external_format = self._normalize_format(date_external_format)

 def apply(
 self,

```

```

 text: str,
) -> str:
 output = str(text or "")

 if not output:
 return ""

 if self.date_external_format == "DD.MM.YYYY":
 return output

 if self.date_external_format == "YYYY-MM-DD":
 output = output.replace("DA-DD-MM-CCYY", "DA-YYYY-MM-DD")
 output = output.replace("'01.01.0001'", "'0001-01-01'")
 output = output.replace("'31.12.9999'", "'9999-12-31'")
 return output

 if self.date_external_format == "YYYYMMDD":
 output = output.replace("DA-DD-MM-CCYY", "DA-YYYYMMDD")
 output = output.replace("'01.01.0001'", "'00010101'")
 output = output.replace("'31.12.9999'", "'99991231'")
 return output

 return output

def _normalize_format(
 self,
 value: str,
) -> str:
 requested = str(value or "").strip().upper()

 if not requested:
 return DEFAULT_DB2_DATE_EXTERNAL_FORMAT

 normalized = DATE_FORMAT_ALIASES.get(requested, requested)

 if normalized not in SUPPORTED_DB2_DATE_FORMATS:
 return DEFAULT_DB2_DATE_EXTERNAL_FORMAT

 return normalized

```

```

=====
FILE: src\idms_db2_phase2\services\final_sequence_resequencer_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.cobol.fixed_format_rules import (
 DEFAULT_LEFT_START,
 DEFAULT_LEFT_STEP,
 DEFAULT_RIGHT_START,
 DEFAULT_RIGHT_STEP,
)
from .fixed_format_line_service import FixedFormatLineService

class FinalSequenceResequencerService:
 """
 Resequences final manual-style COBOL output.

 Generic behavior:
 - Resequences already fixed-format 80-column COBOL lines.
 - Preserves column 7 indicator.
 - Preserves columns 8-72 COBOL body.
 - Rewrites only columns 1-6 and 73-80.
 - Does not reformat or wrap COBOL statements.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 def resequence(

```

```

self,
text: str,
left_start: int = DEFAULT_LEFT_START,
left_step: int = DEFAULT_LEFT_STEP,
right_start: int = DEFAULT_RIGHT_START,
right_step: int = DEFAULT_RIGHT_STEP,
) -> str:
 source = str(text or "")

 if not source:
 return ""

 output: list[str] = []
 left_value = int(left_start)
 right_value = int(right_start)

 for line in source.splitlines():
 if not self.fixed_format.is_fixed_line(line):
 output.append(line)
 continue

 _left, indicator, body, _right = self.fixed_format.split(line)
 output.append(
 self.fixed_format.build(
 body=body.rstrip(),
 left_sequence=left_value,
 right_sequence=right_value,
 indicator=indicator,
)
)

 left_value += int(left_step)
 right_value += int(right_step)

 return "\n".join(output).rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\services\fixed_format_line_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from ..reference.rules.cobol.fixed_format_rules import (
 BODY_END_INDEX,
 BODY_START_INDEX,
 BODY_WIDTH,
 COMMENT_INDICATOR,
 DEBUG_INDICATOR,
 INDICATOR_INDEX,
 LEFT_SEQUENCE_END_INDEX,
 LEFT_SEQUENCE_START_INDEX,
 PAGE_INDICATOR,
 RIGHT_SEQUENCE_END_INDEX,
 RIGHT_SEQUENCE_START_INDEX,
 SPACE_INDICATOR,
 TOTAL_WIDTH,
 VALID_INDICATORS,
)

```

```

class FixedFormatLineService:
 """

```

Fixed-format COBOL line helper service.

Purpose:

- Preserve existing left sequence, indicator, body, and right sequence layout.
- Allow callers to update only the COBOL body area safely.
- Build valid 80-column manual-style lines.

No regex.

No business hardcoding.

"""

```

TOTAL_WIDTH = TOTAL_WIDTH

```

```

BODY_WIDTH = BODY_WIDTH

def is_fixed_line(
 self,
 line: str,
) -> bool:
 text = str(line or "")

 if len(text) != TOTAL_WIDTH:
 return False

 left = text[LEFT_SEQUENCE_START_INDEX:LEFT_SEQUENCE_END_INDEX]
 right = text[RIGHT_SEQUENCE_START_INDEX:RIGHT_SEQUENCE_END_INDEX]

 if not left.isdigit():
 return False

 if not right.isdigit():
 return False

 return True

def split(
 self,
 line: str,
) -> tuple[str, str, str, str]:
 text = str(line or "")

 if self.is_fixed_line(text):
 return (
 text[LEFT_SEQUENCE_START_INDEX:LEFT_SEQUENCE_END_INDEX],
 text[INDICATOR_INDEX : INDICATOR_INDEX + 1],
 text[BODY_START_INDEX:BODY_END_INDEX],
 text[RIGHT_SEQUENCE_START_INDEX:RIGHT_SEQUENCE_END_INDEX],
)

 return (
 "",
 SPACE_INDICATOR,
 text.rstrip(),
 "",
)

def body(
 self,
 line: str,
) -> str:
 _left, _indicator, body, _right = self.split(line)
 return body.rstrip()

def indicator(
 self,
 line: str,
) -> str:
 _left, indicator, _body, _right = self.split(line)
 return indicator

def is_comment_line(
 self,
 line: str,
) -> bool:
 _left, indicator, body, _right = self.split(line)

 if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
 return True

 stripped_body = body.strip()

 if stripped_body.startswith("*"):
 return True

 if stripped_body.startswith("/"):
 return True

 return False

```

```

def build(
 self,
 body: str,
 left_sequence: int | str = "",
 right_sequence: int | str = "",
 indicator: str = SPACE_INDICATOR,
) -> str:
 left = self._sequence_text(left_sequence, width=6)
 right = self._sequence_text(right_sequence, width=8)
 safe_indicator = self._safe_indicator(indicator)
 safe_body = str(body or "").rstrip()

 if len(safe_body) > BODY_WIDTH:
 safe_body = safe_body[:BODY_WIDTH]

 return f"{left}{safe_indicator}{safe_body.ljust(BODY_WIDTH)}{right}"

def replace_body(
 self,
 line: str,
 body: str,
) -> str:
 left, indicator, _old_body, right = self.split(line)

 if not left:
 return str(body or "").rstrip()

 return self.build(
 body=body,
 left_sequence=left,
 right_sequence=right,
 indicator=indicator,
)

def build_comment(
 self,
 comment_body: str,
 left_sequence: int | str = "",
 right_sequence: int | str = "",
) -> str:
 text = str(comment_body or "").strip()

 if not text.startswith("#"):
 text = f"# {text}"

 return self.build(
 body=text,
 left_sequence=left_sequence,
 right_sequence=right_sequence,
 indicator=COMMENT_INDICATOR,
)

def leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

def _safe_indicator(
 self,
 indicator: str,
) -> str:
 value = str(indicator or SPACE_INDICATOR)[:1]

 if value == "d":

```

```

 return DEBUG_INDICATOR

 if value not in VALID_INDICATORS:
 return SPACE_INDICATOR

 return value

def _sequence_text(
 self,
 value: int | str,
 width: int,
) -> str:
 text = str(value or "").strip()

 if not text:
 return " " * width

 if text.isdigit():
 return f"{int(text):0{width}d}"[-width:]

 return text[:width].rjust(width)

=====
FILE: src\idms_db2_phase2\services\include_placement_normalization_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..reference.patterns.normalization.include_placement_patterns import (
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 INCLUDE_BODY_PATTERN,
 INCLUDE_INLINE_PATTERN,
 LINKAGE_SECTION_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 WORKING_STORAGE_PATTERN,
)
from ..reference.rules.normalization.include_placement_rules import (
 SYSTEM_INCLUDE_NAMES,
)
from .fixed_format_line_service import FixedFormatLineService
from .name_normalizer import NameNormalizer

class IncludePlacementNormalizationService:
 """
 Normalizes DB2 INCLUDE block placement.

 Responsibilities:
 - Remove duplicate DCLGEN includes after LINKAGE SECTION.
 - Remove duplicate DCLGEN includes inside PROCEDURE DIVISION.
 - Preserve SQLERROR include paragraph.
 - Preserve first valid include occurrence.

 This service does not define regex patterns and does not hardcode
 business names.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []

 def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

```

```

lines = source.splitlines()
output: list[str] = []
known_includes: set[str] = set()
after_linkage = False
in_procedure = False
index = 0
changed = False

while index < len(lines):
 logical = self._logical(lines[index])

 if LINKAGE_SECTION_PATTERN.match(logical):
 after_linkage = True
 output.append(lines[index].rstrip())
 index += 1
 continue

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure = True
 output.append(lines[index].rstrip())
 index += 1
 continue

 include_name = self._inline_include_name(logical)

 if include_name:
 if self._should_remove_include(
 include_name=include_name,
 known_includes=known_includes,
 after_linkage=after_linkage,
 in_procedure=in_procedure,
):
 changed = True
 index += 1
 continue

 known_includes.add(include_name)
 output.append(lines[index].rstrip())
 index += 1
 continue

 block, next_index, block_include = self._collect_include_block(
 lines=lines,
 start_index=index,
)

 if block_include:
 if self._should_remove_include(
 include_name=block_include,
 known_includes=known_includes,
 after_linkage=after_linkage,
 in_procedure=in_procedure,
):
 changed = True
 index = next_index
 continue

 known_includes.add(block_include)
 output.extend(line.rstrip() for line in block)
 index = next_index
 continue

 output.append(lines[index].rstrip())
 index += 1

if changed:
 self.messages.append(
 "Include placement normalization: removed misplaced duplicate INCLUDE block."
)

return "\n".join(output).rstrip() + "\n"

def _should_remove_include(
 self,
 include_name: str,

```

```

 known_includes: set[str],
 after_linkage: bool,
 in_procedure: bool,
) -> bool:
 normalized = NameNormalizer.normalize(include_name)

 if not normalized:
 return False

 if normalized in SYSTEM_INCLUDE_NAMES:
 return False

 if normalized in known_includes:
 return True

 if after_linkage or in_procedure:
 return True

 return False

def _collect_include_block(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int, str]:
 if start_index >= len(lines):
 return [], start_index, ""

 first_logical = self._logical(lines[start_index])

 if not EXEC_SQL_START_PATTERN.match(first_logical):
 return [], start_index, ""

 block: list[str] = []
 include_name = ""
 index = start_index

 while index < len(lines):
 block.append(lines[index])
 logical = self._logical(lines[index])

 include_match = INCLUDE_BODY_PATTERN.match(logical)

 if include_match:
 include_name = NameNormalizer.normalize(
 include_match.group("include")
)

 index += 1

 if EXEC_SQL_END_PATTERN.match(logical):
 break

 if not include_name:
 return [], start_index, ""

 return block, index, include_name

def _inline_include_name(
 self,
 logical: str,
) -> str:
 match = INCLUDE_INLINE_PATTERN.match(logical)

 if not match:
 return ""

 return NameNormalizer.normalize(match.group("include"))

def _logical(
 self,
 line: str,
) -> str:
 if hasattr(self.fixed_format, "logical"):
 return self.fixed_format.logical(line)

```

```

raw = str(line or "")

if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

return raw.rstrip()

```

```

=====
FILE: src\idms_db2_phase2\services\name_normalizer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re
from difflib import SequenceMatcher

```

```

class NameNormalizer:
 """
 Shared name normalization utility.

 This service is generic and must not contain business records,
 DB2 table names, DB2 columns, DCLGEN names, or host variables.
 """

 @staticmethod
 def normalize(
 value: object,
) -> str:
 if value is None:
 return ""

 text = str(value)
 text = text.replace("\uffeff", "")
 text = text.replace("\xa0", " ")
 text = text.strip().upper()

 while " " in text:
 text = text.replace(" ", " ")

 return text

 @staticmethod
 def compact(
 value: object,
) -> str:
 text = NameNormalizer.normalize(value)
 return re.sub(r"^[A-Z0-9]", "", text)

 @staticmethod
 def to_cobol(
 value: object,
) -> str:
 text = NameNormalizer.normalize(value)
 text = text.replace("_", "-")
 text = re.sub(r"^[A-Z0-9-]+", "-", text)
 text = re.sub(r"-+", "-", text)
 return text.strip("-")

 @staticmethod
 def to_sql(
 value: object,
) -> str:
 text = NameNormalizer.normalize(value)
 text = text.replace("-", "_")
 text = re.sub(r"^[A-Z0-9_]+", "_", text)
 text = re.sub(r"_+", "_", text)
 return text.strip("_")

 @staticmethod
 def remove_dcl_prefix(
 value: object,
) -> str:
 text = NameNormalizer.normalize(value)

```

```

 if text.startswith("DCL"):
 return text[3:]

 return text

 @staticmethod
 def remove_record_suffix(
 value: object,
) -> str:
 text = NameNormalizer.normalize(value)
 return re.sub(r"[_-]\d{4}$", "", text)

 @staticmethod
 def similarity(
 left: object,
 right: object,
) -> float:
 return SequenceMatcher(
 None,
 NameNormalizer.compact(left),
 NameNormalizer.compact(right),
).ratio()

=====
FILE: src\idms_db2_phase2\services\procedure_indentation_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..reference.patterns.normalization.procedure_indentation_patterns import (
 BLANK_PATTERN,
 COMMENT_PATTERN,
 DIVISION_HEADER_PATTERN,
 ELSE_PATTERN,
 END_EVALUATE_PATTERN,
 END_IF_PATTERN,
 END_PROGRAM_PATTERN,
 EVALUATE_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 IF_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 SECTION_HEADER_PATTERN,
 WHEN_PATTERN,
)
from ..reference.rules.normalization.procedure_indentation_rules import (
 AREA_A_INDENT,
 AREA_B_INDENT,
 MAX_COBOL_BODY_WIDTH,
 NESTED_BODY_INDENT,
 SQL_BODY_INDENT,
)

class ProcedureIndentationService:
 """
 Normalizes Procedure Division indentation.

 This service keeps paragraph and section headers in Area A, aligns
 executable statements in Area B, and keeps generated IF/EVALUATE/SQL
 blocks readable.

 It does not define regex patterns and does not hardcode business names.
 """

 def __init__(self) -> None:
 self.messages: list[str] = []

 def normalize(
 self,
 text: str,
) -> str:

```

```

self.messages = []
source = str(text or "")

if not source:
 return ""

lines = source.splitlines()
output: list[str] = []
in_procedure = False
in_exec_sql = False
block_depth = 0
changed = False

for line in lines:
 logical = self._logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure = True
 block_depth = 0
 output.append(self._with_body(line, logical.strip()))
 continue

 if in_procedure and DIVISION_HEADER_PATTERN.match(logical):
 if not PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure = False
 block_depth = 0

 if not in_procedure:
 output.append(line.rstrip())
 continue

 if BLANK_PATTERN.match(logical):
 output.append("")
 continue

 if COMMENT_PATTERN.match(logical):
 output.append(self._with_body(line, logical.strip()))
 continue

 if SECTION_HEADER_PATTERN.match(logical):
 output.append(self._with_body(line, f"{AREA_A_INDENT}{logical.strip()}"))
 continue

 if self._is_paragraph_header(logical):
 output.append(self._with_body(line, f"{AREA_A_INDENT}{logical.strip()}"))
 block_depth = 0
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 new_body = f"{AREA_B_INDENT}EXEC SQL"
 output.append(self._with_body(line, new_body))
 changed = changed or new_body != logical
 continue

 if EXEC_SQL_END_PATTERN.match(logical):
 in_exec_sql = False
 new_body = f"{AREA_B_INDENT}END-EXEC."
 output.append(self._with_body(line, new_body))
 changed = changed or new_body != logical
 continue

 if in_exec_sql:
 new_body = self._sql_body(logical)
 output.append(self._with_body(line, new_body))
 changed = changed or new_body != logical
 continue

 normalized, block_depth = self._normalize_procedure_statement(
 logical=logical,
 block_depth=block_depth,
)
 output.append(self._with_body(line, normalized))
 changed = changed or normalized != logical

if changed:

```

```

 self.messages.append(
 "Procedure indentation normalization: aligned Procedure Division statements."
)

 return "\n".join(output).rstrip() + "\n"

def _normalize_procedure_statement(
 self,
 logical: str,
 block_depth: int,
) -> tuple[str, int]:
 stripped = logical.strip()
 current_depth = max(block_depth, 0)

 if END_IF_PATTERN.match(stripped):
 current_depth = max(current_depth - 1, 0)
 return f"{self._indent(current_depth)}END-IF.", current_depth

 if END_EVALUATE_PATTERN.match(stripped):
 current_depth = max(current_depth - 1, 0)
 return f"{self._indent(current_depth)}END-EVALUATE", current_depth

 if ELSE_PATTERN.match(stripped):
 effective_depth = max(current_depth - 1, 0)
 return f"{self._indent(effective_depth)}ELSE", current_depth

 if WHEN_PATTERN.match(stripped):
 effective_depth = max(current_depth - 1, 0)
 return f"{self._indent(effective_depth)}{stripped}", current_depth

 if IF_PATTERN.match(stripped):
 normalized = f"{self._indent(current_depth)}{stripped}"
 return normalized, current_depth + 1

 if EVALUATE_PATTERN.match(stripped):
 normalized = f"{self._indent(current_depth)}{stripped}"
 return normalized, current_depth + 1

 if END_PROGRAM_PATTERN.match(stripped):
 return f"{AREA_A_INDENT}{stripped}", current_depth

 return f"{self._indent(current_depth)}{stripped}", current_depth

def _sql_body(
 self,
 logical: str,
) -> str:
 stripped = logical.strip()
 upper = stripped.upper()

 if upper.startswith(("UPDATE ", "INSERT ", "DELETE ", "SELECT ")):
 return f"{SQL_BODY_INDENT}{stripped}"

 if upper == "SET" or upper.startswith("SET "):
 return f"{SQL_BODY_INDENT} {stripped}"

 if upper == "WHERE" or upper.startswith("WHERE "):
 return f"{SQL_BODY_INDENT} {stripped}"

 if upper.startswith("AND "):
 return f"{SQL_BODY_INDENT} {stripped}"

 if "=" in stripped:
 return f"{SQL_BODY_INDENT} {stripped}"

 return f"{SQL_BODY_INDENT}{stripped}"

def _indent(
 self,
 depth: int,
) -> str:
 if depth <= 0:
 return AREA_B_INDENT

 return AREA_B_INDENT + (NESTED_BODY_INDENT * depth)

```

```

def _is_paragraph_header(
 self,
 logical: str,
) -> bool:
 stripped = logical.strip()

 if not stripped:
 return False

 if not PARAGRAPH_HEADER_PATTERN.match(stripped):
 return False

 upper = stripped.upper()

 if upper in {
 "CONTINUE.",
 "ELSE",
 "END-IF.",
 "END-EVALUATE",
 "END-EXEC.",
 "STOP RUN.",
 }:
 return False

 if " " in stripped:
 return False

 return stripped.endswith(".")

def _logical(
 self,
 line: str,
) -> str:
 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

def _with_body(
 self,
 original_line: str,
 body: str,
) -> str:
 line = str(original_line or "")
 normalized_body = str(body or "").rstrip()

 if len(line) >= 7 and line[:6].isdigit():
 prefix = line[:7]
 suffix = line[72:] if len(line) > 72 else ""
 return f"{prefix}{normalized_body[:MAX_COBOL_BODY_WIDTH].ljust(MAX_COBOL_BODY_WIDTH)}{suffix}"

 return normalized_body

```

```

=====
FILE: src\idms_db2_phase2\services\program_name_sync_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.normalization.final_cobol_patterns import (
 MOVE_TO_PROGRAM_NAME_PATTERN,
 PROGRAM_ID_PATTERN,
)

```

```

class ProgramNameSyncService:
 """
 Synchronizes PROGRAM-NAME from final PROGRAM-ID.

 This service rewrites only safe literal MOVE statements to PROGRAM-NAME.
 It does not globally replace program names.
 """

```

```

def sync(
 self,
 text: str,
) -> str:
 source = str(text or "")

 program_match = PROGRAM_ID_PATTERN.search(source)

 if not program_match:
 return source

 final_program_id = program_match.group("program").upper()

 def replace_program_name(match) -> str:
 return (
 match.group("prefix")
 + final_program_id
 + match.group("suffix")
)

 return MOVE_TO_PROGRAM_NAME_PATTERN.sub(
 replace_program_name,
 source,
)

=====
FILE: src\idms_db2_phase2\services\retrieval_statement_normalization_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from typing import Any

from ..reference.patterns.normalization.retrieval_statement_patterns import (
 DCL_GROUP_DOT_REFERENCE_PATTERN,
 DCL_GROUP_OF_REFERENCE_PATTERN,
 ERROR_STATUS_MOVE_PATTERN,
 EXEC_SQL_INCLUDE_PATTERN,
 INITIALIZE_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 SW_STATUS_D_PIC_PATTERN,
 WRITE_PATTERN,
)
from ..reference.rules.normalization.retrieval_statement_normalization_rules import (
 RETRIEVAL_INCLUDE_MESSAGE,
 RETRIEVAL_INITIALIZE_MESSAGE,
 RETRIEVAL_STATUS_MESSAGE,
 WRITE_LOOKBACK_LIMIT,
)
from .name_normalizer import NameNormalizer

class RetrievalStatementNormalizationService:
 """
 Performs retrieval/output-program normalization.

 Responsibilities:
 - Ensure DCLGEN INCLUDE exists when DCLGEN groups are referenced.
 - Replace residual ERROR-STATUS moves when a DB2 status flag exists.
 - Add INITIALIZE before WRITE when safe and not already present.

 This service does not define regex patterns and does not hardcode
 program names, table names, records, columns, DCLGEN groups, or host
 variables.
 """

 def __init__(
 self,
 dclgen_repository: Any,
) -> None:
 self.dclgen_repository = dclgen_repository
 self.messages: list[str] = []

```

```

def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 output = self._ensure_required_dclgen_includes(source)
 output = self._replace_error_status_when_flag_exists(output)
 output = self._add_initialize_before_write(output)

 return output.rstrip() + "\n"

def _ensure_required_dclgen_includes(
 self,
 text: str,
) -> str:
 referenced_groups = self._referenced_dclgen_groups(text)
 existing_includes = self._existing_includes(text)
 missing_tables: list[str] = []

 for group in referenced_groups:
 table = self._table_from_group(group)

 if not table:
 continue

 if table in existing_includes:
 continue

 missing_tables.append(table)

 if not missing_tables:
 return text

 lines = text.splitlines()
 output: list[str] = []
 inserted = False

 for line in lines:
 logical = self._logical(line)

 if not inserted and PROCEDURE_DIVISION_PATTERN.match(logical):
 for table in self._unique(missing_tables):
 output.extend(
 [
 "EXEC SQL",
 f" INCLUDE {table}",
 "END-EXEC.",
]
)

 inserted = True
 self.messages.append(RETRIEVAL_INCLUDE_MESSAGE)

 output.append(line.rstrip())

 return "\n".join(output)

def _replace_error_status_when_flag_exists(
 self,
 text: str,
) -> str:
 status_flag = self._find_status_flag(text)

 if not status_flag:
 return text

 output: list[str] = []
 changed = False

 for line in text.splitlines():
 logical = self._logical(line)

```

```

 if ERROR_STATUS_MOVE_PATTERN.match(logical):
 output.append(self._with_body(line, f"MOVE 'Y' TO {status_flag}.")
 changed = True
 continue

 output.append(line.rstrip())

 if changed:
 self.messages.append(RETRIEVAL_STATUS_MESSAGE)

 return "\n".join(output)

def _add_initialize_before_write(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 output: list[str] = []
 changed = False

 for index, line in enumerate(lines):
 logical = self._logical(line)
 write_match = WRITE_PATTERN.match(logical)

 if not write_match:
 output.append(line.rstrip())
 continue

 target = NameNormalizer.normalize(write_match.group("target"))

 if not target:
 output.append(line.rstrip())
 continue

 if self._has_recent_initialize(output, target):
 output.append(line.rstrip())
 continue

 output.append(self._with_body(line, f"INITIALIZE {target}.")
 output.append(line.rstrip())
 changed = True

 if changed:
 self.messages.append(RETRIEVAL_INITIALIZE_MESSAGE)

 return "\n".join(output)

def _referenced_dclgen_groups(
 self,
 text: str,
) -> list[str]:
 groups: list[str] = []

 for match in DCL_GROUP_OF_REFERENCE_PATTERN.finditer(text):
 groups.append(match.group("group"))

 for match in DCL_GROUP_DOT_REFERENCE_PATTERN.finditer(text):
 groups.append(match.group("group"))

 return self._unique(groups)

def _existing_includes(
 self,
 text: str,
) -> list[str]:
 includes: list[str] = []

 for line in text.splitlines():
 logical = self._logical(line)
 match = EXEC_SQL_INCLUDE_PATTERN.match(logical)

 if match:
 includes.append(NameNormalizer.normalize(match.group("table")))

 return self._unique(includes)

```

```

def _table_from_group(
 self,
 group: str,
) -> str:
 normalized_group = NameNormalizer.normalize(group)

 if not normalized_group:
 return ""

 if normalized_group.startswith("DCL"):
 return normalized_group[3:]

 return ""

def _find_status_flag(
 self,
 text: str,
) -> str:
 for line in text.splitlines():
 logical = self._logical(line)
 match = SW_STATUS_D_PIC_PATTERN.match(logical)

 if match:
 return match.group("name")

 return ""

def _has_recent_initialize(
 self,
 output: list[str],
 target: str,
) -> bool:
 recent = output[-WRITE_LOOKBACK_LIMIT:]

 for line in reversed(recent):
 logical = self._logical(line)
 match = INITIALIZE_PATTERN.match(logical)

 if not match:
 continue

 existing_target = NameNormalizer.normalize(match.group("target"))

 if existing_target == target:
 return True

 return False

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

def _logical(
 self,
 line: str,
) -> str:
 raw = str(line or "")

```

```

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

def _with_body(
 self,
 original_line: str,
 body: str,
) -> str:
 line = str(original_line or "")
 normalized_body = str(body or "").rstrip()

 if len(line) >= 7 and line[:6].isdigit():
 prefix = line[:7]
 suffix = line[72:] if len(line) > 72 else ""
 return f"{prefix}{normalized_body[:65].ljust(65)}{suffix}"

 return normalized_body

=====
FILE: src\idms_db2_phase2\services\sqlcode_guard_normalization_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..reference.patterns.normalization.sqlcode_guard_patterns import (
 AND_SQLCODE_NOT_FOUND_PATTERN,
 CONTINUE_PATTERN,
 DB2_REMOVED_OR_SKIPPED_COMMENT_PATTERN,
 END_IF_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 IF_SQLCODE_NOT_ZERO_PATTERN,
 PERFORM_SQLERROR_PATTERN,
)
from ..reference.rules.normalization.sqlcode_guard_normalization_rules import (
 SQLCODE_MANUAL_CONTEXT_LOOKBACK_LIMIT,
 SQLCODE_RECENT_SQL_LOOKBACK_LIMIT,
)
from .fixed_format_line_service import FixedFormatLineService

class SqlcodeGuardNormalizationService:
 """
 Normalizes generated SQLCODE guard blocks.

 Responsibilities:
 - Remove orphaned AND SQLCODE lines.
 - Remove generated SQLCODE checks after skipped/manual redesign blocks.
 - Repair malformed IF / END-IF / PERFORM / END-IF order.
 - Preserve SQLCODE checks that follow real EXEC SQL blocks.

 This service does not define regex patterns and does not hardcode
 business names.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []

 def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

```

```

lines = source.splitlines()
lines = self._repair_malformed_sqlcode_order(lines)
lines = self._remove_orphaned_and_guardrails(lines)
lines = self._remove_guardrails_after_manual_context(lines)
lines = self._complete_sqlcode_guardrails(lines)

return "\n".join(lines).rstrip() + "\n"

def _repair_malformed_sqlcode_order(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 if index + 3 >= len(lines):
 output.append(lines[index].rstrip())
 index += 1
 continue

 first = self._logical(lines[index])
 second = self._logical(lines[index + 1])
 third = self._logical(lines[index + 2])
 fourth = self._logical(lines[index + 3])

 if (
 IF_SQLCODE_NOT_ZERO_PATTERN.match(first)
 and END_IF_PATTERN.match(second)
 and PERFORM_SQLERROR_PATTERN.match(third)
 and END_IF_PATTERN.match(fourth)
):
 output.append(lines[index].rstrip())
 output.append(self._replace_body(lines[index], self._perform_body(lines[index + 2])))
 output.append(lines[index + 1].rstrip())
 index += 4
 changed = True
 continue

 output.append(lines[index].rstrip())
 index += 1

 if changed:
 self.messages.append(
 "SQLCODE guard normalization: repaired malformed SQLCODE guard order."
)

 return output

def _remove_orphaned_and_guardrails(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self._logical(lines[index])

 if not AND_SQLCODE_NOT_FOUND_PATTERN.match(logical):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if self._previous_output_is_sqlcode_if(output):
 output.append(lines[index].rstrip())
 index += 1
 continue

 changed = True
 index = self._skip_sqlcode_tail(lines, index)

 if changed:
 self.messages.append(

```

```

 "SQLCODE guard normalization: removed orphaned SQLCODE continuation."
)

 return output

def _remove_guards_after_manual_context(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self._logical(lines[index])

 if not IF_SQLCODE_NOT_ZERO_PATTERN.match(logical):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if self._has_recent_exec_sql(output):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if not self._has_recent_manual_context(output):
 output.append(lines[index].rstrip())
 index += 1
 continue

 changed = True
 index = self._skip_complete_sqlcode_guard(lines, index)

 if changed:
 self.messages.append(
 "SQLCODE guard normalization: removed generated guard after manual context."
)

 return output

def _complete_sqlcode_guards(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 line = lines[index]
 logical = self._logical(line)

 if not IF_SQLCODE_NOT_ZERO_PATTERN.match(logical):
 output.append(line.rstrip())
 index += 1
 continue

 guard_end = self._find_guard_end(lines, index)

 if guard_end <= index:
 output.append(line.rstrip())
 index += 1
 continue

 guard_lines = lines[index:guard_end]
 output.extend(item.rstrip() for item in guard_lines)

 if not self._guard_has_end_if(guard_lines):
 output.append(
 self._replace_body(
 line,
 self._guard_end_if_body(line),
)
)
 changed = True

```

```

 index = guard_end

 if changed:
 self.messages.append(
 "SQLCODE guard normalization: completed generated SQLCODE guard."
)

 return output

def _find_guard_end(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index + 1
 saw_perform = False

 while index < len(lines):
 logical = self._logical(lines[index])

 if AND_SQLCODE_NOT_FOUND_PATTERN.match(logical):
 index += 1
 continue

 if PERFORM_SQLERROR_PATTERN.match(logical):
 saw_perform = True
 index += 1
 continue

 if END_IF_PATTERN.match(logical):
 return index + 1

 if saw_perform:
 return index

 return index

 return index

def _guard_has_end_if(
 self,
 guard_lines: list[str],
) -> bool:
 return any(
 END_IF_PATTERN.match(self._logical(line))
 for line in guard_lines
)

def _guard_end_if_body(
 self,
 line: str,
) -> str:
 logical = self._logical(line)
 indent = logical[: len(logical) - len(logical.lstrip())]
 return f"{indent}END-IF."

def _perform_body(
 self,
 line: str,
) -> str:
 logical = self._logical(line)
 indent = logical[: len(logical) - len(logical.lstrip())]
 stripped = logical.strip()

 if stripped.upper().endswith("."):
 stripped = stripped[:-1]

 return f"{indent}{stripped}."

def _skip_sqlcode_tail(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index

```

```

while index < len(lines):
 logical = self._logical(lines[index])

 if AND_SQLCODE_NOT_FOUND_PATTERN.match(logical):
 index += 1
 continue

 if PERFORM_SQLERROR_PATTERN.match(logical):
 index += 1
 continue

 if END_IF_PATTERN.match(logical):
 index += 1
 break

 break

return index

def _skip_complete_sqlcode_guard(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index

 while index < len(lines):
 logical = self._logical(lines[index])
 index += 1

 if END_IF_PATTERN.match(logical):
 break

 if PERFORM_SQLERROR_PATTERN.match(logical):
 next_logical = self._logical(lines[index]) if index < len(lines) else ""

 if not END_IF_PATTERN.match(next_logical):
 break

 return index

def _previous_output_is_sqlcode_if(
 self,
 output: list[str],
) -> bool:
 if not output:
 return False

 return IF_SQLCODE_NOT_ZERO_PATTERN.match(self._logical(output[-1])) is not None

def _has_recent_exec_sql(
 self,
 output: list[str],
) -> bool:
 recent = output[-SQLCODE_RECENT_SQL_LOOKBACK_LIMIT:]
 seen_end_exec = False

 for line in reversed(recent):
 logical = self._logical(line)

 if EXEC_SQL_END_PATTERN.match(logical):
 seen_end_exec = True
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 return seen_end_exec

 return False

def _has_recent_manual_context(
 self,
 output: list[str],
) -> bool:
 recent = output[-SQLCODE_MANUAL_CONTEXT_LOOKBACK_LIMIT:]

```

```

 return any(
 DB2_REMOVED_OR_SKIPPED_COMMENT_PATTERN.match(self._logical(line))
 or CONTINUE_PATTERN.match(self._logical(line))
 for line in recent
)

 def _logical(
 self,
 line: str,
) -> str:
 if hasattr(self.fixed_format, "logical"):
 return self.fixed_format.logical(line)

 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

 def _replace_body(
 self,
 line: str,
 body: str,
) -> str:
 clean_body = str(body or "").rstrip()
 raw = str(line or "")

 if len(raw) >= 80 and raw[:6].isdigit():
 prefix = raw[:7]
 suffix = raw[72:] if len(raw) > 72 else ""
 return f"{prefix}{clean_body[:65].ljust(65)}{suffix}"

 return clean_body

```

```

=====
FILE: src\idms_db2_phase2\services\update_audit_assignment_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.normalization.update_statement_patterns import (
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 INSERT_ONLY_AUDIT_HOST_PATTERN,
 UPDATE_AUDIT_COLUMN_PATTERN,
)
from ..reference.rules.normalization.update_statement_normalization_rules import (
 INSERT_ONLY_AUDIT_HOST_PREFIXES,
 UPDATE_TIMESTAMP_HOST_PREFIXES,
)

```

```

class UpdateAuditAssignmentService:
 """
 Normalizes generated update audit assignments.

 For update-only flows, insert-only audit hosts such as TS-CREATE are
 not allowed. This service replaces only clear TS_UPDATE assignments
 that incorrectly reference TS-CREATE host variables.

 This service does not define regex patterns and does not hardcode
 program names, table names, or business fields.
 """

 def __init__(self) -> None:
 self.messages: list[str] = []

 def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

```

```

if not source:
 return ""

lines = source.splitlines()
output: list[str] = []
in_exec_sql = False
in_update_sql = False
changed = False

for line in lines:
 logical = self._logical(line)

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 in_update_sql = False
 output.append(line.rstrip())
 continue

 if in_exec_sql and logical.strip().upper().startswith("UPDATE "):
 in_update_sql = True
 output.append(line.rstrip())
 continue

 if in_exec_sql and in_update_sql:
 normalized = self._normalize_update_audit_line(line)
 changed = changed or normalized != line.rstrip()
 output.append(normalized)

 if EXEC_SQL_END_PATTERN.match(logical):
 in_exec_sql = False
 in_update_sql = False

 continue

 if EXEC_SQL_END_PATTERN.match(logical):
 in_exec_sql = False
 in_update_sql = False
 output.append(line.rstrip())
 continue

 output.append(line.rstrip())

if changed:
 self.messages.append(
 "Update audit assignment normalization: replaced insert-only audit host usage."
)

return "\n".join(output).rstrip() + "\n"

def _normalize_update_audit_line(
 self,
 line: str,
) -> str:
 logical = self._logical(line)

 if not UPDATE_AUDIT_COLUMN_PATTERN.match(logical):
 return line.rstrip()

 if not INSERT_ONLY_AUDIT_HOST_PATTERN.search(logical):
 return line.rstrip()

 normalized_body = self._replace_insert_only_host(logical)
 return self._with_body(line, normalized_body)

def _replace_insert_only_host(
 self,
 body: str,
) -> str:
 result = str(body or "")

 for insert_prefix in INSERT_ONLY_AUDIT_HOST_PREFIXES:
 for update_prefix in UPDATE_TIMESTAMP_HOST_PREFIXES:
 result = result.replace(insert_prefix, update_prefix)
 result = result.replace(insert_prefix.lower(), update_prefix.lower())

```

```

 return result

 def _logical(
 self,
 line: str,
) -> str:
 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

 def _with_body(
 self,
 original_line: str,
 body: str,
) -> str:
 line = str(original_line or "")
 normalized_body = str(body or "").rstrip()

 if len(line) >= 7 and line[:6].isdigit():
 prefix = line[:7]
 suffix = line[72:] if len(line) > 72 else ""
 return f"{prefix}{normalized_body[:65].ljust(65)}{suffix}"

 return normalized_body

```

```

=====
FILE: src\idms_db2_phase2\services\update_date_assignment_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.rules.normalization.update_direct_sql_normalization_rules import (
 DATE_COLUMN_PREFIXES,
)
from .fixed_format_line_service import FixedFormatLineService
from .name_normalizer import NameNormalizer

```

```

class UpdateDateAssignmentService:
 """
 Builds date conversion lines for update changed-field moves.

 This service does not resolve metadata and does not generate SQL.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 def conversion_lines(
 self,
 original_line: str,
 move: dict[str, str],
) -> list[str]:
 if not move:
 return []

 column = NameNormalizer.to_sql(move.get("column", ""))

 if not any(column.startswith(prefix) for prefix in DATE_COLUMN_PREFIXES):
 target = self._without_colon(move.get("host_reference", ""))

 return [
 self.replace_body(
 original_line,
 f"MOVE {move['source']} TO {target}.",
 indicator=" ",
)
]

```

```

target = self._without_colon(move.get("host_reference", ""))
source = move.get("source", "")

return [
 self.replace_body(original_line, "MOVE ZEROES TO DA-CCYYMMDD"),
 self.replace_body(original_line, "MOVE SPACES TO DA-DD-MM-CCYY"),
 self.replace_body(original_line, "EVALUATE TRUE"),
 self.replace_body(original_line, f" WHEN {source} = ZEROES"),
 self.replace_body(original_line, f" MOVE SPACES TO DA-DD-MM-CCYY"),
 self.replace_body(original_line, f" WHEN {source} = SPACES"),
 self.replace_body(original_line, f" MOVE SPACES TO DA-DD-MM-CCYY"),
 self.replace_body(original_line, f" WHEN OTHER"),
 self.replace_body(original_line, f" MOVE {source} TO DA-CCYYMMDD"),
 self.replace_body(
 original_line,
 f" MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
),
 self.replace_body(original_line, "END-EVALUATE"),
 self.replace_body(original_line, f"MOVE DA-DD-MM-CCYY TO {target}."),
]

```

```

def replace_body(
 self,
 line: str,
 body: str,
 indicator: str = " ",
) -> str:
 clean_body = str(body or "").rstrip()
 raw = str(line or "")

 if len(raw) >= 80 and raw[:6].isdigit():
 prefix = raw[:6] + indicator[:1]
 suffix = raw[72:] if len(raw) > 72 else ""
 return f"{prefix}{clean_body[:65].ljust(65)}{suffix}"

 if indicator == "":
 return f"*{clean_body}"

 return clean_body

def _without_colon(
 self,
 reference: str,
) -> str:
 value = str(reference or "").strip()

 if value.startswith(":"):
 return value[1:]

 return value

```

```

=====
FILE: src\idms_db2_phase2\services\update_direct_sql_normalization_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Any

from ..reference.patterns.normalization.update_direct_sql_patterns import (
 CONVERTED_MODIFY_COMMENT_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 MOVE_TO_SOURCE_FIELD_PATTERN,
 NON_SQL_HOST_TARGET_PATTERN,
)
from ..reference.rules.normalization.update_direct_sql_normalization_rules import (
 MODIFY_LOOKBACK_LIMIT,
)
from .fixed_format_line_service import FixedFormatLineService
from .name_normalizer import NameNormalizer
from .update_date_assignment_service import UpdateDateAssignmentService
from .update_metadata_resolution_service import UpdateMetadataResolutionService

```

```

from .update_sql_block_builder_service import UpdateSqlBlockBuilderService
from .update_sql_scan_service import UpdateSqlScanService

```

```

class UpdateDirectSqlNormalizationService:
 """

```

```

 Coordinates direct update SQL normalization.

```

```

 This service delegates scanning, metadata resolution, date conversion,
 and SQL block construction to smaller services.
 """

```

```

def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 mapping_repository: Any | None = None,
 dclgen_repository: Any | None = None,
 table_name_resolver: Any | None = None,
 host_variable_resolver: Any | None = None,
 scan_service: UpdateSqlScanService | None = None,
 metadata_resolution: UpdateMetadataResolutionService | None = None,
 date_assignment: UpdateDateAssignmentService | None = None,
 block_builder: UpdateSqlBlockBuilderService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.scan_service = scan_service or UpdateSqlScanService(
 fixed_format=self.fixed_format,
)
 self.metadata_resolution = metadata_resolution or UpdateMetadataResolutionService(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)
 self.date_assignment = date_assignment or UpdateDateAssignmentService(
 fixed_format=self.fixed_format,
)
 self.block_builder = block_builder or UpdateSqlBlockBuilderService(
 metadata_resolution=self.metadata_resolution,
 fixed_format=self.fixed_format,
)
 self.messages: list[str] = []

```

```

def apply(
 self,
 text: str,
) -> str:
 return self.normalize(text)

```

```

def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 lines = source.splitlines()
 lines = self.scan_service.remove_obtain_calc_select_blocks(lines)
 self.messages.extend(self.scan_service.messages)

 lines = self._rewrite_modify_update_blocks(lines)
 lines = self._normalize_non_sql_host_targets(lines)

 return "\n".join(lines).rstrip() + "\n"

```

```

def _rewrite_modify_update_blocks(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

```

```

 while index < len(lines):

```

```

logical = self._logical(lines[index])
match = CONVERTED_MODIFY_COMMENT_PATTERN.match(logical)

if not match:
 output.append(lines[index].rstrip())
 index += 1
 continue

record = NameNormalizer.normalize(match.group("record"))
table = self.metadata_resolution.table_for_record(record)

if not record or not table or not self.metadata_resolution.has_metadata():
 output.append(lines[index].rstrip())
 index += 1
 continue

generated_sql_start = self.scan_service.next_executable_index(
 lines,
 index + 1,
)
generated_where_columns = self.scan_service.generated_where_columns(
 lines=lines,
 start_index=generated_sql_start,
)
move_key_columns = self._key_columns_from_recent_moves(
 output=output,
 table=table,
)
changed_move = self._changed_move_before_modify(
 output=output,
 record=record,
 table=table,
)
changed_columns = self._changed_columns_for_move(changed_move)
update_columns = self.metadata_resolution.update_columns(
 table=table,
 changed_columns=changed_columns,
)
key_columns = self.metadata_resolution.key_columns(
 record=record,
 table=table,
 move_key_columns=move_key_columns,
 generated_where_columns=generated_where_columns,
)

if not update_columns or not key_columns:
 output.append(lines[index].rstrip())
 index += 1
 continue

block_end = self.scan_service.skip_generated_sql_and_sqlcode(
 lines=lines,
 start_index=index + 1,
)

self._remove_original_changed_move_from_output(
 output=output,
 move=changed_move,
)

output.extend(
 self.date_assignment.conversion_lines(
 original_line=lines[index],
 move=changed_move,
)
)
output.extend(
 self.block_builder.conservative_update_block(
 original_line=lines[index],
 table=table,
 update_columns=update_columns,
 key_columns=key_columns,
)
)

index = block_end

```

```

 changed = True

 if changed:
 self.messages.append(
 "Update direct SQL normalization: rewrote MODIFY block to conservative DB2 UPDATE."
)

 return output

def _key_columns_from_recent_moves(
 self,
 output: list[str],
 table: str,
) -> list[str]:
 recent = output[-MODIFY_LOOKBACK_LIMIT:]
 columns: list[str] = []

 for line in recent:
 logical = self._logical(line)
 host = self._target_host_from_move_continuation(logical)

 if not host:
 continue

 column = self.metadata_resolution.column_for_host(table, host)

 if column:
 columns.append(column)

 return self.metadata_resolution.unique(columns)

def _target_host_from_move_continuation(
 self,
 logical: str,
) -> str:
 text = str(logical or "").strip()
 upper = text.upper()

 if not upper.startswith("TO "):
 return ""

 if " OF DCL" not in upper:
 return ""

 target = text[3:].split(" OF ", 1)[0].strip()
 return NameNormalizer.to_cobol(target)

def _changed_move_before_modify(
 self,
 output: list[str],
 record: str,
 table: str,
) -> dict[str, str]:
 recent = output[-MODIFY_LOOKBACK_LIMIT:]

 for line in reversed(recent):
 logical = self._logical(line)
 match = MOVE_TO_SOURCE_FIELD_PATTERN.match(logical)

 if not match:
 continue

 target = NameNormalizer.normalize(match.group("target"))
 column = self.metadata_resolution.column_for_source_field(
 record,
 target,
)

 if not column:
 continue

 host_reference = self.metadata_resolution.host_reference(
 table,
 column,
)

```

```

 if not host_reference:
 continue

 return {
 "line": line,
 "source": NameNormalizer.normalize(match.group("source")),
 "target": target,
 "column": column,
 "host_reference": host_reference,
 }

 return {}

def _changed_columns_for_move(
 self,
 move: dict[str, str],
) -> list[str]:
 column = NameNormalizer.to_sql(move.get("column", ""))

 if not column:
 return []

 return [column]

def _remove_original_changed_move_from_output(
 self,
 output: list[str],
 move: dict[str, str],
) -> None:
 original_line = move.get("line", "")

 if not original_line:
 return

 start = max(len(output) - MODIFY_LOOKBACK_LIMIT, 0)

 for index in range(len(output) - 1, start - 1, -1):
 if output[index].rstrip() == original_line.rstrip():
 del output[index]
 return

def _normalize_non_sql_host_targets(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 in_exec_sql = False

 for line in lines:
 logical = self._logical(line)

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 output.append(line.rstrip())
 continue

 if EXEC_SQL_END_PATTERN.match(logical):
 in_exec_sql = False
 output.append(line.rstrip())
 continue

 if in_exec_sql:
 output.append(line.rstrip())
 continue

 match = NON_SQL_HOST_TARGET_PATTERN.match(logical)

 if not match:
 output.append(line.rstrip())
 continue

 normalized = (
 f"{match.group('prefix')}}"
 f"{match.group('host')}}"
 f"{match.group('suffix')}}"
)

```

```

 output.append(
 self.block_builder.replace_body(
 line,
 normalized,
)
)

 return output

```

```

def _logical(
 self,
 line: str,
) -> str:
 return self.scan_service.logical(line)

```

```

=====
FILE: src\idms_db2_phase2\services\update_metadata_resolution_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from difflib import SequenceMatcher
from typing import Any

from ..reference.rules.normalization.update_direct_sql_normalization_rules import (
 AUDIT_COLUMN_PREFIXES,
)
from .name_normalizer import NameNormalizer

```

```

class UpdateMetadataResolutionService:
 """
 Resolves metadata required for conservative update SQL.

 Responsibilities:
 - Resolve table for record.
 - Resolve source field to DB2 column.
 - Resolve DCLGEN host references by table and column.
 - Resolve update columns and composite key columns.

 This service does not generate SQL text.
 """

```

```

def __init__(
 self,
 mapping_repository: Any | None = None,
 dclgen_repository: Any | None = None,
 table_name_resolver: Any | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

```

```

def has_metadata(
 self,
) -> bool:
 return bool(
 self.mapping_repository is not None
 and self.dclgen_repository is not None
 and self.table_name_resolver is not None
)

```

```

def table_for_record(
 self,
 record: str,
) -> str:
 if self.table_name_resolver is not None:
 for method_name in ("table_for_record", "resolve_record_table"):
 method = getattr(self.table_name_resolver, method_name, None)

 if callable(method):
 table = method(record)

 if table:

```

```

 return NameNormalizer.to_sql(table)

 if self.mapping_repository is None:
 return ""

 method = getattr(self.mapping_repository, "db2_table_for_record", None)

 if not callable(method):
 return ""

 mapped_table = method(record)

 if not mapped_table:
 return ""

 if self.table_name_resolver is not None:
 resolve = getattr(self.table_name_resolver, "resolve_table", None)

 if callable(resolve):
 resolved = resolve(mapped_table)

 if resolved:
 return NameNormalizer.to_sql(resolved)

 return NameNormalizer.to_sql(mapped_table)

def update_columns(
 self,
 table: str,
 changed_columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in changed_columns:
 normalized = NameNormalizer.to_sql(column)

 if normalized and self.host_reference(table, normalized):
 output.append(normalized)

 for column in self.audit_columns(table):
 normalized = NameNormalizer.to_sql(column)

 if normalized and self.host_reference(table, normalized):
 output.append(normalized)

 return self.unique(output)

def audit_columns(
 self,
 table: str,
) -> list[str]:
 output: list[str] = []

 for column in self.dclgen_columns(table):
 normalized = NameNormalizer.to_sql(column)

 if any(normalized.startswith(prefix) for prefix in AUDIT_COLUMN_PREFIXES):
 output.append(normalized)

 return output

def key_columns(
 self,
 record: str,
 table: str,
 move_key_columns: list[str],
 generated_where_columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in self.mapping_key_columns(record):
 normalized = NameNormalizer.to_sql(column)

 if normalized and self.host_reference(table, normalized):
 output.append(normalized)

```

```

 for column in move_key_columns:
 normalized = NameNormalizer.to_sql(column)

 if normalized and self.host_reference(table, normalized):
 output.append(normalized)

 for column in generated_where_columns:
 normalized = NameNormalizer.to_sql(column)

 if normalized and self.host_reference(table, normalized):
 output.append(normalized)

 return self.unique(output)

def mapping_key_columns(
 self,
 record: str,
) -> list[str]:
 if self.mapping_repository is None:
 return []

 output: list[str] = []

 for method_name in (
 "primary_key_columns_for_record",
 "key_columns_for_record",
):
 method = getattr(self.mapping_repository, method_name, None)

 if callable(method):
 values = method(record)

 if values:
 output.extend(values)

 return output

def column_for_host(
 self,
 table: str,
 host: str,
) -> str:
 if self.dclgen_repository is None:
 return ""

 columns_for_table = getattr(self.dclgen_repository, "columns_for_table", None)

 if not callable(columns_for_table):
 return ""

 target_host = NameNormalizer.to_cobol(host)

 for dclgen_column in columns_for_table(table):
 current_host = NameNormalizer.to_cobol(
 getattr(dclgen_column, "cobol_host_name", "")
 or getattr(dclgen_column, "column_name", "")
)

 if current_host == target_host:
 return NameNormalizer.to_sql(
 getattr(dclgen_column, "column_name", "")
)

 return ""

def column_for_source_field(
 self,
 record: str,
 source_field: str,
) -> str:
 if self.mapping_repository is None:
 return ""

 method = getattr(self.mapping_repository, "column_for_source_field", None)

 if callable(method):

```

```

 column = method(
 record_name=record,
 source_field_name=source_field,
)

 if column:
 return NameNormalizer.to_sql(column)

 rows_method = getattr(self.mapping_repository, "rows_for_record", None)

 if not callable(rows_method):
 return ""

 rows = rows_method(record)
 source = NameNormalizer.normalize(source_field)
 compact_source = NameNormalizer.compact(source)

 return (
 self._column_for_source_exact(rows, source)
 or self._column_for_source_compact(rows, compact_source)
 or self._column_for_source_similarity(rows, source)
)

def dclgen_columns(
 self,
 table: str,
) -> list[str]:
 if self.dclgen_repository is None:
 return []

 method = getattr(self.dclgen_repository, "column_names_for_table", None)

 if callable(method):
 return [
 NameNormalizer.to_sql(column)
 for column in method(table)
 if NameNormalizer.to_sql(column)
]

 return []

def host_reference(
 self,
 table: str,
 column: str,
) -> str:
 if self.dclgen_repository is None:
 return ""

 normalized_table = NameNormalizer.to_sql(table)
 normalized_column = NameNormalizer.to_sql(column)
 columns_for_table = getattr(self.dclgen_repository, "columns_for_table", None)

 if not callable(columns_for_table):
 return ""

 for dclgen_column in columns_for_table(normalized_table):
 current_column = NameNormalizer.to_sql(
 getattr(dclgen_column, "column_name", "")
)

 if current_column != normalized_column:
 continue

 return self._reference_from_dclgen_column(
 table=normalized_table,
 column=dclgen_column,
)

 return ""

def _reference_from_dclgen_column(
 self,
 table: str,
 column: object,
) -> str:

```

```

group = NameNormalizer.to_cobol(getattr(column, "group_name", ""))

if not group:
 group_method = getattr(self.dclgen_repository, "group_for_table", None)

 if callable(group_method):
 group = NameNormalizer.to_cobol(group_method(table))

if not group:
 group = "DCL" + NameNormalizer.to_cobol(table)

host = NameNormalizer.to_cobol(
 getattr(column, "cobol_host_name", "")
 or getattr(column, "column_name", "")
)

if not group or not host:
 return ""

return f":{group}.{host}"

def _column_for_source_exact(
 self,
 rows: list[object],
 source: str,
) -> str:
 for row in rows:
 column = NameNormalizer.to_sql(getattr(row, "new_db2_field_name", ""))

 if not column:
 continue

 for candidate in self._source_candidates(row):
 if NameNormalizer.normalize(candidate) == source:
 return column

 return ""

def _column_for_source_compact(
 self,
 rows: list[object],
 compact_source: str,
) -> str:
 for row in rows:
 column = NameNormalizer.to_sql(getattr(row, "new_db2_field_name", ""))

 if not column:
 continue

 for candidate in self._source_candidates(row):
 if NameNormalizer.compact(candidate) == compact_source:
 return column

 return ""

def _column_for_source_similarity(
 self,
 rows: list[object],
 source: str,
) -> str:
 best_column = ""
 best_score = 0.0

 for row in rows:
 column = NameNormalizer.to_sql(getattr(row, "new_db2_field_name", ""))

 if not column:
 continue

 for candidate in self._source_candidates(row):
 score = SequenceMatcher(
 None,
 NameNormalizer.compact(candidate),
 NameNormalizer.compact(source),
).ratio()

```

```

 if score > best_score:
 best_score = score
 best_column = column

 if best_score >= 0.74:
 return best_column

 return ""

def _source_candidates(
 self,
 row: object,
) -> list[str]:
 names = (
 "cobol_zone",
 "cobol_field_name",
 "idms_field_name",
 "source_field_name",
 "field_name",
 "cobol_field",
)
 output: list[str] = []

 for name in names:
 value = getattr(row, name, "")

 if value:
 output.append(str(value))

 return output

def unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.to_sql(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\services\update_procedure_normalization_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Any

from .fixed_format_line_service import FixedFormatLineService
from .include_placement_normalization_service import (
 IncludePlacementNormalizationService,
)
from .procedure_indentation_service import ProcedureIndentationService
from .sqlcode_guard_normalization_service import SqlcodeGuardNormalizationService
from .update_audit_assignment_service import UpdateAuditAssignmentService
from .update_direct_sql_normalization_service import (
 UpdateDirectSqlNormalizationService,
)
from .update_statement_normalization_service import UpdateStatementNormalizationService

class UpdateProcedureNormalizationService:

```

"""

Coordinates update-program PROCEDURE DIVISION normalization.

This service keeps compatibility with FinalCobolNormalizationComposer, which passes fixed\_format during construction.

This service can also receive repositories and resolvers from the update-program composer so direct UPDATE blocks can be regenerated from Sheet Mapping and DCLGEN metadata.

"""

```
def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 mapping_repository: Any | None = None,
 dclgen_repository: Any | None = None,
 table_name_resolver: Any | None = None,
 host_variable_resolver: Any | None = None,
 include_placement: IncludePlacementNormalizationService | None = None,
 statement_normalization: UpdateStatementNormalizationService | None = None,
 direct_sql_normalization: UpdateDirectSqlNormalizationService | None = None,
 audit_assignment: UpdateAuditAssignmentService | None = None,
 sqlcode_guard: SqlcodeGuardNormalizationService | None = None,
 procedure_indentation: ProcedureIndentationService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver

 self.include_placement = (
 include_placement
 or IncludePlacementNormalizationService(
 fixed_format=self.fixed_format,
)
)
 self.statement_normalization = (
 statement_normalization or UpdateStatementNormalizationService()
)
 self.direct_sql_normalization = (
 direct_sql_normalization
 or UpdateDirectSqlNormalizationService(
 fixed_format=self.fixed_format,
 mapping_repository=self.mapping_repository,
 dclgen_repository=self.dclgen_repository,
 table_name_resolver=self.table_name_resolver,
 host_variable_resolver=self.host_variable_resolver,
)
)
 self.audit_assignment = audit_assignment or UpdateAuditAssignmentService()
 self.sqlcode_guard = (
 sqlcode_guard
 or SqlcodeGuardNormalizationService(
 fixed_format=self.fixed_format,
)
)
 self.procedure_indentation = (
 procedure_indentation or ProcedureIndentationService()
)
 self.messages: list[str] = []

def apply(
 self,
 text: str,
) -> str:
 return self.normalize(text)

def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
```

```

 return output

 output = self.include_placement.normalize(output)
 self.messages.extend(self.include_placement.messages)

 output = self.statement_normalization.normalize(output)
 self.messages.extend(self.statement_normalization.messages)

 output = self.direct_sql_normalization.normalize(output)
 self.messages.extend(self.direct_sql_normalization.messages)

 output = self.audit_assignment.normalize(output)
 self.messages.extend(self.audit_assignment.messages)

 output = self.sqlcode_guard.normalize(output)
 self.messages.extend(self.sqlcode_guard.messages)

 output = self.procedure_indentation.normalize(output)
 self.messages.extend(self.procedure_indentation.messages)

 return output.rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\services\update_sql_block_builder_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from .fixed_format_line_service import FixedFormatLineService
from .name_normalizer import NameNormalizer
from .update_metadata_resolution_service import UpdateMetadataResolutionService

```

```

class UpdateSqlBlockBuilderService:

```

```

 """
 Builds conservative update SQL blocks.

```

```

 Responsibilities:
 - Build audit MOVE statements.
 - Build SET lines.
 - Build WHERE lines.
 - Build SQLCODE check.

```

```

 This service does not scan source and does not resolve source fields.
 """

```

```

 def __init__(
 self,
 metadata_resolution: UpdateMetadataResolutionService,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.metadata_resolution = metadata_resolution
 self.fixed_format = fixed_format or FixedFormatLineService()

```

```

 def conservative_update_block(
 self,
 original_line: str,
 table: str,
 update_columns: list[str],
 key_columns: list[str],
) -> list[str]:
 output = [
 self.replace_body(
 original_line,
 f"DB2:Converted MODIFY for {table}.",
 indicator="*",
),
 self.replace_body(
 original_line,
 f"MOVE 'UPDATE-{table}' TO SQL-LOCATION.",
),
]

```

```

 output.extend(

```

```

 self.audit_move_lines(
 original_line=original_line,
 table=table,
 update_columns=update_columns,
)
)

output.extend(
 [
 self.replace_body(original_line, "EXEC SQL"),
 self.replace_body(original_line, f"UPDATE {table}"),
 self.replace_body(original_line, "SET"),
]
)

output.extend(
 self.set_lines(
 original_line=original_line,
 table=table,
 columns=update_columns,
)
)

output.append(self.replace_body(original_line, "WHERE"))

output.extend(
 self.where_lines(
 original_line=original_line,
 table=table,
 columns=key_columns,
)
)

output.extend(
 [
 self.replace_body(original_line, "END-EXEC."),
 self.replace_body(original_line, "IF SQLCODE NOT = 0"),
 self.replace_body(original_line, " PERFORM SQLERROR."),
 self.replace_body(original_line, "END-IF."),
]
)

return output

def audit_move_lines(
 self,
 original_line: str,
 table: str,
 update_columns: list[str],
) -> list[str]:
 output: list[str] = []

 for column in update_columns:
 normalized = NameNormalizer.to_sql(column)
 reference = self.metadata_resolution.host_reference(table, normalized)

 if not reference:
 continue

 target = self.without_colon(reference)

 if normalized.startswith("TS_UPDATE"):
 output.append(
 self.replace_body(
 original_line,
 f"MOVE TS-TIMESTAMP TO {target}",
)
)

 if normalized.startswith(("ID_USERID", "NR_USERID", "ID_USER", "NR_USER")):
 output.append(
 self.replace_body(
 original_line,
 f"MOVE CS-PROGRAM TO {target}",
)
)

```

```

 return output

def set_lines(
 self,
 original_line: str,
 table: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []

 for index, column in enumerate(columns):
 reference = self.metadata_resolution.host_reference(table, column)

 if not reference:
 continue

 comma = "," if index < len(columns) - 1 else ""
 output.append(
 self.replace_body(
 original_line,
 f"{column} = {reference}{comma}",
)
)

 return output

def where_lines(
 self,
 original_line: str,
 table: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []

 for index, column in enumerate(columns):
 reference = self.metadata_resolution.host_reference(table, column)

 if not reference:
 continue

 prefix = "" if index == 0 else "AND "
 output.append(
 self.replace_body(
 original_line,
 f"{prefix}{column} = {reference}",
)
)

 return output

def replace_body(
 self,
 line: str,
 body: str,
 indicator: str = " ",
) -> str:
 clean_body = str(body or "").rstrip()
 raw = str(line or "")

 if clean_body.lstrip().startswith("*"):
 clean_body = clean_body.lstrip()[1:].lstrip()
 indicator = "*"

 if len(raw) >= 80 and raw[:6].isdigit():
 prefix = raw[:6] + indicator[:1]
 suffix = raw[72:] if len(raw) > 72 else ""
 return f"{prefix}{clean_body[:65].ljust(65)}{suffix}"

 if indicator == "*":
 return f"*{clean_body}"

 return clean_body

def without_colon(
 self,

```

```

 reference: str,
) -> str:
 value = str(reference or "").strip()

 if value.startswith(":"):
 return value[1:]

 return value

```

```

=====
FILE: src\idms_db2_phase2\services\update_sql_scan_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.normalization.update_direct_sql_patterns import (
 AND_SQLCODE_NOT_FOUND_PATTERN,
 CONVERTED_OBTAIN_CALC_COMMENT_PATTERN,
 END_IF_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 IF_SQLCODE_NOT_ZERO_PATTERN,
 PERFORM_SQLERROR_PATTERN,
 SELECT_STATEMENT_PATTERN,
)
from ..reference.rules.normalization.update_direct_sql_normalization_rules import (
 OBTAIN_SELECT_SCAN_LIMIT,
 SQLCODE_GUARD_SCAN_LIMIT,
)
from .fixed_format_line_service import FixedFormatLineService
from .name_normalizer import NameNormalizer

```

```

class UpdateSqlScanService:
 """
 Scans generated update SQL blocks.

 Responsibilities:
 - Detect generated OBTAIN CALC SELECT blocks.
 - Remove unnecessary SELECT blocks before direct UPDATE.
 - Read generated WHERE columns when needed.
 - Skip generated SQLCODE guards attached to generated SQL blocks.

 This service does not hardcode business names.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []

 def remove_obtain_calc_select_blocks(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self.logical(lines[index])
 match = CONVERTED_OBTAIN_CALC_COMMENT_PATTERN.match(logical)

 if not match:
 output.append(lines[index].rstrip())
 index += 1
 continue

 sql_start = self.next_executable_index(lines, index + 1)

 if sql_start < 0:
 output.append(lines[index].rstrip())

```

```

 index += 1
 continue

 if not EXEC_SQL_START_PATTERN.match(self.logical(lines[sql_start])):
 output.append(lines[index].rstrip())
 index += 1
 continue

 if not self.exec_sql_block_contains_select(lines, sql_start):
 output.append(lines[index].rstrip())
 index += 1
 continue

 sql_end = self.skip_exec_sql_block(lines, sql_start)
 index = self.skip_following_sqlcode_guard(lines, sql_end)
 changed = True

 if changed:
 self.messages.append(
 "Update SQL scan: removed generated OBTAIN CALC SELECT block."
)

 return output

def generated_where_columns(
 self,
 lines: list[str],
 start_index: int,
) -> list[str]:
 if start_index < 0:
 return []

 output: list[str] = []
 in_exec_sql = False
 in_where = False
 index = start_index

 while index < len(lines):
 logical = self.logical(lines[index]).strip()
 upper = logical.upper()

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 index += 1
 continue

 if not in_exec_sql:
 break

 if EXEC_SQL_END_PATTERN.match(logical):
 break

 if upper == "WHERE" or upper.startswith("WHERE "):
 in_where = True
 column = self._column_from_condition(
 logical.replace("WHERE", "", 1)
)

 if column:
 output.append(column)

 index += 1
 continue

 if in_where:
 if upper.startswith("AND "):
 column = self._column_from_condition(logical[4:])

 if column:
 output.append(column)

 elif "=" in logical:
 column = self._column_from_condition(logical)

 if column:
 output.append(column)

```

```

 index += 1

 return self.unique(output)

def exec_sql_block_contains_select(
 self,
 lines: list[str],
 start_index: int,
) -> bool:
 index = start_index + 1
 scanned = 0

 while index < len(lines) and scanned < OBTAIN_SELECT_SCAN_LIMIT:
 logical = self.logical(lines[index])

 if EXEC_SQL_END_PATTERN.match(logical):
 return False

 if SELECT_STATEMENT_PATTERN.match(logical):
 return True

 index += 1
 scanned += 1

 return False

def skip_generated_sql_and_sqlcode(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = self.next_executable_index(lines, start_index)

 if index < 0:
 return start_index

 if EXEC_SQL_START_PATTERN.match(self.logical(lines[index])):
 index = self.skip_exec_sql_block(lines, index)

 return self.skip_following_sqlcode_guard(lines, index)

def skip_exec_sql_block(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index

 while index < len(lines):
 logical = self.logical(lines[index])
 index += 1

 if EXEC_SQL_END_PATTERN.match(logical):
 break

 return index

def skip_following_sqlcode_guard(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = self.next_executable_index(lines, start_index)

 if index < 0:
 return start_index

 if not IF_SQLCODE_NOT_ZERO_PATTERN.match(self.logical(lines[index])):
 return start_index

 scanned = 0

 while index < len(lines) and scanned < SQLCODE_GUARD_SCAN_LIMIT:
 logical = self.logical(lines[index])

```

```

 if END_IF_PATTERN.match(logical):
 return index + 1

 if (
 IF_SQLCODE_NOT_ZERO_PATTERN.match(logical)
 or AND_SQLCODE_NOT_FOUND_PATTERN.match(logical)
 or PERFORM_SQLERROR_PATTERN.match(logical)
):
 index += 1
 scanned += 1
 continue

 break

 return index

def next_executable_index(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 index = start_index

 while index < len(lines):
 logical = self.logical(lines[index]).strip()

 if not logical:
 index += 1
 continue

 if logical.startswith("*"):
 index += 1
 continue

 return index

 return -1

def logical(
 self,
 line: str,
) -> str:
 if hasattr(self.fixed_format, "logical"):
 return self.fixed_format.logical(line)

 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

def unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.to_sql(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

def _column_from_condition(
 self,
 text: str,

```

```

) -> str:
 value = str(text or "").strip()

 if "=" not in value:
 return ""

 left = value.split("=", 1)[0].strip()
 left = left.split()[-1].strip()

 return NameNormalizer.to_sql(left)

=====
FILE: src\idms_db2_phase2\services\update_statement_normalization_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..reference.patterns.normalization.update_statement_patterns import (
 DCLGEN_DOT_REFERENCE_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 MANUAL_REDESIGN_COMMENT_PATTERN,
 MOVE_TO_BARE_RECORD_PATTERN,
)
from ..reference.rules.normalization.update_statement_normalization_rules import (
 UPDATE_STATEMENT_NORMALIZATION_RULES,
)

class UpdateStatementNormalizationService:
 """
 Performs generic update-program statement normalization.

 Responsibilities:
 - Convert non-SQL DCLGEN dot references to COBOL OF qualification.
 - Avoid changing SQL column syntax inside EXEC SQL blocks.
 - Remove unsafe generated fragments after manual-redesign context by
 delegating SQLCODE cleanup to the SQLCODE guard service.
 - Preserve business logic.

 This service does not define regex patterns and does not hardcode
 program names, table names, records, columns, DCLGEN groups, or host
 variables.
 """

 def __init__(self) -> None:
 self.messages: list[str] = []

 def normalize(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 lines = source.splitlines()
 lines = self._normalize_non_sql_dclgen_references(lines)
 lines = self._normalize_manual_redesign_spacing(lines)

 return "\n".join(lines).rstrip() + "\n"

 def _normalize_non_sql_dclgen_references(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 in_exec_sql = False
 changed = False

 for line in lines:
 logical = self._logical(line)

```

```

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 output.append(line.rstrip())
 continue

 if EXEC_SQL_END_PATTERN.match(logical):
 in_exec_sql = False
 output.append(line.rstrip())
 continue

 if in_exec_sql:
 output.append(line.rstrip())
 continue

 normalized_body = DCLGEN_DOT_REFERENCE_PATTERN.sub(
 self._replace_dot_reference,
 logical,
)

 if normalized_body != logical:
 changed = True
 output.append(self._with_body(line, normalized_body))
 continue

 output.append(line.rstrip())

 if changed:
 self.messages.append(
 "Update statement normalization: converted non-SQL DCLGEN dot references."
)

 return output

def _normalize_manual_redesign_spacing(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 changed = False
 index = 0

 while index < len(lines):
 output.append(lines[index].rstrip())
 logical = self._logical(lines[index])

 if not MANUAL_REDESIGN_COMMENT_PATTERN.match(logical):
 index += 1
 continue

 if index + 1 >= len(lines):
 index += 1
 continue

 next_logical = self._logical(lines[index + 1])

 if next_logical.strip().upper() == "CONTINUE.":
 index += 1
 continue

 output.append(self._with_body(lines[index], " CONTINUE. "))
 changed = True
 index += 1

 if changed:
 self.messages.append(
 "Update statement normalization: added CONTINUE after manual-redesign comment."
)

 return output

def _replace_dot_reference(
 self,
 match: object,
) -> str:
 group = match.group("group")

```

```

 field = match.group("field")
 return f"{field} OF {group}"

def _logical(
 self,
 line: str,
) -> str:
 raw = str(line or "")

 if len(raw) >= 7 and raw[:6].isdigit():
 return raw[7:72].rstrip()

 return raw.rstrip()

def _with_body(
 self,
 original_line: str,
 body: str,
) -> str:
 line = str(original_line or "")
 normalized_body = str(body or "").rstrip()

 if len(line) >= 7 and line[:6].isdigit():
 prefix = line[:7]
 suffix = line[72:] if len(line) > 72 else ""
 return f"{prefix}{normalized_body[:65].ljust(65)}{suffix}"

 return normalized_body

def rules(
 self,
) -> tuple[str, ...]:
 return tuple(UPDATE_STATEMENT_NORMALIZATION_RULES)

```

```

=====
FILE: src\idms_db2_phase2\testing__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\testing\create_sample_inputs.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\testing\local_conversion_runner.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Any

from ..domain.models import ConversionInput, ConversionResult
from ..infrastructure.file_loader import FileLoader
from ..infrastructure.local_uploaded_file import LocalUploadedFile
from ..infrastructure.runtime_context import RuntimeContext
from ..orchestration.conversion_service import ConversionService
from ..parsers.copybook_parser import CopybookParser
from ..parsers.dclgen_parser import DclgenParser
from ..parsers.sheet_mapping_parser import SheetMappingParser

```

```

@dataclass(slots=True)
class LocalConversionRunResult:
 """

```

```
Result for local runner execution.
"""
```

```
conversion_result: ConversionResult
output_cobol_path: Path
diagnostics: list[str]
```

```
class LocalConversionRunner:
 """
 Shared local conversion runner.

 Responsibilities:
 - Validate local runtime file paths.
 - Load input files.
 - Parse Sheet Mapping, DCLGEN, and optional Copybook.
 - Invoke ConversionService.
 - Write generated COBOL to output folder.

 This runner does not implement conversion rules.
 """

 def __init__(
 self,
 file_loader: FileLoader | None = None,
 sheet_mapping_parser: SheetMappingParser | None = None,
 dclgen_parser: DclgenParser | None = None,
 copybook_parser: CopybookParser | None = None,
 conversion_service: ConversionService | None = None,
) -> None:
 self.file_loader = file_loader or FileLoader()
 self.sheet_mapping_parser = sheet_mapping_parser or SheetMappingParser()
 self.dclgen_parser = dclgen_parser or DclgenParser()
 self.copybook_parser = copybook_parser or CopybookParser()
 self.conversion_service = conversion_service or ConversionService()

 def run(
 self,
 runtime_context: RuntimeContext,
 logger: Any | None = None,
) -> LocalConversionRunResult:
 runtime_context.validate_required_files()

 (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
) = self.load_inputs(
 runtime_context=runtime_context,
 logger=logger,
)

 conversion_result = self.conversion_service.convert(
 ConversionInput(
 sheet_mapping_rows=sheet_rows,
 dclgen_columns=dclgen_columns,
 copybook_fields=copybook_fields,
 idms_cobol_text=idms_cobol_text,
 target_program_id=runtime_context.target_program_id,
 auto_fix_pic_length_mismatches=(
 runtime_context.auto_fix_pic_length_mismatches
),
)
)

 output_cobol_path = self._write_output(
 runtime_context=runtime_context,
 conversion_result=conversion_result,
)

 self._log(logger, "DB2 COBOL generation completed.")
 self._log(logger, f"Output file created: {output_cobol_path}")

 return LocalConversionRunResult(
```

```

 conversion_result=conversion_result,
 output_cobol_path=output_cobol_path,
 diagnostics=diagnostics,
)

def load_inputs(
 self,
 runtime_context: RuntimeContext,
 logger: Any | None = None,
) -> tuple[list, list, list, str, list[str]]:
 diagnostics: list[str] = []

 diagnostics.append("START LOAD INPUTS")
 self._log(logger, "START LOAD INPUTS")

 sheet_rows = self._load_sheet_mapping(
 runtime_context=runtime_context,
 diagnostics=diagnostics,
 logger=logger,
)
 dclgen_columns = self._load_dclgen(
 runtime_context=runtime_context,
 diagnostics=diagnostics,
 logger=logger,
)
 copybook_fields = self._load_copybook(
 runtime_context=runtime_context,
 diagnostics=diagnostics,
 logger=logger,
)
 idms_cobol_text = self._load_idms_cobol(
 runtime_context=runtime_context,
 diagnostics=diagnostics,
 logger=logger,
)

 return (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
)

def _load_sheet_mapping(
 self,
 runtime_context: RuntimeContext,
 diagnostics: list[str],
 logger: Any | None,
) -> list:
 assert runtime_context.sheet_mapping_path is not None

 sheet_file = LocalUploadedFile(runtime_context.sheet_mapping_path)
 sheet_rows = self.sheet_mapping_parser.parse_uploaded_file(sheet_file)

 diagnostics.append(f"Sheet Mapping file: {runtime_context.sheet_mapping_path}")
 diagnostics.append(f"Sheet Mapping rows: {len(sheet_rows)}")
 diagnostics.extend(getattr(self.sheet_mapping_parser, "diagnostics", []))
 self._log(logger, f"Sheet Mapping rows: {len(sheet_rows)}")

 return sheet_rows

def _load_dclgen(
 self,
 runtime_context: RuntimeContext,
 diagnostics: list[str],
 logger: Any | None,
) -> list:
 dclgen_texts: list[str] = []

 diagnostics.append(f"DCLGEN file count: {len(runtime_context.dclgen_paths)}")
 self._log(logger, f"DCLGEN file count: {len(runtime_context.dclgen_paths)}")

 for index, dclgen_path in enumerate(runtime_context.dclgen_paths, start=1):
 dclgen_file = LocalUploadedFile(dclgen_path)
 dclgen_text = self.file_loader.read_uploaded_text(dclgen_file)

```

```

 dclgen_texts.append(dclgen_text)

 diagnostics.append(f"DCLGEN {index}: {dclgen_path}")
 diagnostics.append(f"DCLGEN {index} text length: {len(dclgen_text)}")
 self._log(logger, f"DCLGEN {index}: {dclgen_path}")

 dclgen_columns = self.dclgen_parser.parse_many_texts(dclgen_texts)

 diagnostics.append(f"DCLGEN total columns: {len(dclgen_columns)}")
 diagnostics.extend(getattr(self.dclgen_parser, "diagnostics", []))
 self._log(logger, f"DCLGEN total columns: {len(dclgen_columns)}")

 return dclgen_columns

def _load_copybook(
 self,
 runtime_context: RuntimeContext,
 diagnostics: list[str],
 logger: Any | None,
) -> list:
 copybook_text_parts: list[str] = []

 diagnostics.append(f"Copybook file count: {len(runtime_context.copybook_paths)}")
 self._log(logger, f"Copybook file count: {len(runtime_context.copybook_paths)}")

 for index, copybook_path in enumerate(runtime_context.copybook_paths, start=1):
 copybook_file = LocalUploadedFile(copybook_path)
 copybook_text = self.file_loader.read_uploaded_text(copybook_file)
 copybook_text_parts.append(copybook_text)

 diagnostics.append(f"Copybook {index}: {copybook_path}")
 diagnostics.append(f"Copybook {index} text length: {len(copybook_text)}")
 self._log(logger, f"Copybook {index}: {copybook_path}")

 copybook_text = "\n".join(copybook_text_parts)

 if not copybook_text.strip():
 diagnostics.append("Copybook total fields: 0")
 return []

 copybook_fields = self.copybook_parser.parse(copybook_text)

 diagnostics.append(f"Copybook total fields: {len(copybook_fields)}")
 diagnostics.extend(getattr(self.copybook_parser, "diagnostics", []))
 self._log(logger, f"Copybook total fields: {len(copybook_fields)}")

 return copybook_fields

def _load_idms_cobol(
 self,
 runtime_context: RuntimeContext,
 diagnostics: list[str],
 logger: Any | None,
) -> str:
 assert runtime_context.idms_cobol_source_path is not None

 source_file = LocalUploadedFile(runtime_context.idms_cobol_source_path)
 idms_cobol_text = self.file_loader.read_uploaded_text(source_file)

 diagnostics.append(f"IDMS COBOL source file: {runtime_context.idms_cobol_source_path}")
 diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")
 self._log(logger, f"IDMS COBOL source text length: {len(idms_cobol_text)}")

 return idms_cobol_text

def _write_output(
 self,
 runtime_context: RuntimeContext,
 conversion_result: ConversionResult,
) -> Path:
 runtime_context.ensure_output_dir()

 assert runtime_context.idms_cobol_source_path is not None

 date_time = datetime.now().strftime("%d-%m-%Y_%H%M%S")
 code_name = runtime_context.idms_cobol_source_path.stem

```

```

 output_cobol_path = runtime_context.output_dir / f"{code_name}_{date_time}.cbl"

 output_cobol_path.write_text(
 conversion_result.converted_cobol or "",
 encoding="utf-8",
)

 return output_cobol_path

def _log(
 self,
 logger: Any | None,
 message: str,
) -> None:
 if logger is None:
 return

 logger.info(message)

=====
FILE: src\idms_db2_phase2\testing\run_conversion.py
ENCODING: utf-8
=====

from __future__ import annotations

import argparse
import sys
from pathlib import Path

CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]

for path in [PROJECT_ROOT, SRC_DIR]:
 if str(path) not in sys.path:
 sys.path.insert(0, str(path))

from idms_db2_phase2.config.path_settings import (
 DEFAULT_COPYBOOK_CANDIDATE_PATHS,
 DEFAULT_DCLGEN_CANDIDATE_PATHS,
 DEFAULT_OUTPUT_DIR,
 DEFAULT_RETRIEVAL_SOURCE_PATH,
 DEFAULT_SHEET_MAPPING_PATH,
 DEFAULT_UPDATE_SOURCE_PATH,
 LOGS_DIR,
 PROJECT_ROOT,
 SRC_DIR,
 existing_files,
)
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory
from idms_db2_phase2.infrastructure.runtime_context import RuntimeContext
from idms_db2_phase2.testing.local_conversion_runner import LocalConversionRunner

def main() -> None:
 args = parse_args()
 runtime_context = build_runtime_context(args)

 logger = LoggerFactory.create_logger(
 name="idms_db2_local_conversion",
 logs_dir=LOGS_DIR,
)

 runner = LocalConversionRunner()
 result = runner.run(
 runtime_context=runtime_context,
 logger=logger,
)

 print_summary(
 runtime_context=runtime_context,
 output_cobol_path=result.output_cobol_path,
 validation_messages=result.conversion_result.validation_messages,
 diagnostics=result.diagnostics,
)

```

```

)

def parse_args() -> argparse.Namespace:
 parser = argparse.ArgumentParser(
 description="Run local IDMS to DB2 COBOL conversion.",
)
 parser.add_argument(
 "--mode",
 choices=["retrieval", "update"],
 default="retrieval",
 help="Default source file mode when --source is not supplied.",
)
 parser.add_argument(
 "--source",
 default="",
 help="IDMS COBOL source path.",
)
 parser.add_argument(
 "--sheet-mapping",
 default=str(DEFAULT_SHEET_MAPPING_PATH),
 help="Sheet Mapping CSV/XLSX path.",
)
 parser.add_argument(
 "--dclgen",
 action="append",
 default=[],
 help="DCLGEN file path. Can be supplied multiple times.",
)
 parser.add_argument(
 "--copybook",
 action="append",
 default=[],
 help="Optional Copybook file path. Can be supplied multiple times.",
)
 parser.add_argument(
 "--output-dir",
 default=str(DEFAULT_OUTPUT_DIR),
 help="Output directory.",
)
 parser.add_argument(
 "--target-program-id",
 default="",
 help="Target PROGRAM-ID.",
)
 parser.add_argument(
 "--auto-fix-pic-length-mismatches",
 action="store_true",
 help="Enable conservative PIC length auto-fix.",
)
 return parser.parse_args()

def build_runtime_context(
 args: argparse.Namespace,
) -> RuntimeContext:
 source_path = Path(args.source) if args.source else default_source_for_mode(args.mode)

 dclgen_paths = [
 Path(path)
 for path in args.dclgen
]

 if not dclgen_paths:
 dclgen_paths = existing_files(DEFAULT_DCLGEN_CANDIDATE_PATHS)

 copybook_paths = [
 Path(path)
 for path in args.copybook
]

 if not copybook_paths:
 copybook_paths = existing_files(DEFAULT_COPYBOOK_CANDIDATE_PATHS)

 return RuntimeContext(
 project_root=PROJECT_ROOT,

```

```

 input_dir=source_path.parent,
 output_dir=Path(args.output_dir),
 sheet_mapping_path=Path(args.sheet_mapping),
 dclgen_paths=dclgen_paths,
 copybook_paths=copybook_paths,
 idms_cobol_source_path=source_path,
 target_program_id=str(args.target_program_id or "").strip(),
 auto_fix_pic_length_mismatches=bool(args.auto_fix_pic_length_mismatches),
)

```

```

def default_source_for_mode(
 mode: str,
) -> Path:
 if str(mode or "").lower() == "update":
 return DEFAULT_UPDATE_SOURCE_PATH

 return DEFAULT_RETRIEVAL_SOURCE_PATH

```

```

def print_summary(
 runtime_context: RuntimeContext,
 output_cobol_path: Path,
 validation_messages: list[str],
 diagnostics: list[str],
) -> None:
 print("DB2 COBOL generation completed.")
 print(f"Output file created: {output_cobol_path}")
 print("")
 print("Input Summary")
 print("")
 print(f"Project Root: {runtime_context.project_root}")
 print(f"SRC Directory: {SRC_DIR}")
 print(f"Sheet Mapping: {runtime_context.sheet_mapping_path}")
 print(f"IDMS COBOL Source: {runtime_context.idms_cobol_source_path}")
 print(f"DCLGEN Files: {len(runtime_context.dclgen_paths)}")
 print(f"Copybook Files: {len(runtime_context.copybook_paths)}")
 print(f"Target PROGRAM-ID: {runtime_context.target_program_id}")
 print("")
 print("Validation Messages")
 print("")

 if validation_messages:
 for message in validation_messages:
 print(f"- {message}")
 else:
 print("No validation messages.")

 print("")
 print("Diagnostics")
 print("")

 for diagnostic in diagnostics:
 print(f"- {diagnostic}")

```

```

if __name__ == "__main__":
 main()

```

```

=====
FILE: src\idms_db2_phase2\testing\run_retrieval.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import sys
from datetime import datetime
from pathlib import Path

CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]

for path in [PROJECT_ROOT, SRC_DIR]:

```

```

 if str(path) not in sys.path:
 sys.path.insert(0, str(path))

from idms_db2_phase2.config.path_settings import (
 DEFAULT_COPYBOOK_CANDIDATE_PATHS,
 DEFAULT_DCLGEN_CANDIDATE_PATHS,
 DEFAULT_OUTPUT_DIR,
 DEFAULT_RETRIEVAL_SOURCE_PATH,
 DEFAULT_SHEET_MAPPING_PATH,
 LOGS_DIR,
 PROJECT_ROOT,
 SRC_DIR,
 ensure_runtime_directories,
 existing_files,
)
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser

TARGET_PROGRAM_ID = "VMDZ4420"
AUTO_FIX_PIC_LENGTH_MISMATCHES = False

SHEET_MAPPING_PATH = DEFAULT_SHEET_MAPPING_PATH
IDMS_COBOL_SOURCE_PATH = DEFAULT_RETRIEVAL_SOURCE_PATH
OUTPUT_DIR = DEFAULT_OUTPUT_DIR
DCLGEN_PATHS = existing_files(DEFAULT_DCLGEN_CANDIDATE_PATHS)
COPYBOOK_PATHS = existing_files(DEFAULT_COPYBOOK_CANDIDATE_PATHS)

def validate_file_exists(
 file_path: Path,
 label: str,
) -> None:
 if not file_path.exists():
 raise FileNotFoundError(f"{label} file not found: {file_path}")

 if not file_path.is_file():
 raise FileNotFoundError(f"{label} path is not a file: {file_path}")

def validate_inputs() -> None:
 ensure_runtime_directories()

 validate_file_exists(SHEET_MAPPING_PATH, "Sheet Mapping")
 validate_file_exists(IDMS_COBOL_SOURCE_PATH, "IDMS COBOL Source")

 if not DCLGEN_PATHS:
 searched = "\n".join(str(path) for path in DEFAULT_DCLGEN_CANDIDATE_PATHS)
 raise ValueError(
 "At least one DCLGEN file path is required. "
 "No DCLGEN candidate file was found. Searched:\n"
 f"{searched}"
)

 for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
 validate_file_exists(dclgen_path, f"DCLGEN {index}")

 for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
 validate_file_exists(copybook_path, f"Copybook {index}")

 OUTPUT_DIR.mkdir(
 parents=True,
 exist_ok=True,
)

def load_inputs(
 logger,
) -> tuple[list, list, list, str, list[str]]:
 diagnostics: list[str] = []

```

```

sheet_parser = SheetMappingParser()
dclgen_parser = DclgenParser()
copybook_parser = CopybookParser()
file_loader = FileLoader()

diagnostics.append("START LOAD INPUTS")
logger.info("START LOAD INPUTS")

sheet_file = LocalUploadedFile(SHEET_MAPPING_PATH)
sheet_rows = sheet_parser.parse_uploaded_file(sheet_file)

diagnostics.append(f"Sheet Mapping file: {SHEET_MAPPING_PATH}")
diagnostics.append(f"Sheet Mapping rows: {len(sheet_rows)}")
logger.info(f"Sheet Mapping rows: %s", len(sheet_rows))

if hasattr(sheet_parser, "diagnostics"):
 diagnostics.extend(sheet_parser.diagnostics)

dclgen_texts: list[str] = []
diagnostics.append(f"DCLGEN file count: {len(DCLGEN_PATHS)}")
logger.info(f"DCLGEN file count: %s", len(DCLGEN_PATHS))

for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
 dclgen_file = LocalUploadedFile(dclgen_path)
 dclgen_text = file_loader.read_uploaded_text(dclgen_file)
 dclgen_texts.append(dclgen_text)
 diagnostics.append(f"DCLGEN {index}: {dclgen_path}")
 diagnostics.append(f"DCLGEN {index} text length: {len(dclgen_text)}")
 logger.info(f"DCLGEN %s: %s", index, dclgen_path)

dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
diagnostics.append(f"DCLGEN total columns: {len(dclgen_columns)}")
logger.info(f"DCLGEN total columns: %s", len(dclgen_columns))

if hasattr(dclgen_parser, "diagnostics"):
 diagnostics.extend(dclgen_parser.diagnostics)

copybook_text_parts: list[str] = []
diagnostics.append(f"Copybook file count: {len(COPYBOOK_PATHS)}")
logger.info(f"Copybook file count: %s", len(COPYBOOK_PATHS))

for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
 copybook_file = LocalUploadedFile(copybook_path)
 copybook_text = file_loader.read_uploaded_text(copybook_file)
 copybook_text_parts.append(copybook_text)
 diagnostics.append(f"Copybook {index}: {copybook_path}")
 logger.info(f"Copybook %s: %s", index, copybook_path)

copybook_text = "\n".join(copybook_text_parts)

if copybook_text.strip():
 copybook_fields = copybook_parser.parse(copybook_text)
else:
 copybook_fields = []

diagnostics.append(f"Copybook total fields: {len(copybook_fields)}")
logger.info(f"Copybook total fields: %s", len(copybook_fields))

if hasattr(copybook_parser, "diagnostics"):
 diagnostics.extend(copybook_parser.diagnostics)

source_file = LocalUploadedFile(IDMS_COBOL_SOURCE_PATH)
idms_cobol_text = file_loader.read_uploaded_text(source_file)

diagnostics.append(f"IDMS COBOL source file: {IDMS_COBOL_SOURCE_PATH}")
diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")
logger.info(f"IDMS COBOL source text length: %s", len(idms_cobol_text))

return (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
)

```

```

def run_conversion() -> None:
 validate_inputs()

 logger = LoggerFactory.create_logger(
 name="idms_db2_retrieval_conversion",
 logs_dir=LOGS_DIR,
)

 (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
) = load_inputs(logger)

 service = ConversionService()

 result = service.convert(
 ConversionInput(
 sheet_mapping_rows=sheet_rows,
 dclgen_columns=dclgen_columns,
 copybook_fields=copybook_fields,
 idms_cobol_text=idms_cobol_text,
 target_program_id=TARGET_PROGRAM_ID,
 auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES,
)
)

 date_time = datetime.now().strftime("%d-%m-%Y_%H%M%S")
 code_name = IDMS_COBOL_SOURCE_PATH.stem
 output_cobol_path = OUTPUT_DIR / f"{code_name}_{date_time}.cbl"
 output_cobol_path.write_text(
 result.converted_cobol or "",
 encoding="utf-8",
)

 logger.info("DB2 COBOL generation completed.")
 logger.info("Output file created: %s", output_cobol_path)

 print("DB2 COBOL generation completed.")
 print(f"Output file created: {output_cobol_path}")
 print("")
 print("Input Summary")
 print("")
 print(f"Project Root: {PROJECT_ROOT}")
 print(f"SRC Directory: {SRC_DIR}")
 print(f"Sheet Mapping Rows: {len(sheet_rows)}")
 print(f"DCLGEN Columns: {len(dclgen_columns)}")
 print(f"Copybook Fields: {len(copybook_fields)}")
 print(f"COBOL Text Length: {len(idms_cobol_text)}")
 print(f"Target PROGRAM-ID: {TARGET_PROGRAM_ID}")
 print("")
 print("Validation Messages")
 print("")

 if result.validation_messages:
 for message in result.validation_messages:
 print(f"- {message}")
 logger.warning(message)
 else:
 print("No validation messages.")

 print("")
 print("Diagnostics")
 print("")

 for diagnostic in diagnostics:
 print(f"- {diagnostic}")
 logger.info(diagnostic)

if __name__ == "__main__":
 run_conversion()

```

```
=====
FILE: src\idms_db2_phase2\testing\run_update.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import sys
from datetime import datetime
from pathlib import Path

CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]

for path in [PROJECT_ROOT, SRC_DIR]:
 if str(path) not in sys.path:
 sys.path.insert(0, str(path))

from idms_db2_phase2.config.path_settings import (
 DEFAULT_COPYBOOK_CANDIDATE_PATHS,
 DEFAULT_DCLGEN_CANDIDATE_PATHS,
 DEFAULT_OUTPUT_DIR,
 DEFAULT_SHEET_MAPPING_PATH,
 DEFAULT_UPDATE_SOURCE_PATH,
 LOGS_DIR,
 PROJECT_ROOT,
 SRC_DIR,
 ensure_runtime_directories,
 existing_files,
)
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser

TARGET_PROGRAM_ID = "VMDZ1567"
AUTO_FIX_PIC_LENGTH_MISMATCHES = False

SHEET_MAPPING_PATH = DEFAULT_SHEET_MAPPING_PATH
IDMS_COBOL_SOURCE_PATH = DEFAULT_UPDATE_SOURCE_PATH
OUTPUT_DIR = DEFAULT_OUTPUT_DIR
DCLGEN_PATHS = existing_files(DEFAULT_DCLGEN_CANDIDATE_PATHS)
COPYBOOK_PATHS = existing_files(DEFAULT_COPYBOOK_CANDIDATE_PATHS)

def validate_file_exists(
 file_path: Path,
 label: str,
) -> None:
 if not file_path.exists():
 raise FileNotFoundError(f"{label} file not found: {file_path}")

 if not file_path.is_file():
 raise FileNotFoundError(f"{label} path is not a file: {file_path}")

def validate_inputs() -> None:
 ensure_runtime_directories()

 validate_file_exists(SHEET_MAPPING_PATH, "Sheet Mapping")
 validate_file_exists(IDMS_COBOL_SOURCE_PATH, "IDMS COBOL Source")

 if not DCLGEN_PATHS:
 searched = "\n".join(str(path) for path in DEFAULT_DCLGEN_CANDIDATE_PATHS)
 raise ValueError(
 "At least one DCLGEN file path is required. "
 "No DCLGEN candidate file was found. Searched:\n"
 f"{searched}"
)
```

```

)

 for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
 validate_file_exists(dclgen_path, f"DCLGEN {index}")

 for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
 validate_file_exists(copybook_path, f"Copybook {index}")

 OUTPUT_DIR.mkdir(
 parents=True,
 exist_ok=True,
)

def load_inputs(
 logger,
) -> tuple[list, list, list, str, list[str]]:
 diagnostics: list[str] = []

 sheet_parser = SheetMappingParser()
 dclgen_parser = DclgenParser()
 copybook_parser = CopybookParser()
 file_loader = FileLoader()

 diagnostics.append("START LOAD INPUTS")
 logger.info("START LOAD INPUTS")

 sheet_file = LocalUploadedFile(SHEET_MAPPING_PATH)
 sheet_rows = sheet_parser.parse_uploaded_file(sheet_file)

 diagnostics.append(f"Sheet Mapping file: {SHEET_MAPPING_PATH}")
 diagnostics.append(f"Sheet Mapping rows: {len(sheet_rows)}")
 logger.info(f"Sheet Mapping rows: %s", len(sheet_rows))

 if hasattr(sheet_parser, "diagnostics"):
 diagnostics.extend(sheet_parser.diagnostics)

 dclgen_texts: list[str] = []
 diagnostics.append(f"DCLGEN file count: {len(DCLGEN_PATHS)}")
 logger.info(f"DCLGEN file count: %s", len(DCLGEN_PATHS))

 for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
 dclgen_file = LocalUploadedFile(dclgen_path)
 dclgen_text = file_loader.read_uploaded_text(dclgen_file)
 dclgen_texts.append(dclgen_text)
 diagnostics.append(f"DCLGEN {index}: {dclgen_path}")
 diagnostics.append(f"DCLGEN {index} text length: {len(dclgen_text)}")
 logger.info(f"DCLGEN %s: %s", index, dclgen_path)

 dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
 diagnostics.append(f"DCLGEN total columns: {len(dclgen_columns)}")
 logger.info(f"DCLGEN total columns: %s", len(dclgen_columns))

 if hasattr(dclgen_parser, "diagnostics"):
 diagnostics.extend(dclgen_parser.diagnostics)

 copybook_text_parts: list[str] = []
 diagnostics.append(f"Copybook file count: {len(COPYBOOK_PATHS)}")
 logger.info(f"Copybook file count: %s", len(COPYBOOK_PATHS))

 for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
 copybook_file = LocalUploadedFile(copybook_path)
 copybook_text = file_loader.read_uploaded_text(copybook_file)
 copybook_text_parts.append(copybook_text)
 diagnostics.append(f"Copybook {index}: {copybook_path}")
 logger.info(f"Copybook %s: %s", index, copybook_path)

 copybook_text = "\n".join(copybook_text_parts)

 if copybook_text.strip():
 copybook_fields = copybook_parser.parse(copybook_text)
 else:
 copybook_fields = []

 diagnostics.append(f"Copybook total fields: {len(copybook_fields)}")
 logger.info(f"Copybook total fields: %s", len(copybook_fields))

```

```

if hasattr(copybook_parser, "diagnostics"):
 diagnostics.extend(copybook_parser.diagnostics)

source_file = LocalUploadedFile(IDMS_COBOL_SOURCE_PATH)
idms_cobol_text = file_loader.read_uploaded_text(source_file)

diagnostics.append(f"IDMS COBOL source file: {IDMS_COBOL_SOURCE_PATH}")
diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")
logger.info("IDMS COBOL source text length: %s", len(idms_cobol_text))

return (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
)

def run_conversion() -> None:
 validate_inputs()

 logger = LoggerFactory.create_logger(
 name="idms_db2_update_conversion",
 logs_dir=LOGS_DIR,
)

 (
 sheet_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text,
 diagnostics,
) = load_inputs(logger)

 service = ConversionService()

 result = service.convert(
 ConversionInput(
 sheet_mapping_rows=sheet_rows,
 dclgen_columns=dclgen_columns,
 copybook_fields=copybook_fields,
 idms_cobol_text=idms_cobol_text,
 target_program_id=TARGET_PROGRAM_ID,
 auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES,
)
)

 date_time = datetime.now().strftime("%d-%m-%Y_%H%M%S")
 code_name = IDMS_COBOL_SOURCE_PATH.stem
 output_cobol_path = OUTPUT_DIR / f"{code_name}_{date_time}.cbl"
 output_cobol_path.write_text(
 result.converted_cobol or "",
 encoding="utf-8",
)

 logger.info("DB2 COBOL generation completed.")
 logger.info("Output file created: %s", output_cobol_path)

 print("DB2 COBOL generation completed.")
 print(f"Output file created: {output_cobol_path}")
 print("")
 print("Input Summary")
 print("")
 print(f"Project Root: {PROJECT_ROOT}")
 print(f"SRC Directory: {SRC_DIR}")
 print(f"Sheet Mapping Rows: {len(sheet_rows)}")
 print(f"DCLGEN Columns: {len(dclgen_columns)}")
 print(f"Copybook Fields: {len(copybook_fields)}")
 print(f"COBOL Text Length: {len(idms_cobol_text)}")
 print(f"Target PROGRAM-ID: {TARGET_PROGRAM_ID}")
 print("")
 print("Validation Messages")
 print("")

```

```

if result.validation_messages:
 for message in result.validation_messages:
 print(f"- {message}")
 logger.warning(message)
else:
 print("No validation messages.")

print("")
print("Diagnostics")
print("")

for diagnostic in diagnostics:
 print(f"- {diagnostic}")
 logger.info(diagnostic)

if __name__ == "__main__":
 run_conversion()

=====
FILE: src\idms_db2_phase2\transformers__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\transformers\cobol_transformer.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..domain.models import IdmsOperation
from ..parsers.cobol_parser import CobolParser
from ..reference.patterns.cobol.structure_patterns import (
 DIVISION_PATTERN,
 PROGRAM_ID_PATTERN,
)
from ..reference.patterns.db2.infrastructure_patterns import (
 SQL_ERROR_PARAGRAPH_PATTERN,
)
from ..reference.patterns.idms.declarative_patterns import (
 IDMS_DECLARATIVE_PATTERNS,
)
from ..transformers.idms_statement_transformer import IdmsStatementTransformer

class CobolTransformer:
 """
 Converts IDMS COBOL statements to DB2-compatible COBOL.

 Responsibilities:
 - Preserve original COBOL business flow.
 - Convert compiler option to DB2-compatible option.
 - Replace target PROGRAM-ID.
 - Remove residual IDMS declarative/control lines.
 - Remove or convert residual IDMS executable lines.
 """

 def __init__(
 self,
 idms_statement_transformer: IdmsStatementTransformer,
 cobol_parser: CobolParser | None = None,
) -> None:
 self.idms_statement_transformer = idms_statement_transformer
 self.cobol_parser = cobol_parser or CobolParser()
 self.messages: list[str] = []

 def transform(
 self,
 cobol_text: str,
 target_program_id: str = "",
) -> tuple[str, list[str], list[IdmsOperation]]:

```

```

self.messages = []

source = str(cobol_text or "")

if not source.strip():
 return "", ["COBOL source text is empty."], []

operations = self.cobol_parser.analyze(source)
output_lines: list[str] = []
current_division = ""
opened_sets: list[str] = []
sql_error_paragraph = self._sql_error_paragraph(source)

for line in source.replace("\r\n", "\n").replace("\r", "\n").splitlines():
 logical = self.cobol_parser.logical_line(line)
 division = self._division_from_line(logical)

 if division:
 current_division = division

 if self._is_idms_declarative_line(logical):
 output_lines.append("*DB2: Removed IDMS declarative/control statement.")
 continue

 transformed_lines, opened_set = self.idms_statement_transformer.transform_line(
 line=logical,
 current_division=current_division,
 sql_error_paragraph=sql_error_paragraph,
)

 if transformed_lines is not None:
 output_lines.extend(transformed_lines)

 if opened_set:
 opened_sets.append(opened_set)

 continue

 output_lines.append(line)

converted = "\n".join(output_lines).rstrip() + "\n"
converted = self._replace_program_id(
 source=converted,
 target_program_id=target_program_id,
)
converted = self._ensure_cbl_arith_extend(converted)

return converted, self.messages, operations

def _replace_program_id(
 self,
 source: str,
 target_program_id: str,
) -> str:
 target = str(target_program_id or "").strip().upper()

 if not target:
 return source

 def replace_program_id(match) -> str:
 line = match.group(0)
 return line.replace(match.group(1), target)

 return PROGRAM_ID_PATTERN.sub(replace_program_id, source, count=1)

def _ensure_cbl_arith_extend(
 self,
 source: str,
) -> str:
 lines = source.splitlines()

 if not lines:
 return source

 first_non_blank_index = 0

```

```

for index, line in enumerate(lines):
 if line.strip():
 first_non_blank_index = index
 break

first_line = lines[first_non_blank_index].strip().upper()

if first_line.startswith("CBL ") and "ARITH(EXTEND)" in first_line:
 return source.rstrip() + "\n"

if first_line.startswith("CBL "):
 lines[first_non_blank_index] = "CBL ARITH(EXTEND)"
 return "\n".join(lines).rstrip() + "\n"

lines.insert(first_non_blank_index, "CBL ARITH(EXTEND)")
return "\n".join(lines).rstrip() + "\n"

def _division_from_line(
 self,
 logical: str,
) -> str:
 match = DIVISION_PATTERN.match(logical)

 if not match:
 return ""

 return match.group(1).upper()

def _sql_error_paragraph(
 self,
 source: str,
) -> str:
 match = SQL_ERROR_PARAGRAPH_PATTERN.search(source)

 if not match:
 return "SQLERROR"

 return match.group(1).upper().replace("SQL-ERROR", "SQLERROR")

def _is_idms_declarative_line(
 self,
 logical: str,
) -> bool:
 stripped = str(logical or "").strip()

 if not stripped:
 return False

 for pattern in IDMS_DECLARATIVE_PATTERNS:
 if pattern.search(stripped):
 return True

 return False

```

```

=====
FILE: src\idms_db2_phase2\transformers\field_reference_rewriter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.cobol.field_reference_patterns import (
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 QUALIFIED_REFERENCE_PATTERN,
)
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.host_variable_resolver import HostVariableResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

```

```

class FieldReferenceRewriter:
 """
 Rewrites safe qualified IDMS field references to DCLGEN host references.

```

Supported:

- FIELD OF IDMS-RECORD
- FIELD IN IDMS-RECORD

This transformer does not rewrite bare words globally.  
"""

```
def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

def rewrite(
 self,
 text: str,
) -> str:
 self.messages = []
 source = str(text or "")

 if not source:
 return ""

 output_lines: list[str] = []
 inside_exec_sql = False

 for line in source.splitlines():
 logical = line.strip()

 if EXEC_SQL_START_PATTERN.match(logical):
 inside_exec_sql = True
 output_lines.append(line)
 continue

 if EXEC_SQL_END_PATTERN.match(logical):
 inside_exec_sql = False
 output_lines.append(line)
 continue

 if inside_exec_sql or self._is_comment_or_blank(logical):
 output_lines.append(line)
 continue

 output_lines.append(self._rewrite_line(line))

 return "\n".join(output_lines).rstrip() + "\n"

def _rewrite_line(
 self,
 line: str,
) -> str:
 def replace_reference(match) -> str:
 field_name = NameNormalizer.normalize(match.group("field"))
 record_name = NameNormalizer.normalize(match.group("record"))

 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 self.messages.append(
 f"Field rewrite skipped: no DB2 table for record {record_name}."
)
 return match.group(0)

 column_name = self.mapping_repository.db2_column_for_source_field(
 record_name=record_name,
 source_field_name=field_name,
)

 if not column_name:
 self.messages.append(
```

```

 f"Field rewrite skipped: no DB2 column for "
 f"{field_name} OF {record_name}."
)
 return match.group(0)

 host_reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=column_name,
)

 if not host_reference:
 self.messages.append(
 f"Field rewrite skipped: no DCLGEN host for "
 f"{column_name} on {table_name}."
)
 return match.group(0)

 return host_reference

 return QUALIFIED_REFERENCE_PATTERN.sub(replace_reference, line)

def _is_comment_or_blank(
 self,
 line: str,
) -> bool:
 stripped = str(line or "").strip()

 if not stripped:
 return True

 if stripped.startswith("*"):
 return True

 if stripped.startswith("/"):
 return True

 return False

```

```

=====
FILE: src\idms_db2_phase2\transformers\idms_statement_transformer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..generators.sql_error_generator import SqlErrorGenerator
from ..generators.sql_generator import SqlGenerator
from ..reference.patterns.idms.statement_patterns import (
 BIND_STATEMENT_PATTERN,
 COMMIT_PATTERN,
 CONNECT_STATEMENT_PATTERN,
 DISCONNECT_STATEMENT_PATTERN,
 ERASE_PATTERN,
 FIND_CURRENT_PATTERN,
 FIND_FIRST_PATTERN,
 FINISH_PATTERN,
 MODIFY_PATTERN,
 OBTAIN_CALC_PATTERN,
 OBTAIN_CALC_REVERSED_PATTERN,
 OBTAIN_FIRST_NEXT_PATTERN,
 READY_PATTERN,
 STORE_PATTERN,
 USAGE_MODE_PATTERN,
)
from ..resolvers.cursor_name_resolver import CursorNameResolver
from ..resolvers.table_name_resolver import TableNameResolver
from ..services.name_normalizer import NameNormalizer

```

```

class IdmsStatementTransformer:
 """
 Converts executable IDMS statements to DB2-compatible COBOL.

 Regex patterns live in reference/patterns.
 SQL text generation lives in SqlGenerator.
 """

```

```
"""
```

```
def __init__(
 self,
 sql_generator: SqlGenerator,
 sql_error_generator: SqlErrorGenerator,
 table_name_resolver: TableNameResolver,
 cursor_name_resolver: CursorNameResolver,
) -> None:
 self.sql_generator = sql_generator
 self.sql_error_generator = sql_error_generator
 self.table_name_resolver = table_name_resolver
 self.cursor_name_resolver = cursor_name_resolver
 self.messages: list[str] = []

def transform_line(
 self,
 line: str,
 current_division: str = "",
 sql_error_paragraph: str = "SQLERROR",
) -> tuple[list[str] | None, str]:
 stripped_line = str(line or "").strip()
 upper = stripped_line.upper()

 if not stripped_line:
 return None, ""

 converted = self._convert_declarative_or_control(
 upper=upper,
 stripped_line=stripped_line,
 current_division=current_division,
)

 if converted is not None:
 return converted, ""

 converted = self._convert_obtain_calc(
 upper=upper,
 stripped_line=stripped_line,
 current_division=current_division,
)

 if converted is not None:
 return converted, ""

 converted, opened_set = self._convert_obtain_first_next(
 upper=upper,
 stripped_line=stripped_line,
 current_division=current_division,
)

 if converted is not None:
 return converted, opened_set

 converted = self._convert_store(
 upper=upper,
 stripped_line=stripped_line,
 current_division=current_division,
 sql_error_paragraph=sql_error_paragraph,
)

 if converted is not None:
 return converted, ""

 converted = self._convert_modify(
 upper=upper,
 stripped_line=stripped_line,
 current_division=current_division,
 sql_error_paragraph=sql_error_paragraph,
)

 if converted is not None:
 return converted, ""

 converted = self._convert_erase(
 upper=upper,
```

```

 stripped_line=stripped_line,
 current_division=current_division,
 sql_error_paragraph=sql_error_paragraph,
)

 if converted is not None:
 return converted, ""

 converted = self._convert_commit_or_finish(upper)

 if converted is not None:
 return converted, ""

 return None, ""

def _convert_declarative_or_control(
 self,
 upper: str,
 stripped_line: str,
 current_division: str,
) -> list[str] | None:
 if READY_PATTERN.search(upper):
 return self._removed_statement("READY")

 if BIND_STATEMENT_PATTERN.search(upper):
 return self._removed_statement("BIND")

 if USAGE_MODE_PATTERN.search(upper):
 return self._removed_statement("USAGE MODE")

 if CONNECT_STATEMENT_PATTERN.search(upper):
 return self._removed_statement("CONNECT")

 if DISCONNECT_STATEMENT_PATTERN.search(upper):
 return self._removed_statement("DISCONNECT")

 if current_division != "PROCEDURE":
 return None

 return None

def _convert_obtain_calc(
 self,
 upper: str,
 stripped_line: str,
 current_division: str,
) -> list[str] | None:
 match = OBTAIN_CALC_PATTERN.search(upper)

 if not match:
 match = OBTAIN_CALC_REVERSED_PATTERN.search(upper)

 if not match:
 return None

 record = NameNormalizer.normalize(match.group("record"))
 cobol_record = NameNormalizer.to_cobol(record)

 if current_division != "PROCEDURE":
 return [
 f"*DB2: OBTAIN CALC ignored outside PROCEDURE DIVISION: {stripped_line}",
]

 lines = [
 f"*DB2: Converted OBTAIN CALC for {cobol_record}.",
]
 lines.extend(self.sql_generator.select_by_key(record))
 lines.extend(self.sql_error_generator.sqlcode_check())
 return lines

def _convert_obtain_first_next(
 self,
 upper: str,
 stripped_line: str,
 current_division: str,
) -> tuple[list[str] | None, str]:

```

```

match = OBTAIN_FIRST_NEXT_PATTERN.search(upper)

if not match:
 match = FIND_FIRST_PATTERN.search(upper)

if not match:
 return None, ""

record = NameNormalizer.normalize(match.group("record"))
set_name = NameNormalizer.normalize(match.group("set"))

if current_division != "PROCEDURE":
 return [
 f"*DB2: cursor retrieval ignored outside PROCEDURE DIVISION: {stripped_line}",
], ""

return [
 f"*DB2: Converted cursor retrieval for {NameNormalizer.to_cobol(record)}.",
 f"*DB2: Cursor paragraphs will be generated for set {NameNormalizer.to_cobol(set_name)}.",
 "CONTINUE.",
], set_name

def _convert_store(
 self,
 upper: str,
 stripped_line: str,
 current_division: str,
 sql_error_paragraph: str,
) -> list[str] | None:
 match = STORE_PATTERN.search(upper)

 if not match:
 return None

 record = NameNormalizer.normalize(match.group("record"))

 if current_division != "PROCEDURE":
 return [
 f"*DB2: STORE ignored outside PROCEDURE DIVISION: {stripped_line}",
]

 lines = [
 f"*DB2: Converted STORE for {NameNormalizer.to_cobol(record)}.",
]
 lines.extend(self.sql_generator.insert(record))
 lines.extend(self.sql_error_generator.sqlcode_check(sql_error_paragraph))
 return lines

def _convert_modify(
 self,
 upper: str,
 stripped_line: str,
 current_division: str,
 sql_error_paragraph: str,
) -> list[str] | None:
 match = MODIFY_PATTERN.search(upper)

 if not match:
 return None

 record = NameNormalizer.normalize(match.group("record"))

 if current_division != "PROCEDURE":
 return [
 f"*DB2: MODIFY ignored outside PROCEDURE DIVISION: {stripped_line}",
]

 lines = [
 f"*DB2: Converted MODIFY for {NameNormalizer.to_cobol(record)}.",
]
 lines.extend(self.sql_generator.update(record))
 lines.extend(self.sql_error_generator.sqlcode_check(sql_error_paragraph))
 return lines

def _convert_erase(
 self,

```

```

 upper: str,
 stripped_line: str,
 current_division: str,
 sql_error_paragraph: str,
) -> list[str] | None:
 match = ERASE_PATTERN.search(upper)

 if not match:
 return None

 record = NameNormalizer.normalize(match.group("record"))

 if current_division != "PROCEDURE":
 return [
 f"*DB2: ERASE ignored outside PROCEDURE DIVISION: {stripped_line}",
]

 lines = [
 f"*DB2: Converted ERASE for {NameNormalizer.to_cobol(record)}.",
]
 lines.extend(self.sql_generator.delete(record))
 lines.extend(self.sql_error_generator.sqlcode_check(sql_error_paragraph))
 return lines

 def _convert_commit_or_finish(
 self,
 upper: str,
) -> list[str] | None:
 if COMMIT_PATTERN.search(upper):
 return self.sql_generator.commit()

 if FINISH_PATTERN.search(upper):
 return [
 "*DB2: Converted FINISH to COMMIT.",
 *self.sql_generator.commit(),
]

 return None

 def _removed_statement(
 self,
 statement_name: str,
) -> list[str]:
 return [
 f"*DB2: Removed IDMS {statement_name} statement.",
 "CONTINUE.",
]

```

```

=====
FILE: src\idms_db2_phase2\transformers\pic_length_auto_fixer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from ..reference.patterns.cobol.pic_clause_patterns import (
 MOVE_STATEMENT_PATTERN,
 WORKING_STORAGE_PIC_PATTERN,
)
from ..services.name_normalizer import NameNormalizer

```

```

class PicLengthAutoFixer:

```

```

 """
 Expands target PIC length when a generated MOVE would otherwise
 move a larger elementary source into a smaller elementary target.

```

```

 This transformer is conservative:

```

- It only uses clear MOVE source/target pairs.
- It only updates target declarations that already exist in converted COBOL.
- It does not invent fields.
- It does not hardcode business fields.

```

 """

```

```

 def __init__(

```

```

 self,
) -> None:
 self.messages: list[str] = []

def fix(
 self,
 source_cobol_text: str,
 converted_cobol_text: str,
) -> str:
 self.messages = []

 if not converted_cobol_text:
 return converted_cobol_text

 combined_text = "\n".join(
 [
 str(source_cobol_text or ""),
 str(converted_cobol_text or ""),
]
)

 pic_lengths = self._collect_pic_lengths(combined_text)

 if not pic_lengths:
 self.messages.append("PIC length auto-fix: no PIC fields found.")
 return converted_cobol_text

 move_pairs = self._collect_moves(converted_cobol_text)

 if not move_pairs:
 self.messages.append("PIC length auto-fix: no MOVE source/target pairs found.")
 return converted_cobol_text

 required_lengths: dict[str, int] = {}

 for source_name, target_name in move_pairs:
 source_length = pic_lengths.get(source_name)
 target_length = pic_lengths.get(target_name)

 if not source_length or not target_length:
 continue

 if source_length <= target_length:
 continue

 required_lengths[target_name] = max(
 required_lengths.get(target_name, target_length),
 source_length,
)

 if not required_lengths:
 self.messages.append("PIC length auto-fix: no target PIC expansion required.")
 return converted_cobol_text

 return self._apply_pic_expansions(
 text=converted_cobol_text,
 required_lengths=required_lengths,
)

def _collect_pic_lengths(
 self,
 text: str,
) -> dict[str, int]:
 output: dict[str, int] = {}

 for line in str(text or "").splitlines():
 match = WORKING_STORAGE_PIC_PATTERN.match(line)

 if not match:
 continue

 name = NameNormalizer.to_cobol(match.group("name"))
 pic = str(match.group("pic") or "")
 length = self._pic_length(pic)

 if name and length:

```

```

 output[name] = length

 return output

def _collect_moves(
 self,
 text: str,
) -> list[tuple[str, str]]:
 output: list[tuple[str, str]] = []

 for line in str(text or "").splitlines():
 match = MOVE_STATEMENT_PATTERN.match(line)

 if not match:
 continue

 source_name = NameNormalizer.to_cobol(match.group("source"))
 target_name = NameNormalizer.to_cobol(match.group("target"))

 if source_name and target_name:
 output.append((source_name, target_name))

 return output

def _apply_pic_expansions(
 self,
 text: str,
 required_lengths: dict[str, int],
) -> str:
 output_lines: list[str] = []

 for line in str(text or "").splitlines():
 match = WORKING_STORAGE_PIC_PATTERN.match(line)

 if not match:
 output_lines.append(line)
 continue

 name = NameNormalizer.to_cobol(match.group("name"))
 required_length = required_lengths.get(name)

 if not required_length:
 output_lines.append(line)
 continue

 pic = str(match.group("pic") or "")
 updated_pic = self._replace_pic_length(
 pic=pic,
 required_length=required_length,
)

 if updated_pic == pic:
 output_lines.append(line)
 continue

 output_lines.append(
 f"{match.group('prefix')}{updated_pic}{match.group('suffix')}"
)
 self.messages.append(
 f"PIC length auto-fix: expanded {name} to length {required_length}."
)

 return "\n".join(output_lines).rstrip() + "\n"

def _pic_length(
 self,
 pic: str,
) -> int:
 text = str(pic or "").upper().strip()

 if not text:
 return 0

 if "(" in text and ")" in text:
 start = text.find("(")
 end = text.find(")", start + 1)

```

```

 if end > start:
 value = text[start + 1 : end].strip()

 if value.isdigit():
 return int(value)

 return text.count("9") + text.count("X")

def _replace_pic_length(
 self,
 pic: str,
 required_length: int,
) -> str:
 text = str(pic or "").upper().strip()

 if not text:
 return text

 if text.startswith("S9"):
 return f"S9({required_length})"

 if text.startswith("9"):
 return f"9({required_length})"

 if text.startswith("X"):
 return f"X({required_length})"

 return text

```

```

=====
FILE: src\idms_db2_phase2\ui__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\ui\app_state.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import streamlit as st

```

```

SESSION_DEFAULTS = {
 "sheet_mapping_rows": [],
 "dclgen_columns": [],
 "copybook_fields": [],
 "idms_cobol_text": "",
 "idms_cobol_source_name": "",
 "converted_cobol": "",
 "converted_cobol_file_name": "converted_db2_cobol.cbl",
 "validation_messages": [],
 "operations": [],
 "metadata_overview": {},
 "record_summary": [],
 "column_summary": [],
 "relationship_summary": [],
 "sheet_mapping_preview": [],
 "generated": False,
 "loaded": False,
 "diagnostics": [],
 "uploaded_file_names": {},
 "auto_fix_pic_length_mismatches": False,
}

```

```

class AppState:
 """
 Streamlit session-state helper.

```

This module owns UI state defaults only.  
 It does not parse files and does not run conversion logic.  
 """

```
@staticmethod
def initialize() -> None:
 for key, value in SESSION_DEFAULTS.items():
 if key in st.session_state:
 continue

 if isinstance(value, list):
 st.session_state[key] = []
 continue

 if isinstance(value, dict):
 st.session_state[key] = {}
 continue

 st.session_state[key] = value

@staticmethod
def reset_generated_output() -> None:
 st.session_state.converted_cobol = ""
 st.session_state.converted_cobol_file_name = "converted_db2_cobol.cbl"
 st.session_state.validation_messages = []
 st.session_state.operations = []
 st.session_state.metadata_overview = {}
 st.session_state.record_summary = []
 st.session_state.column_summary = []
 st.session_state.relationship_summary = []
 st.session_state.sheet_mapping_preview = []
 st.session_state.generated = False

@staticmethod
def set_loaded_inputs(
 sheet_mapping_rows,
 dclgen_columns,
 copybook_fields,
 idms_cobol_text: str,
 idms_cobol_source_name: str,
 diagnostics: list[str],
 uploaded_file_names: dict[str, object],
) -> None:
 st.session_state.sheet_mapping_rows = sheet_mapping_rows or []
 st.session_state.dclgen_columns = dclgen_columns or []
 st.session_state.copybook_fields = copybook_fields or []
 st.session_state.idms_cobol_text = idms_cobol_text or ""
 st.session_state.idms_cobol_source_name = idms_cobol_source_name or ""
 st.session_state.diagnostics = diagnostics or []
 st.session_state.uploaded_file_names = uploaded_file_names or {}
 st.session_state.loaded = True

 AppState.reset_generated_output()

@staticmethod
def set_conversion_result(
 result,
 converted_cobol_file_name: str,
 diagnostics: list[str],
) -> None:
 converted_cobol = getattr(result, "converted_cobol", "") or ""

 st.session_state.converted_cobol = converted_cobol
 st.session_state.converted_cobol_file_name = (
 converted_cobol_file_name or "converted_db2_cobol.cbl"
)

 st.session_state.validation_messages = (
 getattr(result, "validation_messages", []) or []
)
 st.session_state.operations = getattr(result, "operations", []) or []
 st.session_state.metadata_overview = (
 getattr(result, "metadata_overview", {}) or {}
)
 st.session_state.record_summary = (
 getattr(result, "record_summary", []) or []
)
```

```

)
 st.session_state.column_summary = (
 getattr(result, "column_summary", []) or []
)
 st.session_state.relationship_summary = (
 getattr(result, "relationship_summary", []) or []
)
 st.session_state.sheet_mapping_preview = (
 getattr(result, "sheet_mapping_preview", []) or []
)

 st.session_state.generated = bool(converted_cobol)
 st.session_state.diagnostics = diagnostics or []

=====
FILE: src\idms_db2_phase2\ui\conversion_controller.py
ENCODING: utf-8
=====

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime

from ..domain.models import ConversionInput
from ..orchestration.conversion_service import ConversionService

@dataclass(slots=True)
class UiConversionRequest:
 """
 UI conversion request.

 UI layer passes already parsed input metadata to the conversion service.
 """

 sheet_mapping_rows: list
 dclgen_columns: list
 copybook_fields: list
 idms_cobol_text: str
 idms_cobol_source_name: str
 target_program_id: str
 auto_fix_pic_length_mismatches: bool
 diagnostics: list[str]

@dataclass(slots=True)
class UiConversionResponse:
 """
 UI conversion response.

 Carries service result and output file name.
 """

 result: object
 converted_cobol_file_name: str
 diagnostics: list[str]

class UiConversionController:
 """
 Runs conversion for the UI.

 This controller delegates conversion to ConversionService.
 It does not implement conversion rules.
 """

 def __init__(
 self,
 conversion_service: ConversionService | None = None,
) -> None:
 self.conversion_service = conversion_service or ConversionService()

 def generate(
 self,

```

```

 request: UiConversionRequest,
) -> UiConversionResponse:
 diagnostics = list(request.diagnostics or [])
 diagnostics.append("START GENERATE DB2 COBOL")

 result = self.conversion_service.convert(
 ConversionInput(
 sheet_mapping_rows=request.sheet_mapping_rows,
 dclgen_columns=request.dclgen_columns,
 copybook_fields=request.copybook_fields,
 idms_cobol_text=request.idms_cobol_text,
 target_program_id=str(request.target_program_id or "").strip(),
 auto_fix_pic_length_mismatches=(
 request.auto_fix_pic_length_mismatches
),
)
)

 source_name = request.idms_cobol_source_name or "converted_db2_cobol"
 stem = source_name.rsplit(".", 1)[0]
 timestamp = datetime.now().strftime("%d-%m-%Y_%H%M%S")

 return UiConversionResponse(
 result=result,
 converted_cobol_file_name=f"{stem}_{timestamp}.cbl",
 diagnostics=diagnostics,
)

```

```

=====
FILE: src\idms_db2_phase2\ui\input_loader.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from ..infrastructure.file_loader import FileLoader
from ..parsers.copybook_parser import CopybookParser
from ..parsers.dclgen_parser import DclgenParser
from ..parsers.sheet_mapping_parser import SheetMappingParser

```

```

@dataclass(slots=True)
class LoadedInputBundle:
 """
 Parsed UI input bundle.

 This object carries parsed inputs plus diagnostics.
 """

 sheet_mapping_rows: list
 dclgen_columns: list
 copybook_fields: list
 idms_cobol_text: str
 idms_cobol_source_name: str
 diagnostics: list[str]
 uploaded_file_names: dict[str, object]

```

```

class UiInputLoader:
 """
 Loads and parses uploaded UI inputs.

 This class coordinates parsers only.
 It does not perform conversion.
 """

 def __init__(
 self,
 file_loader: FileLoader | None = None,
 sheet_mapping_parser: SheetMappingParser | None = None,
 dclgen_parser: DclgenParser | None = None,
 copybook_parser: CopybookParser | None = None,
):

```

```

) -> None:
 self.file_loader = file_loader or FileLoader()
 self.sheet_mapping_parser = sheet_mapping_parser or SheetMappingParser()
 self.dclgen_parser = dclgen_parser or DclgenParser()
 self.copybook_parser = copybook_parser or CopybookParser()

def load(
 self,
 sheet_mapping_file: Any,
 dclgen_files: list[Any] | None,
 copybook_files: list[Any] | None,
 idms_cobol_source_file: Any,
) -> LoadedInputBundle:
 diagnostics: list[str] = []
 uploaded_file_names: dict[str, object] = {}

 diagnostics.append("START LOAD INPUTS")

 sheet_rows = self._load_sheet_mapping(
 sheet_mapping_file=sheet_mapping_file,
 diagnostics=diagnostics,
 uploaded_file_names=uploaded_file_names,
)

 dclgen_columns = self._load_dclgen_files(
 dclgen_files=dclgen_files,
 diagnostics=diagnostics,
 uploaded_file_names=uploaded_file_names,
)

 copybook_fields = self._load_copybook_files(
 copybook_files=copybook_files,
 diagnostics=diagnostics,
 uploaded_file_names=uploaded_file_names,
)

 idms_cobol_text, idms_cobol_source_name = self._load_idms_cobol_source(
 idms_cobol_source_file=idms_cobol_source_file,
 diagnostics=diagnostics,
 uploaded_file_names=uploaded_file_names,
)

 return LoadedInputBundle(
 sheet_mapping_rows=sheet_rows,
 dclgen_columns=dclgen_columns,
 copybook_fields=copybook_fields,
 idms_cobol_text=idms_cobol_text,
 idms_cobol_source_name=idms_cobol_source_name,
 diagnostics=diagnostics,
 uploaded_file_names=uploaded_file_names,
)

def _load_sheet_mapping(
 self,
 sheet_mapping_file: Any,
 diagnostics: list[str],
 uploaded_file_names: dict[str, object],
) -> list:
 if sheet_mapping_file is None:
 diagnostics.append("Sheet Mapping file not uploaded.")
 uploaded_file_names["sheet_mapping_file"] = ""
 return []

 uploaded_file_names["sheet_mapping_file"] = str(
 sheet_mapping_file.name or ""
)

 try:
 rows = self.sheet_mapping_parser.parse_uploaded_file(
 sheet_mapping_file
)
 diagnostics.append(f"Sheet Mapping uploaded: {sheet_mapping_file.name}")
 diagnostics.append(f"Sheet Mapping parsed rows: {len(rows)}")
 diagnostics.extend(
 getattr(self.sheet_mapping_parser, "diagnostics", [])
)

```

```

 return rows

 except Exception as exc:
 diagnostics.append(f"Sheet Mapping parse failed: {exc}")
 return []

def _load_dclgen_files(
 self,
 dclgen_files: list[Any] | None,
 diagnostics: list[str],
 uploaded_file_names: dict[str, object],
) -> list:
 dclgen_file_names: list[str] = []
 dclgen_texts: list[str] = []

 if not dclgen_files:
 diagnostics.append("DCLGEN file not uploaded.")
 uploaded_file_names["dclgen_files"] = []
 return []

 diagnostics.append(f"DCLGEN file count: {len(dclgen_files)}")

 for index, file in enumerate(dclgen_files, start=1):
 file_name = str(file.name or "")
 dclgen_file_names.append(file_name)

 try:
 text = self.file_loader.read_uploaded_text(file)
 dclgen_texts.append(text)
 diagnostics.append(f"DCLGEN {index}: {file_name}")
 diagnostics.append(f"DCLGEN {index} text length: {len(text)}")

 except Exception as exc:
 diagnostics.append(f"DCLGEN read failed for {file_name}: {exc}")

 uploaded_file_names["dclgen_files"] = dclgen_file_names

 try:
 if hasattr(self.dclgen_parser, "parse_many_texts"):
 columns = self.dclgen_parser.parse_many_texts(dclgen_texts)
 else:
 columns = []

 for index, text in enumerate(dclgen_texts, start=1):
 parsed_columns = self.dclgen_parser.parse(
 text=text,
 source_label=f"DCLGEN file {index}",
)
 columns.extend(parsed_columns)

 diagnostics.append(f"DCLGEN parsed columns: {len(columns)}")
 diagnostics.extend(getattr(self.dclgen_parser, "diagnostics", []))
 return columns

 except Exception as exc:
 diagnostics.append(f"DCLGEN parse failed: {exc}")
 return []

def _load_copybook_files(
 self,
 copybook_files: list[Any] | None,
 diagnostics: list[str],
 uploaded_file_names: dict[str, object],
) -> list:
 copybook_file_names: list[str] = []
 copybook_text_parts: list[str] = []

 if not copybook_files:
 diagnostics.append("Copybook file not uploaded.")
 uploaded_file_names["copybook_files"] = []
 return []

 diagnostics.append(f"Copybook file count: {len(copybook_files)}")

 for index, file in enumerate(copybook_files, start=1):
 file_name = str(file.name or "")

```

```

 copybook_file_names.append(file_name)

 try:
 text = self.file_loader.read_uploaded_text(file)
 copybook_text_parts.append(text)
 diagnostics.append(f"Copybook {index}: {file_name}")
 diagnostics.append(f"Copybook {index} text length: {len(text)}")

 except Exception as exc:
 diagnostics.append(f"Copybook read failed for {file_name}: {exc}")

uploaded_file_names["copybook_files"] = copybook_file_names

copybook_text = "\n".join(copybook_text_parts)

if not copybook_text.strip():
 diagnostics.append("Copybook text is empty after upload.")
 return []

try:
 try:
 fields = self.copybook_parser.parse(
 text=copybook_text,
 source_label="Copybook files",
)
 except TypeError:
 fields = self.copybook_parser.parse(copybook_text)

 diagnostics.append(f"Copybook parsed fields: {len(fields)}")
 diagnostics.extend(getattr(self.copybook_parser, "diagnostics", []))
 return fields

except Exception as exc:
 diagnostics.append(f"Copybook parse failed: {exc}")
 return []

def _load_idms_cobol_source(
 self,
 idms_cobol_source_file: Any,
 diagnostics: list[str],
 uploaded_file_names: dict[str, object],
) -> tuple[str, str]:
 if idms_cobol_source_file is None:
 diagnostics.append("IDMS COBOL source file not uploaded.")
 uploaded_file_names["idms_cobol_source_file"] = ""
 return "", ""

 uploaded_file_names["idms_cobol_source_file"] = str(
 idms_cobol_source_file.name or ""
)

 try:
 text = self.file_loader.read_uploaded_text(idms_cobol_source_file)
 source_name = str(idms_cobol_source_file.name or "")

 diagnostics.append(f"IDMS COBOL source uploaded: {source_name}")
 diagnostics.append(f"IDMS COBOL source text length: {len(text)}")

 return text, source_name

 except Exception as exc:
 diagnostics.append(f"IDMS COBOL source read failed: {exc}")
 return "", ""

```

```

=====
FILE: src\idms_db2_phase2\ui\main_page.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import streamlit as st

from .app_state import AppState
from .conversion_controller import UiConversionController, UiConversionRequest

```

```

from .input_loader import UiInputLoader
from .result_tabs import (
 render_diagnostics_tab,
 render_generated_cobol_tab,
 render_metadata_tab,
 render_sheet_mapping_tab,
 render_validation_tab,
)
from .status_panel import render_current_status, render_download_section

def render_main_page() -> None:
 AppState.initialize()

 tabs = st.tabs(
 [
 "Main",
 "Sheet Mapping",
 "Metadata",
 "Generated DB2 COBOL",
 "Validation",
 "Diagnostics",
]
)

 with tabs[0]:
 render_main_tab()

 with tabs[1]:
 render_sheet_mapping_tab()

 with tabs[2]:
 render_metadata_tab()

 with tabs[3]:
 render_generated_cobol_tab()

 with tabs[4]:
 render_validation_tab()

 with tabs[5]:
 render_diagnostics_tab()

def render_main_tab() -> None:
 st.markdown("## Upload Inputs")

 st.info(
 "Upload Sheet Mapping, one or more DCLGEN files, optional Copybook files, "
 "and the IDMS COBOL source text file to generate DB2 embedded SQL COBOL."
)

 col1, col2 = st.columns(2)

 with col1:
 sheet_mapping_file = st.file_uploader(
 "Sheet Mapping Excel or CSV",
 type=["xlsx", "csv"],
 key="sheet_mapping_file",
 help=(
 "Upload Sheet Mapping as .xlsx or .csv. "
 "If your file is .xls, save the file as .xlsx or .csv first."
),
)

 dclgen_files = st.file_uploader(
 "DCLGEN text file or files",
 type=["txt", "cbl", "cpy"],
 accept_multiple_files=True,
 key="dclgen_files",
)

 with col2:
 copybook_files = st.file_uploader(
 "Optional Copybook text file or files",
 type=["txt", "cbl", "cpy"],

```

```

 accept_multiple_files=True,
 key="copybook_files",
)

 idms_cobol_source_file = st.file_uploader(
 "IDMS COBOL source code",
 type=["txt", "cbl", "cob"],
 key="idms_cobol_source_file",
)

 st.markdown("## Conversion Options")

 target_program_id = st.text_input(
 "Target PROGRAM-ID",
 value="",
 help="Optional. If provided, generated COBOL PROGRAM-ID will be replaced.",
)

 st.session_state.auto_fix_pic_length_mismatches = st.checkbox(
 "Auto-fix PIC length mismatches",
 value=st.session_state.auto_fix_pic_length_mismatches,
 help=(
 "If enabled, the converter can expand generated target PIC lengths "
 "when a clear MOVE source/target mismatch is detected."
),
)

 col_load, col_generate = st.columns(2)

 with col_load:
 if st.button("Load and Analyze Inputs", type="primary"):
 load_and_analyze_inputs(
 sheet_mapping_file=sheet_mapping_file,
 dclgen_files=dclgen_files,
 copybook_files=copybook_files,
 idms_cobol_source_file=idms_cobol_source_file,
)

 with col_generate:
 if st.button("Generate DB2 COBOL"):
 generate_db2_cobol(target_program_id=target_program_id)

 render_current_status()
 render_download_section()

def load_and_analyze_inputs(
 sheet_mapping_file,
 dclgen_files,
 copybook_files,
 idms_cobol_source_file,
) -> None:
 loader = UiInputLoader()

 bundle = loader.load(
 sheet_mapping_file=sheet_mapping_file,
 dclgen_files=dclgen_files,
 copybook_files=copybook_files,
 idms_cobol_source_file=idms_cobol_source_file,
)

 AppState.set_loaded_inputs(
 sheet_mapping_rows=bundle.sheet_mapping_rows,
 dclgen_columns=bundle.dclgen_columns,
 copybook_fields=bundle.copybook_fields,
 idms_cobol_text=bundle.idms_cobol_text,
 idms_cobol_source_name=bundle.idms_cobol_source_name,
 diagnostics=bundle.diagnostics,
 uploaded_file_names=bundle.uploaded_file_names,
)

 required_inputs_loaded = (
 bool(bundle.sheet_mapping_rows)
 and bool(bundle.dclgen_columns)
 and bool(str(bundle.idms_cobol_text or "").strip())
)

```

```

if required_inputs_loaded:
 st.success("Inputs loaded and analyzed.")
else:
 st.warning(
 "Inputs were processed, but one or more required inputs are missing "
 "or parsed as empty. Review the Diagnostics tab."
)

def generate_db2_cobol(
 target_program_id: str,
) -> None:
 if not st.session_state.loaded:
 st.warning("Load and analyze valid inputs before generating DB2 COBOL.")
 return

 if not st.session_state.sheet_mapping_rows:
 st.error("Sheet Mapping rows are empty. Reload Sheet Mapping input.")
 return

 if not st.session_state.dclgen_columns:
 st.error("DCLGEN columns are empty. Reload DCLGEN input.")
 return

 if not str(st.session_state.idms_cobol_text or "").strip():
 st.error("IDMS COBOL source is empty. Reload COBOL input.")
 return

 controller = UiConversionController()

 response = controller.generate(
 UiConversionRequest(
 sheet_mapping_rows=st.session_state.sheet_mapping_rows,
 dclgen_columns=st.session_state.dclgen_columns,
 copybook_fields=st.session_state.copybook_fields,
 idms_cobol_text=st.session_state.idms_cobol_text,
 idms_cobol_source_name=st.session_state.idms_cobol_source_name,
 target_program_id=target_program_id,
 auto_fix_pic_length_mismatches=(
 st.session_state.auto_fix_pic_length_mismatches
),
 diagnostics=st.session_state.diagnostics,
)
)

 AppState.set_conversion_result(
 result=response.result,
 converted_cobol_file_name=response.converted_cobol_file_name,
 diagnostics=response.diagnostics,
)

 if st.session_state.generated and st.session_state.converted_cobol:
 st.success("DB2 COBOL generated. Download is now available.")
 else:
 st.warning(
 "DB2 COBOL was not generated. Review the Validation and Diagnostics tabs."
)

=====
FILE: src\idms_db2_phase2\ui\result_tabs.py
ENCODING: utf-8
=====

from __future__ import annotations

from dataclasses import asdict, is_dataclass

import streamlit as st

def render_sheet_mapping_tab() -> None:
 st.markdown("## Sheet Mapping Preview")

 preview = st.session_state.sheet_mapping_preview

```

```

if not preview:
 rows = st.session_state.sheet_mapping_rows or []
 preview = [
 _row_to_dict(row)
 for row in rows[:20]
]

if not preview:
 st.info("No Sheet Mapping rows loaded.")
 return

st.dataframe(preview, use_container_width=True)

def render_metadata_tab() -> None:
 st.markdown("## Metadata Overview")

 if st.session_state.metadata_overview:
 st.json(st.session_state.metadata_overview)
 else:
 st.info("Generate DB2 COBOL to view metadata overview.")

 st.markdown("## Record Summary")

 if st.session_state.record_summary:
 st.dataframe(st.session_state.record_summary, use_container_width=True)
 else:
 st.info("No record summary available.")

 st.markdown("## Column Summary")

 if st.session_state.column_summary:
 st.dataframe(st.session_state.column_summary, use_container_width=True)
 else:
 st.info("No column summary available.")

 st.markdown("## Relationship Summary")

 if st.session_state.relationship_summary:
 st.dataframe(st.session_state.relationship_summary, use_container_width=True)
 else:
 st.info("No relationship summary available.")

def render_generated_cobol_tab() -> None:
 st.markdown("## Generated DB2 COBOL")

 if not st.session_state.converted_cobol:
 st.info("No generated COBOL available.")
 return

 st.text_area(
 "Final DB2 COBOL Code",
 value=st.session_state.converted_cobol,
 height=760,
 key="generated_tab_final_db2_cobol_code",
)

def render_validation_tab() -> None:
 st.markdown("## Validation")

 if not st.session_state.validation_messages:
 st.success("No validation messages.")
 return

 for message in st.session_state.validation_messages:
 st.warning(message)

def render_diagnostics_tab() -> None:
 st.markdown("## Diagnostics")
 st.markdown("### Uploaded Files")
 st.json(st.session_state.uploaded_file_names)

```

```

st.markdown("### Parser Diagnostics")
diagnostics = st.session_state.diagnostics or []

if not diagnostics:
 st.info("No diagnostics available. Click Load and Analyze Inputs first.")
 return

st.code(
 "\n".join(diagnostics),
 language="text",
)

def _row_to_dict(
 row,
) -> dict[str, object]:
 if row is None:
 return {}

 if isinstance(row, dict):
 return dict(row)

 if isinstance(row, dict):
 return dict(row)

 if hasattr(row, "__dict__"):
 return dict(row.__dict__)

 output: dict[str, object] = {}

 for name in dir(row):
 if name.startswith("_"):
 continue

 value = getattr(row, name, None)

 if callable(value):
 continue

 output[name] = value

 return output

```

```

=====
FILE: src\idms_db2_phase2\ui\status_panel.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import streamlit as st

def render_current_status() -> None:
 st.markdown("## Current Status")

 col1, col2, col3, col4 = st.columns(4)

 with col1:
 st.metric(
 "Sheet Mapping Rows",
 len(st.session_state.sheet_mapping_rows),
)

 with col2:
 st.metric(
 "DCLGEN Columns",
 len(st.session_state.dclgen_columns),
)

 with col3:
 st.metric(
 "Copybook Fields",
 len(st.session_state.copybook_fields),
)

```

```

with col4:
 st.metric(
 "IDMS COBOL Length",
 len(st.session_state.idms_cobol_text),
)

if st.session_state.loaded:
 st.success("Inputs loaded. Review diagnostics before generating.")

if st.session_state.generated:
 st.success("DB2 COBOL generated. Download is available below.")

def render_download_section() -> None:
 st.markdown("## Download Generated Output")

 if not st.session_state.generated or not st.session_state.converted_cobol:
 st.info("Generate DB2 COBOL to enable download.")
 return

 st.success("Generated DB2 COBOL is ready.")

 st.download_button(
 label="Download Generated DB2 COBOL",
 data=st.session_state.converted_cobol,
 file_name=st.session_state.converted_cobol_file_name,
 mime="text/plain",
 type="primary",
 key="download_generated_cobol",
)

=====
FILE: src\idms_db2_phase2\validators__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\validators\input_validator.py
ENCODING: utf-8
=====

from __future__ import annotations

from ..domain.models import ConversionInput
from ..reference.rules.validation.validation_messages import (
 INPUT_VALIDATION_MESSAGES,
)

class InputValidator:
 """
 Validates minimum required conversion inputs.

 This validator checks only input availability.
 It does not validate generated COBOL production quality.
 """

 def validate(
 self,
 conversion_input: ConversionInput,
) -> list[str]:
 messages: list[str] = []

 if not conversion_input.sheet_mapping_rows:
 messages.append(INPUT_VALIDATION_MESSAGES["missing_sheet_mapping"])

 if not conversion_input.dclgen_columns:
 messages.append(INPUT_VALIDATION_MESSAGES["missing_dclgen"])

 if not str(conversion_input.idms_cobol_text or "").strip():
 messages.append(INPUT_VALIDATION_MESSAGES["missing_idms_cobol"])

```

```
return messages
```

```
=====
FILE: src\idms_db2_phase2\validators\mapping_validator.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
from ..reference.rules.validation.validation_messages import (
 MAPPING_VALIDATION_MESSAGES,
)
from ..repositories.dclgen_repository import DclgenRepository
from ..repositories.mapping_repository import MappingRepository
from ..resolvers.table_name_resolver import TableNameResolver
```

```
class MappingValidator:
```

```
 """
```

```
 Validates Sheet Mapping and DCLGEN consistency.
```

```
 This validator checks metadata availability and resolver alignment.
 It does not generate SQL and does not transform COBOL.
```

```
 """
```

```
 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
```

```
 def validate(
 self,
) -> list[str]:
 messages: list[str] = []

 if self.mapping_repository.count() == 0:
 messages.append(MAPPING_VALIDATION_MESSAGES["missing_mapping_rows"])

 if self.dclgen_repository.count() == 0:
 messages.append(MAPPING_VALIDATION_MESSAGES["missing_dclgen_columns"])

 messages.extend(self._validate_record_table_resolution())
 messages.extend(self._validate_table_column_resolution())

 return self._unique_messages(messages)
```

```
 def _validate_record_table_resolution(
 self,
) -> list[str]:
 messages: list[str] = []

 for record in self.mapping_repository.records():
 table = self.table_name_resolver.table_for_record(record)

 if table:
 continue

 messages.append(
 f"{MAPPING_VALIDATION_MESSAGES['missing_record_mapping']} "
 f"Record={record}"
)

 return messages
```

```
 def _validate_table_column_resolution(
 self,
) -> list[str]:
 messages: list[str] = []
```

```

for table in self.mapping_repository.tables():
 resolved_table = self.table_name_resolver.resolve_table(table)

 if not resolved_table:
 continue

 columns = self.mapping_repository.db2_columns_for_table(table)

 if columns:
 continue

 messages.append(
 f"{MAPPING_VALIDATION_MESSAGES['missing_column_mapping']} "
 f"Table={resolved_table}"
)

return messages

def _unique_messages(
 self,
 messages: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for message in messages:
 clean_message = str(message or "").strip()

 if not clean_message:
 continue

 if clean_message in seen:
 continue

 seen.add(clean_message)
 output.append(clean_message)

 return output

```

```

=====
FILE: src\idms_db2_phase2\validators\production_validator.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from ..reference.patterns.cobol.sequence_patterns import strip_sequence_numbers
from ..reference.patterns.db2.sql_patterns import HOST_REFERENCE_PATTERN
from ..reference.patterns.validation.production_patterns import (
 ERROR_DB2_PATTERN,
 FORBIDDEN_EXECUTABLE_IDMS_PATTERNS,
 FORBIDDEN_IDMS_DECLARATIVE_PATTERNS,
 NO_FETCH_HOST_VARIABLES_PATTERN,
 TODO_DB2_PATTERN,
 TODO_HOST_VARIABLE_PATTERN,
 UNABLE_TO_DECLARE_CURSOR_PATTERN,
)
from ..reference.rules.validation.db2_validation_rules import REQUIRED_DB2_TOKENS
from ..reference.rules.validation.validation_messages import (
 PRODUCTION_VALIDATION_MESSAGES,
)
from ..repositories.dclgen_repository import DclgenRepository
from ..services.name_normalizer import NameNormalizer

```

```

class ProductionValidator:
 """
 Performs production-focused validation for generated DB2 COBOL.

 It detects:
 - Missing required DB2 constructs.
 - TODO host variables.
 - Generated DB2 error markers.
 - Residual executable IDMS statements.
 - Residual IDMS declarative/control statements.
 """

```

- Generated DCLGEN host variables not found in uploaded DCLGEN metadata.

This validator ignores commented fixed-format lines.

Important:

- Regex patterns are imported from reference/patterns.
- Validation messages are imported from reference/rules.
- No regex is defined inside this validator.

"""

```
def __init__(
 self,
 dclgen_repository: DclgenRepository,
) -> None:
 self.dclgen_repository = dclgen_repository

def validate(
 self,
 converted_cobol_text: str,
) -> list[str]:
 messages: list[str] = []
 text = str(converted_cobol_text or "")

 if not text.strip():
 return [PRODUCTION_VALIDATION_MESSAGES["empty_converted_cobol"]]

 self._validate_required_db2_tokens(text, messages)
 self._validate_no_todo_or_generated_error(text, messages)
 self._validate_forbidden_idms_patterns(text, messages)
 self._validate_forbidden_idms_declaratives(text, messages)
 self._validate_generated_dclgen_host_variables(text, messages)

 return self._unique_messages(messages)

def _validate_required_db2_tokens(
 self,
 text: str,
 messages: list[str],
) -> None:
 upper_text = text.upper()

 for token in REQUIRED_DB2_TOKENS:
 if token.upper() in upper_text:
 continue

 messages.append(
 f"Production validation: required DB2 token missing: {token}"
)

def _validate_no_todo_or_generated_error(
 self,
 text: str,
 messages: list[str],
) -> None:
 if TODO_HOST_VARIABLE_PATTERN.search(text):
 messages.append(PRODUCTION_VALIDATION_MESSAGES["todo_host_variable"])

 if TODO_DB2_PATTERN.search(text):
 messages.append(PRODUCTION_VALIDATION_MESSAGES["todo_db2"])

 if ERROR_DB2_PATTERN.search(text):
 messages.append(PRODUCTION_VALIDATION_MESSAGES["error_db2"])

 if UNABLE_TO_DECLARE_CURSOR_PATTERN.search(text):
 messages.append(
 PRODUCTION_VALIDATION_MESSAGES["unable_to_declare_cursor"]
)

 if NO_FETCH_HOST_VARIABLES_PATTERN.search(text):
 messages.append(
 PRODUCTION_VALIDATION_MESSAGES["no_fetch_host_variables"]
)

def _validate_forbidden_idms_patterns(
 self,
 text: str,
```

```

 messages: list[str],
) -> None:
 for line_number, line in enumerate(text.splitlines(), start=1):
 logical = self._logical_line(line)

 if self._is_comment_or_blank(logical):
 continue

 if self._inside_exec_sql_boundary_line(logical):
 continue

 for pattern in FORBIDDEN_EXECUTABLE_IDMS_PATTERNS:
 if not pattern.search(logical):
 continue

 messages.append(
 f"{PRODUCTION_VALIDATION_MESSAGES['residual_executable_idms']} "
 f"at line {line_number}: {logical}"
)
 break

def _validate_forbidden_idms_declaratives(
 self,
 text: str,
 messages: list[str],
) -> None:
 for line_number, line in enumerate(text.splitlines(), start=1):
 logical = self._logical_line(line)

 if self._is_comment_or_blank(logical):
 continue

 for pattern in FORBIDDEN_IDMS_DECLARATIVE_PATTERNS:
 if not pattern.search(logical):
 continue

 messages.append(
 f"{PRODUCTION_VALIDATION_MESSAGES['residual_idms_declarative']} "
 f"at line {line_number}: {logical}"
)
 break

def _validate_generated_dclgen_host_variables(
 self,
 text: str,
 messages: list[str],
) -> None:
 valid_references = self.dclgen_repository.valid_host_references()

 if not valid_references:
 if HOST_REFERENCE_PATTERN.search(text):
 messages.append(PRODUCTION_VALIDATION_MESSAGES["no_dclgen_hosts"])
 return

 normalized_valid_references = self._normalized_reference_set(
 valid_references
)

 for line in text.splitlines():
 logical = self._logical_line(line)

 if self._is_comment_or_blank(logical):
 continue

 for match in HOST_REFERENCE_PATTERN.finditer(logical):
 raw_reference = self._expand_host_reference_from_line(
 line=logical,
 host_start_index=match.start(),
)
 normalized_reference = self._normalize_host_reference(raw_reference)

 if not normalized_reference:
 continue

 if normalized_reference in normalized_valid_references:
 continue

```

```

 messages.append(
 f"{PRODUCTION_VALIDATION_MESSAGES['unknown_dclgen_host']}: "
 f"{raw_reference}"
)

def _expand_host_reference_from_line(
 self,
 line: str,
 host_start_index: int,
) -> str:
 """
 Expands a host reference from the source line.

 Reason:
 The imported HOST_REFERENCE_PATTERN may match only the first token
 after ':' for dot-qualified DCLGEN references, for example:

 :DCLDZBFARTV.NR-ID

 This method expands the reference safely without defining validator
 regex patterns.
 """
 text = str(line or "")

 if host_start_index < 0 or host_start_index >= len(text):
 return ""

 if text[host_start_index] != ":":
 return ""

 index = host_start_index + 1
 output: list[str] = []

 while index < len(text):
 character = text[index]

 if character == ",":
 break

 if character == ")":
 break

 if character == ";":
 break

 output.append(character)
 index += 1

 candidate = "".join(output).strip()

 while candidate.endswith("."):
 candidate = candidate[:-1].rstrip()

 return candidate

def _normalize_host_reference(
 self,
 value: str,
) -> str:
 text = str(value or "").strip()

 if not text:
 return ""

 text = text.replace(".", " ")
 text = " ".join(text.split())

 if " OF " in text.upper():
 parts = text.upper().split(" OF ", 1)
 host = NameNormalizer.to_cobol(parts[0])
 group = NameNormalizer.to_cobol(parts[1])
 return f"{host} OF {group}"

 parts = text.split()

```

```

 if len(parts) == 2:
 first = NameNormalizer.to_cobol(parts[0])
 second = NameNormalizer.to_cobol(parts[1])

 if first.startswith("DCL"):
 return f"{second} OF {first}"

 if second.startswith("DCL"):
 return f"{first} OF {second}"

 return NameNormalizer.to_cobol(text)

def _normalized_reference_set(
 self,
 references: set[str],
) -> set[str]:
 output: set[str] = set()

 for reference in references:
 normalized = self._normalize_host_reference(reference)

 if normalized:
 output.add(normalized)

 return output

def _logical_line(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

def _is_comment_or_blank(
 self,
 logical: str,
) -> bool:
 stripped = str(logical or "").strip()

 if not stripped:
 return True

 if stripped.startswith("*"):
 return True

 if stripped.startswith("/"):
 return True

 return False

def _inside_exec_sql_boundary_line(
 self,
 logical: str,
) -> bool:
 upper = str(logical or "").strip().upper()

 if upper.startswith("EXEC SQL"):
 return True

 if upper.startswith("END-EXEC"):
 return True

 return False

def _unique_messages(
 self,
 messages: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for message in messages:
 clean_message = str(message or "").strip()

 if not clean_message:
 continue

```

```

 if clean_message in seen:
 continue

 seen.add(clean_message)
 output.append(clean_message)

 return output

=====
FILE: tests\test_architecture_boundaries.py
ENCODING: utf-8
=====

from __future__ import annotations

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]

RUNTIME_LAYER_FOLDERS = [
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "parsers",
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "services",
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "transformers",
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "generators",
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "validators",
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "composers",
]

RULES_FOLDER = PROJECT_ROOT / "src" / "idms_db2_phase2" / "reference" / "rules"
PATTERNS_FOLDER = (
 PROJECT_ROOT / "src" / "idms_db2_phase2" / "reference" / "patterns"
)

def test_runtime_layers_do_not_define_regex_patterns() -> None:
 for folder in RUNTIME_LAYER_FOLDERS:
 if not folder.exists():
 continue

 for path in folder.rglob("*.py"):
 text = path.read_text(encoding="utf-8")

 assert "re.compile(" not in text, (
 f"Regex pattern found in runtime layer: {path}. "
 "Move regex definitions to reference/patterns."
)

def test_rules_do_not_import_runtime_layers() -> None:
 if not RULES_FOLDER.exists():
 return

 forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.services",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
]

 for path in RULES_FOLDER.rglob("*.py"):
 text = path.read_text(encoding="utf-8")

 for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in rules file {path}: {forbidden_import}"
)

def test_patterns_do_not_import_runtime_layers() -> None:
 if not PATTERNS_FOLDER.exists():
 return

```

```

forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.repositories",
 "idms_db2_phase2.resolvers",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
 "idms_db2_phase2.services",
]

for path in PATTERNS_FOLDER.rglob("*.py"):
 text = path.read_text(encoding="utf-8")

 for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in pattern file {path}: {forbidden_import}"
)

=====
FILE: tests\test_batch10_conversion_pipeline.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.orchestration.conversion_service import ConversionService

def test_conversion_service_returns_validation_messages_for_missing_inputs() -> None:
 service = ConversionService()

 result = service.convert(ConversionInput())

 assert result.converted_cobol == ""
 assert "Sheet Mapping is required and must contain rows." in result.validation_messages
 assert "At least one DCLGEN file is required." in result.validation_messages
 assert (
 "IDMS COBOL source text file is required and must contain readable COBOL code."
 in result.validation_messages
)

def test_conversion_service_end_to_end_obtain_calc_and_store() -> None:
 service = ConversionService()

 conversion_input = ConversionInput(
 sheet_mapping_rows=[
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
],
 dclgen_columns=[
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="CD_NAME",
),
],
)

```

```

 host_variable="CD-NAME",
),
],
idms_cobol_text=""
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
DATA DIVISION.
WORKING-STORAGE SECTION.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
STORE VMB-FAR.
END PROGRAM OLDPROG.
""",
 target_program_id="NEWPROG",
)

result = service.convert(conversion_input)

assert result.converted_cobol
assert "PROGRAM-ID. NEWPROG." in result.converted_cobol
assert "EXEC SQL" in result.converted_cobol
assert "INCLUDE SQLCA" in result.converted_cobol
assert "INCLUDE SQLERRWS" in result.converted_cobol
assert "INCLUDE DZBFARTV" in result.converted_cobol
assert "FROM DZBFARTV" in result.converted_cobol
assert "INSERT INTO DZBFARTV" in result.converted_cobol
assert result.operations
assert result.metadata_overview["sheet_mapping_rows"] == 2
assert result.record_summary
assert result.column_summary

```

```

=====
FILE: tests\test_batch11_infrastructure_runner.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory
from idms_db2_phase2.parsers.text_loader import TextLoader

def test_local_uploaded_file_reads_bytes(tmp_path: Path) -> None:
 file_path = tmp_path / "sample.txt"
 file_path.write_text("HELLO", encoding="utf-8")

 uploaded = LocalUploadedFile(file_path)

 assert uploaded.name == "sample.txt"
 assert uploaded.getvalue() == b"HELLO"

def test_file_loader_reads_text_uploaded_file(tmp_path: Path) -> None:
 file_path = tmp_path / "sample.txt"
 file_path.write_text("HELLO", encoding="utf-8")

 uploaded = LocalUploadedFile(file_path)
 text = FileLoader().read_uploaded_text(uploaded)

 assert text == "HELLO"

def test_text_loader_is_file_loader() -> None:
 assert isinstance(TextLoader(), FileLoader)

def test_logger_factory_creates_log_file(tmp_path: Path) -> None:
 logger = LoggerFactory.create_logger(
 name="test_logger_factory",
 logs_dir=tmp_path,
)

```

```

logger.info("hello")

log_files = list(tmp_path.glob("idms_db2_conversion_*.log"))

assert log_files

=====
FILE: tests\test_batch12_ui_imports.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: tests\test_batch13_ui_split.py
ENCODING: utf-8
=====

from __future__ import annotations

def test_ui_module_imports() -> None:
 from idms_db2_phase2.app import main
 from idms_db2_phase2.ui.app_state import AppState
 from idms_db2_phase2.ui.conversion_controller import UiConversionController
 from idms_db2_phase2.ui.input_loader import UiInputLoader
 from idms_db2_phase2.ui.main_page import render_main_page
 from idms_db2_phase2.ui.result_tabs import render_validation_tab
 from idms_db2_phase2.ui.status_panel import render_current_status

 assert main is not None
 assert AppState is not None
 assert UiConversionController is not None
 assert UiInputLoader is not None
 assert render_main_page is not None
 assert render_validation_tab is not None
 assert render_current_status is not None

=====
FILE: tests\test_batch14_architecture_quality.py
ENCODING: utf-8
=====

from __future__ import annotations

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
PACKAGE_ROOT = PROJECT_ROOT / "src" / "idms_db2_phase2"

RUNTIME_LAYER_FOLDERS = [
 PACKAGE_ROOT / "parsers",
 PACKAGE_ROOT / "services",
 PACKAGE_ROOT / "transformers",
 PACKAGE_ROOT / "generators",
 PACKAGE_ROOT / "validators",
 PACKAGE_ROOT / "composers",
 PACKAGE_ROOT / "orchestration",
 PACKAGE_ROOT / "ui",
]

REFERENCE_RULES_FOLDER = PACKAGE_ROOT / "reference" / "rules"
REFERENCE_PATTERNS_FOLDER = PACKAGE_ROOT / "reference" / "patterns"

ALLOWED_REGEX_RUNTIME_FILES = set()

FORBIDDEN_ROOT_IMPORTS = [
 "from patterns",
 "import patterns",
 "from rules",
 "import rules",

```

```

 "from catalogs",
 "import catalogs",
]

FORBIDDEN_NAME_PARTS = [
 "feedback",
 "cleanup",
]

FORBIDDEN_EXACT_NAME_PARTS = [
 "_fix_",
 "_fix.",
 "_fixer_",
 "_fixer.",
]

ALLOWED_TECHNICAL_NAMES = {
 "pic_length_auto_fixer.py",
}

MAX_PREFERRED_LINES = 500

def _python_files(root: Path) -> list[Path]:
 if not root.exists():
 return []

 return [
 path
 for path in root.rglob("*.py")
 if path.is_file()
]

def _read(path: Path) -> str:
 return path.read_text(encoding="utf-8")

def test_runtime_layers_do_not_define_regex_patterns() -> None:
 for folder in RUNTIME_LAYER_FOLDERS:
 for path in _python_files(folder):
 if path.name in ALLOWED_REGEX_RUNTIME_FILES:
 continue

 text = _read(path)

 assert "re.compile(" not in text, (
 f"Regex pattern found in runtime layer: {path}. "
 "Move regex definitions to src/idms_db2_phase2/reference/patterns."
)

def test_internal_modules_do_not_import_root_patterns_rules_or_catalogs() -> None:
 for path in _python_files(PACKAGE_ROOT):
 text = _read(path)

 for forbidden in FORBIDDEN_ROOT_IMPORTS:
 assert forbidden not in text, (
 f"Root-level import '{forbidden}' found in {path}. "
 "Internal modules must import from ..reference.* using relative imports."
)

def test_rules_do_not_import_runtime_layers() -> None:
 forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.services",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
 "idms_db2_phase2.orchestration",
 "idms_db2_phase2.ui",
]

 for path in _python_files(REFERENCE_RULES_FOLDER):

```

```

text = _read(path)

for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in rules file {path}: {forbidden_import}"
)

def test_patterns_do_not_import_runtime_layers() -> None:
 forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.repositories",
 "idms_db2_phase2.resolvers",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
 "idms_db2_phase2.services",
 "idms_db2_phase2.orchestration",
 "idms_db2_phase2.ui",
]

 for path in _python_files(REFERENCE_PATTERNS_FOLDER):
 text = _read(path)

 for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in pattern file {path}: {forbidden_import}"
)

def test_new_production_modules_use_professional_names() -> None:
 for path in _python_files(PACKAGE_ROOT):
 if path.name in ALLOWED_TECHNICAL_NAMES:
 continue

 normalized_path = str(path).replace("\\", "/").lower()

 for forbidden in FORBIDDEN_NAME_PARTS:
 assert forbidden not in normalized_path, (
 f"Non-professional naming found in path: {path}. "
 "Use domain-responsibility names such as normalization, composer, "
 "resolver, validator, generator, or transformer."
)

 for forbidden_exact in FORBIDDEN_EXACT_NAME_PARTS:
 assert forbidden_exact not in normalized_path, (
 f"Non-professional fix-style naming found in path: {path}. "
 "Use domain-responsibility names instead."
)

def test_python_files_stay_within_preferred_size() -> None:
 oversized_files: list[str] = []

 for path in _python_files(PACKAGE_ROOT):
 line_count = len(_read(path).splitlines())

 if line_count > MAX_PREFERRED_LINES:
 oversized_files.append(f"{path} has {line_count} lines")

 assert not oversized_files, (
 "Files exceeded preferred maintainability limit:\n"
 + "\n".join(oversized_files)
)

```

```

=====
FILE: tests\test_batch14_core_imports.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

```

```

PROJECT_ROOT = Path(__file__).resolve().parents[1]
PACKAGE_ROOT = PROJECT_ROOT / "src" / "idms_db2_phase2"

RUNTIME_LAYER_FOLDERS = [
 PACKAGE_ROOT / "parsers",
 PACKAGE_ROOT / "services",
 PACKAGE_ROOT / "transformers",
 PACKAGE_ROOT / "generators",
 PACKAGE_ROOT / "validators",
 PACKAGE_ROOT / "composers",
 PACKAGE_ROOT / "orchestration",
 PACKAGE_ROOT / "ui",
]

REFERENCE_RULES_FOLDER = PACKAGE_ROOT / "reference" / "rules"
REFERENCE_PATTERNS_FOLDER = PACKAGE_ROOT / "reference" / "patterns"

ALLOWED_REGEX_RUNTIME_FILES = set()

FORBIDDEN_ROOT_IMPORTS = [
 "from patterns",
 "import patterns",
 "from rules",
 "import rules",
 "from catalogs",
 "import catalogs",
]

FORBIDDEN_NAME_PARTS = [
 "feedback",
 "cleanup",
]

FORBIDDEN_EXACT_NAME_PARTS = [
 "_fix_",
 "_fix.",
 "_fixer_",
 "_fixer.",
]

ALLOWED_TECHNICAL_NAMES = {
 "pic_length_auto_fixer.py",
}

MAX_PREFERRED_LINES = 500

def _python_files(root: Path) -> list[Path]:
 if not root.exists():
 return []

 return [
 path
 for path in root.rglob("*.py")
 if path.is_file()
]

def _read(path: Path) -> str:
 return path.read_text(encoding="utf-8")

def test_runtime_layers_do_not_define_regex_patterns() -> None:
 for folder in RUNTIME_LAYER_FOLDERS:
 for path in _python_files(folder):
 if path.name in ALLOWED_REGEX_RUNTIME_FILES:
 continue

 text = _read(path)

 assert "re.compile(" not in text, (
 f"Regex pattern found in runtime layer: {path}. "
 "Move regex definitions to src/idms_db2_phase2/reference/patterns."
)

```

```

def test_internal_modules_do_not_import_root_patterns_rules_or_catalogs() -> None:
 for path in _python_files(PACKAGE_ROOT):
 text = _read(path)

 for forbidden in FORBIDDEN_ROOT_IMPORTS:
 assert forbidden not in text, (
 f"Root-level import '{forbidden}' found in {path}. "
 "Internal modules must import from ..reference.* using relative imports."
)

def test_rules_do_not_import_runtime_layers() -> None:
 forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.services",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
 "idms_db2_phase2.orchestration",
 "idms_db2_phase2.ui",
]

 for path in _python_files(REFERENCE_RULES_FOLDER):
 text = _read(path)

 for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in rules file {path}: {forbidden_import}"
)

def test_patterns_do_not_import_runtime_layers() -> None:
 forbidden_imports = [
 "idms_db2_phase2.parsers",
 "idms_db2_phase2.repositories",
 "idms_db2_phase2.resolvers",
 "idms_db2_phase2.transformers",
 "idms_db2_phase2.generators",
 "idms_db2_phase2.validators",
 "idms_db2_phase2.composers",
 "idms_db2_phase2.services",
 "idms_db2_phase2.orchestration",
 "idms_db2_phase2.ui",
]

 for path in _python_files(REFERENCE_PATTERNS_FOLDER):
 text = _read(path)

 for forbidden_import in forbidden_imports:
 assert forbidden_import not in text, (
 f"Runtime import found in pattern file {path}: {forbidden_import}"
)

def test_new_production_modules_use_professional_names() -> None:
 for path in _python_files(PACKAGE_ROOT):
 if path.name in ALLOWED_TECHNICAL_NAMES:
 continue

 normalized_path = str(path).replace("\\", "/").lower()

 for forbidden in FORBIDDEN_NAME_PARTS:
 assert forbidden not in normalized_path, (
 f"Non-professional naming found in path: {path}. "
 "Use domain-responsibility names such as normalization, composer, "
 "resolver, validator, generator, or transformer."
)

 for forbidden_exact in FORBIDDEN_EXACT_NAME_PARTS:
 assert forbidden_exact not in normalized_path, (
 f"Non-professional fix-style naming found in path: {path}. "
 "Use domain-responsibility names instead."
)

```

```

def test_python_files_stay_within_preferred_size() -> None:
 oversized_files: list[str] = []

 for path in _python_files(PACKAGE_ROOT):
 line_count = len(_read(path).splitlines())

 if line_count > MAX_PREFERRED_LINES:
 oversized_files.append(f"{path} has {line_count} lines")

 assert not oversized_files, (
 "Files exceeded preferred maintainability limit:\n"
 + "\n".join(oversized_files)
)

=====
FILE: tests\test_batch15_compatibility_imports.py
ENCODING: utf-8
=====

from __future__ import annotations

def test_root_pattern_compatibility_imports() -> None:
 from patterns.cobol_patterns import PROGRAM_ID_PATTERN
 from patterns.dclgen_patterns import DECLARE_TABLE_PATTERN
 from patterns.idms_patterns import OBTAIN_CALC_PATTERN, STORE_PATTERN
 from patterns.sql_patterns import EXEC_SQL_PATTERN, UPDATE_SQL_PATTERN
 from patterns.validation_patterns import TODO_DB2_PATTERN

 assert PROGRAM_ID_PATTERN.search("PROGRAM-ID. TEST01.")
 assert DECLARE_TABLE_PATTERN.search("EXEC SQL DECLARE DZBFARTV TABLE")
 assert OBTAIN_CALC_PATTERN.search("OBTAIN VMB-FAR CALC.")
 assert STORE_PATTERN.search("STORE VMB-FAR.")
 assert EXEC_SQL_PATTERN.match("EXEC SQL")
 assert UPDATE_SQL_PATTERN.search("UPDATE DZBFARTV")
 assert TODO_DB2_PATTERN.search("TODO DB2 mapping missing")

def test_root_rule_compatibility_imports() -> None:
 from rules.authority_rules import INPUT_AUTHORITY_RULES
 from rules.conversion_rules import CONVERSION_RULES
 from rules.cursor_rules import CURSOR_PARAGRAPH_NUMBERING
 from rules.validation_rules import INPUT_VALIDATION_MESSAGES

 assert INPUT_AUTHORITY_RULES
 assert CONVERSION_RULES
 assert CURSOR_PARAGRAPH_NUMBERING
 assert INPUT_VALIDATION_MESSAGES

def test_root_catalog_compatibility_imports() -> None:
 from catalogs.db2_sql_types import DB2_SQL_TYPE_STARTERS
 from catalogs.dclgen_schema import DCLGEN_GROUP_PREFIX
 from catalogs.output_sections import SQLCA_INCLUDE_NAME
 from catalogs.sheet_mapping_schema import SHEET_MAPPING_MODEL_FIELD_MAP

 assert DB2_SQL_TYPE_STARTERS
 assert DCLGEN_GROUP_PREFIX == "DCL"
 assert SQLCA_INCLUDE_NAME == "SQLCA"
 assert SHEET_MAPPING_MODEL_FIELD_MAP

```

```

=====
FILE: tests\test_batch16_repository_health.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: tests\test_batch17_sqlerror_and_snapshots.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.orchestration.conversion_service import ConversionService
```

```
def test_sql_error_generator_adds_paragraph_before_end_program() -> None:
 generator = SqlErrorGenerator()
```

```
 text = (
 "IDENTIFICATION DIVISION.\n"
 "PROGRAM-ID. TEST.\n"
 "END PROGRAM TEST.\n"
)
```

```
 result = generator.ensure_sql_error_paragraph(text)
```

```
 assert "SQLERROR." in result
 assert "INCLUDE SQLERROR" in result
 assert result.index("SQLERROR.") < result.index("END PROGRAM")
```

```
def test_sql_error_generator_normalizes_sql_error_name() -> None:
 generator = SqlErrorGenerator()
```

```
 text = """
PROCEDURE DIVISION.
PERFORM SQL-ERROR.
SQL-ERROR.
 EXEC SQL
 INCLUDE SQLERROR
 END-EXEC.
"""
```

```
 result = generator.ensure_sql_error_paragraph(text)
```

```
 assert "SQL-ERROR" not in result
 assert "PERFORM SQLERROR" in result
 assert "SQLERROR." in result
```

```
def test_end_to_end_snapshot_contains_required_db2_sections() -> None:
 service = ConversionService()
```

```
 conversion_input = ConversionInput(
 sheet_mapping_rows=[
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
],
 dclgen_columns=[
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
```

```

 group_name="DCLDZBFARTV",
 column_name="CD_NAME",
 host_variable="CD-NAME",
),
],
idms_cobol_text=""
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
DATA DIVISION.
WORKING-STORAGE SECTION.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
STORE VMB-FAR.
END PROGRAM OLDPROG.
""",
 target_program_id="NEWPROG",
)

result = service.convert(conversion_input)
text = result.converted_cobol

```

```

assert "CBL ARITH(EXTEND)" in text
assert "PROGRAM-ID. NEWPROG." in text
assert "EXEC SQL" in text
assert "INCLUDE SQLCA" in text
assert "INCLUDE SQLERRWS" in text
assert "INCLUDE SQLERROR" in text
assert "INCLUDE DZBFARTV" in text
assert "SQL-LOCATION" in text
assert "FROM DZBFARTV" in text
assert "INSERT INTO DZBFARTV" in text
assert "SQLERROR." in text
assert "END PROGRAM OLDPROG." in text
assert result.operations

```

```

=====
FILE: tests\test_batch19_runtime_runner.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: tests\test_batch4_validators.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.validators.input_validator import InputValidator
from idms_db2_phase2.validators.mapping_validator import MappingValidator
from idms_db2_phase2.validators.production_validator import ProductionValidator

```

```

def test_input_validator_detects_missing_inputs() -> None:
 validator = InputValidator()

 messages = validator.validate(ConversionInput())

 assert "Sheet Mapping is required and must contain rows." in messages
 assert "At least one DCLGEN file is required." in messages
 assert (
 "IDMS COBOL source text file is required and must contain readable COBOL code."
 in messages
)

```

```

def test_mapping_validator_accepts_resolved_table_and_columns() -> None:
 mapping_repository = MappingRepository(
[

```

```

 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
)
]
)
dclgen_repository = DclgenRepository(
 [
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
)
]
)
table_name_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
validator = MappingValidator(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)

messages = validator.validate()

assert messages == []

def test_mapping_validator_detects_missing_dclgen_columns() -> None:
 mapping_repository = MappingRepository(
 [
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
)
]
)
 dclgen_repository = DclgenRepository([])
 table_name_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
 validator = MappingValidator(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)

 messages = validator.validate()

 assert "Mapping validation: DCLGEN columns are missing." in messages

def test_production_validator_detects_missing_db2_tokens() -> None:
 dclgen_repository = DclgenRepository([])
 validator = ProductionValidator(dclgen_repository=dclgen_repository)

 messages = validator.validate("IDENTIFICATION DIVISION.")

 assert "Production validation: required DB2 token missing: EXEC SQL" in messages
 assert "Production validation: required DB2 token missing: SQLCA" in messages
 assert "Production validation: required DB2 token missing: END-EXEC" in messages

def test_production_validator_detects_residual_idms_statement() -> None:
 dclgen_repository = DclgenRepository([])
 validator = ProductionValidator(dclgen_repository=dclgen_repository)

```

```

text = """
EXEC SQL
INCLUDE SQLCA
END-EXEC.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
"""

```

```

messages = validator.validate(text)

```

```

assert any(
 "residual executable IDMS statement remains" in message
 for message in messages
)

```

```

def test_production_validator_accepts_valid_host_reference() -> None:

```

```

 dclgen_repository = DclgenRepository(
 [
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
)
]
)

```

```

 validator = ProductionValidator(dclgen_repository=dclgen_repository)

```

```

text = """
EXEC SQL
INCLUDE SQLCA
END-EXEC.
EXEC SQL
SELECT NR_ID
INTO :DCLDZBFARTV.NR-ID
FROM DZBFARTV
END-EXEC.
"""

```

```

messages = validator.validate(text)

```

```

assert not any(
 "was not found in uploaded DCLGEN columns" in message
 for message in messages
)

```

```

=====
FILE: tests\test_batch5_transformers.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.domain.models import DclgenColumn, SheetMappingRow
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.resolvers.update_sql_plan_resolver import UpdateSqlPlanResolver
from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
from idms_db2_phase2.transformers.field_reference_rewriter import FieldReferenceRewriter
from idms_db2_phase2.transformers.idms_statement_transformer import (
 IdmsStatementTransformer,
)
from idms_db2_phase2.transformers.pic_length_auto_fixer import PicLengthAutoFixer

```

```

def build_context():
 mapping_repository = MappingRepository(
 [

```

```

 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
]
)

dclgen_repository = DclgenRepository(
 [
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="CD_NAME",
 host_variable="CD-NAME",
),
]
)

table_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
column_resolver = ColumnNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
host_resolver = HostVariableResolver(
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
update_plan_resolver = UpdateSqlPlanResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
 column_name_resolver=column_resolver,
 host_variable_resolver=host_resolver,
)
sql_generator = SqlGenerator(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
 column_name_resolver=column_resolver,
 host_variable_resolver=host_resolver,
 update_sql_plan_resolver=update_plan_resolver,
)
statement_transformer = IdmsStatementTransformer(
 sql_generator=sql_generator,
 sql_error_generator=SqlErrorGenerator(),
 table_name_resolver=table_resolver,
 cursor_name_resolver=CursorNameResolver(),
)

return {
 "mapping_repository": mapping_repository,
 "dclgen_repository": dclgen_repository,
 "table_resolver": table_resolver,
 "host_resolver": host_resolver,
 "statement_transformer": statement_transformer,
 "sql_generator": sql_generator,
}

```

```

}

def test_sql_generator_select_by_key() -> None:
 context = build_context()
 lines = context["sql_generator"].select_by_key("VMB-FAR")
 text = "\n".join(lines)

 assert "SELECT" in text
 assert "FROM DZBFARTV" in text
 assert "NR_ID = :NR-ID OF DCLDZBFARTV" in text

def test_sql_generator_insert() -> None:
 context = build_context()
 lines = context["sql_generator"].insert("VMB-FAR")
 text = "\n".join(lines)

 assert "INSERT INTO DZBFARTV" in text
 assert ":NR-ID OF DCLDZBFARTV" in text

def test_sql_generator_update() -> None:
 context = build_context()
 lines = context["sql_generator"].update(
 "VMB-FAR",
 changed_source_fields=["CD-NAME"],
)
 text = "\n".join(lines)

 assert "UPDATE DZBFARTV" in text
 assert "CD_NAME = :CD-NAME OF DCLDZBFARTV" in text
 assert "WHERE NR_ID = :NR-ID OF DCLDZBFARTV" in text

def test_idms_statement_transformer_converts_obtain_calc() -> None:
 context = build_context()

 lines, _opened_set = context["statement_transformer"].transform_line(
 line="OBTAIN VMB-FAR CALC.",
 current_division="PROCEDURE",
 sql_error_paragraph="SQLERROR",
)
 text = "\n".join(lines or [])

 assert "Converted OBTAIN CALC" in text
 assert "SELECT" in text
 assert "FROM DZBFARTV" in text

def test_idms_statement_transformer_converts_store_to_insert() -> None:
 context = build_context()

 lines, _opened_set = context["statement_transformer"].transform_line(
 line="STORE VMB-FAR.",
 current_division="PROCEDURE",
 sql_error_paragraph="SQLERROR",
)
 text = "\n".join(lines or [])

 assert "Converted STORE" in text
 assert "INSERT INTO DZBFARTV" in text

def test_idms_statement_transformer_removes_bind() -> None:
 context = build_context()

 lines, _opened_set = context["statement_transformer"].transform_line(
 line="BIND RUN-UNIT.",
 current_division="PROCEDURE",
 sql_error_paragraph="SQLERROR",
)
 text = "\n".join(lines or [])

 assert "Removed IDMS BIND statement" in text
 assert "CONTINUE." in text

```

```

def test_cobol_transformer_converts_program_id_and_idms_statement() -> None:
 context = build_context()
 transformer = CobolTransformer(
 idms_statement_transformer=context["statement_transformer"],
)

 source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
END PROGRAM OLDPROG.
"""

 converted, messages, operations = transformer.transform(
 cobol_text=source,
 target_program_id="NEWPROG",
)

 assert "PROGRAM-ID. NEWPROG." in converted
 assert "Converted OBTAIN CALC" in converted
 assert len(operations) == 1
 assert messages == []

def test_field_reference_rewriter_rewrites_qualified_reference() -> None:
 context = build_context()

 rewriter = FieldReferenceRewriter(
 mapping_repository=context["mapping_repository"],
 table_name_resolver=context["table_resolver"],
 host_variable_resolver=context["host_resolver"],
)

 result = rewriter.rewrite("MOVE NR-ID OF VMB-FAR TO WS-NR-ID.")

 assert ":NR-ID OF DCLDZBFARTV" in result

def test_pic_length_auto_fixer_expands_target_pic() -> None:
 fixer = PicLengthAutoFixer()

 source = """
01 WS-SOURCE PIC 9(8).
01 WS-TARGET PIC 9(6).
"""

 converted = """
01 WS-TARGET PIC 9(6).
PROCEDURE DIVISION.
MOVE WS-SOURCE TO WS-TARGET.
"""

 result = fixer.fix(
 source_cobol_text=source,
 converted_cobol_text=converted,
)

 assert "PIC 9(8)" in result
 assert fixer.messages

=====
FILE: tests\test_batch6_layout_infrastructure.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.composers.cobol_formatter import CobolFormatter
from idms_db2_phase2.composers.fixed_format_composer import FixedFormatComposer
from idms_db2_phase2.composers.manual_layout_composer import ManualLayoutComposer
from idms_db2_phase2.composers.manual_style_preserver import ManualStylePreserver
from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow

```

```

from idms_db2_phase2.generators.db2_infrastructure_generator import (
 Db2InfrastructureGenerator,
)
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.final_sequence_resequencer_service import (
 FinalSequenceResequencerService,
)
from idms_db2_phase2.services.fixed_format_line_service import FixedFormatLineService

def build_context():
 mapping_repository = MappingRepository(
 [
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
]
)

 dclgen_repository = DclgenRepository(
 [
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="CD_NAME",
 host_variable="CD-NAME",
),
]
)

 table_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
 column_resolver = ColumnNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
 host_resolver = HostVariableResolver(
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
 cursor_resolver = CursorNameResolver()

 return {
 "table_resolver": table_resolver,
 "column_resolver": column_resolver,
 "host_resolver": host_resolver,
 "cursor_resolver": cursor_resolver,
 }

def test_fixed_format_line_service_builds_80_column_line() -> None:

```

```

service = FixedFormatLineService()

line = service.build(
 body="DISPLAY 'HELLO'.",
 left_sequence=10,
 right_sequence=1000,
)

assert len(line) == 80
assert line[:6] == "000010"
assert line[72:80] == "00001000"

def test_fixed_format_composer_formats_lines() -> None:
 composer = FixedFormatComposer()

 result = composer.format(
 "IDENTIFICATION DIVISION.\nPROGRAM-ID. TEST.\n"
)

 lines = result.splitlines()

 assert len(lines[0]) == 80
 assert lines[0][:6] == "000010"
 assert lines[0][72:80] == "00001000"

def test_final_sequence_resequencer_updates_existing_sequences() -> None:
 fixed_format = FixedFormatLineService()
 resequencer = FinalSequenceResequencerService(fixed_format=fixed_format)

 source = "\n".join(
 [
 fixed_format.build("IDENTIFICATION DIVISION.", 10, 1000),
 fixed_format.build("PROGRAM-ID. TEST.", 90, 9000),
]
)

 result = resequencer.resequence(source)
 lines = result.splitlines()

 assert lines[0][:6] == "000010"
 assert lines[0][72:80] == "00001000"
 assert lines[1][:6] == "000020"
 assert lines[1][72:80] == "00002000"

def test_manual_style_preserver_normalizes_blank_lines() -> None:
 preserver = ManualStylePreserver()

 result = preserver.preserve(
 original_text="",
 converted_text="A\n\n\n\nB\n",
)

 assert "\n\n\n\n" not in result

def test_cobol_formatter_removes_trailing_spaces() -> None:
 formatter = CobolFormatter()

 result = formatter.format("A \nB \n")

 assert result == "A\nB\n"

def test_manual_layout_keeps_end_program_last() -> None:
 composer = ManualLayoutComposer()

 source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. TEST.
END PROGRAM TEST.
* DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS
"""

```

```

result = composer.compose(source)

assert result.rstrip().endswith("END PROGRAM TEST.")

def test_db2_infrastructure_generator_inserts_sqlca_and_dclgen_include() -> None:
 context = build_context()

 generator = Db2InfrastructureGenerator(
 table_name_resolver=context["table_resolver"],
 column_name_resolver=context["column_resolver"],
 host_variable_resolver=context["host_resolver"],
 cursor_name_resolver=context["cursor_resolver"],
)

 cobol = (
 "IDENTIFICATION DIVISION.\n"
 "PROGRAM-ID. TEST.\n"
 "DATA DIVISION.\n"
 "WORKING-STORAGE SECTION.\n"
 "PROCEDURE DIVISION.\n"
 "END PROGRAM TEST.\n"
)

 updated, messages = generator.apply(
 cobol_text=cobol,
 operations=[
 IdmsOperation(
 operation="OBTAIN_FIRST",
 record_name="VMB-FAR",
 set_name="AR-VMB-FAR",
)
],
)

 assert "INCLUDE SQLCA" in updated
 assert "INCLUDE SQLERRWS" in updated
 assert "INCLUDE DZBFARTV" in updated
 assert "SQL-LOCATION" in updated
 assert "DECLARE DZBFARC1 CURSOR WITH HOLD FOR" in updated
 assert "FOR READ ONLY" in updated
 assert messages

```

```

=====
FILE: tests\test_batch7_cursor_generation.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.composers.cursor_flow_composer import CursorFlowComposer
from idms_db2_phase2.composers.cursor_order_composer import CursorOrderComposer
from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow
from idms_db2_phase2.generators.cursor_paragraph_generator import (
 CursorParagraphGenerator,
)
from idms_db2_phase2.generators.db2_infrastructure_generator import (
 Db2InfrastructureGenerator,
)
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

def build_context():
 mapping_repository = MappingRepository(
 [
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
)
]
)

```

```

 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
]
)

dclgen_repository = DclgenRepository(
[
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="CD_NAME",
 host_variable="CD-NAME",
),
]
)

table_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
column_resolver = ColumnNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
host_resolver = HostVariableResolver(
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)
cursor_resolver = CursorNameResolver()

infrastructure_generator = Db2InfrastructureGenerator(
 table_name_resolver=table_resolver,
 column_name_resolver=column_resolver,
 host_variable_resolver=host_resolver,
 cursor_name_resolver=cursor_resolver,
)

return {
 "host_resolver": host_resolver,
 "infrastructure_generator": infrastructure_generator,
}

```

```

def test_cursor_paragraph_generator_creates_open_fetch_close() -> None:
 context = build_context()

```

```

 operations = [
 IdmsOperation(
 operation="OBTAIN_FIRST",
 record_name="VMB-FAR",
 set_name="AR-VMB-FAR",
)
]

```

```

cobol = (
 "IDENTIFICATION DIVISION.\n"
 "PROGRAM-ID. TEST.\n"
 "DATA DIVISION.\n"
 "WORKING-STORAGE SECTION.\n"
 "PROCEDURE DIVISION.\n"
 "END PROGRAM TEST.\n"
)

```

```

)

updated, _messages = context["infrastructure_generator"].apply(
 cobol_text=cobol,
 operations=operations,
)

paragraph_generator = CursorParagraphGenerator(
 db2_infrastructure_generator=context["infrastructure_generator"],
 host_variable_resolver=context["host_resolver"],
 sql_error_generator=SqlErrorGenerator(),
)

updated, messages = paragraph_generator.apply(
 cobol_text=updated,
 operations=operations,
)

assert "710-OPEN-DZBFARC1." in updated
assert "720-FETCH-DZBFARC1." in updated
assert "730-CLOSE-DZBFARC1." in updated
assert "FETCH DZBFARC1" in updated
assert ":NR-ID OF DCLDZBFARTV" in updated
assert messages

def test_cursor_order_composer_removes_parent_order_by() -> None:
 composer = CursorOrderComposer()

 source = """
EXEC SQL
DECLARE C1 CURSOR WITH HOLD FOR
SELECT
 COL_A,
 COL_B
FROM TAB1
ORDER BY
 COL_B DESC
FOR READ ONLY
END-EXEC.
"""

 updated = composer.compose(source)

 assert "ORDER BY" not in updated
 assert "COL_B DESC" not in updated
 assert "FOR READ ONLY" in updated

def test_cursor_order_composer_keeps_child_order_by_when_where_exists() -> None:
 composer = CursorOrderComposer()

 source = """
EXEC SQL
DECLARE C2 CURSOR WITH HOLD FOR
SELECT
 COL_A,
 COL_B
FROM TAB1
WHERE PARENT_ID = :PARENT-ID
ORDER BY
 COL_B DESC
FOR READ ONLY
END-EXEC.
"""

 updated = composer.compose(source)

 assert "ORDER BY" in updated
 assert "COL_B DESC" in updated

def test_cursor_flow_composer_rewrites_sqlcode_loop() -> None:
 composer = CursorFlowComposer()

 source = """

```

```

PROCEDURE DIVISION.
PERFORM 710-OPEN-DZBFARC1.
PERFORM 720-FETCH-DZBFARC1.
PERFORM 300-PROCESS-ROW
UNTIL SQLCODE = 100.
720-FETCH-DZBFARC1.
 EXEC SQL
 FETCH DZBFARC1
 END-EXEC.
 EVALUATE SQLCODE
 WHEN ZERO
 CONTINUE
 WHEN 100
 SET WS-DZBFARC1-EOC=YES TO TRUE.
 END-EVALUATE.
END PROGRAM TEST.
"""

```

```

updated = composer.compose(source)

```

```

assert "PERFORM 720-FETCH-DZBFARC1 UNTIL WS-DZBFARC1-EOC=YES." in updated
assert "PERFORM 730-CLOSE-DZBFARC1." in updated
assert "PERFORM 300-PROCESS-ROW." in updated
assert "UNTIL SQLCODE = 100" not in updated

```

```

=====
FILE: tests\test_batch8_final_normalization.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.composers.db2_date_comparison_composer import (
 Db2DateComparisonComposer,
)
from idms_db2_phase2.composers.final_cobol_normalization_composer import (
 FinalCobolNormalizationComposer,
 FinalCobolNormalizationConfig,
)
from idms_db2_phase2.services.cobol_area_alignment_service import (
 CobolAreaAlignmentService,
)
from idms_db2_phase2.services.cursor_select_minimizer_service import (
 CursorSelectMinimizerService,
)
from idms_db2_phase2.services.db2_date_format_service import Db2DateFormatService
from idms_db2_phase2.services.program_name_sync_service import ProgramNameSyncService
from idms_db2_phase2.services.update_procedure_normalization_service import (
 UpdateProcedureNormalizationService,
)

```

```

def test_program_name_sync_uses_final_program_id() -> None:
 source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. VMDZ4420.
PROCEDURE DIVISION.
MOVE 'VM4BD420' TO PROGRAM-NAME.
"""

```

```

updated = ProgramNameSyncService().sync(source)

```

```

assert "MOVE 'VMDZ4420' TO PROGRAM-NAME" in updated
assert "VM4BD420" not in updated

```

```

def test_db2_date_format_can_be_configured_to_iso() -> None:
 source = """
MOVE DA-CREVTDPD-479EVEF OF DCLDZEVEFTV TO DA-DD-MM-CCYY
WHEN DA-DD-MM-CCYY = '01.01.0001'
WHEN DA-DD-MM-CCYY = '31.12.9999'
"""

```

```

updated = Db2DateFormatService(
 date_external_format="YYYY-MM-DD",

```

```

).apply(source)

assert "DA-YYYY-MM-DD" in updated
assert "'0001-01-01'" in updated
assert "'9999-12-31'" in updated

def test_cursor_select_minimizer_removes_order_by_only_column() -> None:
 source = """
EXEC SQL
 DECLARE C1 CURSOR WITH HOLD FOR
 SELECT
 COL_A,
 COL_B
 FROM TAB1
 ORDER BY
 COL_B DESC
 FOR READ ONLY
END-EXEC.
"""

 updated = CursorSelectMinimizerService().minimize(source)

 assert "COL_A" in updated
 assert "COL_B," not in updated
 assert "COL_B DESC" in updated

def test_update_procedure_normalization_converts_dot_reference() -> None:
 source = """
PROCEDURE DIVISION.
MOVE 1 TO DCLDZBFARTV.NR-ID.
EXEC SQL
 UPDATE DZBFARTV
 SET NR_ID = :DCLDZBFARTV.NR-ID
END-EXEC.
"""

 updated = UpdateProcedureNormalizationService().apply(source)

 assert "MOVE 1 TO NR-ID OF DCLDZBFARTV." in updated
 assert ":DCLDZBFARTV.NR-ID" in updated

def test_cobol_area_alignment_indents_procedure_statement() -> None:
 source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. TEST.
PROCEDURE DIVISION.
INITIALIZE DCLDZBFARTV.
END PROGRAM TEST.
"""

 updated = CobolAreaAlignmentService().align(source)

 assert " INITIALIZE DCLDZBFARTV." in updated

def test_db2_date_comparison_generates_helper_logic() -> None:
 source = """
DATA DIVISION.
WORKING-STORAGE SECTION.
PROCEDURE DIVISION.
IF DA-START-DATE OF DCLTAB1 > PARMDATE
 CONTINUE
END-IF.
"""

 updated = Db2DateComparisonComposer().compose(source)

 assert "DB2 DATE CONVERSION WORKING STORAGE" in updated
 assert "HELP-DA-START-DATE" in updated
 assert "IF HELP-DA-START-DATE > PARMDATE" in updated

def test_final_cobol_normalization_runs_core_steps() -> None:

```

```

source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. VMDZ4420.
PROCEDURE DIVISION.
MOVE 'OLDPROG' TO PROGRAM-NAME.
MOVE DCLTAB1.COL-A TO WS-A.
"""

```

```

composer = FinalCobolNormalizationComposer(
 config=FinalCobolNormalizationConfig(
 resequence_final_output=False,
)
)
updated = composer.compose(source)

assert "MOVE 'VMDZ4420' TO PROGRAM-NAME" in updated
assert "MOVE COL-A OF DCLTAB1 TO WS-A." in updated

```

```

=====
FILE: tests\test_batch9_program_normalization.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.composers.retrieval_program_normalization_composer import (
 RetrievalProgramNormalizationComposer,
)
from idms_db2_phase2.composers.sqlcode_wrapper_normalization_composer import (
 SqlcodeWrapperNormalizationComposer,
)
from idms_db2_phase2.composers.update_program_normalization_composer import (
 UpdateProgramNormalizationComposer,
)
from idms_db2_phase2.composers.update_restart_conversion_composer import (
 UpdateRestartConversionComposer,
)
from idms_db2_phase2.domain.models import DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

```

```

def build_context():
 mapping_repository = MappingRepository(
 [
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="NR-ID",
 idms_key="CALC",
 db2_key="PRIMARY KEY",
 new_db2_record="DZBFARTB",
 new_db2_field_name="NR_ID",
),
 SheetMappingRow(
 cobol_record_idms="VMB-FAR",
 cobol_zone="CD-NAME",
 new_db2_record="DZBFARTB",
 new_db2_field_name="CD_NAME",
),
]
)

 dclgen_repository = DclgenRepository(
 [
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",
 column_name="NR_ID",
 host_variable="NR-ID",
),
 DclgenColumn(
 table_name="DZBFARTV",
 group_name="DCLDZBFARTV",

```

```

 column_name="CD_NAME",
 host_variable="CD-NAME",
),
]
)

table_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
host_resolver = HostVariableResolver(
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_resolver,
)

return {
 "mapping_repository": mapping_repository,
 "dclgen_repository": dclgen_repository,
 "table_resolver": table_resolver,
 "host_resolver": host_resolver,
}

def test_retrieval_program_normalization_adds_missing_dclgen_include() -> None:
 context = build_context()
 composer = RetrievalProgramNormalizationComposer(
 dclgen_repository=context["dclgen_repository"],
)

 source = """
WORKING-STORAGE SECTION.
EXEC SQL
 INCLUDE SQLCA
END-EXEC.
PROCEDURE DIVISION.
MOVE 1 TO DCLDZBFARTV.NR-ID.
"""

 updated = composer.compose(converted_text=source)

 assert "INCLUDE DZBFARTV" in updated
 assert composer.messages

def test_retrieval_program_normalization_replaces_error_status_when_flag_exists() -> None:
 context = build_context()
 composer = RetrievalProgramNormalizationComposer(
 dclgen_repository=context["dclgen_repository"],
)

 source = """
WORKING-STORAGE SECTION.
Ø1 SW-STATUS-D PIC X.
PROCEDURE DIVISION.
MOVE 'Ø3Ø7' TO ERROR-STATUS.
"""

 updated = composer.compose(converted_text=source)

 assert "MOVE 'Y' TO SW-STATUS-D." in updated
 assert "ERROR-STATUS" not in updated

def test_retrieval_program_normalization_adds_initialize_before_write() -> None:
 context = build_context()
 composer = RetrievalProgramNormalizationComposer(
 dclgen_repository=context["dclgen_repository"],
)

 source = """
PROCEDURE DIVISION.
MOVE A TO UIT-A.
WRITE UITRECORD.
"""

 updated = composer.compose(converted_text=source)

```

```

assert "INITIALIZE UITRECORD." in updated
assert "WRITE UITRECORD." in updated

def test_update_program_normalization_rewrites_obtain_calc_select() -> None:
 context = build_context()
 composer = UpdateProgramNormalizationComposer(
 mapping_repository=context["mapping_repository"],
 dclgen_repository=context["dclgen_repository"],
 table_name_resolver=context["table_resolver"],
 host_variable_resolver=context["host_resolver"],
)

 source = """
PROCEDURE DIVISION.
*DB2: Converted OBTAIN CALC for VMB-FAR.
EXEC SQL
 SELECT NR_ID
 FROM DZBFARTV
END-EXEC.
IF SQLCODE NOT = 0
 PERFORM SQLERROR
END-IF.
"""

 updated = composer.compose(source)

 assert "Removed OBTAIN CALC SELECT for VMB-FAR" in updated
 assert "Direct UPDATE will use mapped composite key WHERE clause" in updated
 assert "SELECT NR_ID" not in updated

def test_update_restart_conversion_replaces_skipped_block_with_manual_comment() -> None:
 composer = UpdateRestartConversionComposer()

 source = """
PROCEDURE DIVISION.
*DB2: UPDATE conversion skipped for CTRL-REC.
EXEC SQL
 UPDATE T1
END-EXEC.
IF SQLCODE NOT = 0
 PERFORM SQLERROR
END-IF.
"""

 updated = composer.compose(source)

 assert "Manual redesign required for CTRL-REC" in updated
 assert "Missing restart/control mapping metadata" in updated
 assert "UPDATE T1" not in updated

def test_sqlcode_wrapper_normalization_removes_redundant_wrapper() -> None:
 composer = SqlcodeWrapperNormalizationComposer()

 source = """
IF NOT SQLCODE = 100
*DB2: Removed redundant OBTAIN NEXT after cursor EOC loop.
CONTINUE.
IF SQLCODE = 100
 CONTINUE
END-IF.
END-IF.
"""

 updated = composer.compose(source)

 assert "Removed redundant SQLCODE wrapper after cursor EOC loop" in updated
 assert "IF NOT SQLCODE = 100" not in updated

```

=====

EXPORT SUMMARY

=====

Project folder: IDMS-DB2-Module  
Text/code files processed: 229